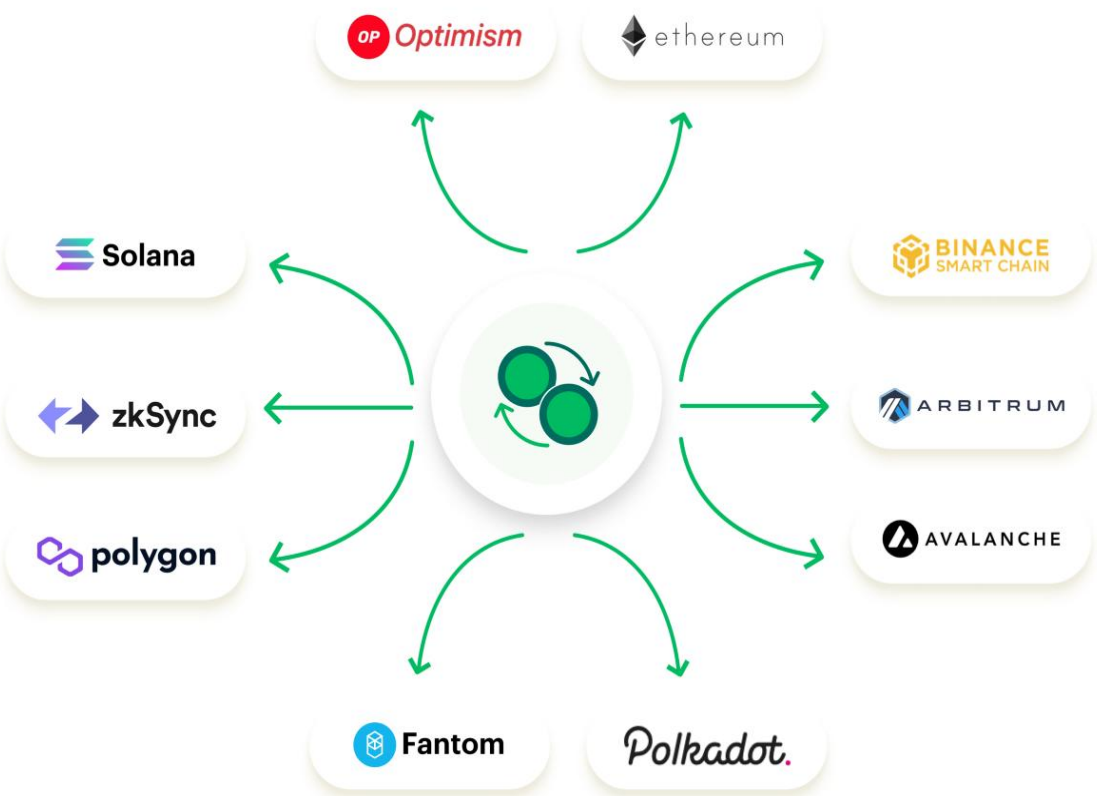
# **Lecture 9** Bridge, Layer2, and Other Consnesus

# Why do we need bridges?

- **Each blockchain is isolated.**
- Separate community (miners, users)
- Separate token



# Interoperability in blockchains

- **Interconnecting blockchains**
- **Increased liquidity**
  - Smooth exchange of assets between different blockchain networks.
- **Enhanced functionality**
  - Access to smart contracts, enhanced privacy, or improved scalability on other blockchains.

# Solutions of connecting blockchain

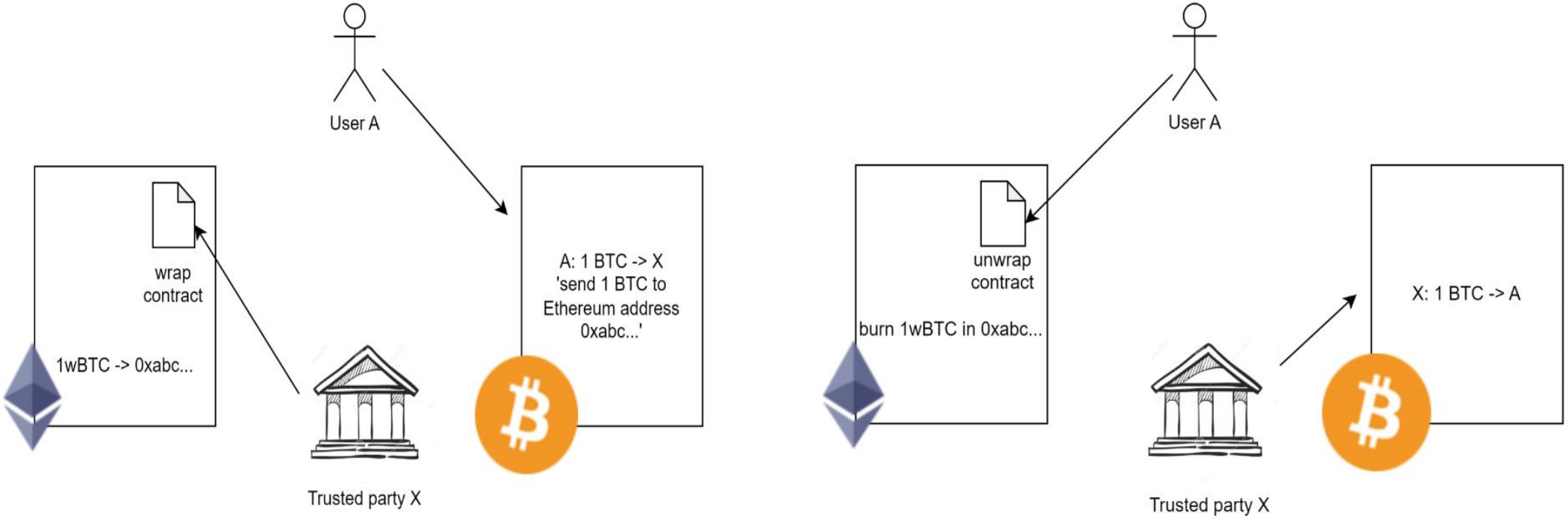
- Wrapped tokens
- Blockchain bridge design
- Interoperability without bridge

# Wrapped tokens

- **Wrapped tokens are digital assets that mirror the value of another cryptocurrency from a different blockchain.**
- **They are created to enhance interoperability between different blockchains, allowing assets from one blockchain to be used on another.**
- **Example**
  - wrapped Bitcoin (wBTC) is a new token in the Ethereum blockchain (tracks value of BTC).

# WBTC as an example

- Move BTC from Bitcoin blockchain to Ethereum blockchain.
- Lock BTC on Bitcoin and get equivalent amount of WBTC on Ethereum (1 to 1).
- Trust the entity that holds the BTC Locked BTC is on-chain in Bitcoin so everyone (both Bitcoin and Ethereum users) can see it.
  - Trust the entity with locked BTC that they do not move it.



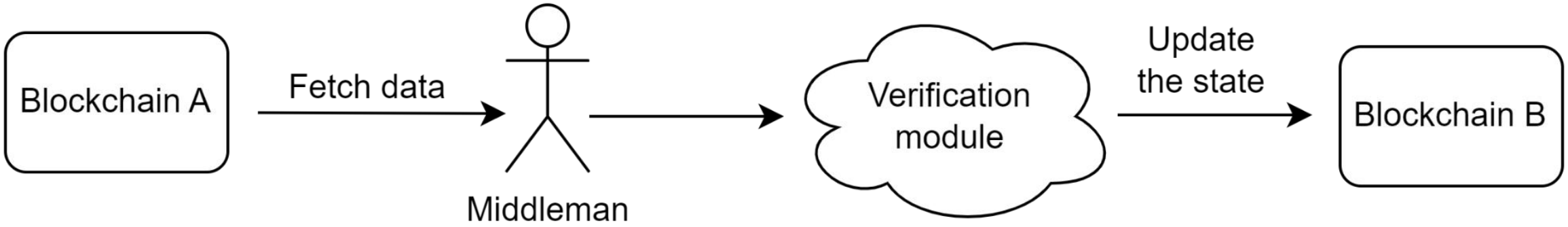
# Bridges enabled DeFi applications

- Cross chain token exchange
- Cross-chain collateralized loans
- Cross-chain yield harvesting

**To enable these applications, we need a general-purpose messaging protocols among blockchains, aka cross-chain bridges.**

# Messaging protocols

- **Headers include the Merkle root of all transactions**
- **Moving headers to other blockchains is enough**
- **Verification module can be on-chain or off-chain**



# Design space

## ■ Middleman

- Relayer: simply relay the data
- Validator: validate and sign the data
- Any oracle network design

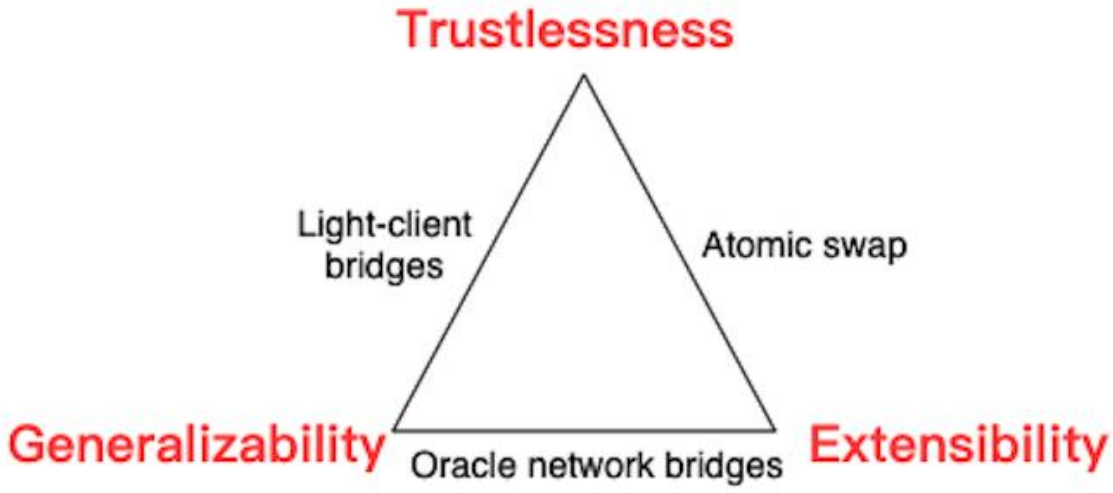
## ■ Verification module

- On-chain light client
- Signature verification
- Oracle contract

# Desired properties

- **Trustlessness: equivalent security to the underlying blockchains**
- **Extensibility: able to be supported on any blockchain**
- **Generalizability: capable of handling arbitrary applications**

Interoperability trilemma



# Bridge designs

## Oracle-network bridges

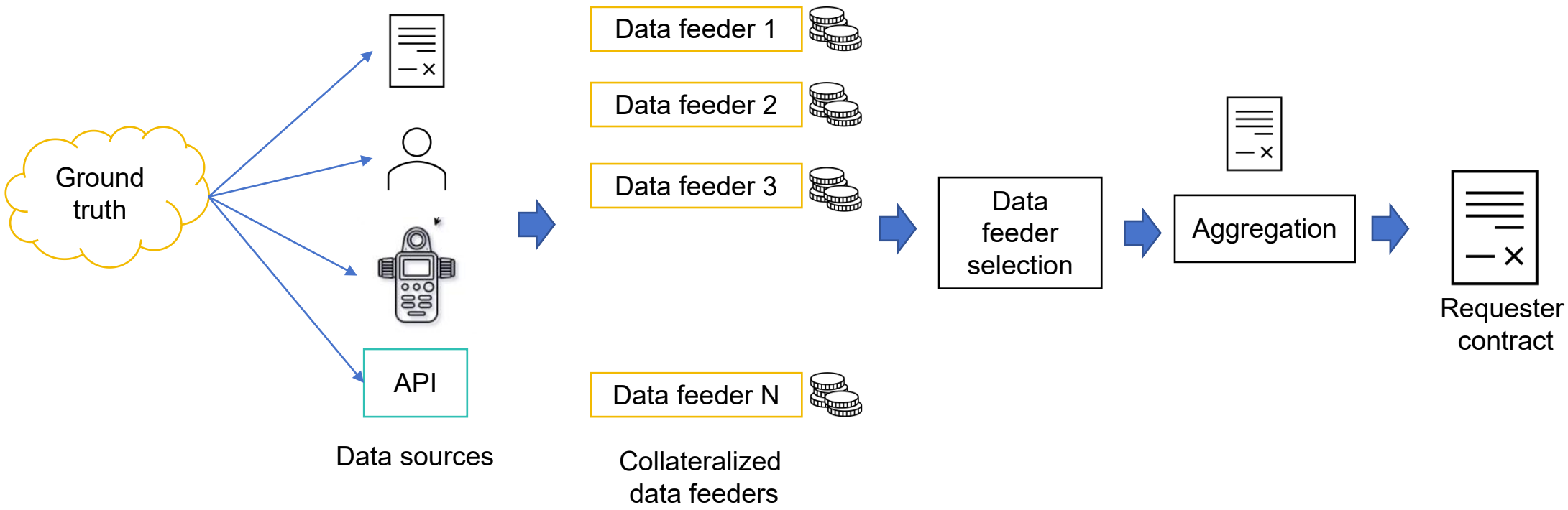
- Layer 0: Oracle and relayer cooperate to verify data
- Axelar: full consensus for verifying data

## Light-client bridges

- Cosmos IBC: light clients embedded in Cosmos-SDK
- ZkBridge: verifier contract checks the validity of the data via ZKP

# Oracles

- **Oracles are a general-purpose fabric connecting blockchains with other off chain systems.**

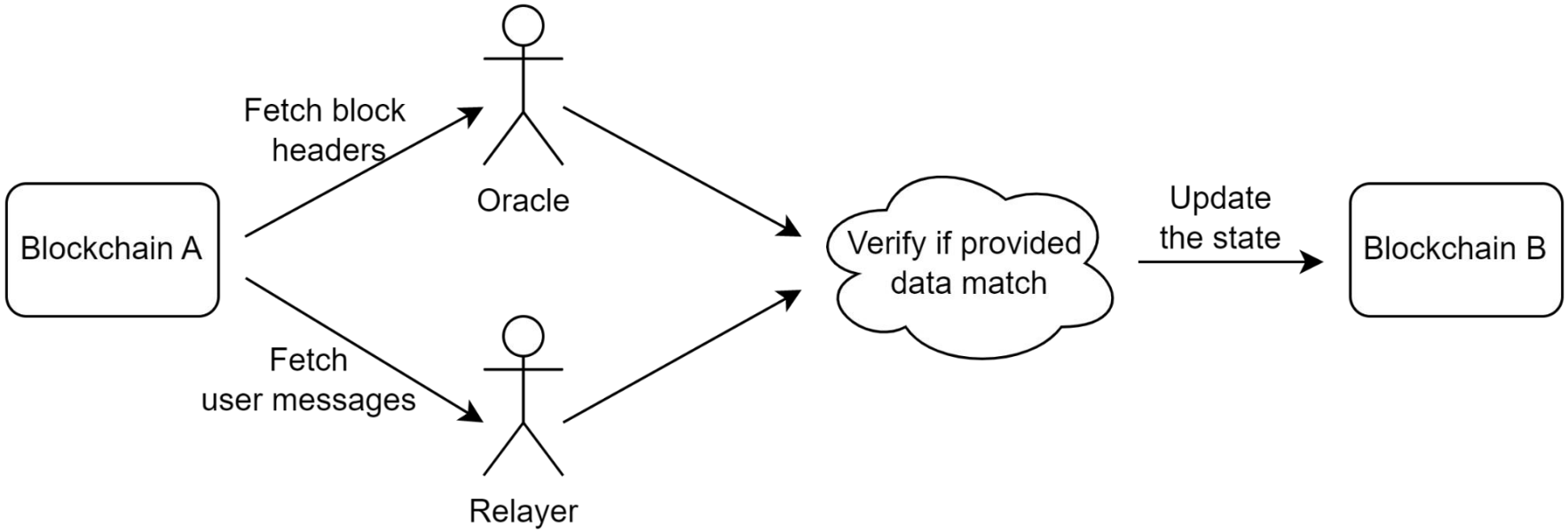


# Oracle-network bridges

- **Middleman: a set of validators or an oracle network.**
- **Verification module: verifies signatures.**
- **Examples:**
  - Layer 0: Oracle and relayer cooperate to verify data
  - Axelar: full consensus for verifying data

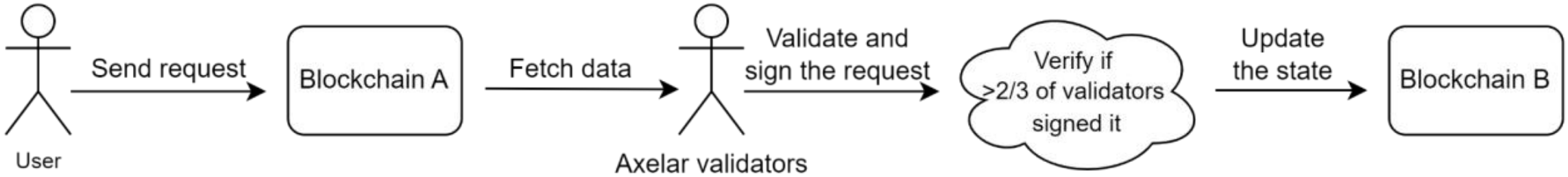
# Layer 0

- An oracle like ChainLink verifies the block headers.
- Nodes called relayers provide proof of inclusion of a message.
- Security assumption: relayer and oracle assumed not to collude.



# Axelar

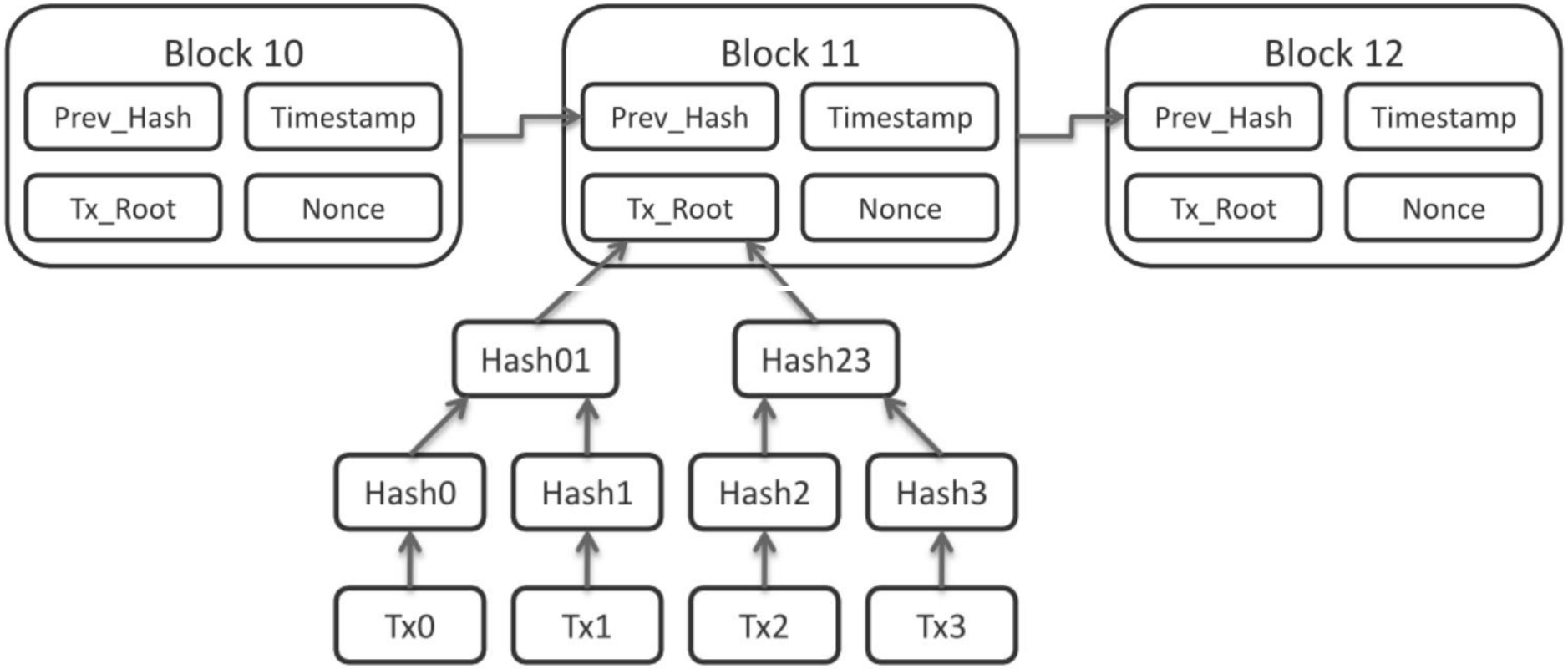
- **Is a Cosmos-based chain.**
- **Uses quadratic voting.**
- **A validator set of 50 validators.**



# Light-client bridges

- **How to get access to a blockchain's data from another blockchain?**
  - Need to run a full node on the destination blockchain
  - Too expensive
- **Use a light client implementation instead.**
- **Access to all transactions and data of the source blockchain on the destination blockchain.**

# Blockchain light clients

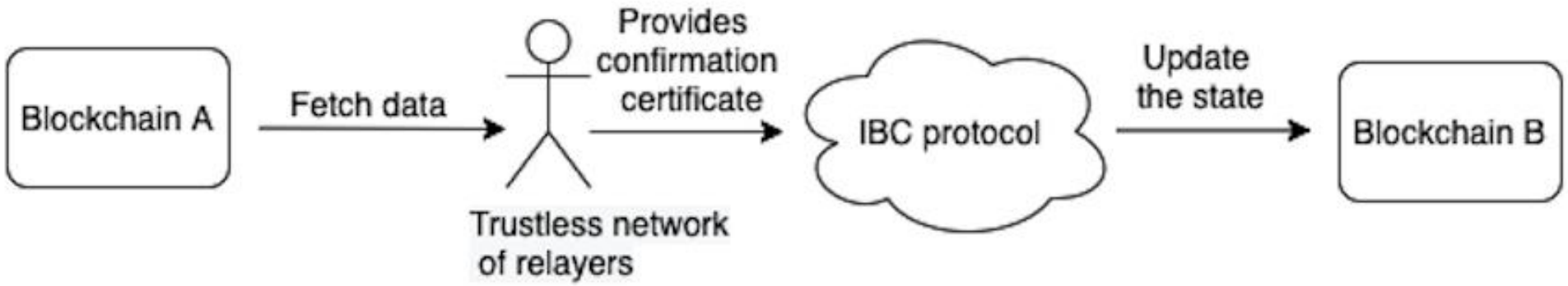


# Light-client bridges

- **Light clients are blockchain nodes that can verify the confirmation of certain transactions without running a full node.**
- **Middleman: trustless relayers.**
- **Verification module: on-chain light clients.**
- **Examples:**
  - Cosmos IBC: light clients embedded in Cosmos-SDK
  - Zk Bridges: verifier contract checks the validity of the data via ZKP

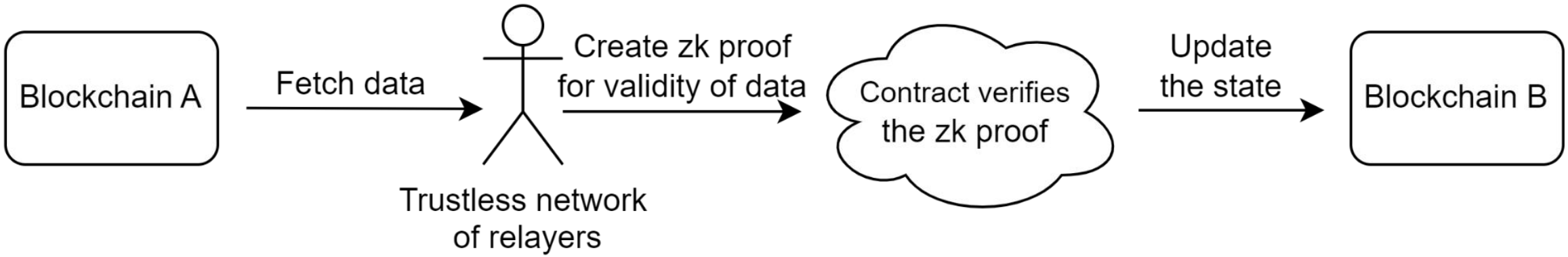
# Cosmos IBC

- The confirmation certificate is a set of valid signatures by the source blockchain nodes.
- IBC protocol is a part of Cosmos consensus mechanism.
- The validators of destination chain verify the validity of data.



# ZK bridges

- **On-chain smart contract verifies the data.**
- **Verification cost reduced by using zero-knowledge proofs.**



# Interoperability not via bridges

## Hashed timelock contract (HTLC)

- HTLC stores a pair (x, t), where x is a hash puzzle and t is a timeout
- If the contract receives the matching secret s such that x = H(s), before time t has elapsed, then the transaction will be executed.
- If the contract does not receive the matching secret s before time t has elapsed, then the transaction will be aborted.

# Hashed timelock contract (HTLC)

## Hashlock key

- Restrict spending an output until some specific data is provided by the recipient.
- The recipient can spend the output by providing that data and a valid signature in a predetermined time limit.

## Timelock key

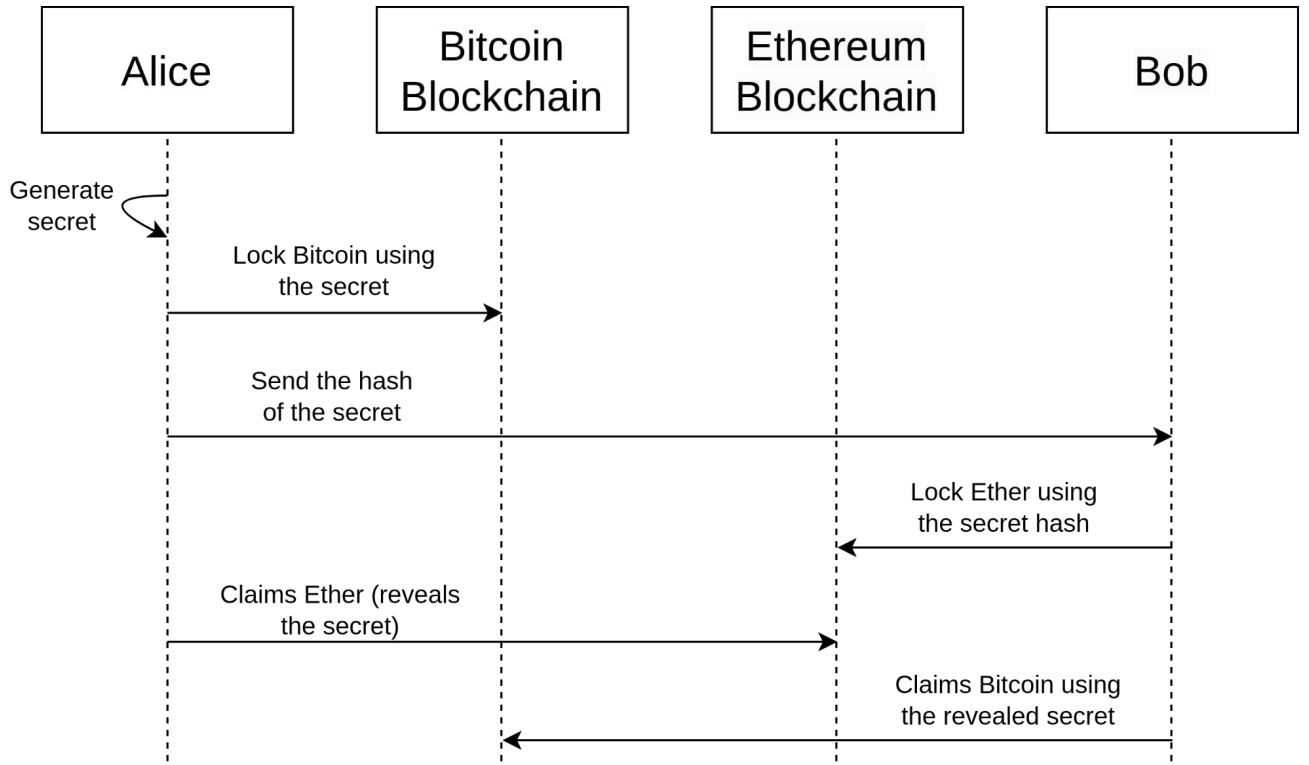
- If the proof is not submitted within a time limit, the coins are returned to the original owner.
- In other words, the owner can spend the output by providing a valid signature after the time limit is passed.

# Atomic swap

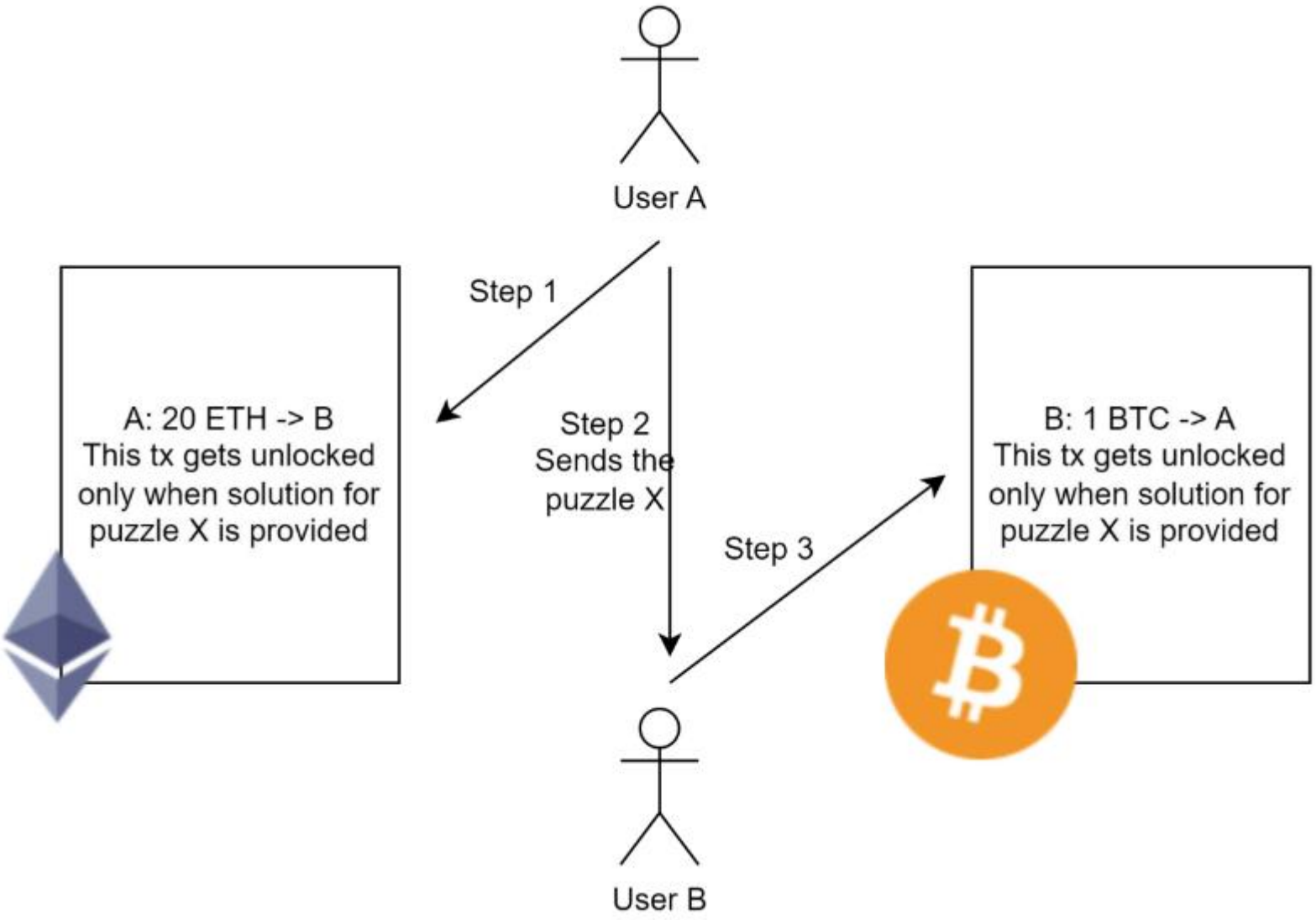
- **Atomic swap is like a link that joins two blockchains for transactions. It ensures that a transaction on one blockchain happens only if a transaction on another blockchain happens too. In simple terms, either both transactions occur, or none occur at all.**

Atomic swap

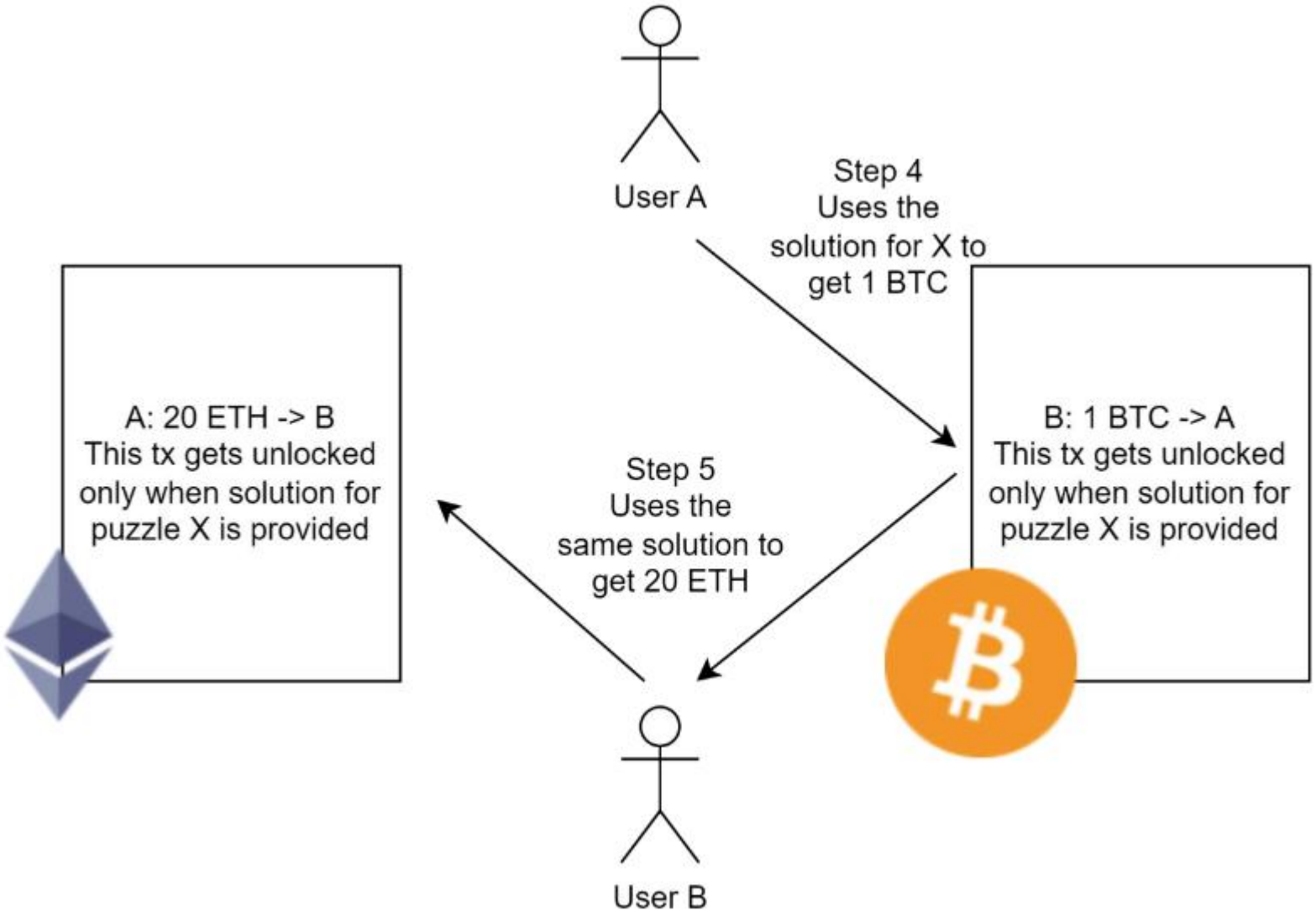
overview



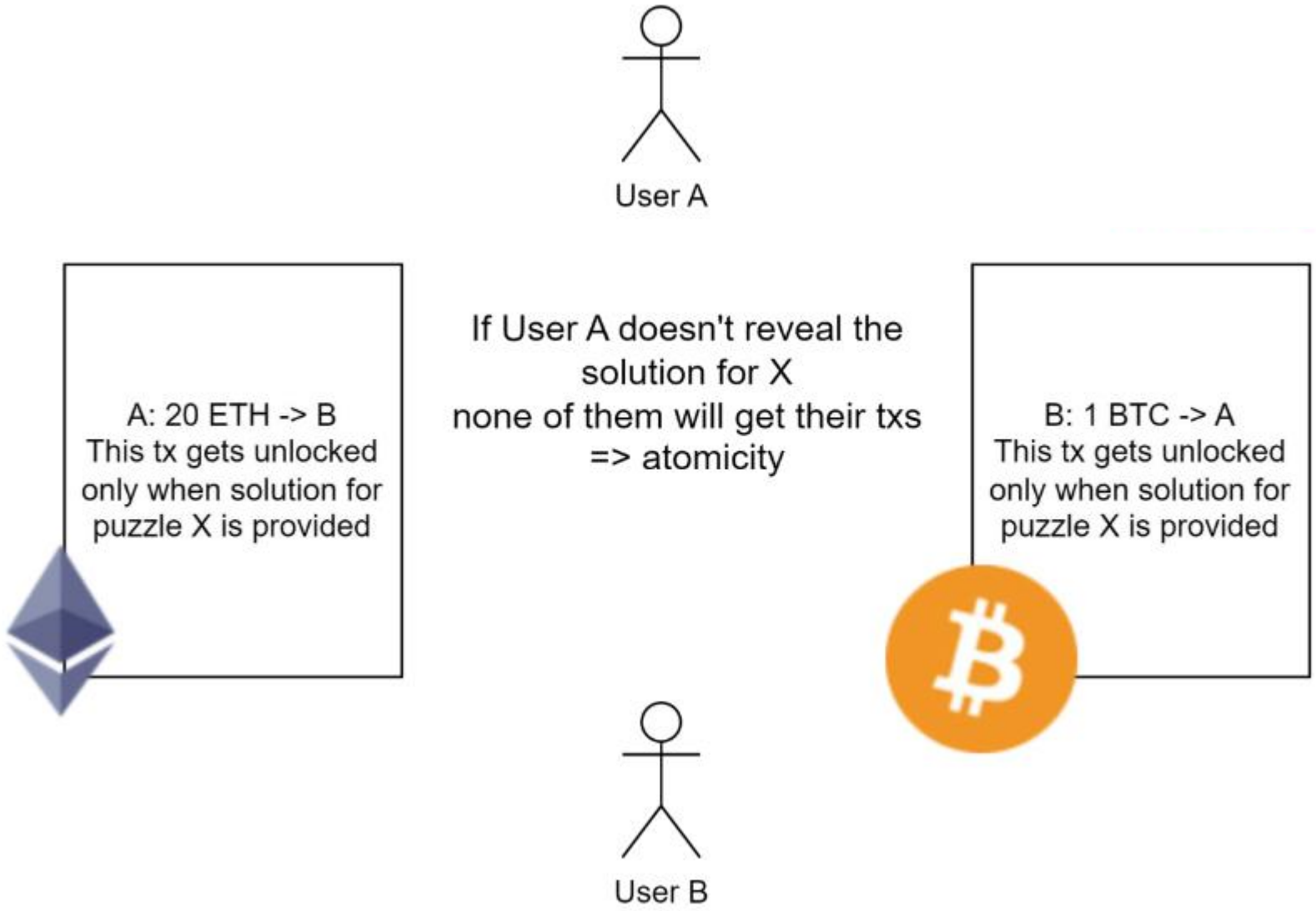
# Atomic swap


User A
Step 1
A: 20 ETH -> B
This tx gets unlocked only when solution for puzzle X is provided
Step 2
Sends the puzzle X
Step 3
B: 1 BTC -> A
This tx gets unlocked only when solution for puzzle X is provided
User B

# Atomic swap (commit)

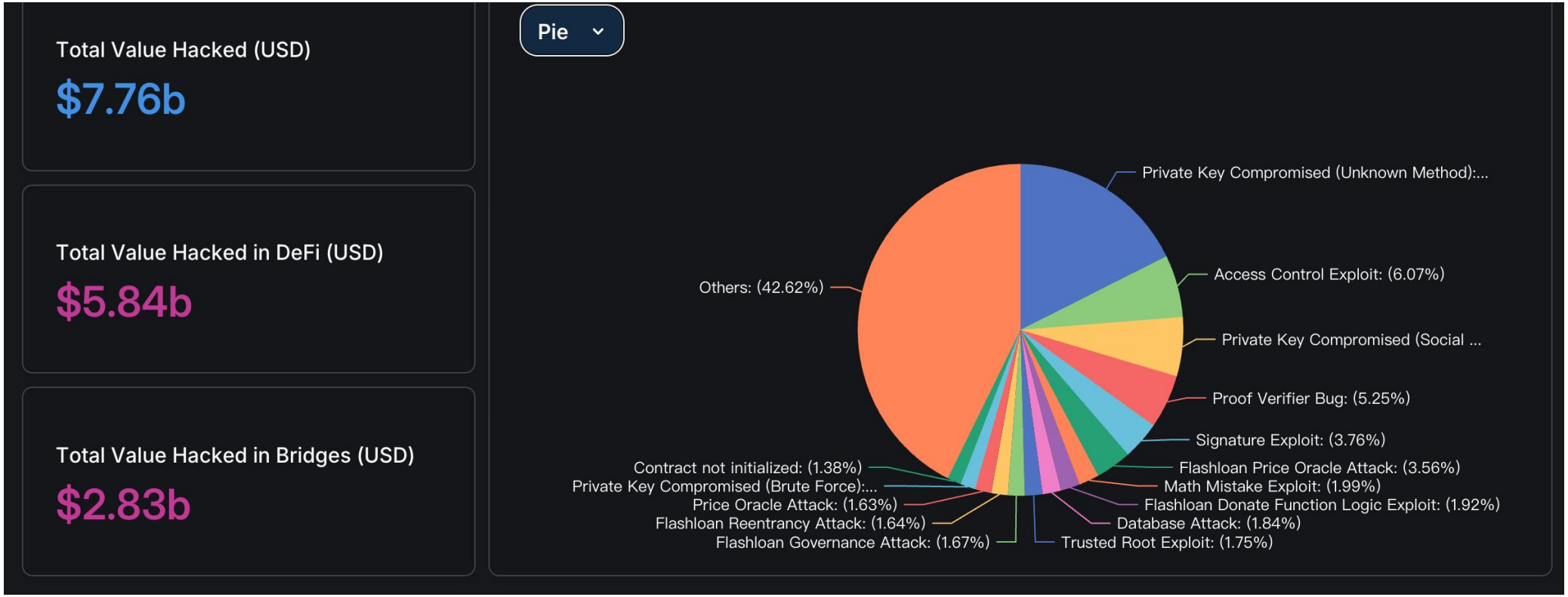


# Atomic swap (abort)



# Bridge attacks

- **To date, cross-chain bridges have been hacked for more than $2.8 billion - representing  almost 40% of the entire value hacked in Web3.**



# Cross-chain bridge vulnerabilities

- Unsecure private key management
- Unaudited smart contracts
- Single network dependency

# Unsecure private key management

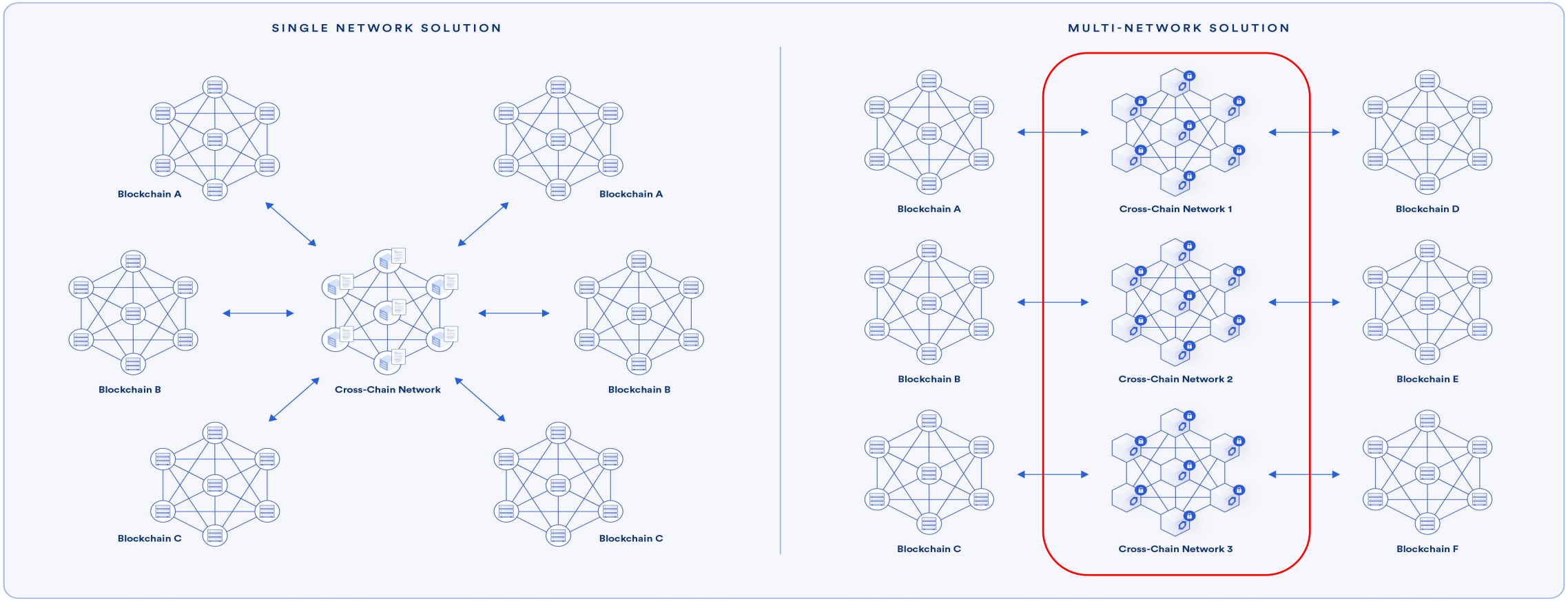
- **Private keys are often what manage a cross-chain bridge's operations. The bridge operator or operators who individually hold a unique private key are tasked with coming to consensus and confirming the message or value being sent cross-chain by a user and delivering it accurately across chains.**
  - Harmony Bridge (2022.06): two out of the five private keys needed to facilitate, approve, and execute transactions on the Harmony Bridge multisig were compromised.
  - Multichain Bridge Hack (2023 07): the cross-chain bridge's compromised private keys, which led to unauthorized withdrawals, were all under the control of Multichain's CEO.
  - Orbit Chain (2024.01): seven out of ten of the bridge's multisig private keys were compromised, enabling attackers to drain funds.

# Unaudited smart contracts

- **In the vast majority of cross-chain use cases, smart contracts on multiple chains are leveraged to power the minting, burning, locking, or unlocking of tokens as they move across chains. These smart contracts are connected through cross-chain bridges, often in collaboration with token issuers.**
  - Qubit (2022.01): a logic error in the bridge's code allowed an attacker to withdraw tokens on BNB Chain when none were deposited on Ethereum.
  - Wormhole Bridge (2022.02): the verification steps outlined in the bridge′ s smart contract were exploited to mint 120,000 wETH on Solana without the appropriate collateralization.
  - Socket Interoperability Protocol (2024.01): a flaw in the protocol's smart contracts affected wallets with infinite approvals to Socket's smart contracts.

# Single network dependency

- Some cross-chain bridges rely on a single network of validators for all bridging operations.



# PoW Issues

Scalability

High Transaction Fee

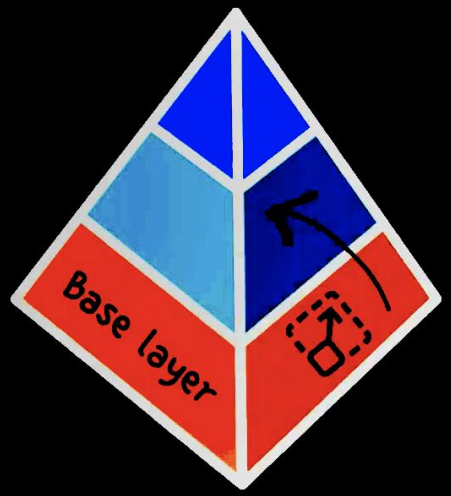
Energy Waste

Network Centralization

# Solution?

Two ways to Scale

- Base Layer
- Outsourcing work to a new layer



# Base Layer Optimizations

# Base Layer Optimizations

Sharding

Directed Acyclic Graph

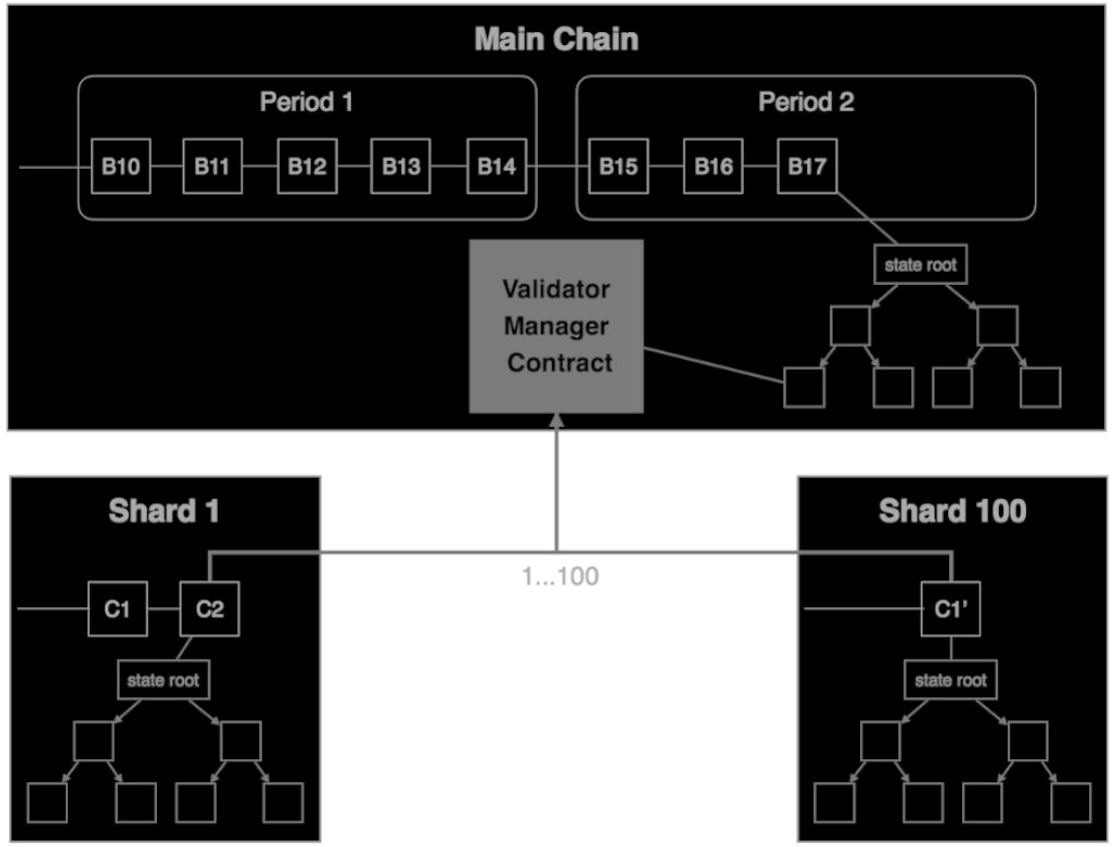
Alternative Consensus Algorithms

# Base Layer Solutions

Sharding

Directed Acyclic Graph

Alternative Consensus Algorithms



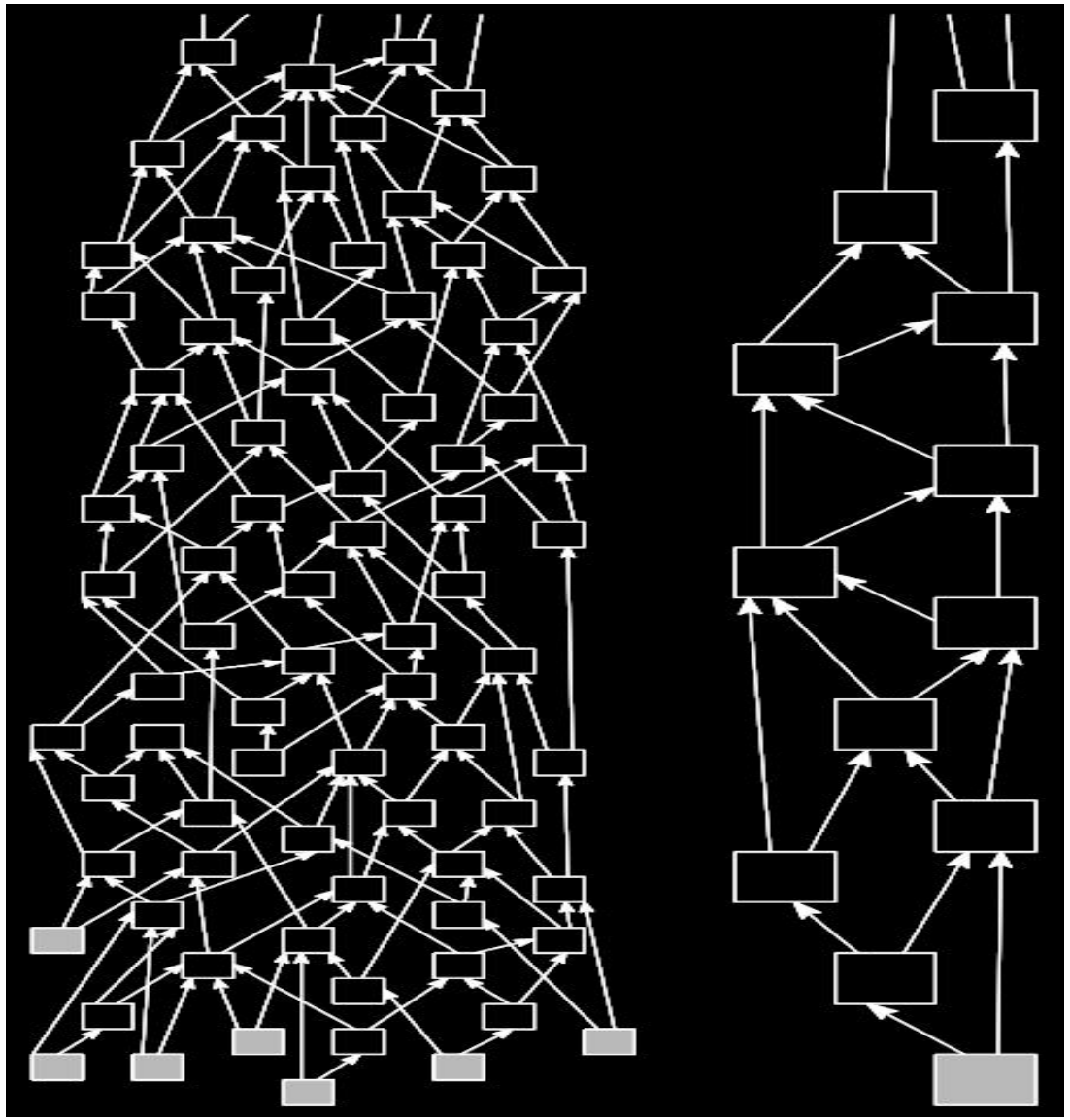
# Base Layer Solutions

Sharding

Directed Acyclic Graph

Alternative Consensus Algorithms

IOTA



# Base Layer Solutions

Sharding

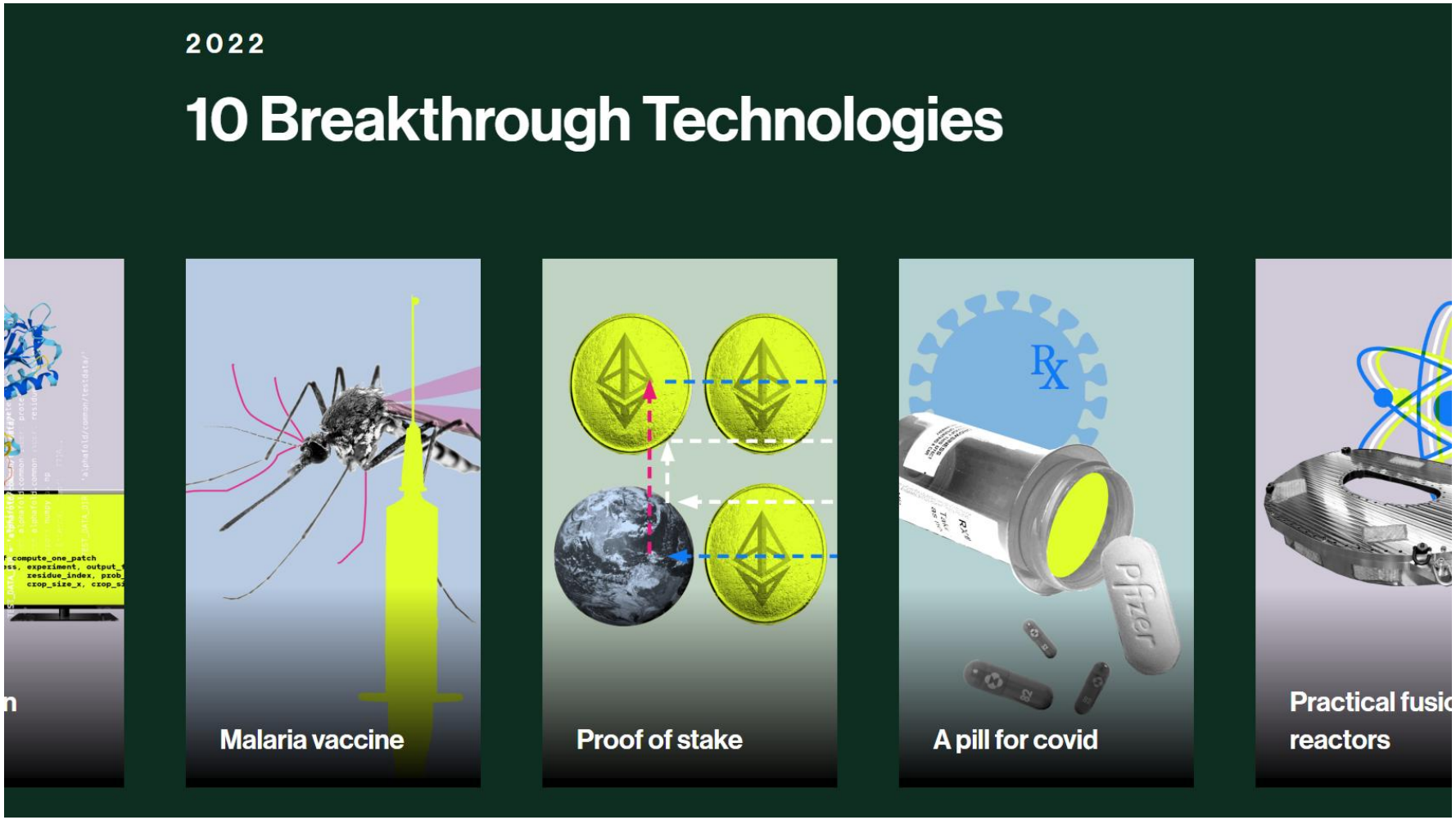
Directed Acyclic Graph

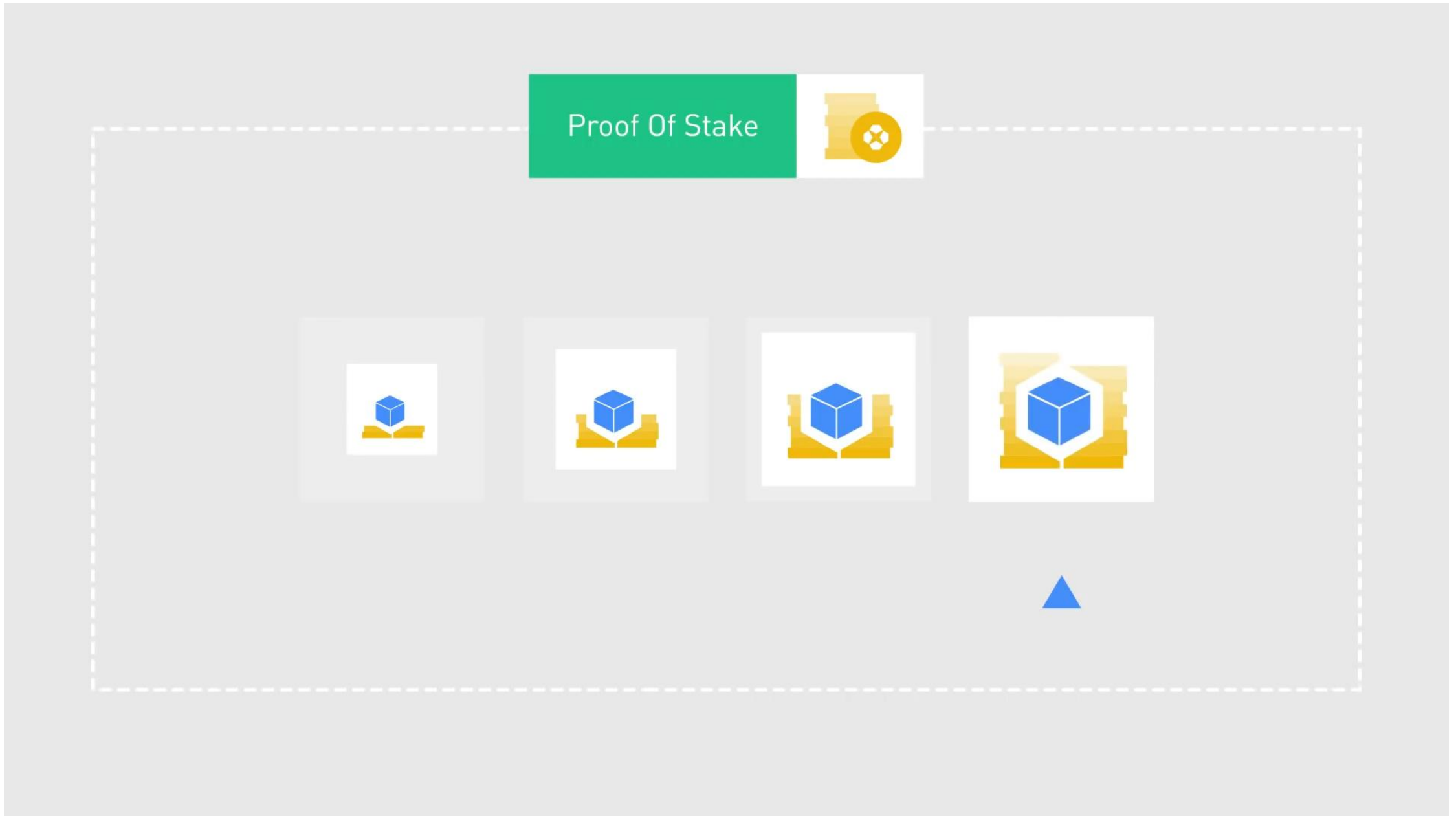
Alternative Consensus Algorithms

# PoS

Proof of Stake

# MIT Technology Review






Proof-of-Stake

# Proof of Stake (PoS)

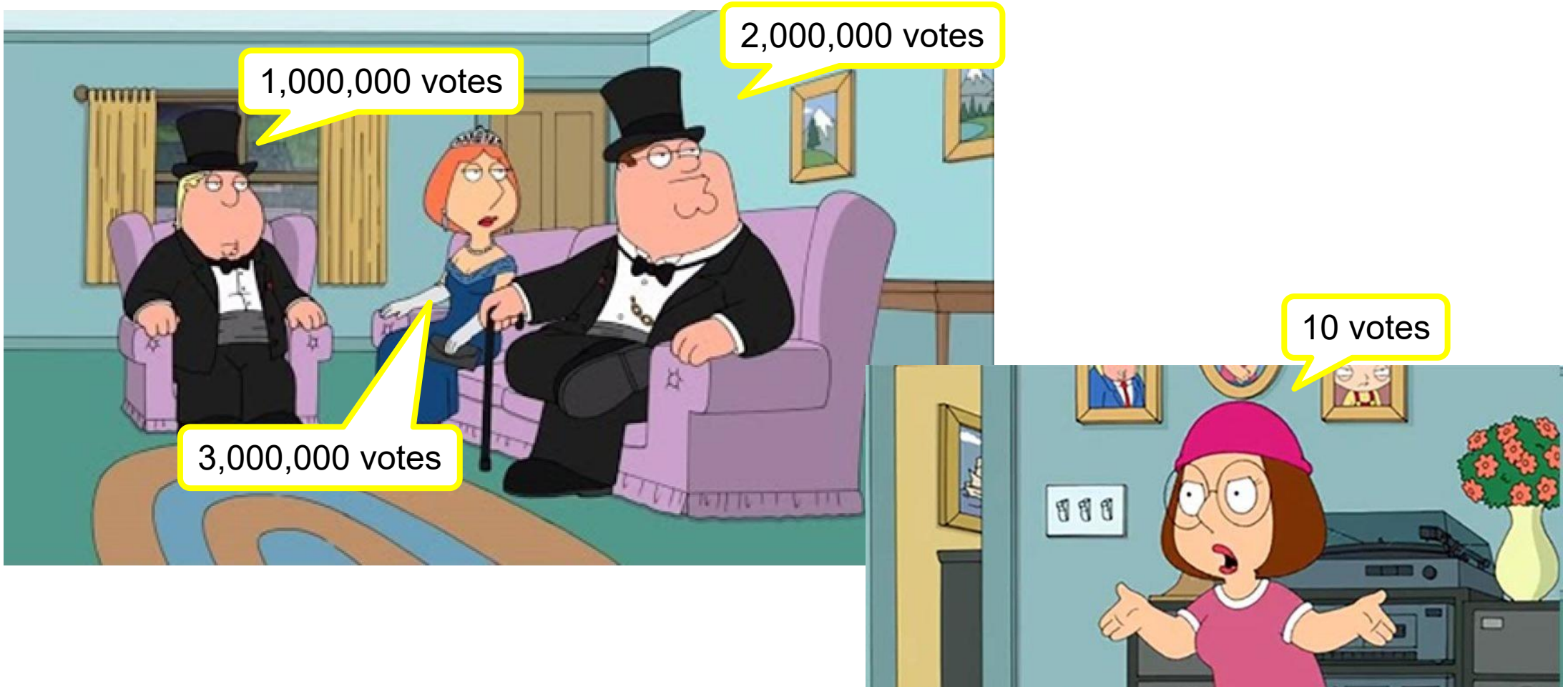
Your vote ~ your holdings

Your holdings also hard to fake

Rich people not likely to wreck things

Not secure from corruption …

# Anti-Sybil: Proof of Stake (PoS)



# Why PoS

- PoS offers more security for the same cost
- Attacks are much easier to recover from in proof of stake
- Proof of stake is more decentralized than ASICs

PoS Introduction from Vitalik: https://vitalik.ca/general/2020/11/06/pos2020.html

# Why Not PoS

- Closed System
  - Higher Wealth Concentration
  - Centralization Future
    - Mandatory KYC?
    - Easier Censorship Attacks?
    - Tax Consequences?
- Nothing at Stake Attack
- Long Range Attack

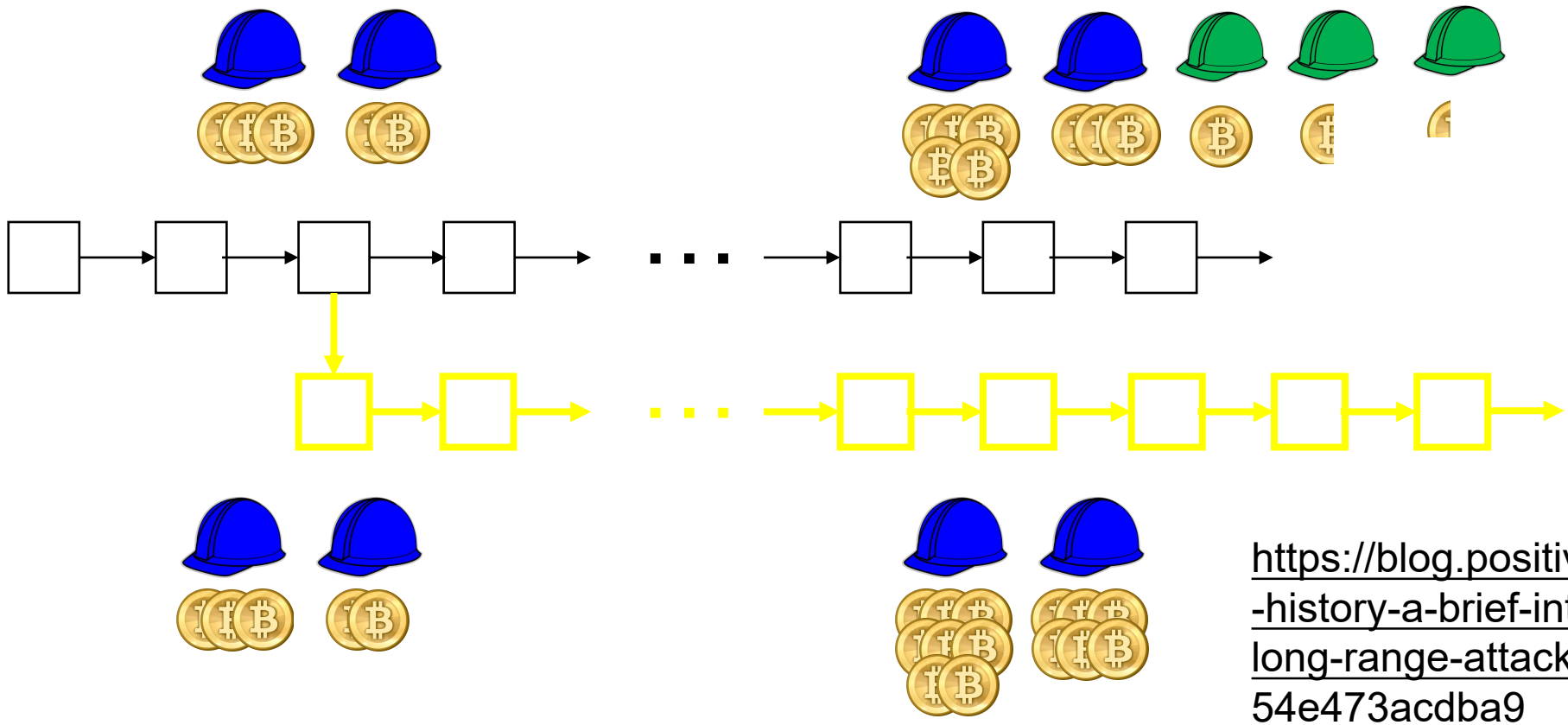
# PoS Problem 1: Nothing at Stake

Reason:
Voting on a PoS blockchain requires no resources and therefore has no opportunity cost

Result:
Rational miners should simply mine on every competing branch they see, so as to maximize the amount of mining returns they get.

# PoS Problem 2: Long Range Attack

Every version of a blockchain that exists or has ever existed could be revived



https://blog.positive.com/rewriting-history-a-brief-introduction-to-long-range-attacks-54e473acdba9

# PoS Blockchains

PoS blockchains

- Cardano
- Solana
- Algorand
- Tezos
- Celo
- Ethereum 2.0

# Ethereum 2.0

**Joe Lubin**

You can think of Ethereum 1.0 as a prototype. We had to release something that we knew wouldn't be scalable to prove that you could build decentralized applications.

**Vitalik Buterin**

ETH 2.0 is finalized except for security audits.

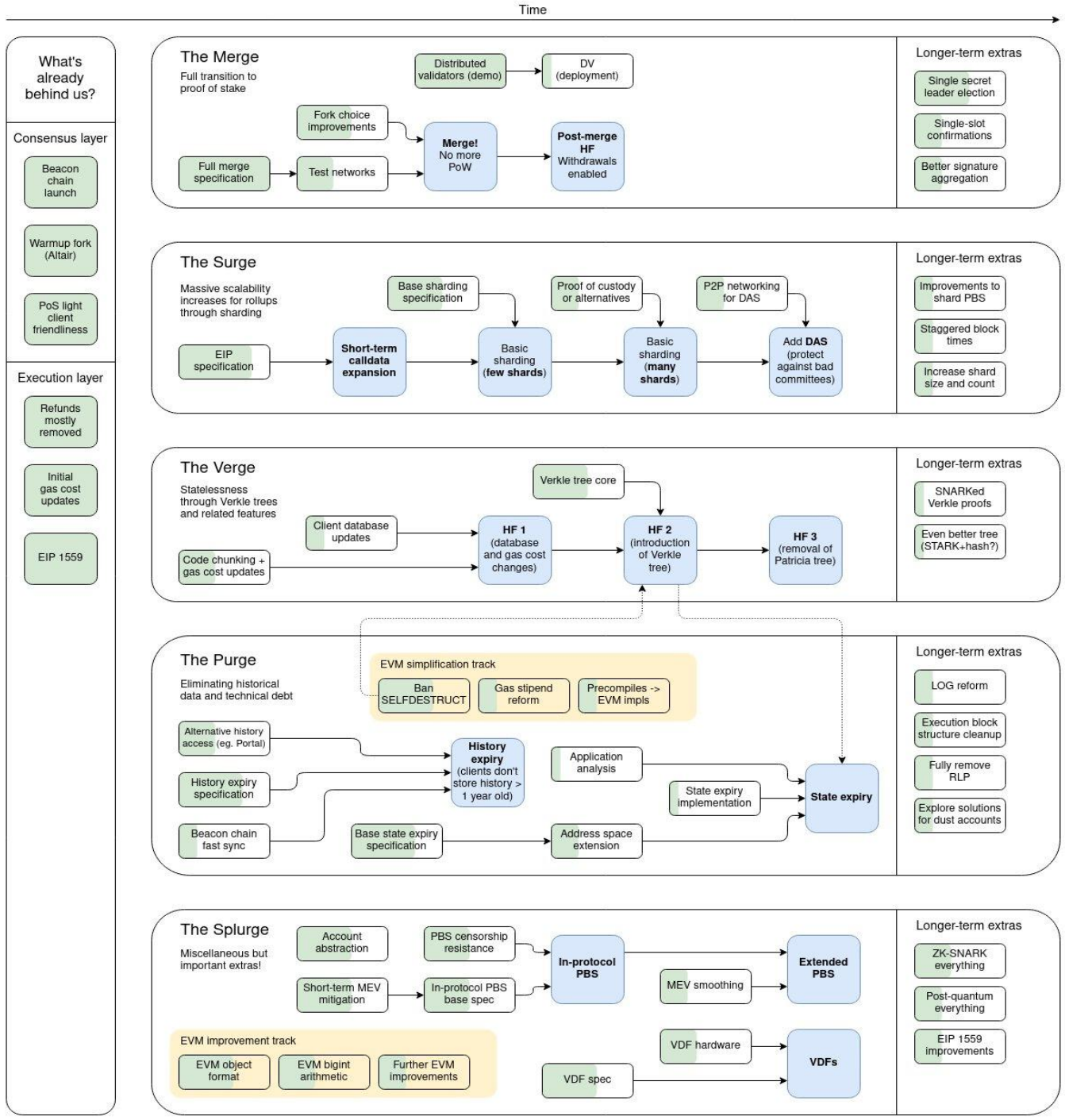
# Ethereum 2.0

- Has been discussed and developed since 2018
- The first phase rolled out at the end of 2019
- Meant to help Ethereum scale, and it has two major goals:
    - Introduce a PoS consensus to eliminate the PoW Cost
    - Introduce sharding to improve the speed and throughput

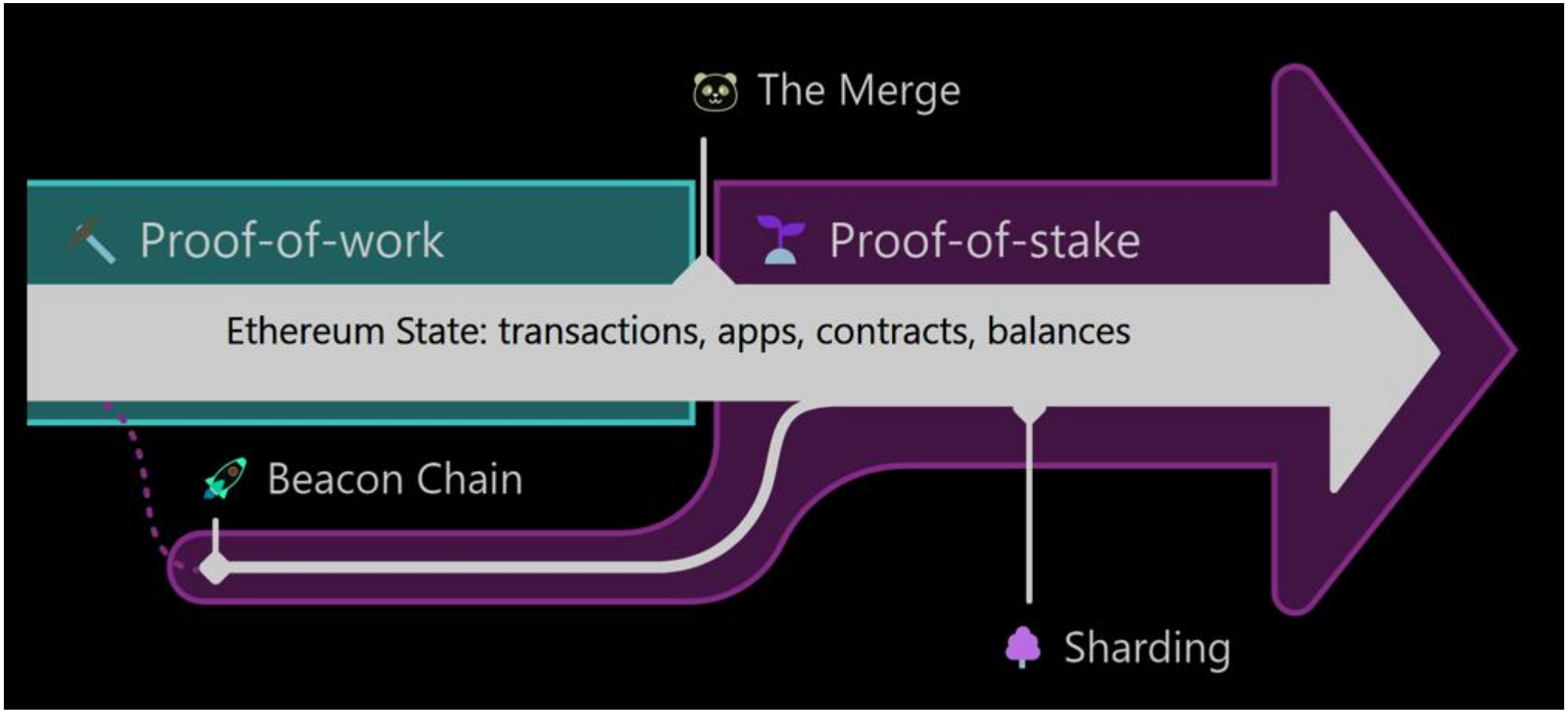
# Ethereum 2.0

- Five Stages
  1. The Merge
  2. The Surge
  3. The Verge
  4. The Purge
  5. The Splurge

Vitalik at ETH Communit Conference
Paris, 2022



# Stage 1: The Merge



# Stage 1: The Merge

- The merging of the execution layer (current mainnet) and the Beacon Chain (new POS layer)
- $ETH:
  - Now becomes a deflationary asset (reduced issuance)
  - Cuts power usage by 99%
- Merge Date: September 19th, 2022

# Beacon Chain

A separate but connected chain (Sidechain)

Adopts CASPER PoS Consensus Algorithm

Consensus is achieved through a set of validators (nodes)

- Each validator “stakes” 32 ETH to participate in the consensus process
- Validators propose new blocks and validate new transactions through voting
- Validators are rewarded for good behavior and penalized for bad behavior

# CASPER: Solution to “Nothing at Stake”

**Wrong-Voting Penalty**

allows users to submit evidence of voting on the wrong chain by miners in order to penalize that miner for wrong-voting

# CASPER: Solution to “Long Range Attack”

## Roundabout Way

Requiring that only blocks with a certain range of prior blocks can be disputed, while the rest are a part of the “main chain”.

## Leads to Weak Subjectivity

A node that comes online for the first time will have to ask a trusted source what the hash of the valid chain is.

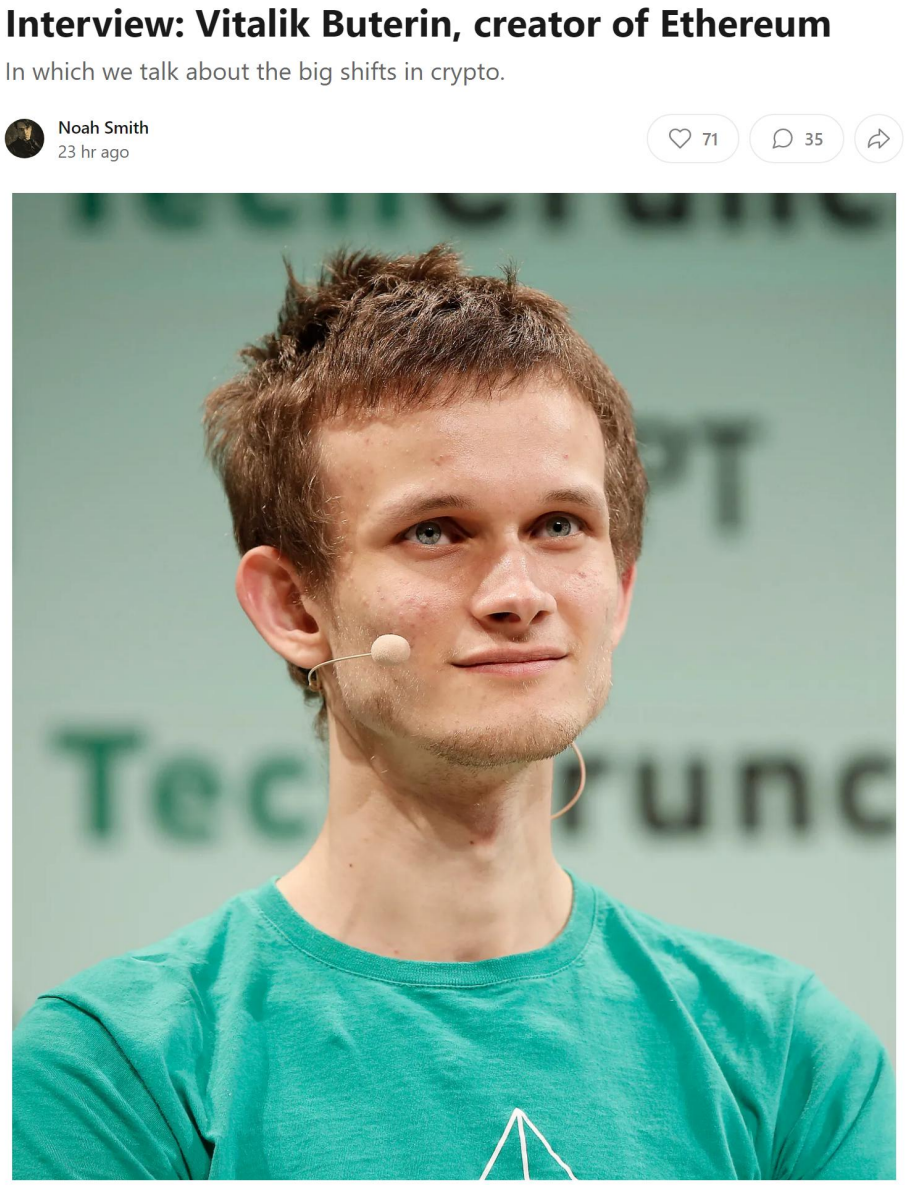
need to depend on trusted nodes to broadcast the correct block hash.

Vitalik Buterin

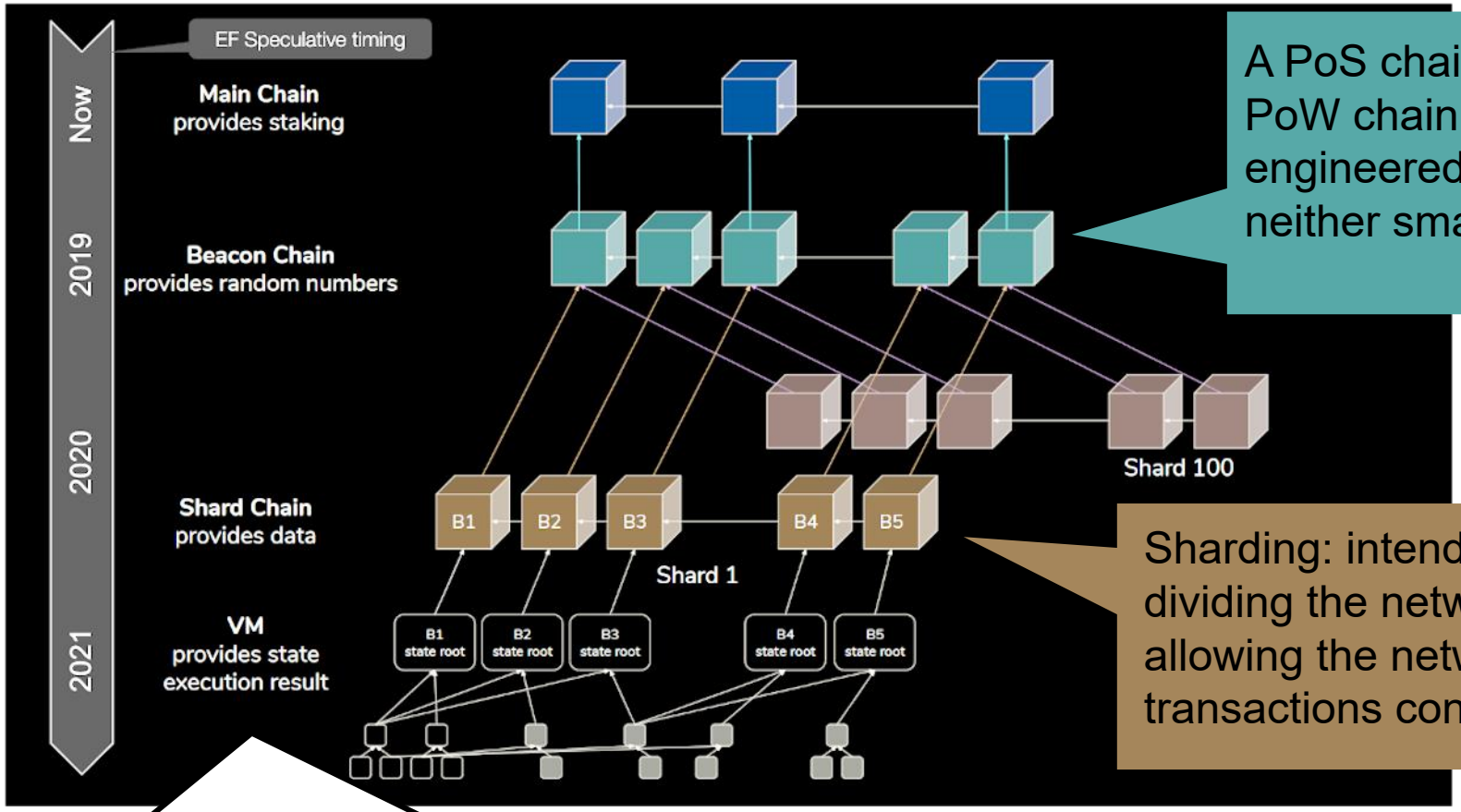
https://medium.com/@abhisharm/understanding-proof-of-stake-through-its-flaws-part-3-long-range-attacks-672a3d413501

# Vitalik Inteview

- https://noahpinion.substack.com/p/interview-vitalik-buterin-creator



# Ethereum 2.0 Roadmap




A PoS chain running parallel to Ethereum's PoW chain. Initially, the Beacon Chain will be engineered for simplicity and will support neither smart contracts nor accounts.

Sharding: intended to help transactions scale by dividing the network across multiple shards, allowing the network to process many transactions concurrently.

The eWASM is essentially a rebuilt EVM that supports PoS and sharding. It will introduce smart contracts, accounts, and more to Ethereum 2.0.

# ETH 2.0 and its impact on $ETH issuance




https://twitter.com/Montana_Wong/status/1556706440726716416

# Debates on Decentralizations in ETH

- Designer of Bit Gold, Precursor of BitCoin
- Inventor of Smart Contract Concept (1994)
- Real Original Gangster
- “Trust Minimization”






Nick Szabo

Computer Scientist & Legal Scholar

*https://www.primitiveslane.org/post/a-brief-study-of-the-shifts-in-nick-szabos-attitude-towards-ethereum*

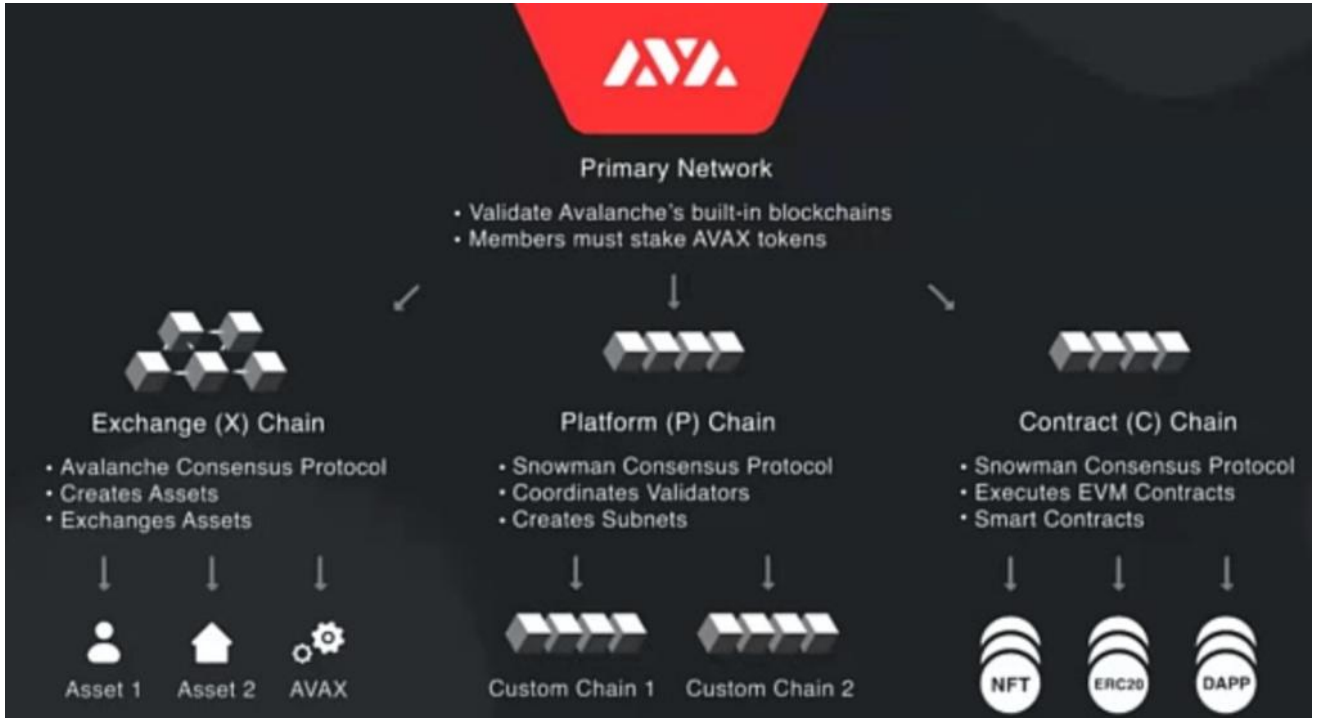
# Algorand

- Pure Proof-of-Stake (PPoS)
- A user secretly checks whether they were selected to participate by evaluating a Verifiable Random Function (VRF)
- Rank 31st in Market Cap (as of Oct 2022)



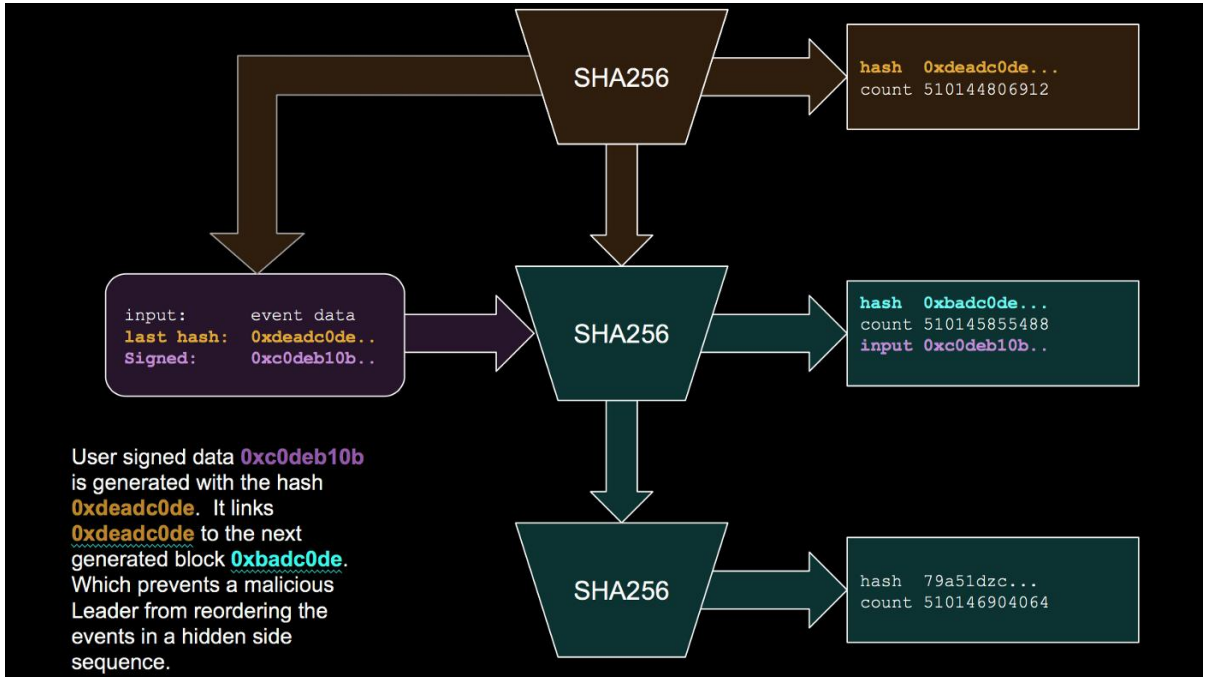
# Avalanche

- Led by Emin Gün Sirer (Cornell), PoS to resisit Sybil Attack
- Rank 17th in Market Cap (as of Oct 2022)



# Solana ($SOL)

- DPoS + Proof-of-History (PoH)
- Rank 10th in Market Cap (as of Oct 2022)






# Binance Smart Chain (BSC)

- Supports EVM
- Proof of Stake Authority = PDoS + PoA
  - PoA refers to the picking of validators by a central authority
  - Validators picked by Binance...
- Ultra-fast Network Throughput: 3 seconds Block Time
- Pretty Centralized

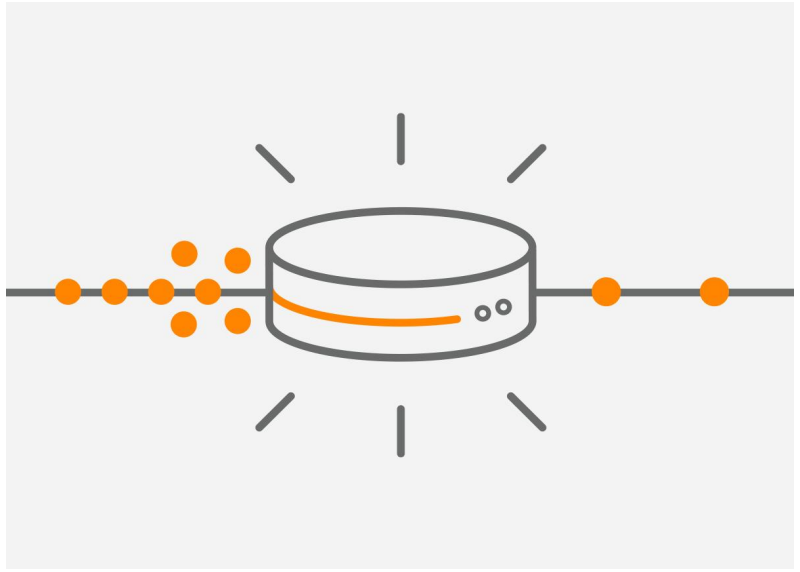


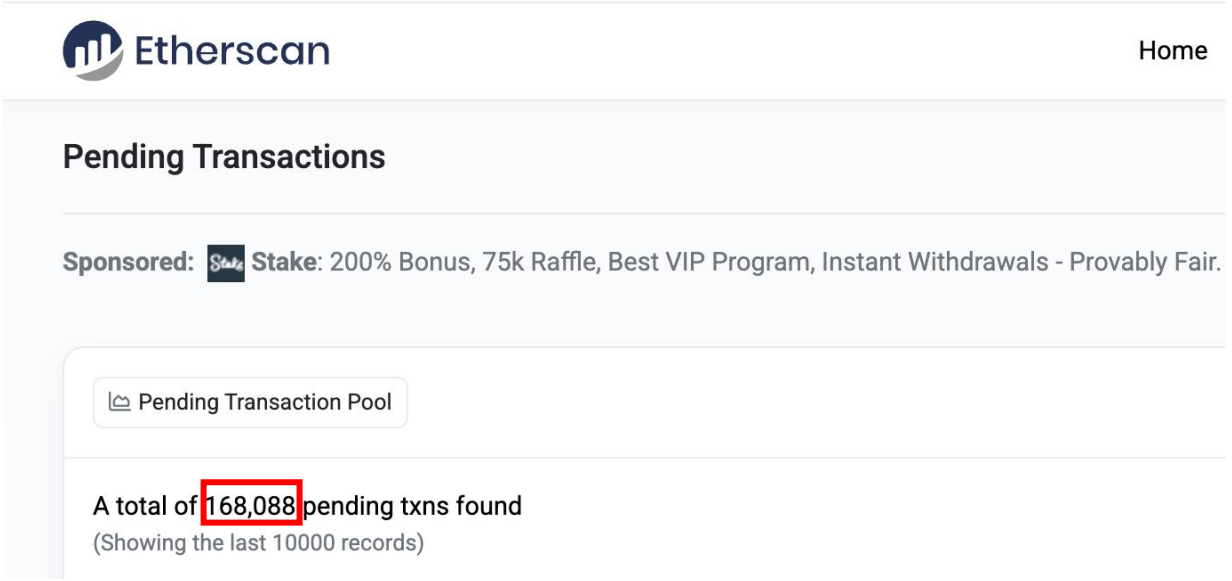
# Layer 2

# Current Issue of Blockchain System

## Blockchain Network Congestion

- High transaction fee
- Long waiting time before confirmation
- Reduce User Experience





# Blockchain Network Congestion

**Reason:** Low Transaction Processing Speed

**Measurement Metric:** Throughout - Transactions Per Second (TPS)

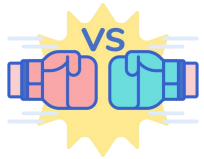
**Blockchain Networks**



3~7 TPS
(block interval: 10 minutes)



10~25 TPS
(block interval: 12 seconds)




**Mainstream Payment Networks**



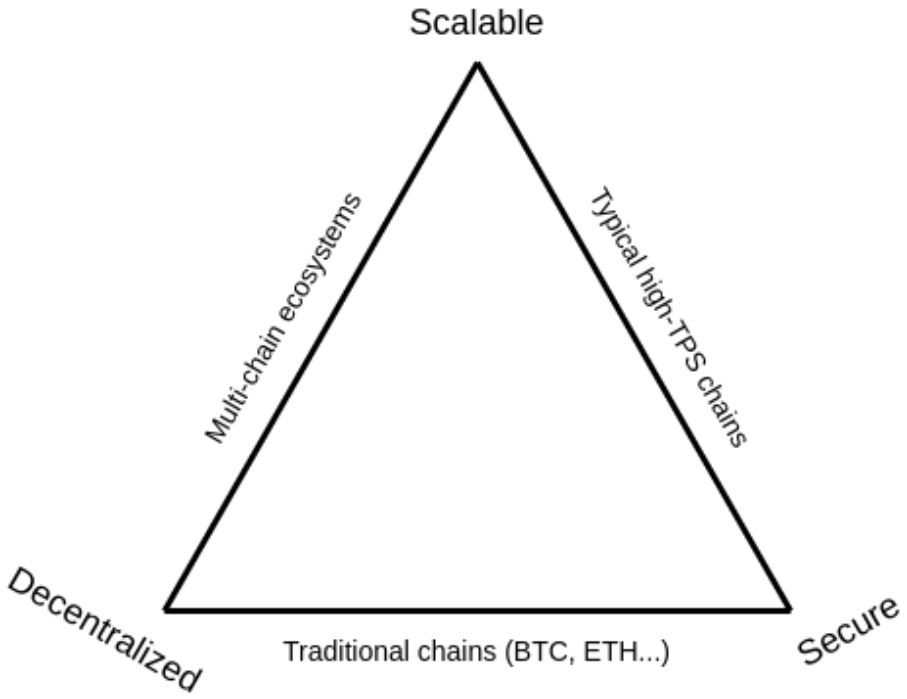

10K~50K TPS

# The Scalability Trilemma

3 properties that blockchain system try to have:

- **Scalability**
- **Decentralization**
- **Security**

In practice, a blockchain system can only fully achieve 2 of these 3 properties.

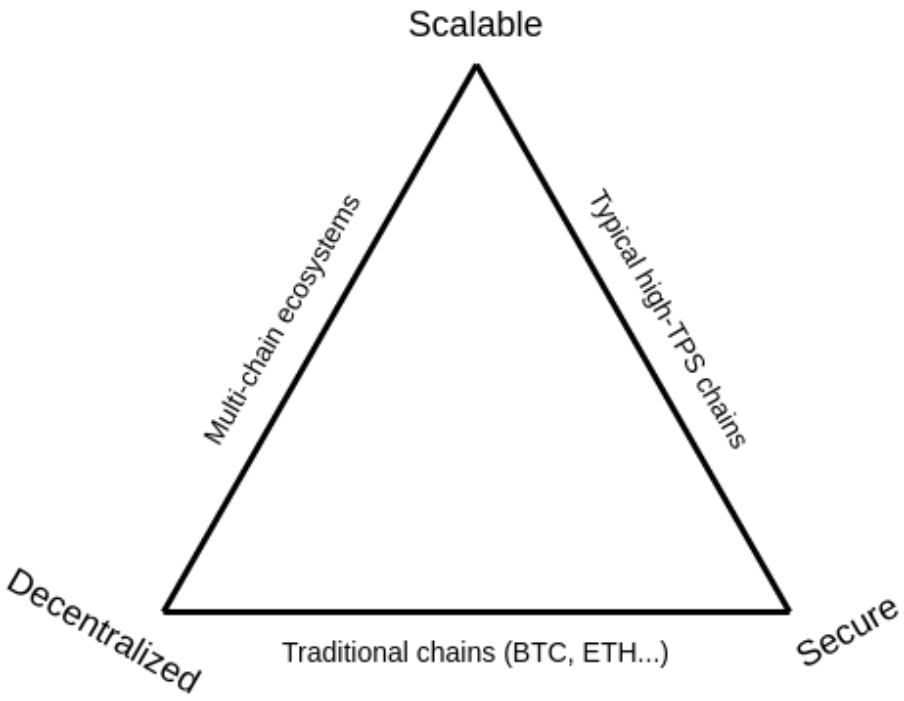



https://vitalik.eth.limo/general/2021/04/07/sharding.html

# The Scalability Trilemma

3 properties blockchain system try to have:

- **Scalability:**
  A scalable blockchain can process a high volume of transactions quickly and efficiently, without significantly increasing transaction costs or times.




- Scalability = Performance
  => High TPS + Low latency

- Benefits of High Scalability:
  - faster payment

# The Scalability Trilemma

3 properties blockchain system try to have:

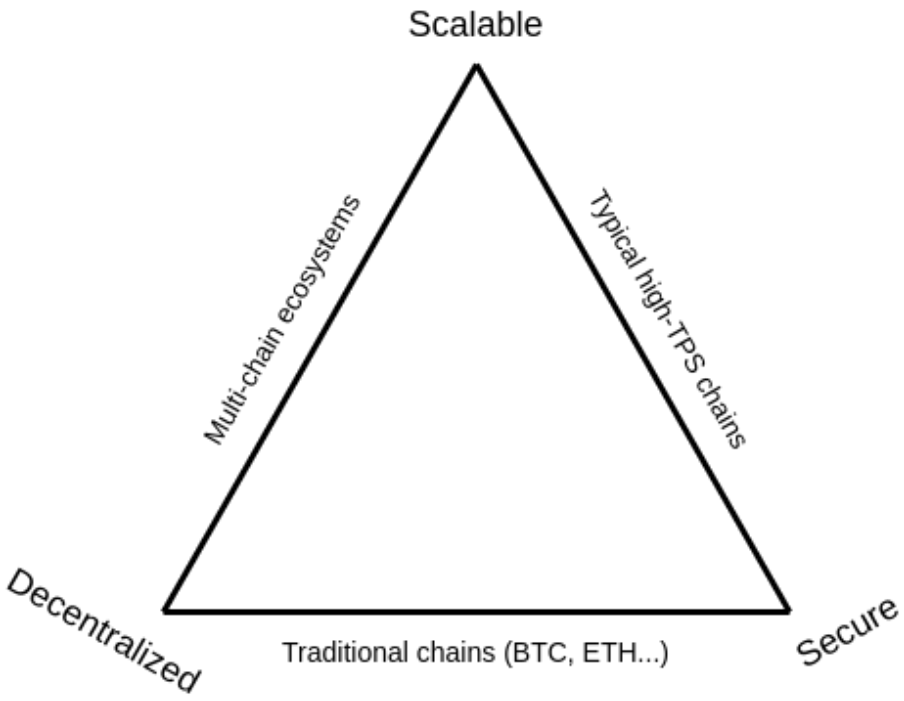
- **Decentralization:**

  Your normal laptop can run a full node.

  Not only "industrial-grade" computers to be blockchain nodes

- Target: Lower resource usage for maintaining a node.

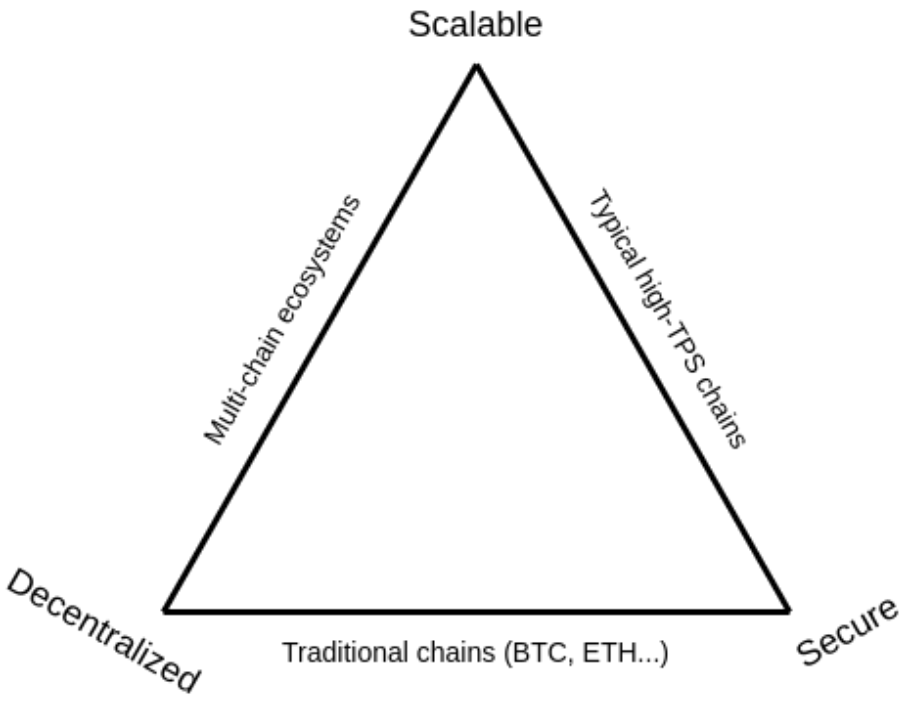
- Constrained Resource:
  - Computation Power ( Laptop CPU )
  - Network Bandwidth
  - Disk Storage ( **Main Bottleneck today** )



# The Scalability Trilemma

3 properties blockchain system try to have:

- **Security:**
  The chain can resist a large percentage of participating nodes trying to attack it.
- Vitalik: ideally 50%; anything above 25% is fine, 5% is definitely not fine.




- Solution for Security:
  - Pick a consensus with a high fault tolerance and enforce it with strict consensus thresholds.
  - Decentralization as a security booster.
  - Add Economic punishment to make attacking unprofitable.

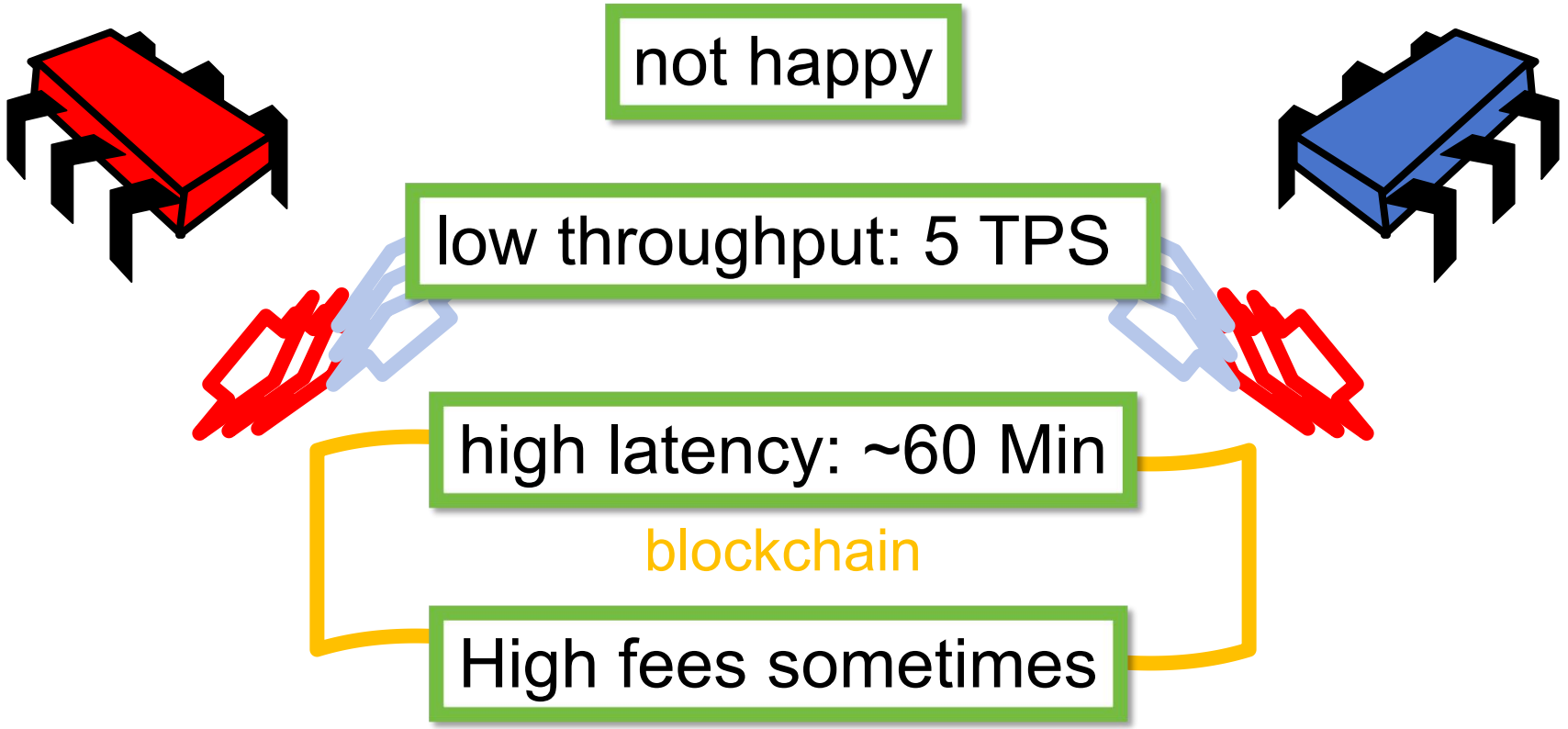
# Ways To Scale Blockchain

The way is a bit like scaling traditional databases

List of the ways:

1. **Payment Channel**
2. **Layer 2**

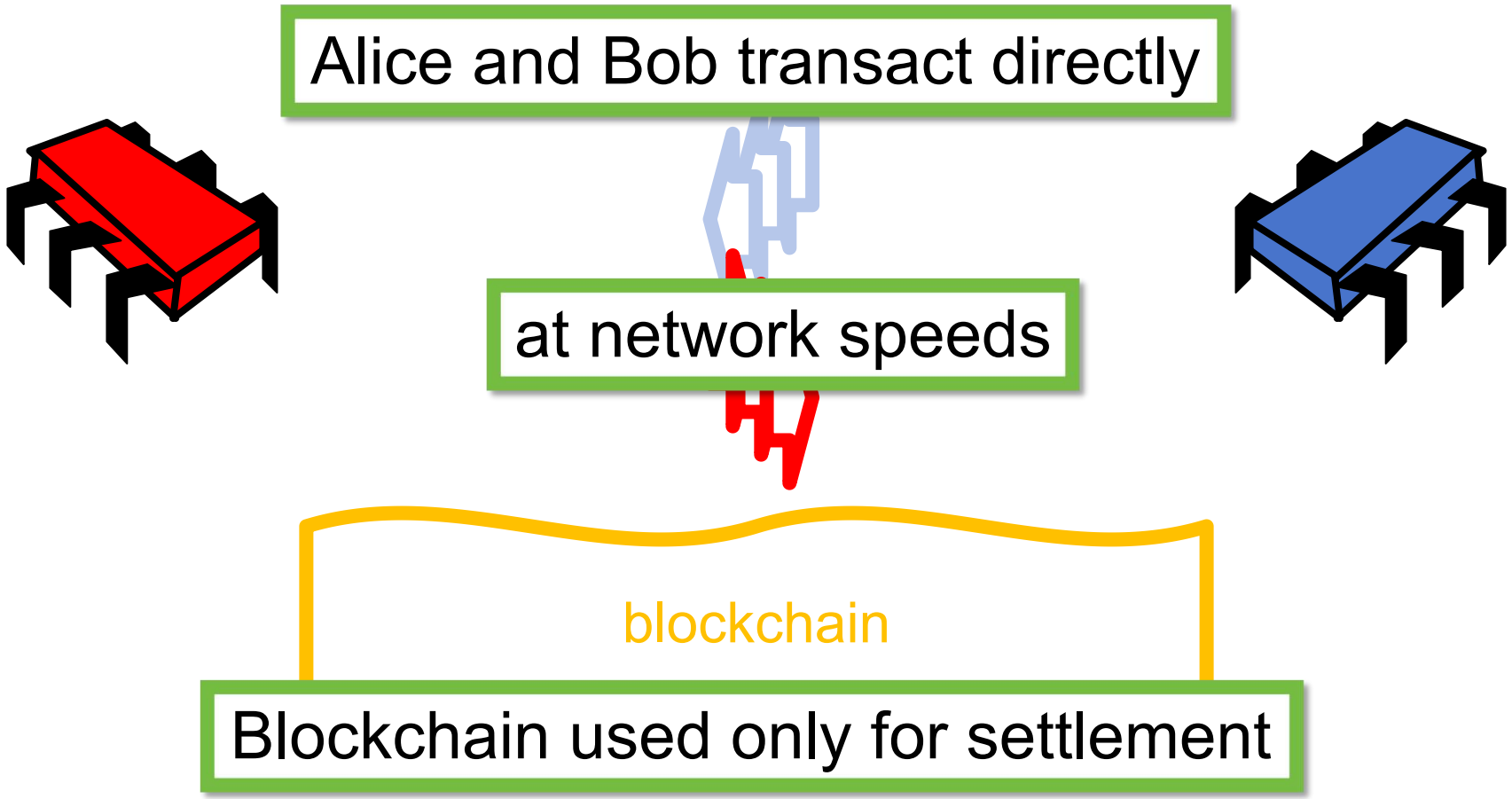
# Alice and Bob do a lot of Bitcoin micro-transactions



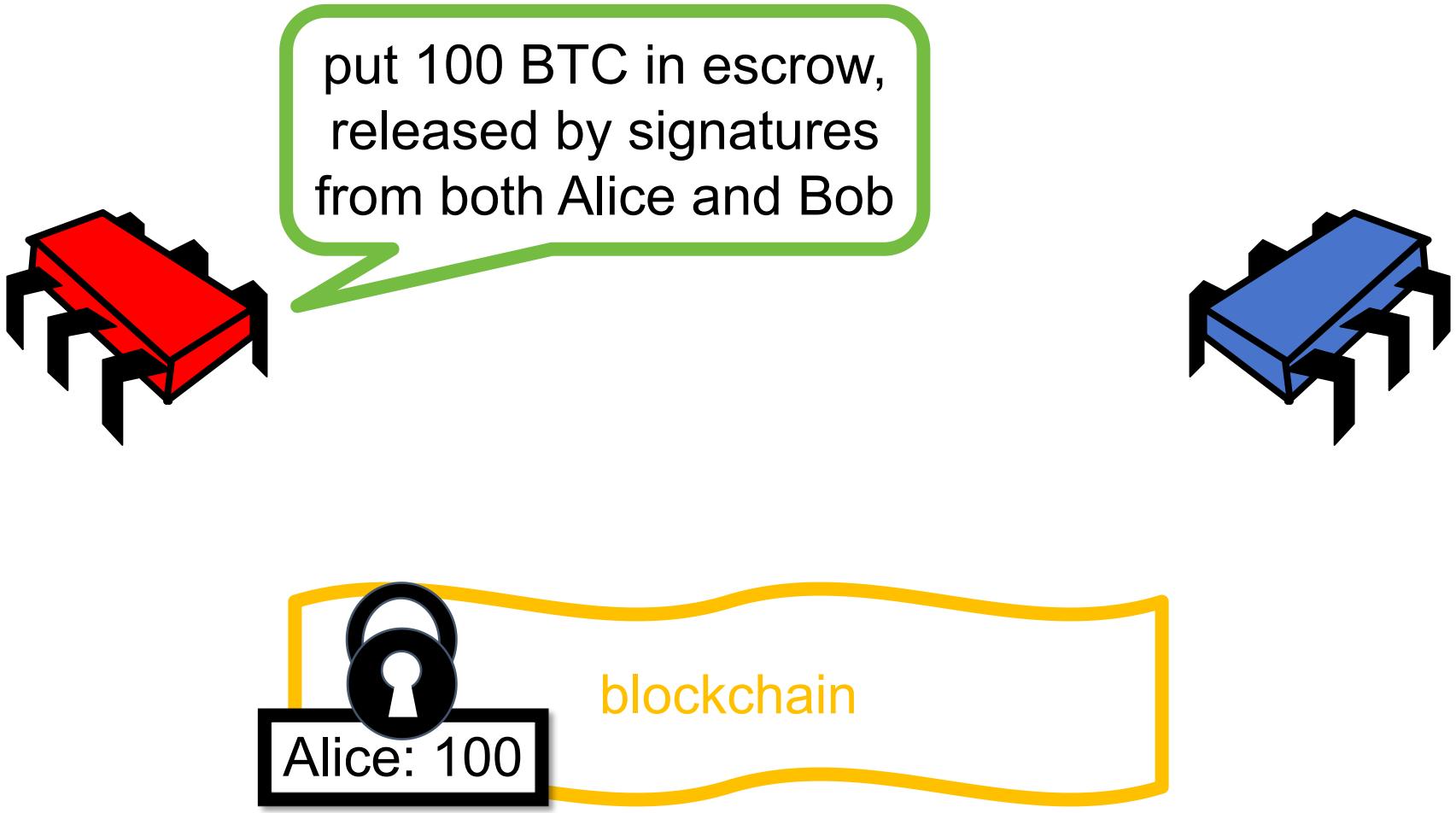
# Off-chain Payment Channels

- Promises orders of magnitude improvement
- Builds on existing Bitcoin (or Ethereum) infrastructure
- Hope to “save” Bitcoin

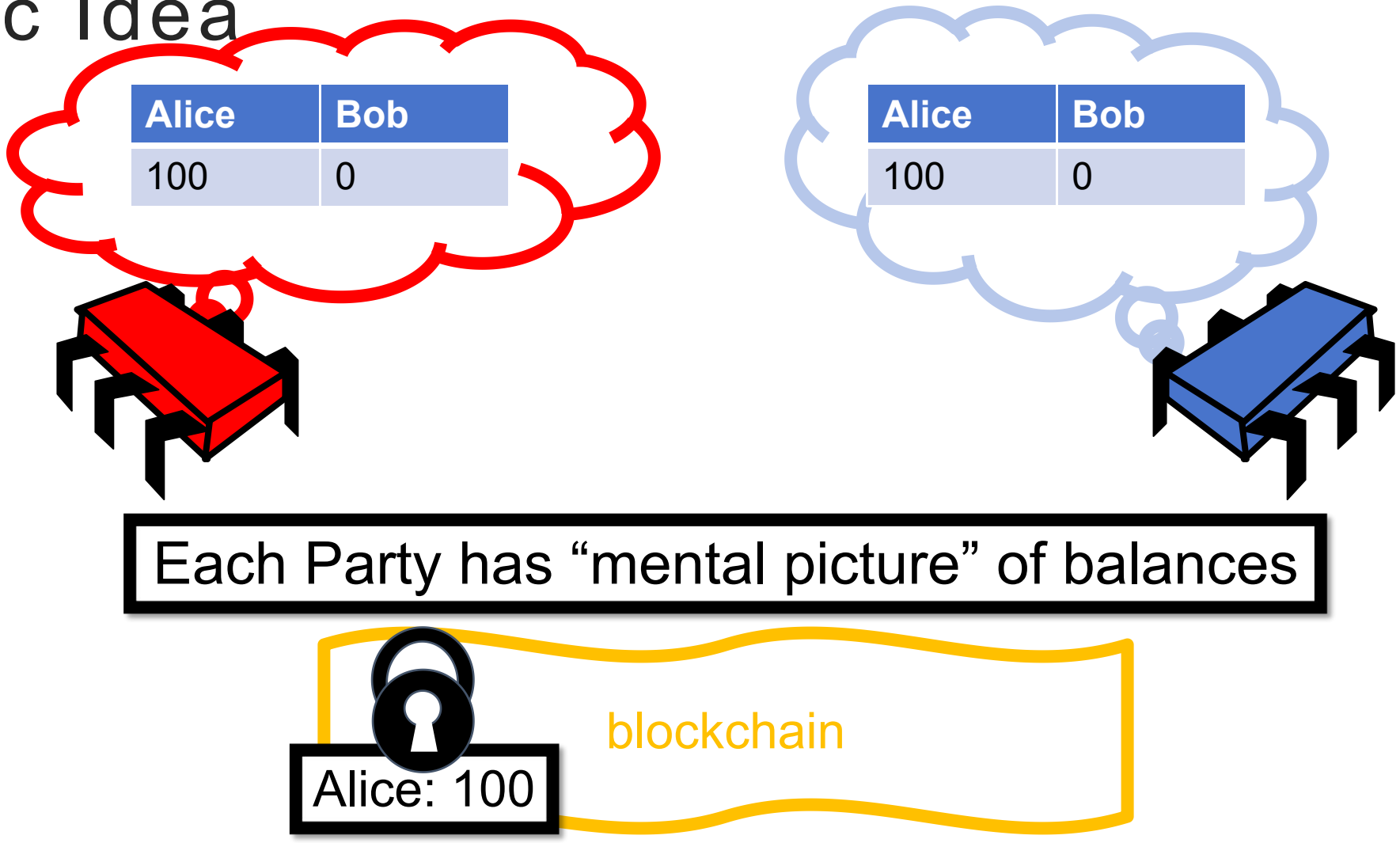
# Basic Idea


Alice and Bob transact directly
at network speeds
blockchain
Blockchain used only for settlement

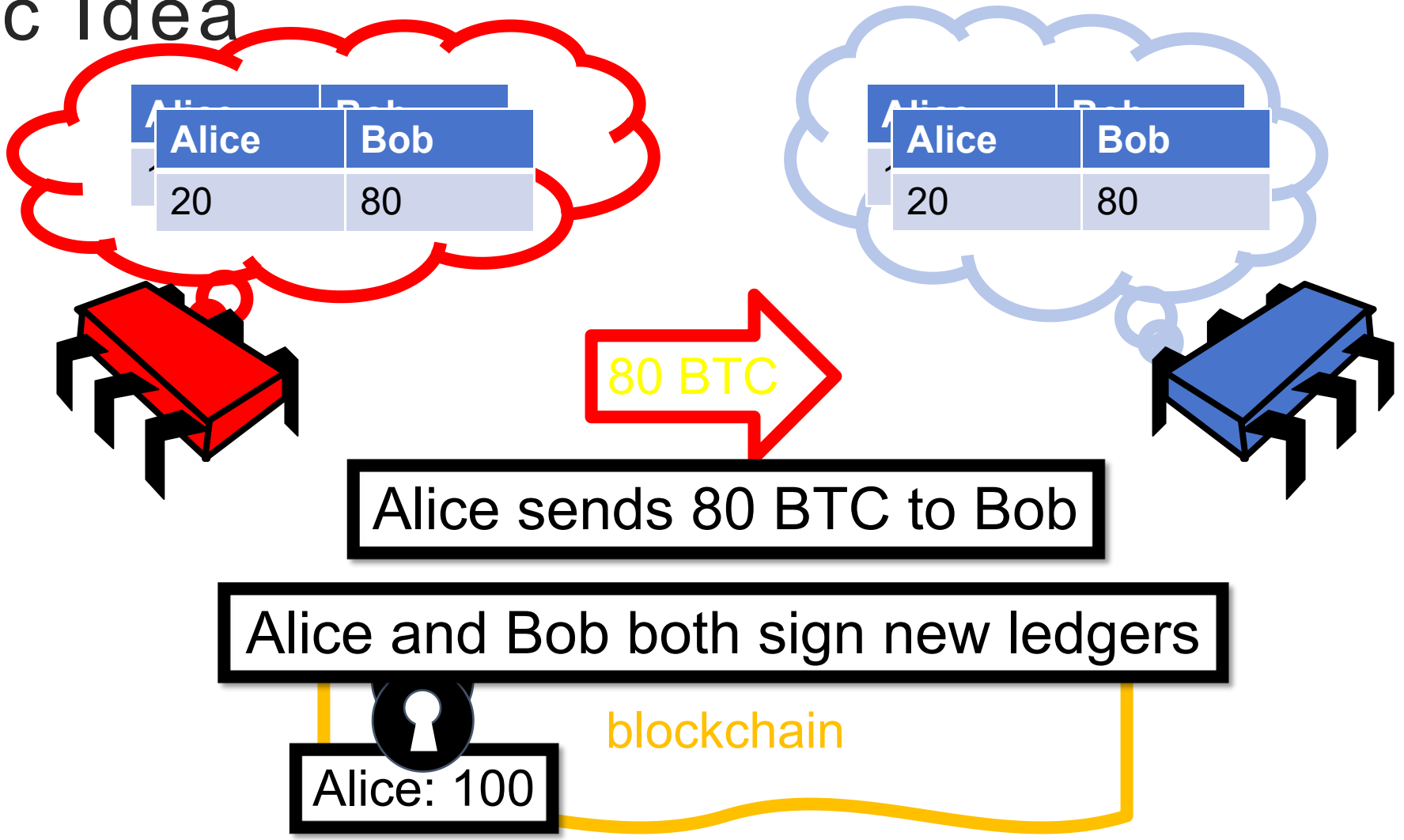
# Basic Idea


put 100 BTC in escrow, released by signatures from both Alice and Bob
Alice: 100
blockchain

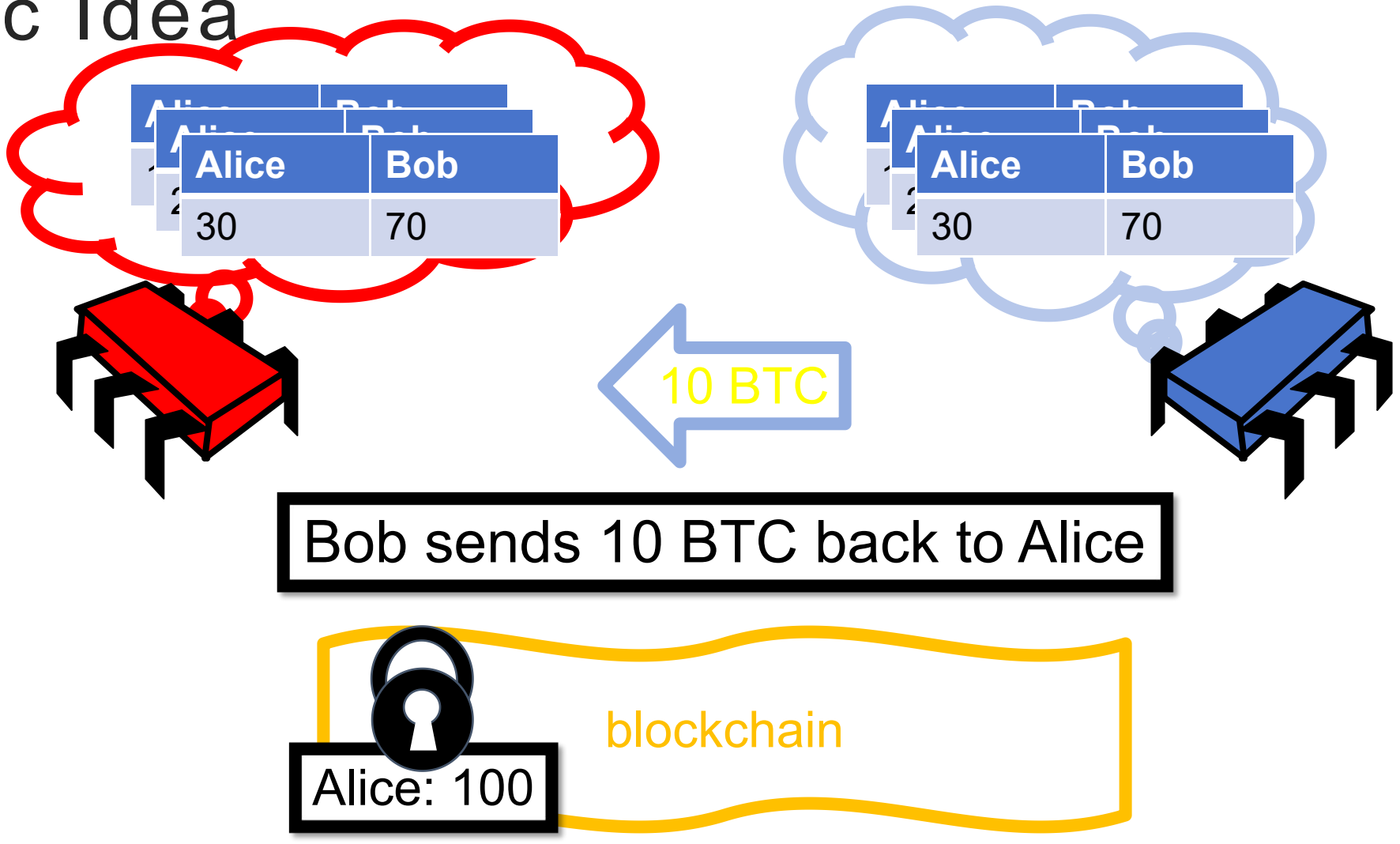
# Basic Idea


Alice
Bob
100
0
Alice
Bob
100
0
Each Party has “mental picture” of balances
blockchain
Alice: 100

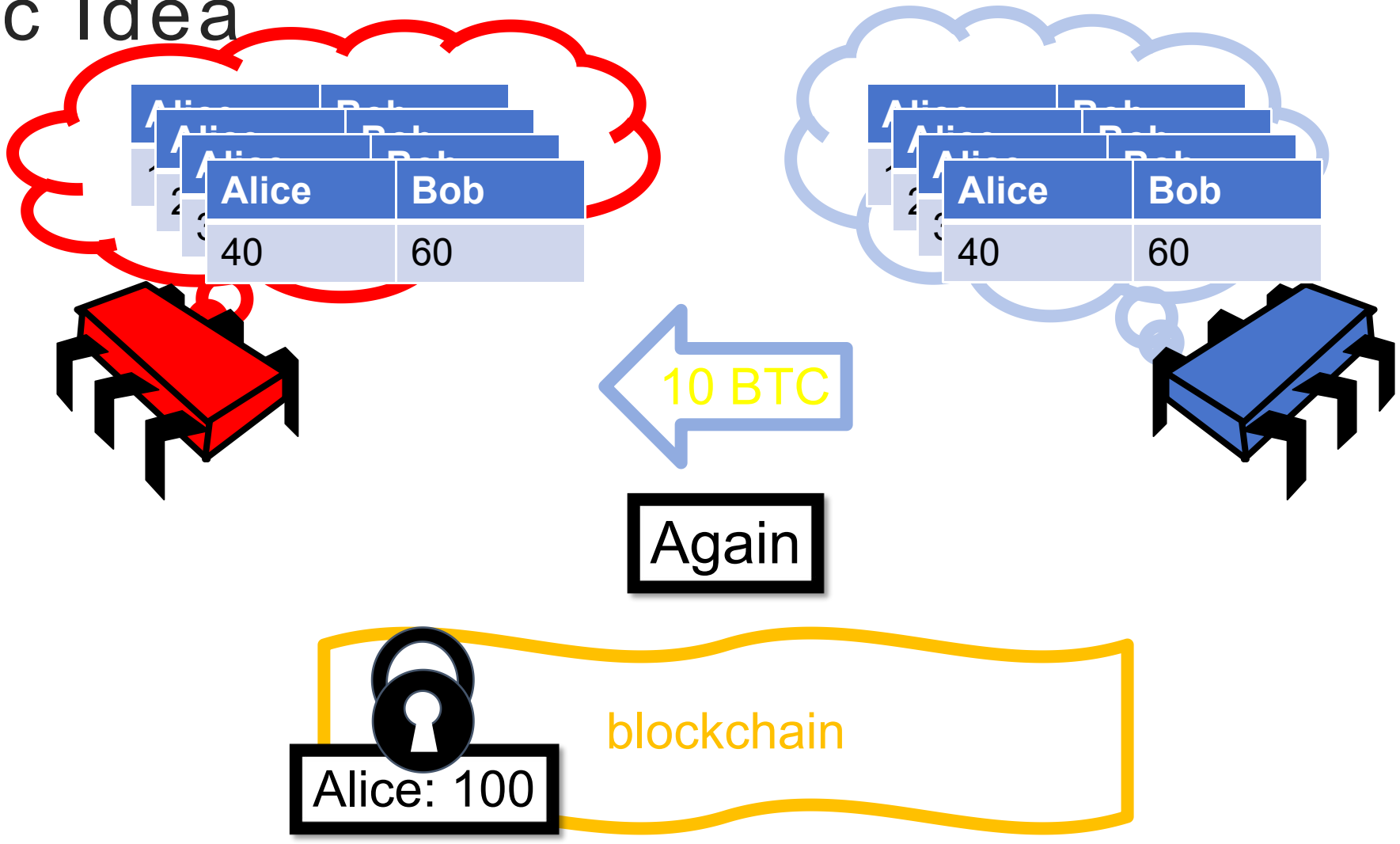
# Basic Idea


Alice
Bob
20
80
Alice
Bob
20
80
80 BTC
Alice sends 80 BTC to Bob
Alice and Bob both sign new ledgers
blockchain
Alice: 100

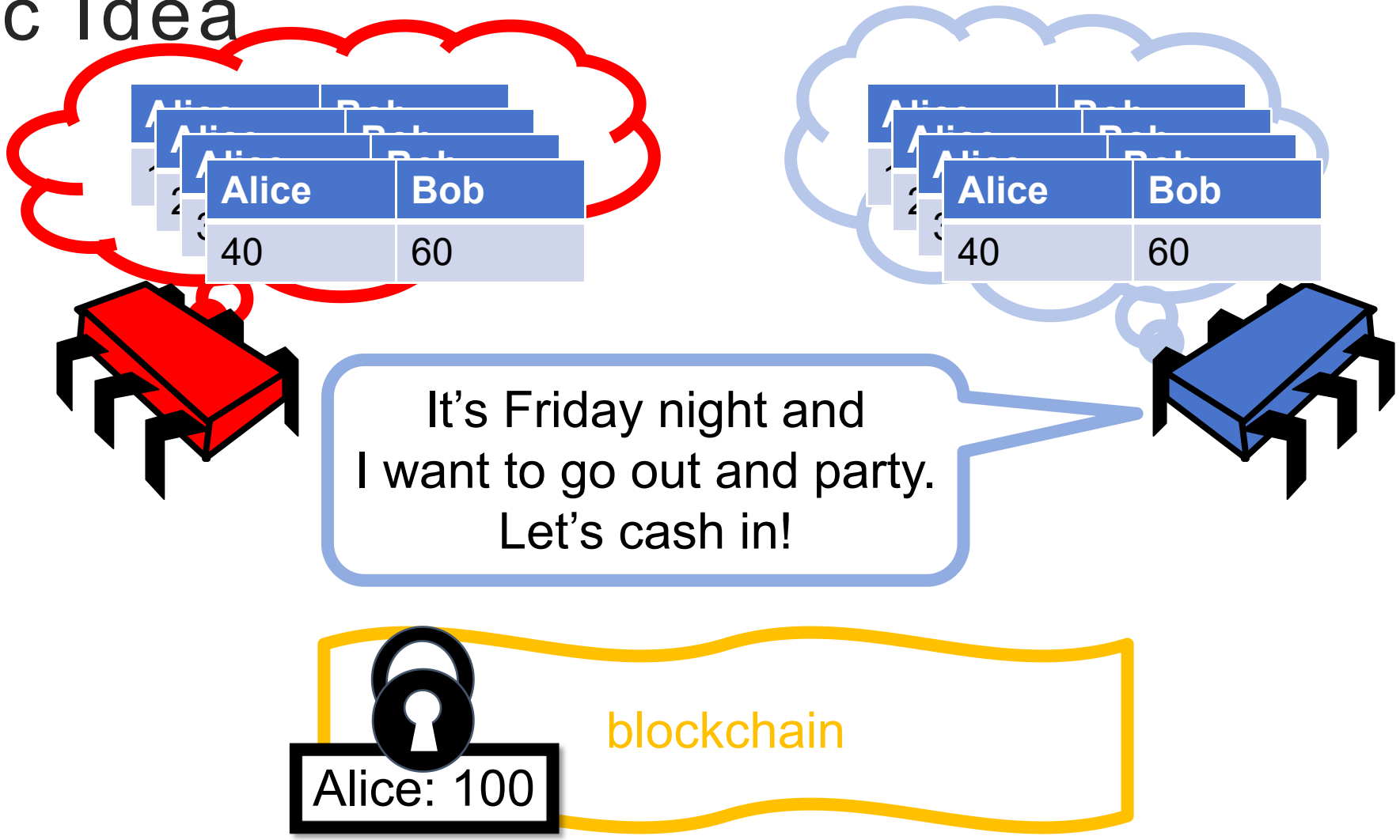
# Basic Idea


Alice
Bob
30
70
Alice
Bob
30
70
10 BTC
Bob sends 10 BTC back to Alice
Alice: 100
blockchain

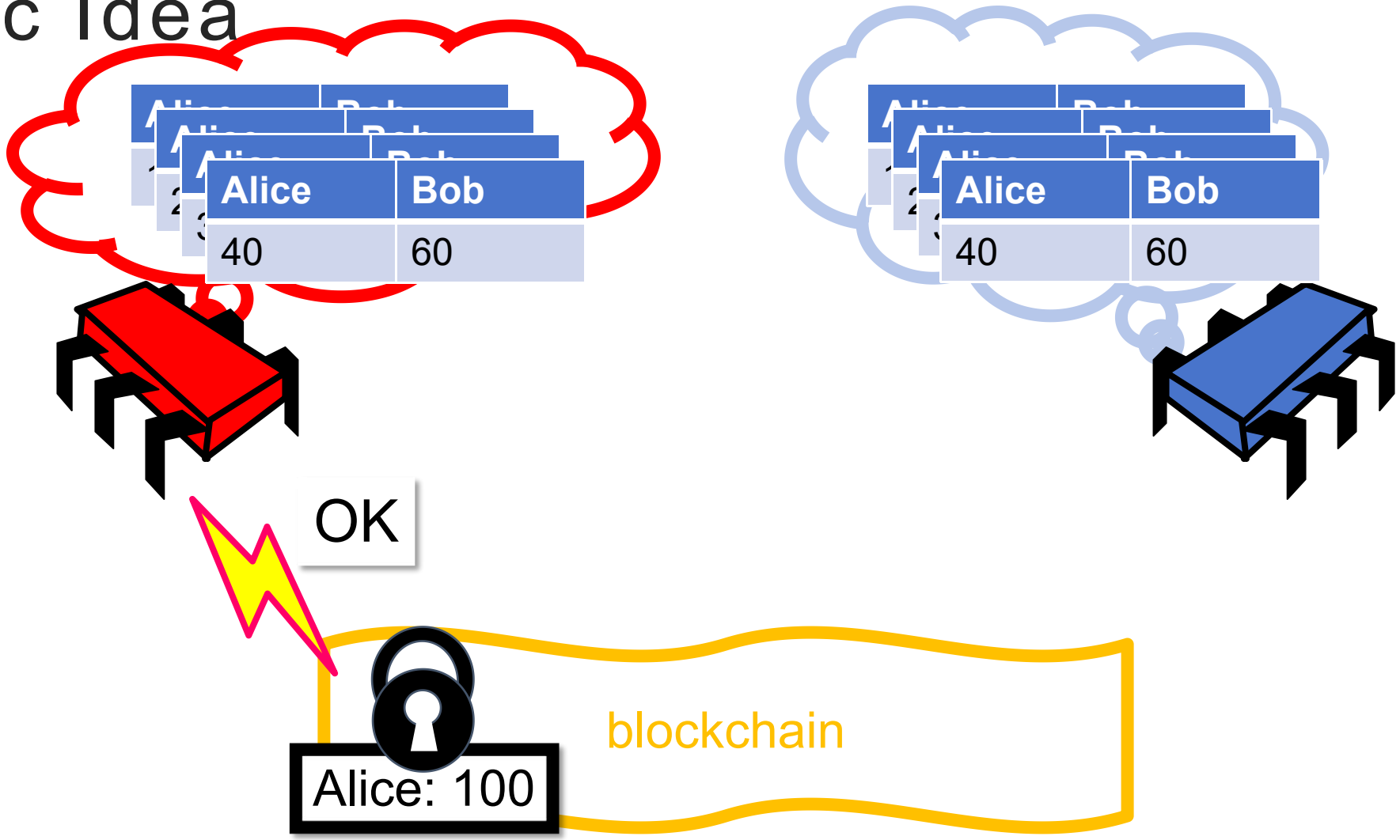
# Basic Idea


Alice
Bob
40
60
10 BTC
Alice
Bob
40
60
Again
Alice: 100
blockchain

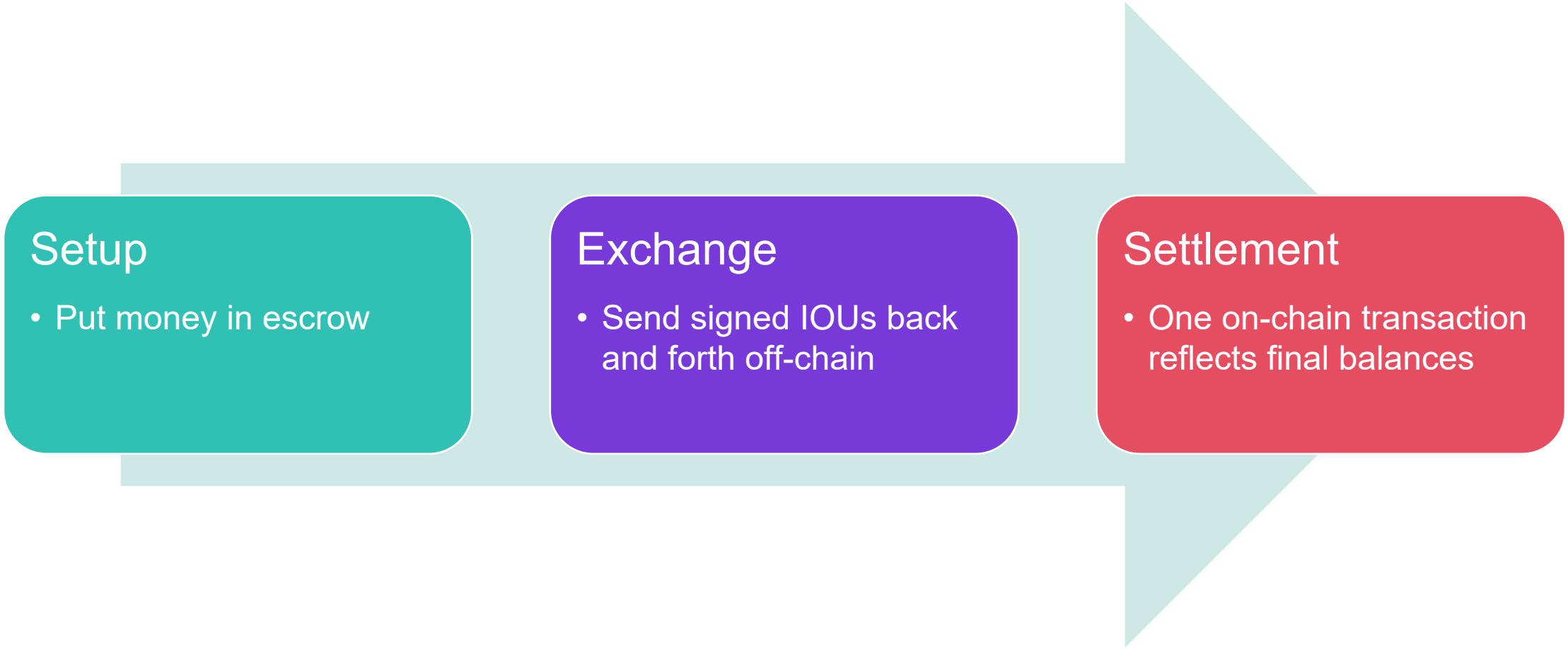
# Basic Idea


Alice
Bob
40
60
Alice
Bob
40
60
It’s Friday night and
I want to go out and party.
Let’s cash in!
Alice: 100
blockchain

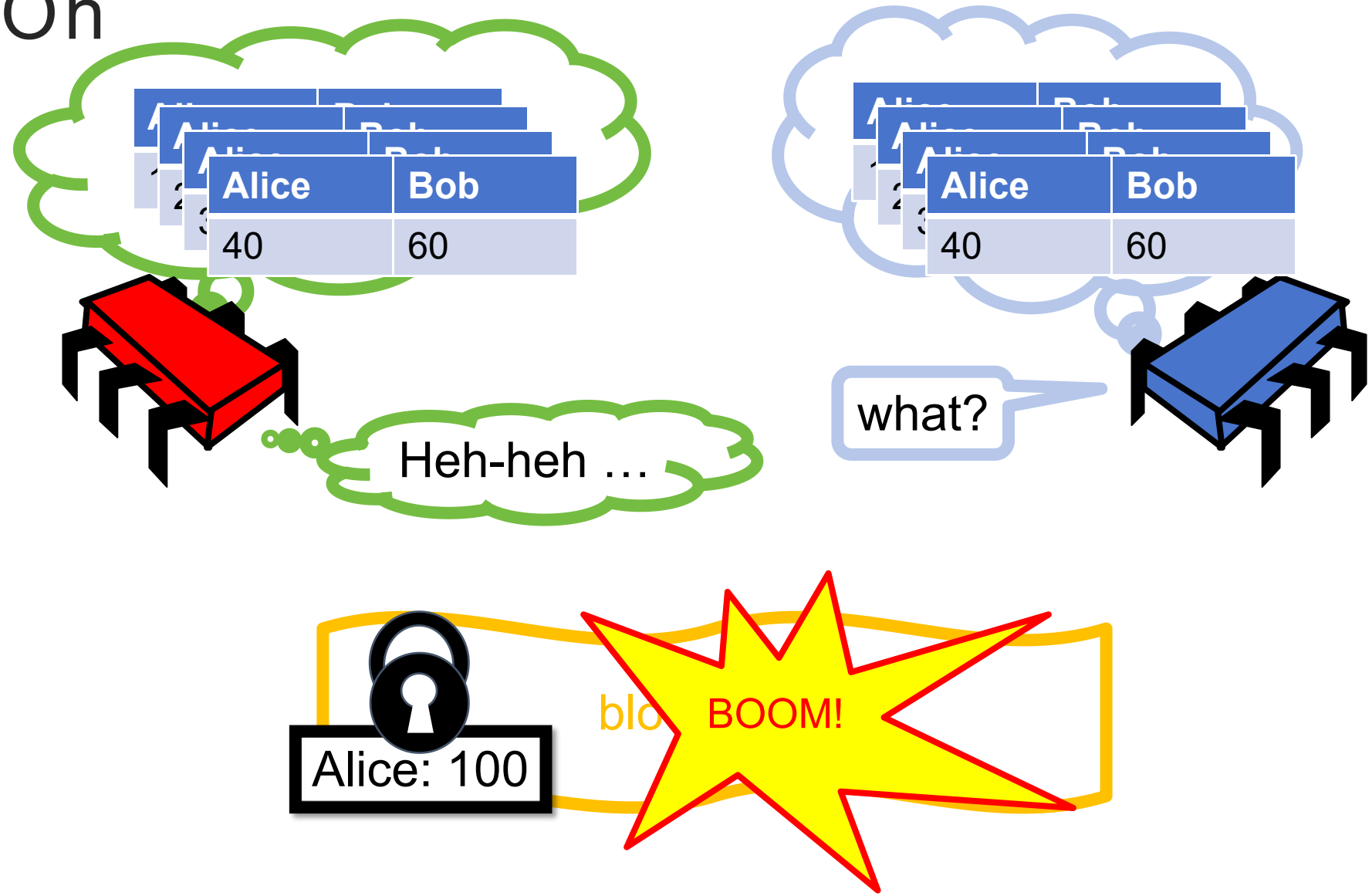
# Basic Idea



# Phases


Setup
• Put money in escrow
Exchange
• Send signed IOUs back and forth off-chain
Settlement
• One on-chain transaction reflects final balances

# Uh, Oh



# Let’s Do That Again

This Time with Security

# nLockTime

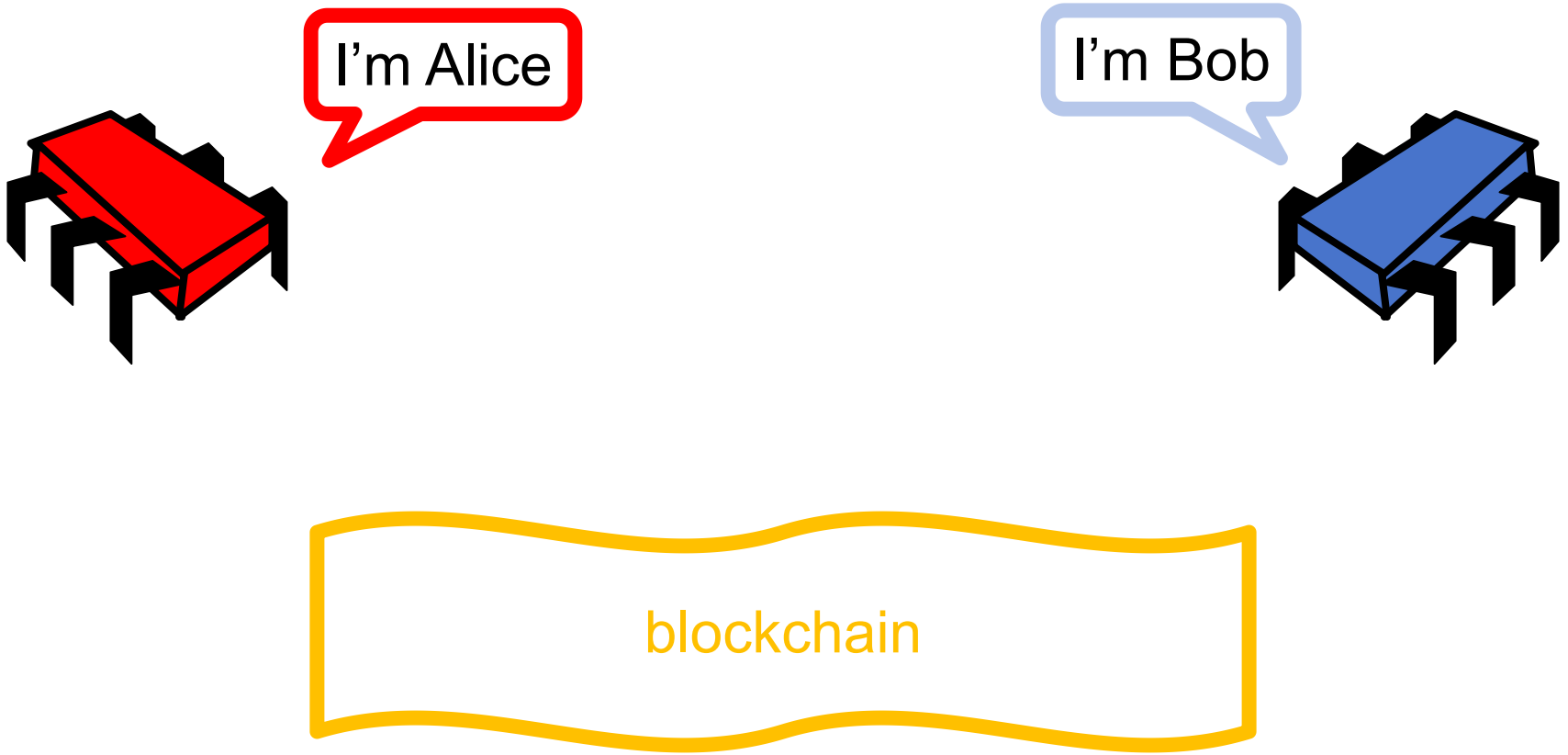
A timelock makes a txn valid only after some time in the future

“pay Alice 10 BTC after 7 days”

Specified in terms of chain height or Unix timestamps



# Unidirectional Channel



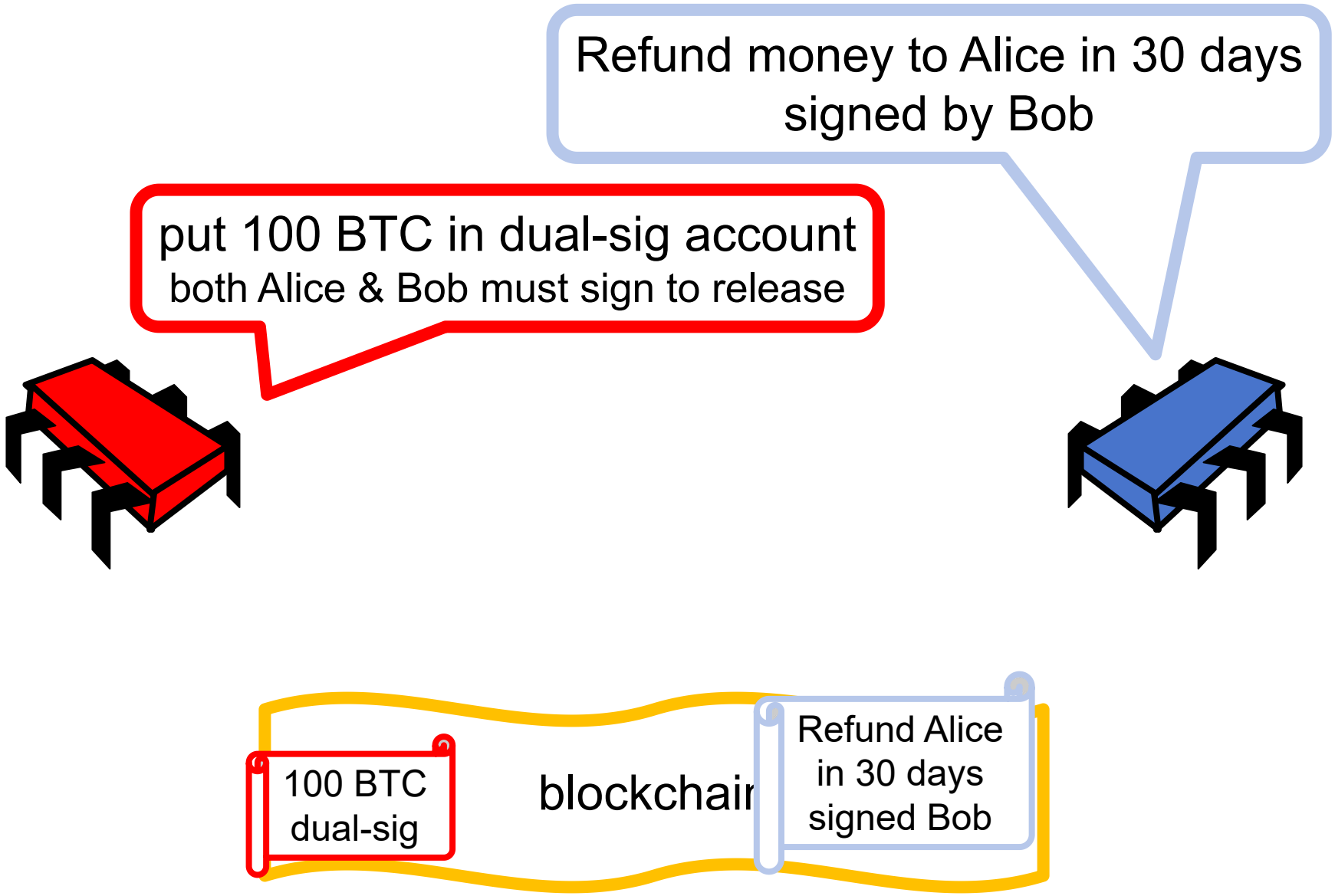
# Setup


put 100 BTC in dual-sig account
both Alice & Bob must sign to release
blockchain

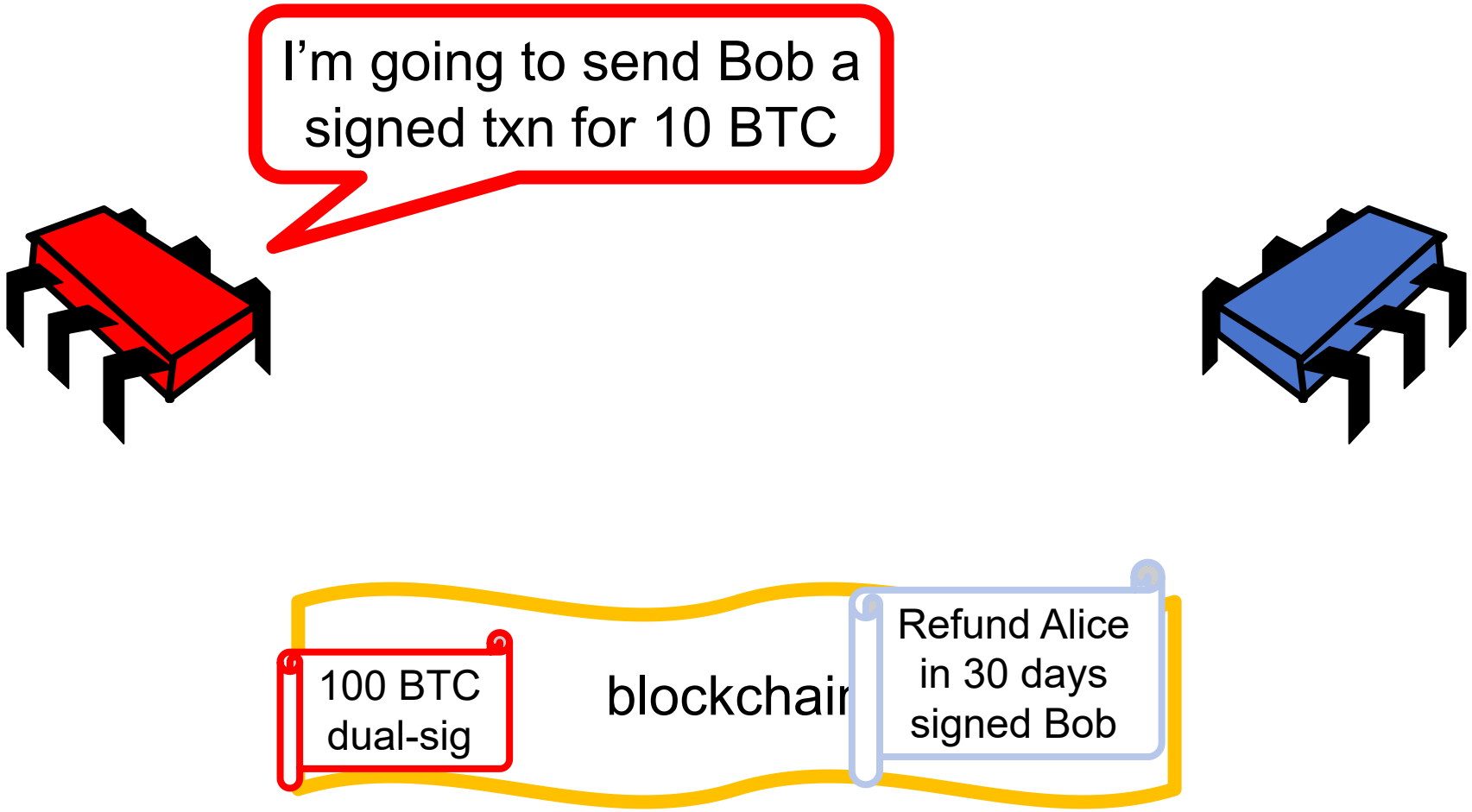
# Setup


Refund money to Alice in 30 days
signed by Bob
put 100 BTC in dual-sig account
both Alice & Bob must sign to release
blockchain

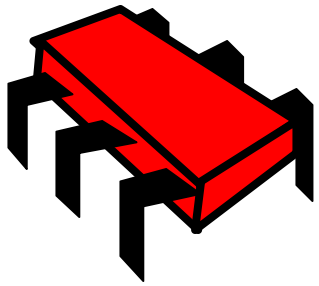
# Setup


Refund money to Alice in 30 days
signed by Bob
put 100 BTC in dual-sig account
both Alice & Bob must sign to release
100 BTC
dual-sig
blockchain
Refund Alice
in 30 days
signed Bob

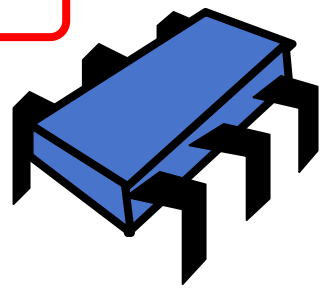
# Exchange


I’m going to send Bob a signed txn for 10 BTC
100 BTC dual-sig
blockchain
Refund Alice in 30 days signed Bob

# Exchange



pay 10 to Bob,
90 to Alice
signed Alice

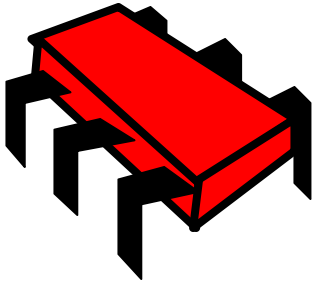


100 BTC
dual-sig

blockchain

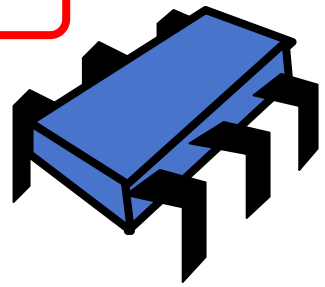
Refund Alice
in 30 days
signed Bob

# Exchange



I could deposit this on the blockchain right now, but I won't

pay 10 to Bob,
90 to Alice
signed Alice

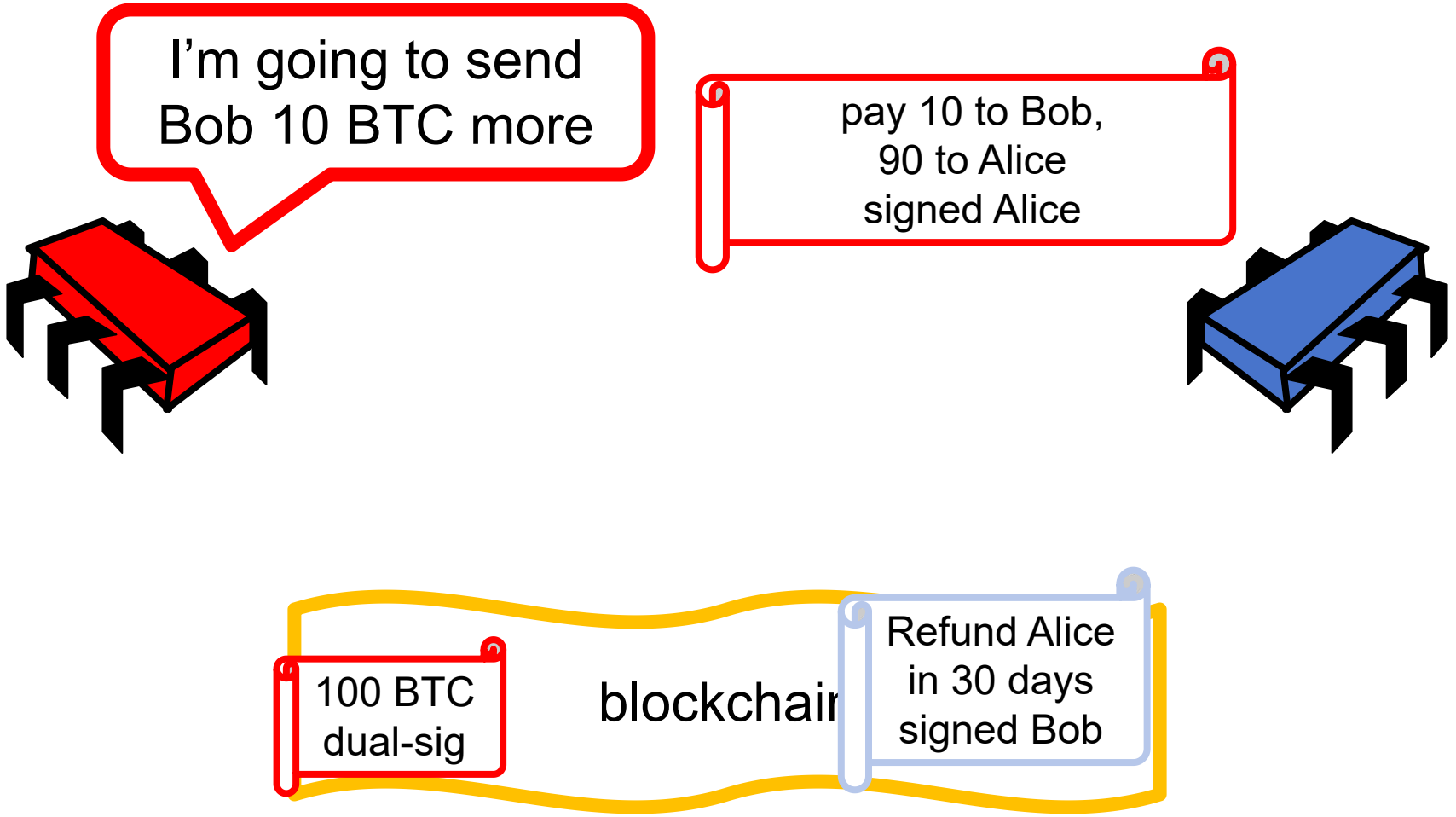


100 BTC
dual-sig

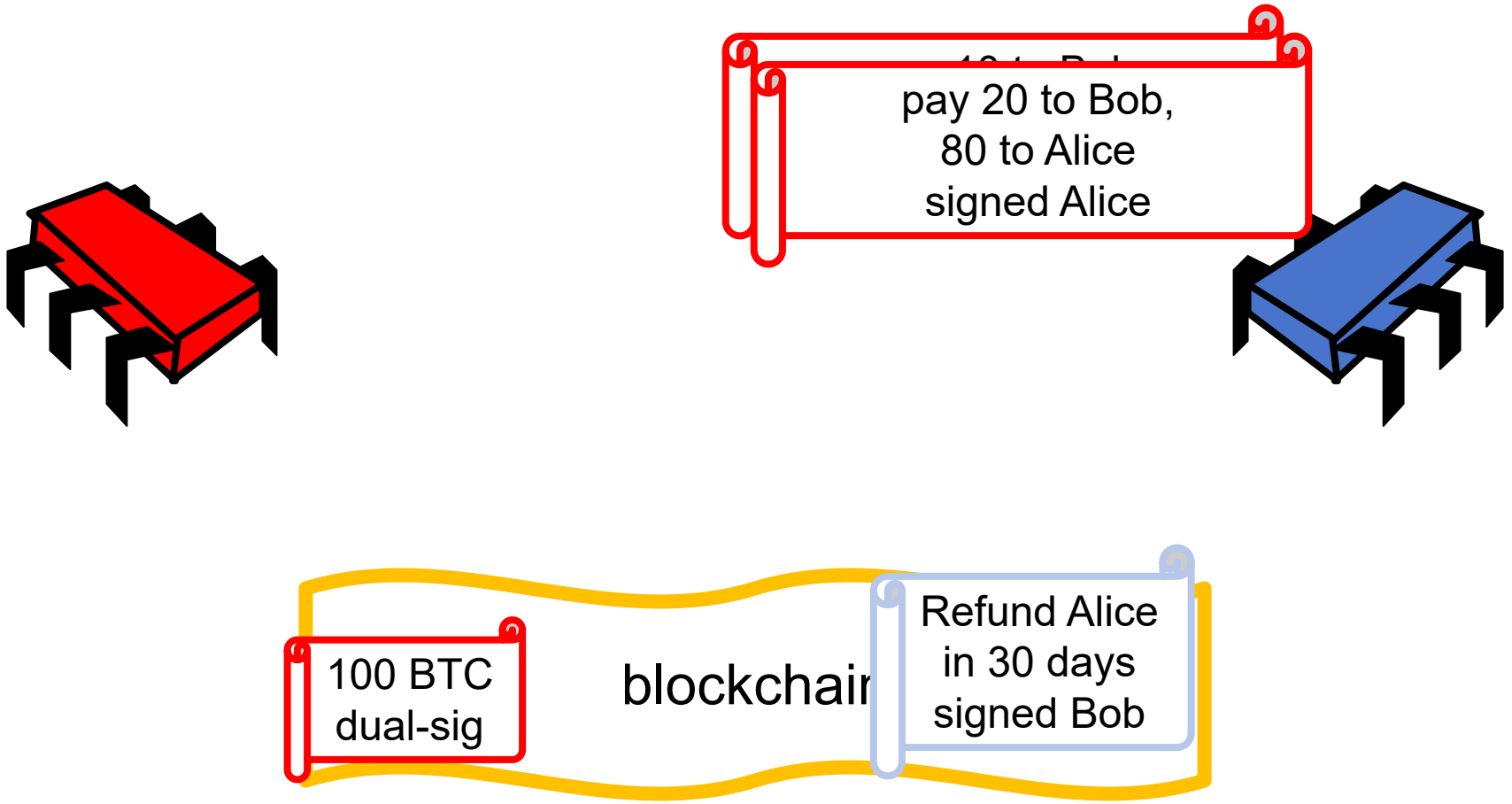
blockchain

Refund Alice
in 30 days
signed Bob

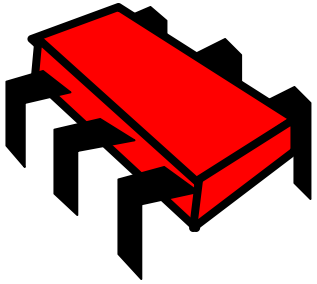
# Exchange

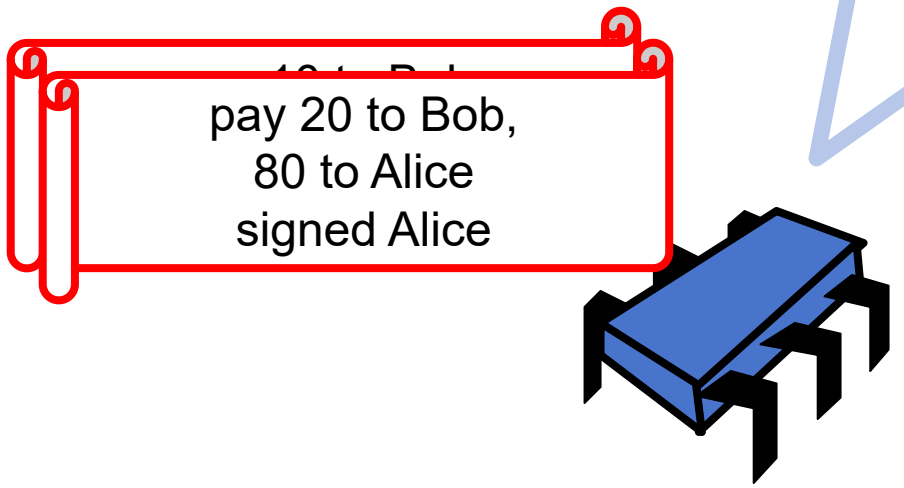

I’m going to send
Bob 10 BTC more
pay 10 to Bob,
90 to Alice
signed Alice
100 BTC
dual-sig
blockchain
Refund Alice
in 30 days
signed Bob

# Exchange

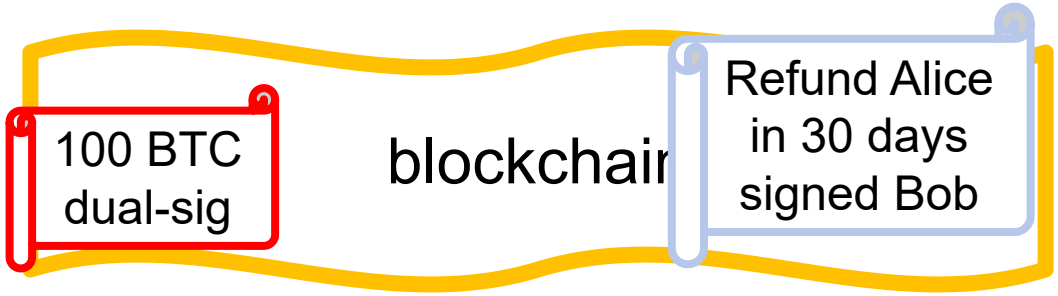

pay 20 to Bob,
80 to Alice
signed Alice
100 BTC
dual-sig
blockchain
Refund Alice
in 30 days
signed Bob

# Exchange

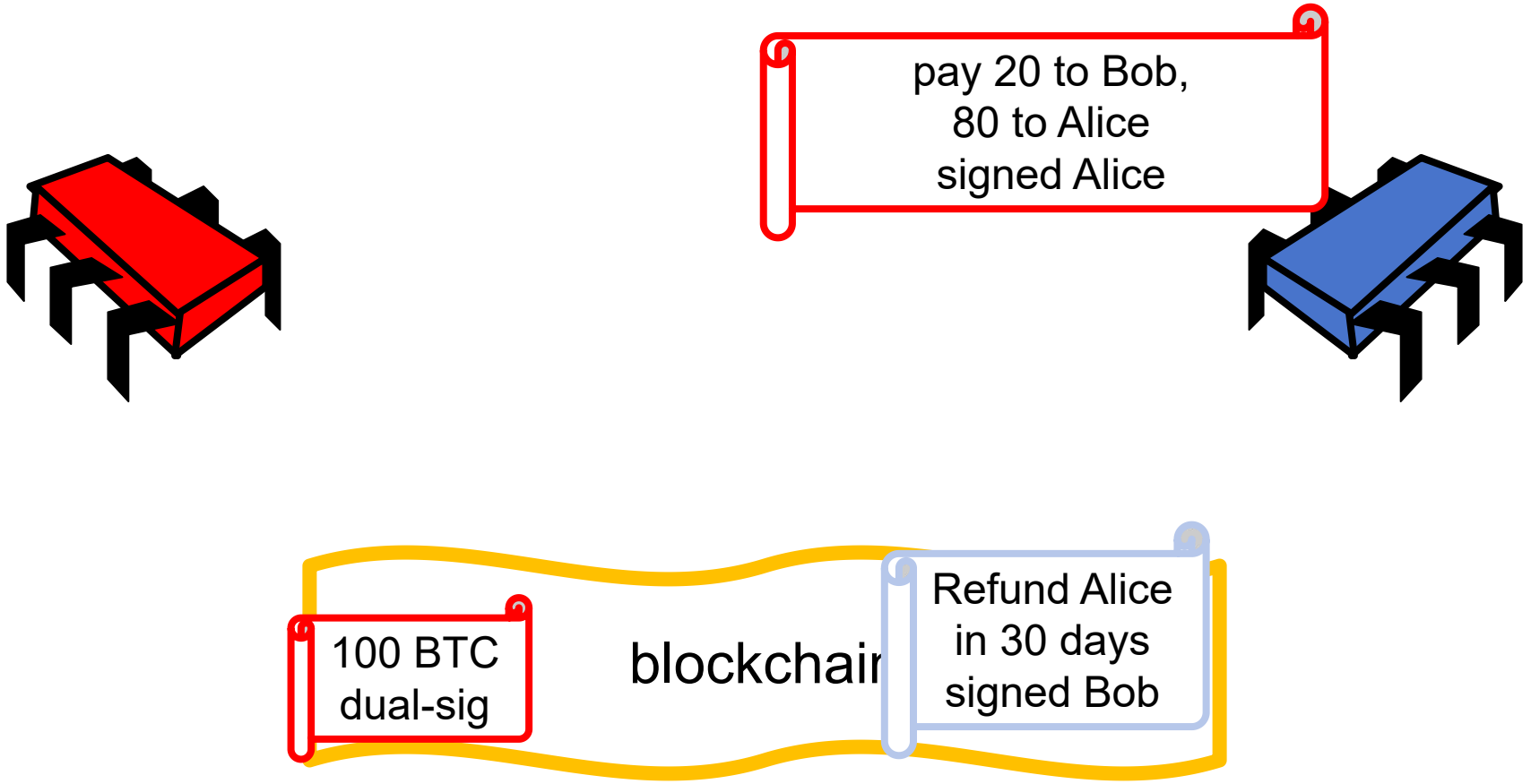




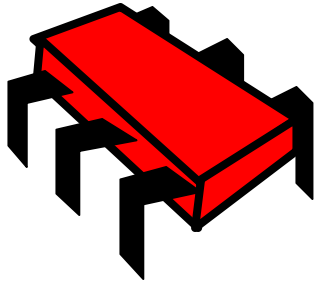

The 2nd txn is more valuable so I will keep that one and discard the other



# Exchange

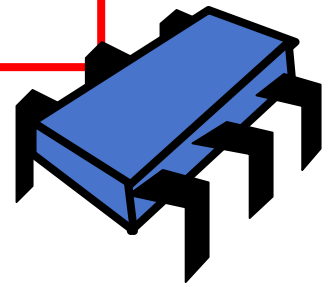

pay 20 to Bob,
80 to Alice
signed Alice
100 BTC
dual-sig
blockchain
Refund Alice
in 30 days
signed Bob

# Settlement



Closing time!
I will sign and publish most recent txn on the blockchain now

pay 20 to Bob,
80 to Alice
signed Alice

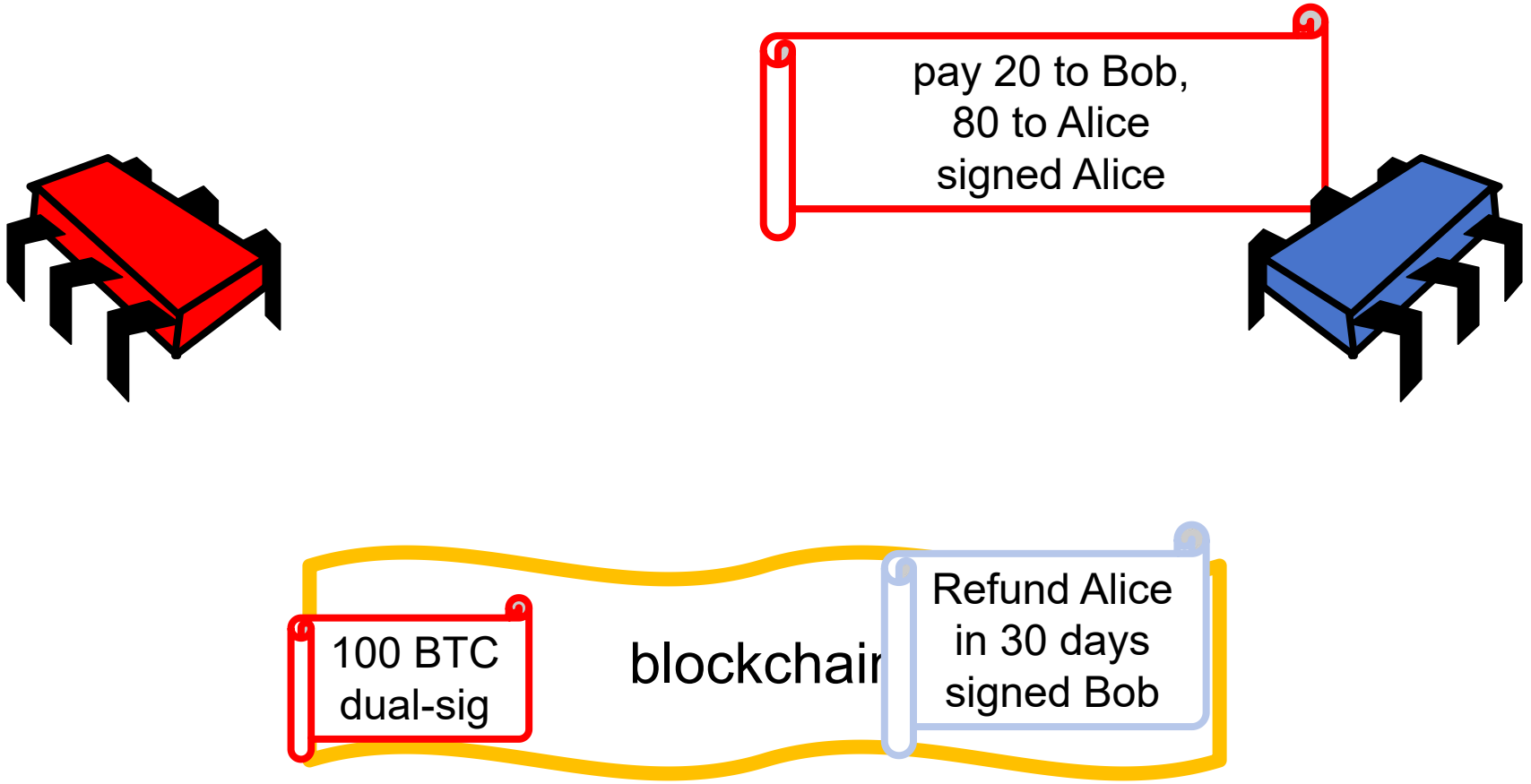


100 BTC
dual-sig

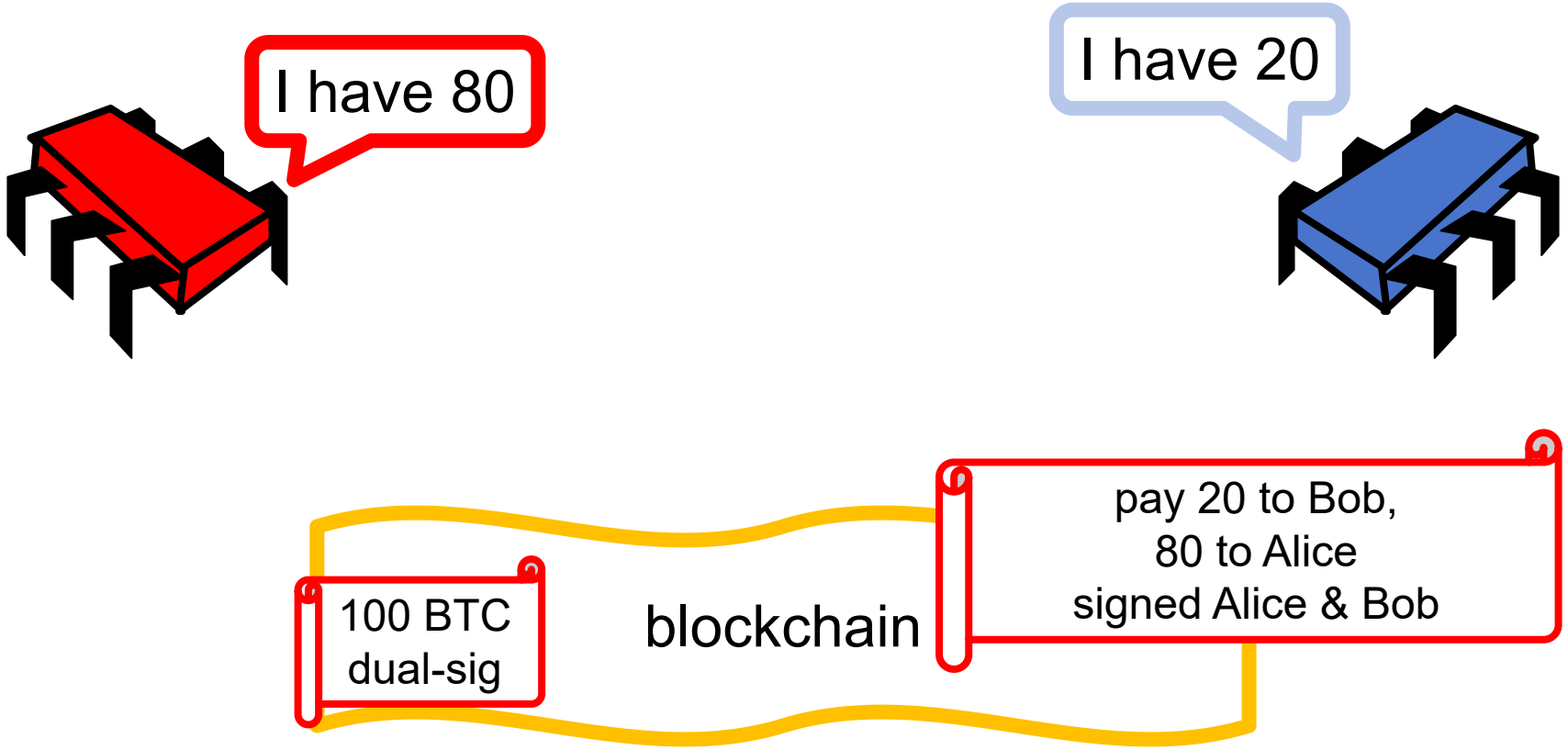
blockchain

Refund Alice
in 30 days
signed Bob

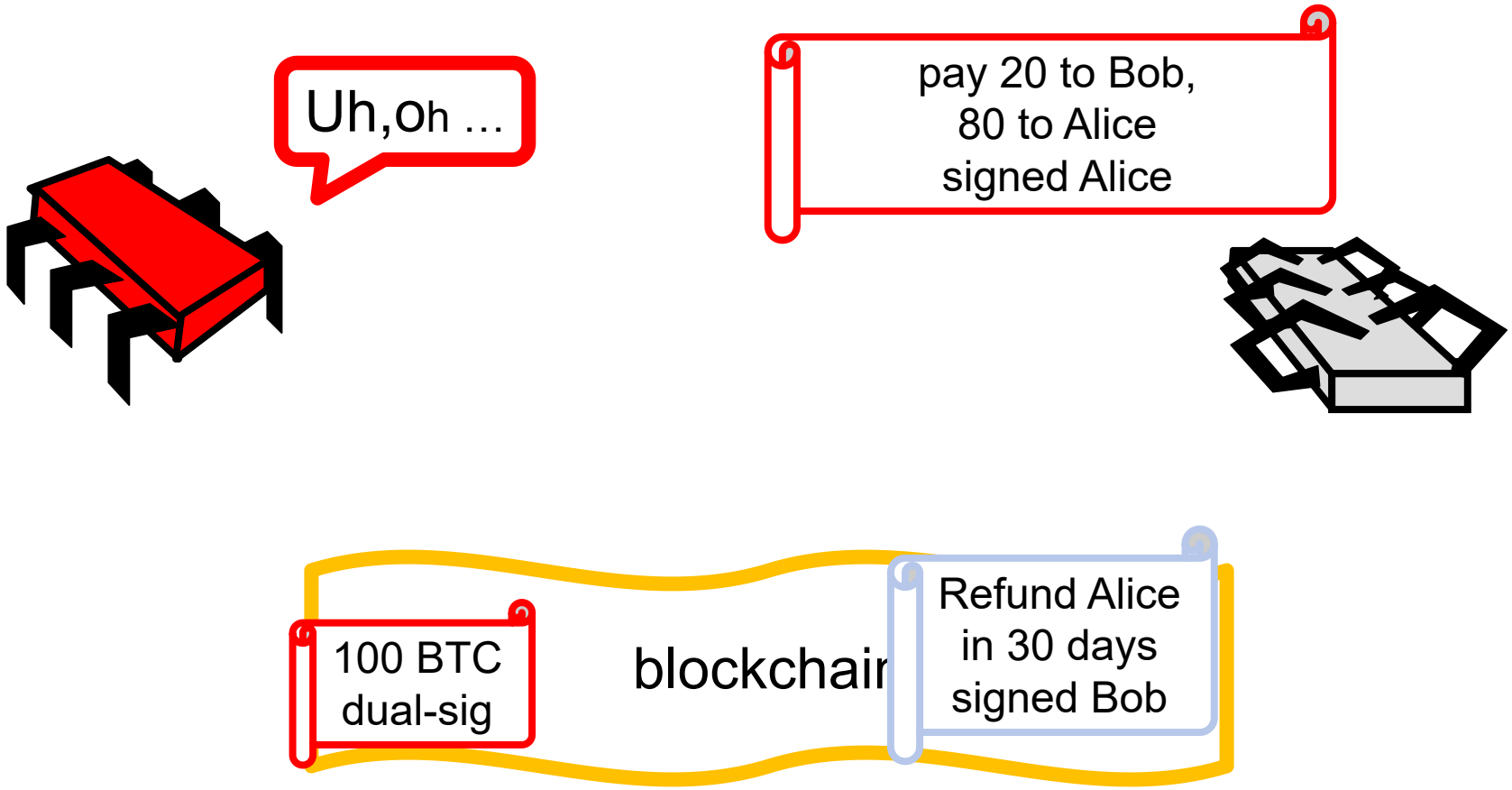
# Settlement


pay 20 to Bob,
80 to Alice
signed Alice
100 BTC
dual-sig
blockchain
Refund Alice
in 30 days
signed Bob

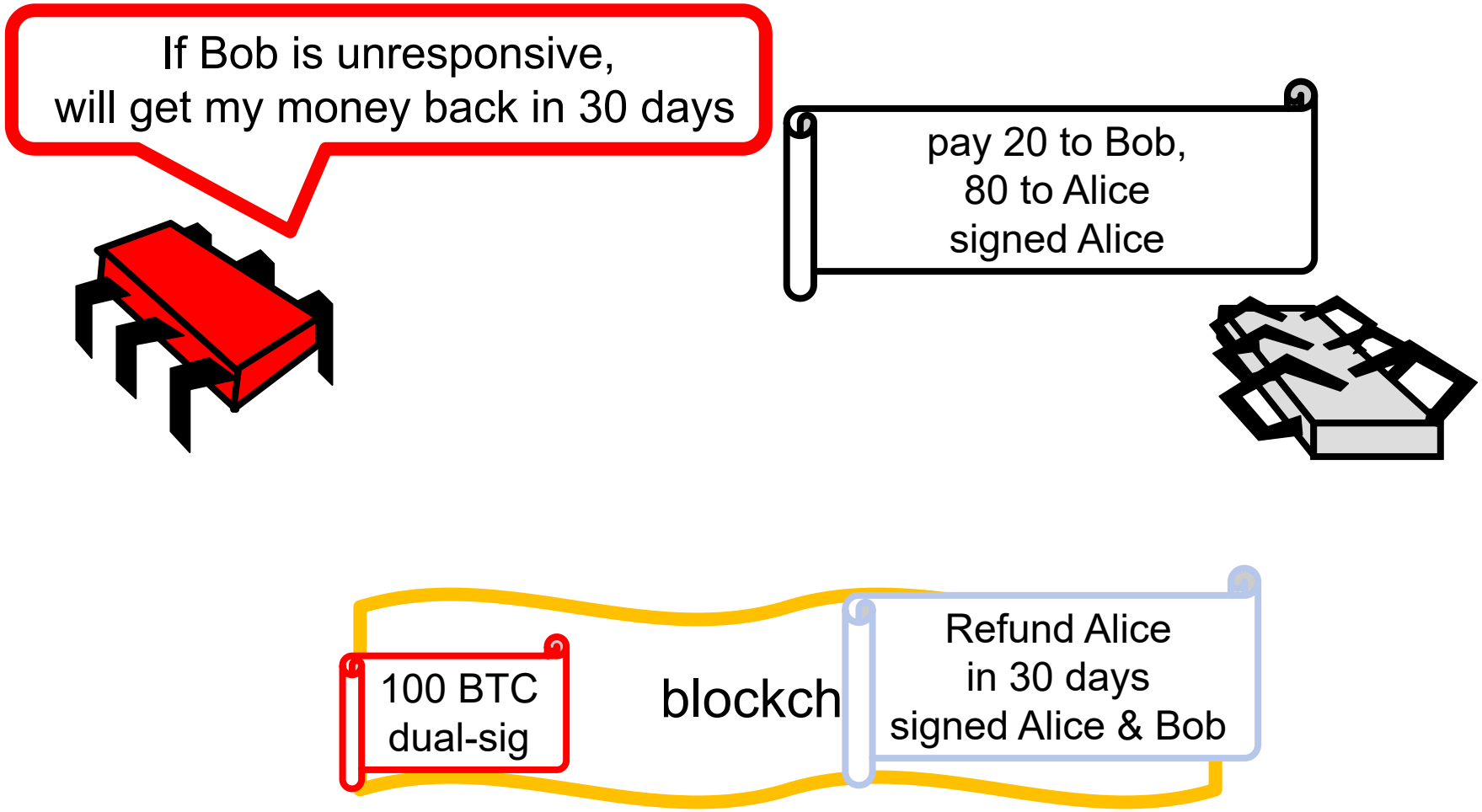
# Settlement


I have 80
I have 20
100 BTC
dual-sig
blockchain
pay 20 to Bob,
80 to Alice
signed Alice & Bob

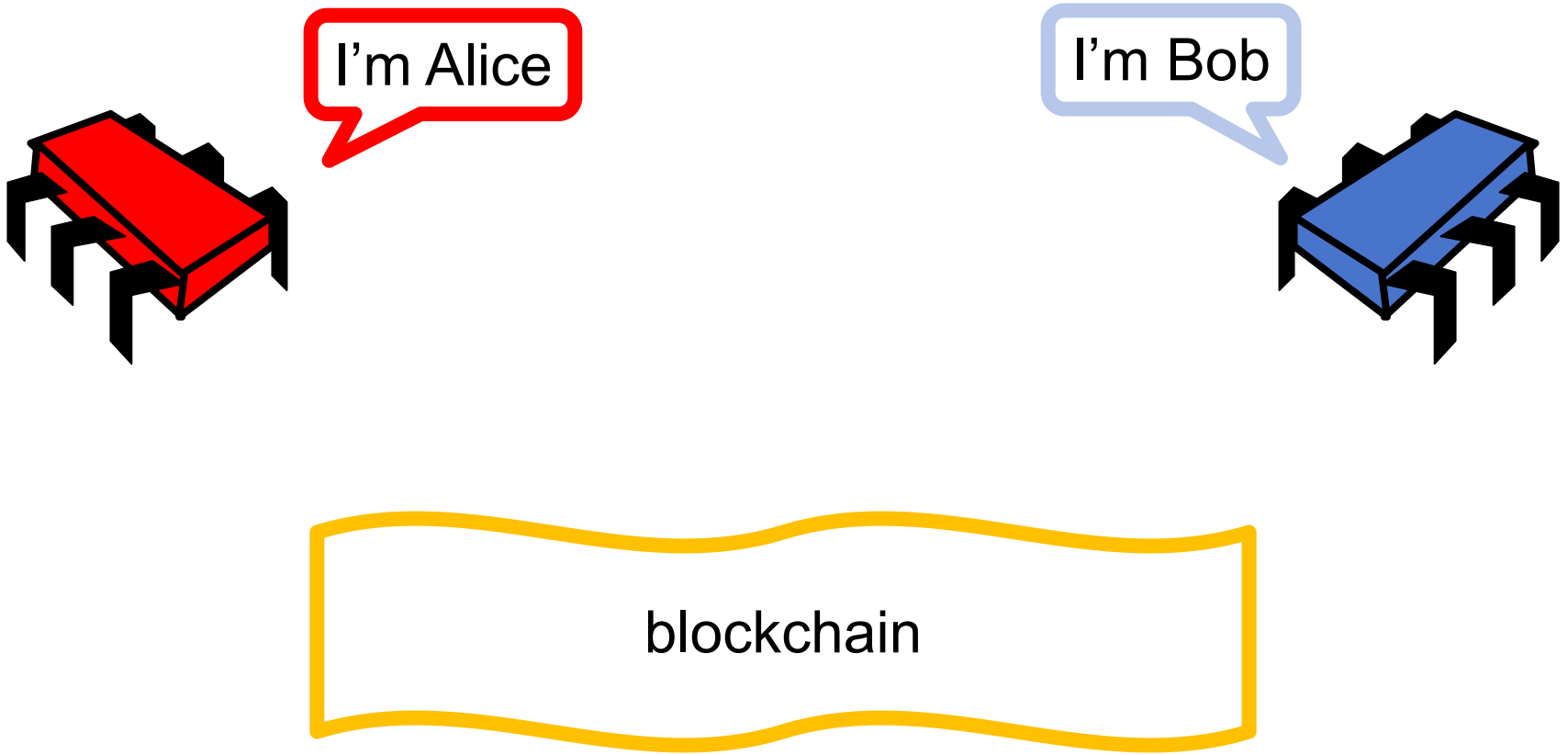
# Settlement


Uh,oh …
pay 20 to Bob,
80 to Alice
signed Alice
100 BTC
dual-sig
blockchain
Refund Alice
in 30 days
signed Bob

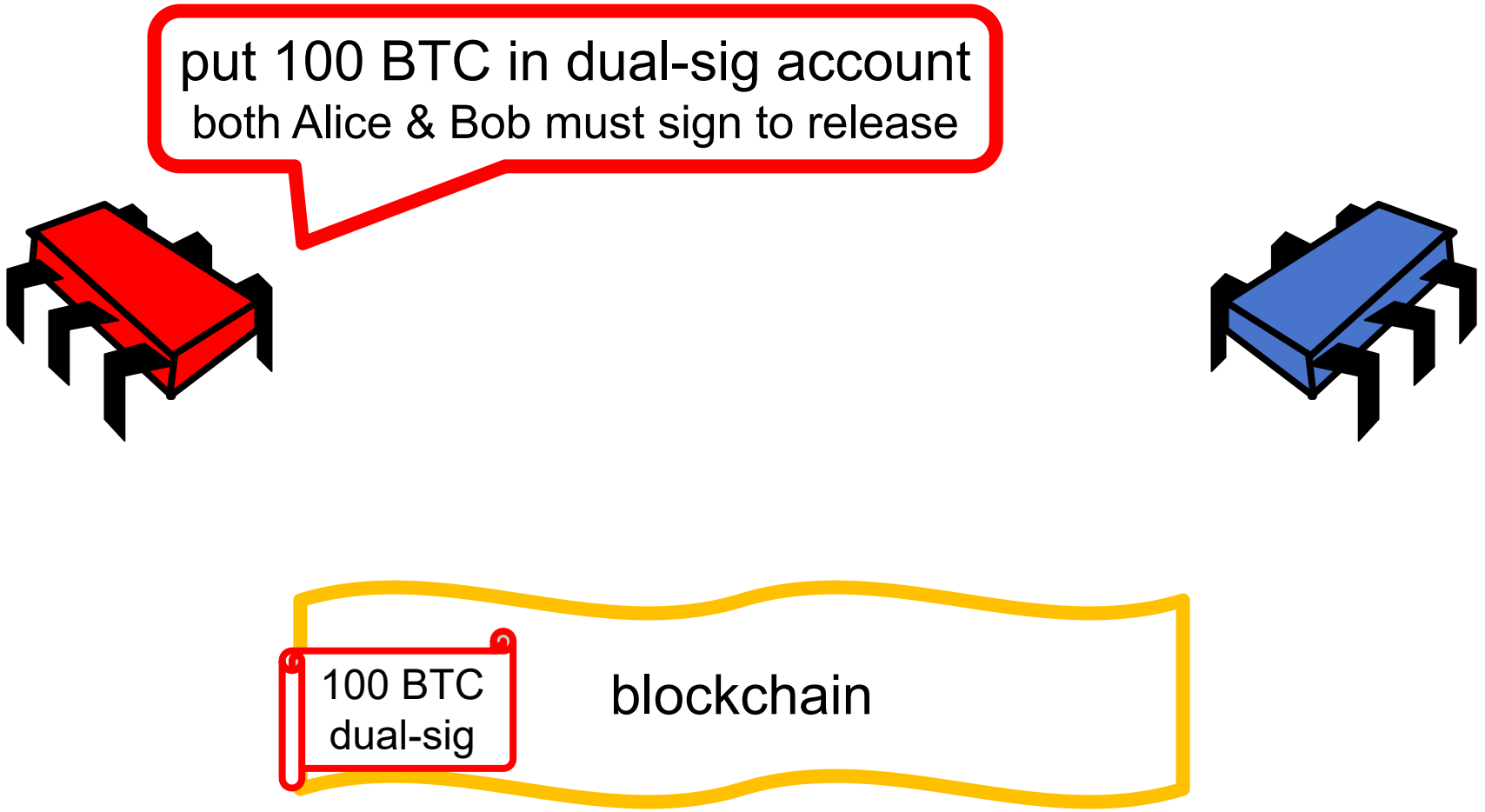
# Settlement


If Bob is unresponsive,
will get my money back in 30 days
pay 20 to Bob,
80 to Alice
signed Alice
100 BTC
dual-sig
blockch
Refund Alice
in 30 days
signed Alice & Bob

# Bidirectional Channel



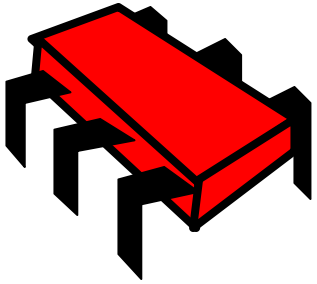
# Setup

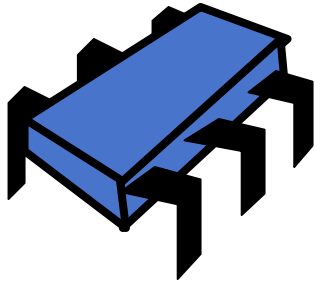

put 100 BTC in dual-sig account
both Alice & Bob must sign to release
100 BTC
dual-sig
blockchain

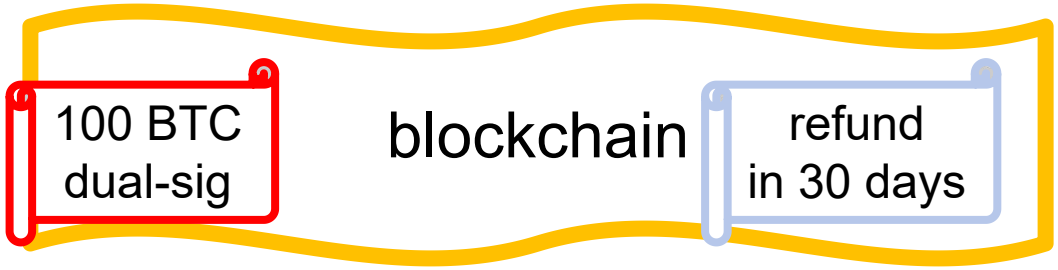
# Setup

Refund money to Alice in 30 days
signed by Bob

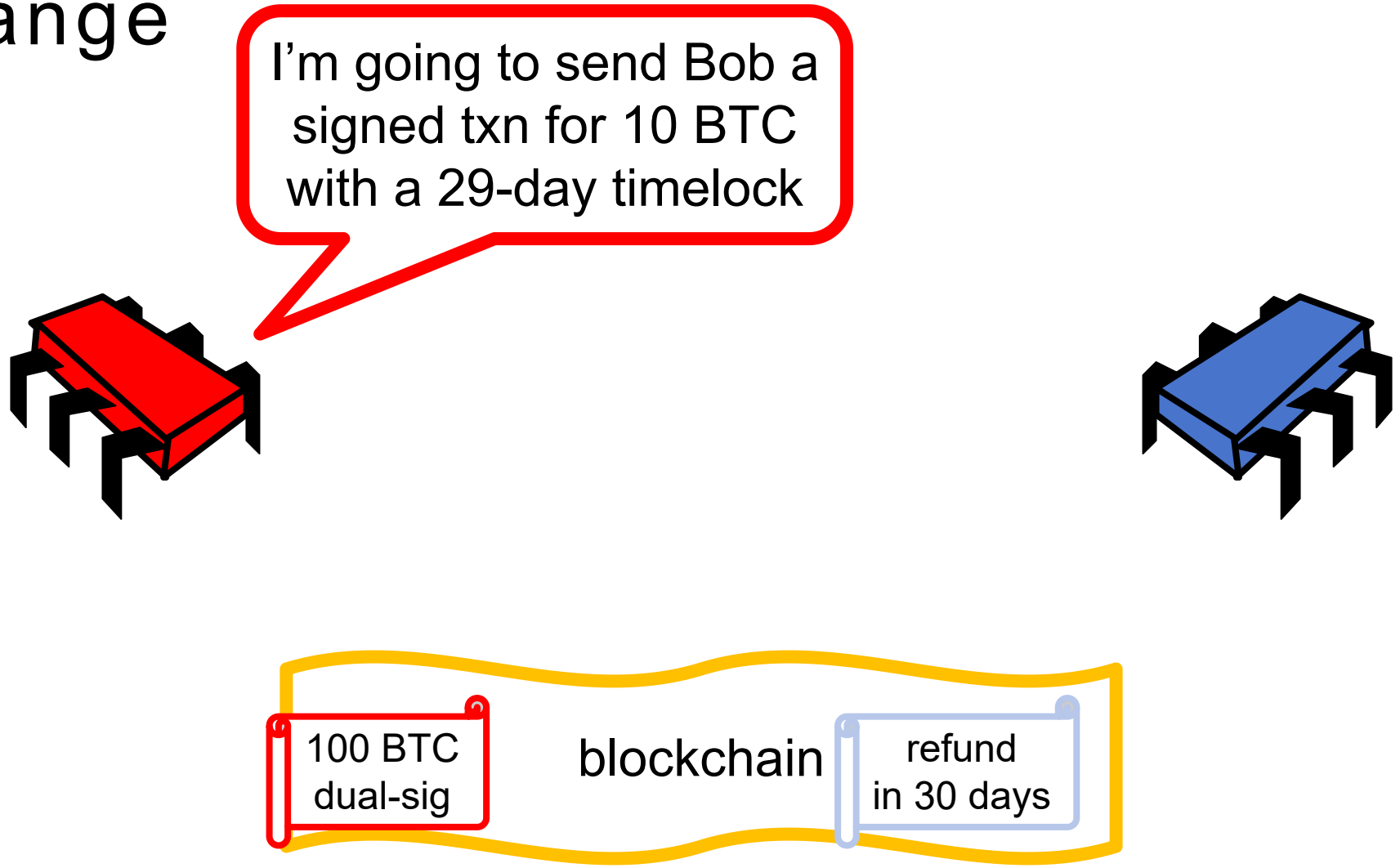
put 100 BTC in dual-sig account
both Alice & Bob must sign to release



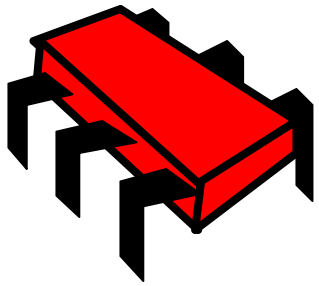




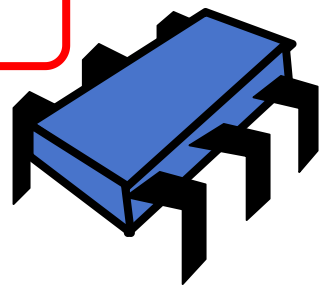
# Exchange


I’m going to send Bob a signed txn for 10 BTC with a 29-day timelock
100 BTC dual-sig
blockchain
refund in 30 days

# Exchange



pay 10 to Bob,
90 to Alice
in 29 days
signed Alice

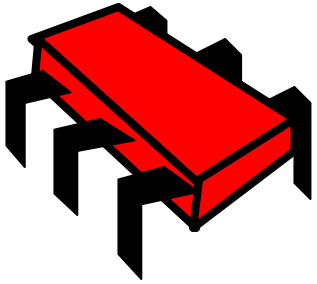


100 BTC
dual-sig

blockchain

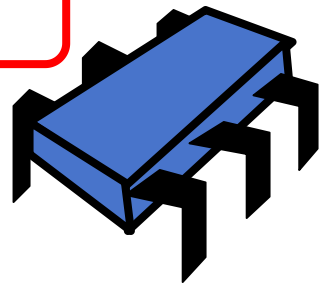
refund
in 30 days

# Exchange



pay 10 to Bob,
90 to Alice
in 29 days
signed Alice

Timelock must be less than refund timelock!

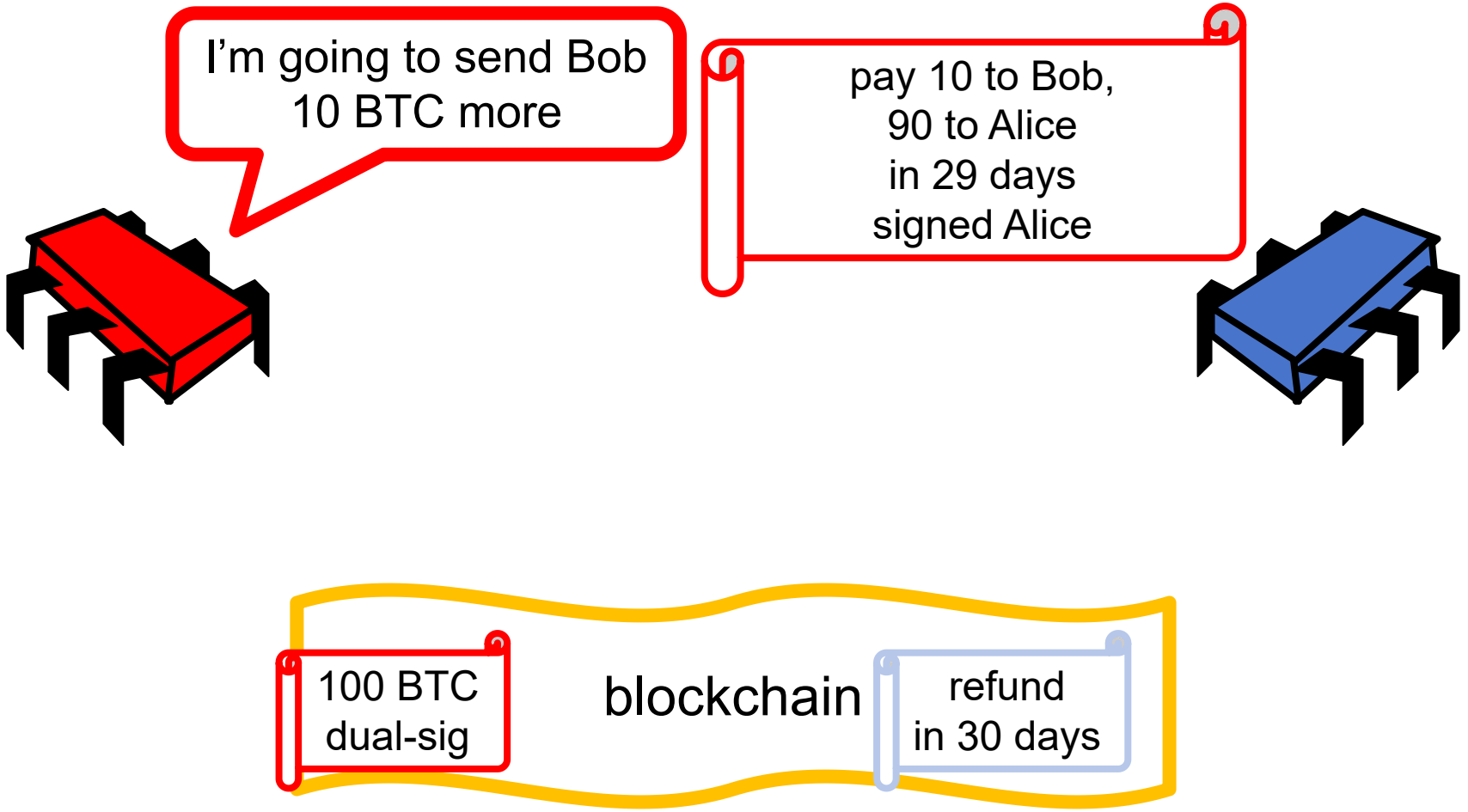


100 BTC
dual-sig

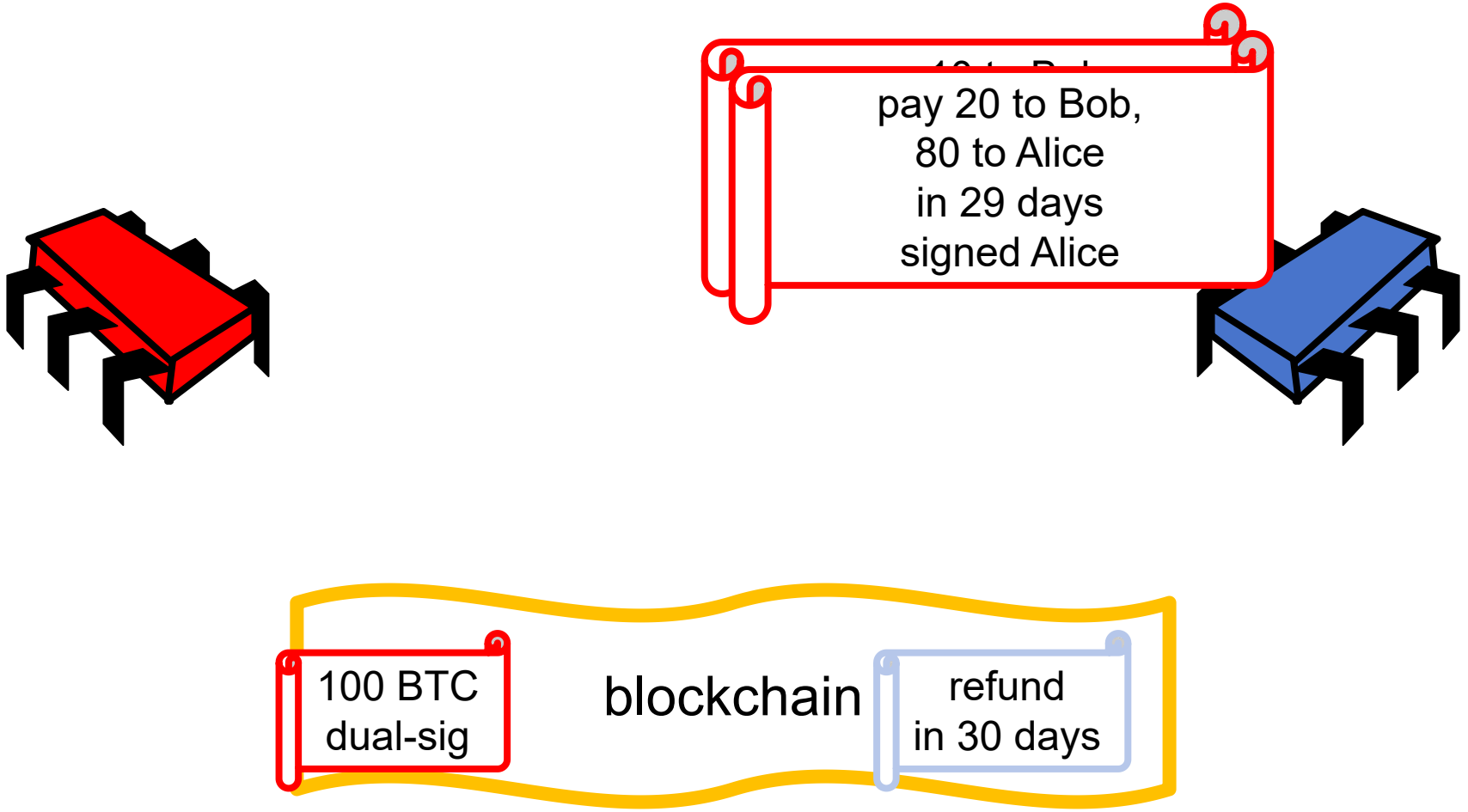
blockchain

refund
in 30 days

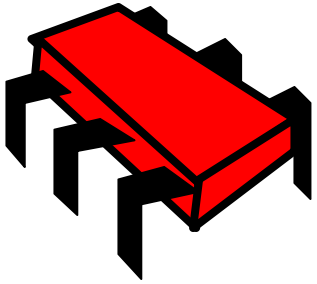
# Exchange

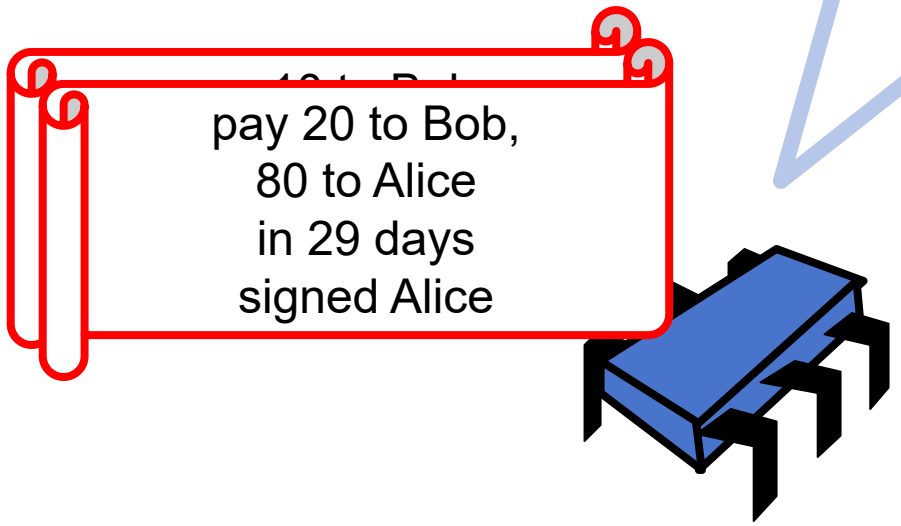

I’m going to send Bob
10 BTC more
pay 10 to Bob,
90 to Alice
in 29 days
signed Alice
100 BTC
dual-sig
blockchain
refund
in 30 days

# Exchange

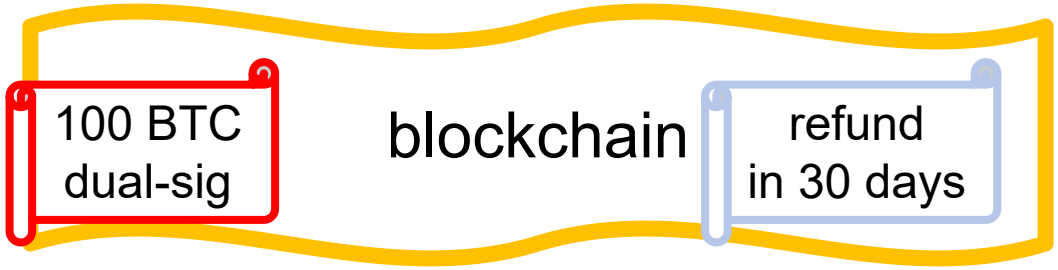

pay 20 to Bob,
80 to Alice
in 29 days
signed Alice
100 BTC
dual-sig
blockchain
refund
in 30 days

# Exchange

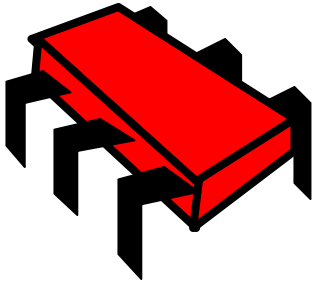


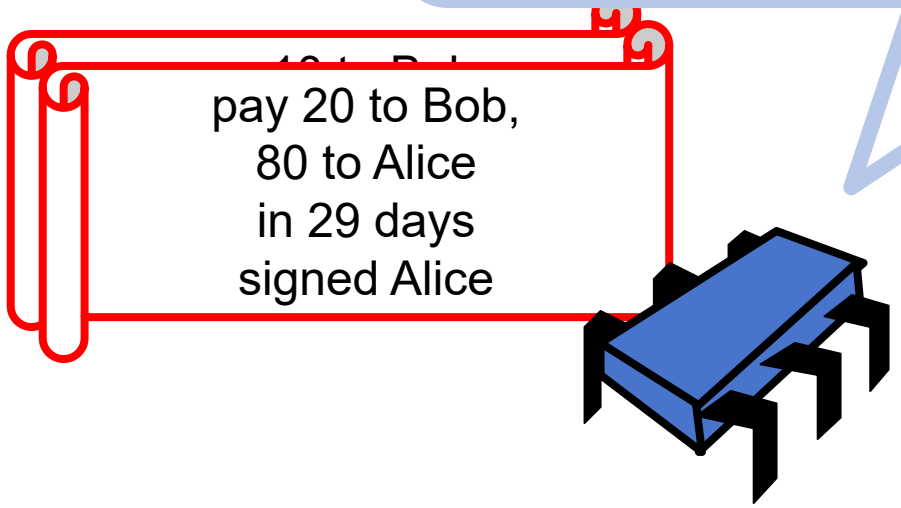



This timelock is the same as the last one …

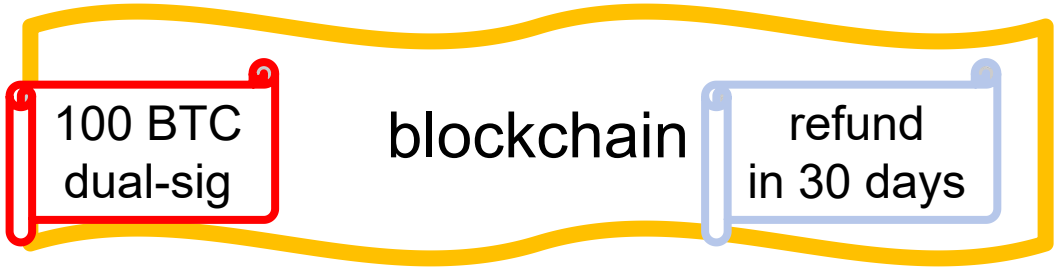


# Exchange

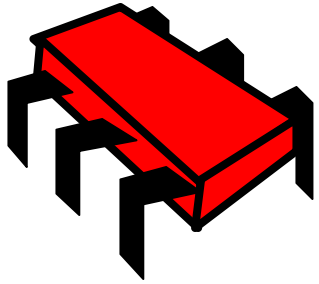




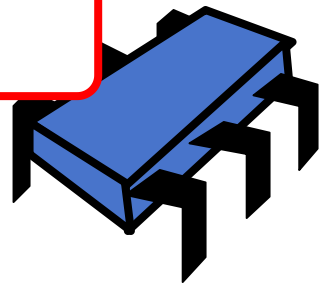

The 2nd txn is more valuable so I will keep that one and discard the other



# Exchange



pay 20 to Bob,
80 to Alice
in 29 days
signed Alice

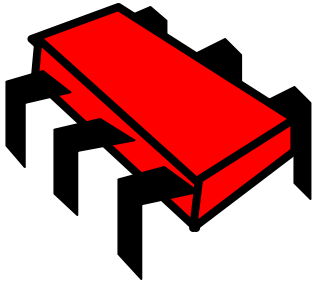


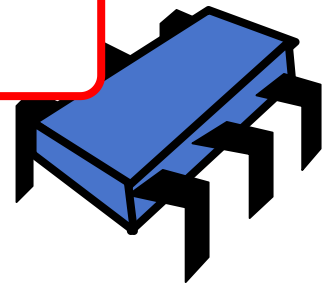
100 BTC
dual-sig

blockchain

refund
in 30 days

# Exchange





I'm going to send 5 back to Alice

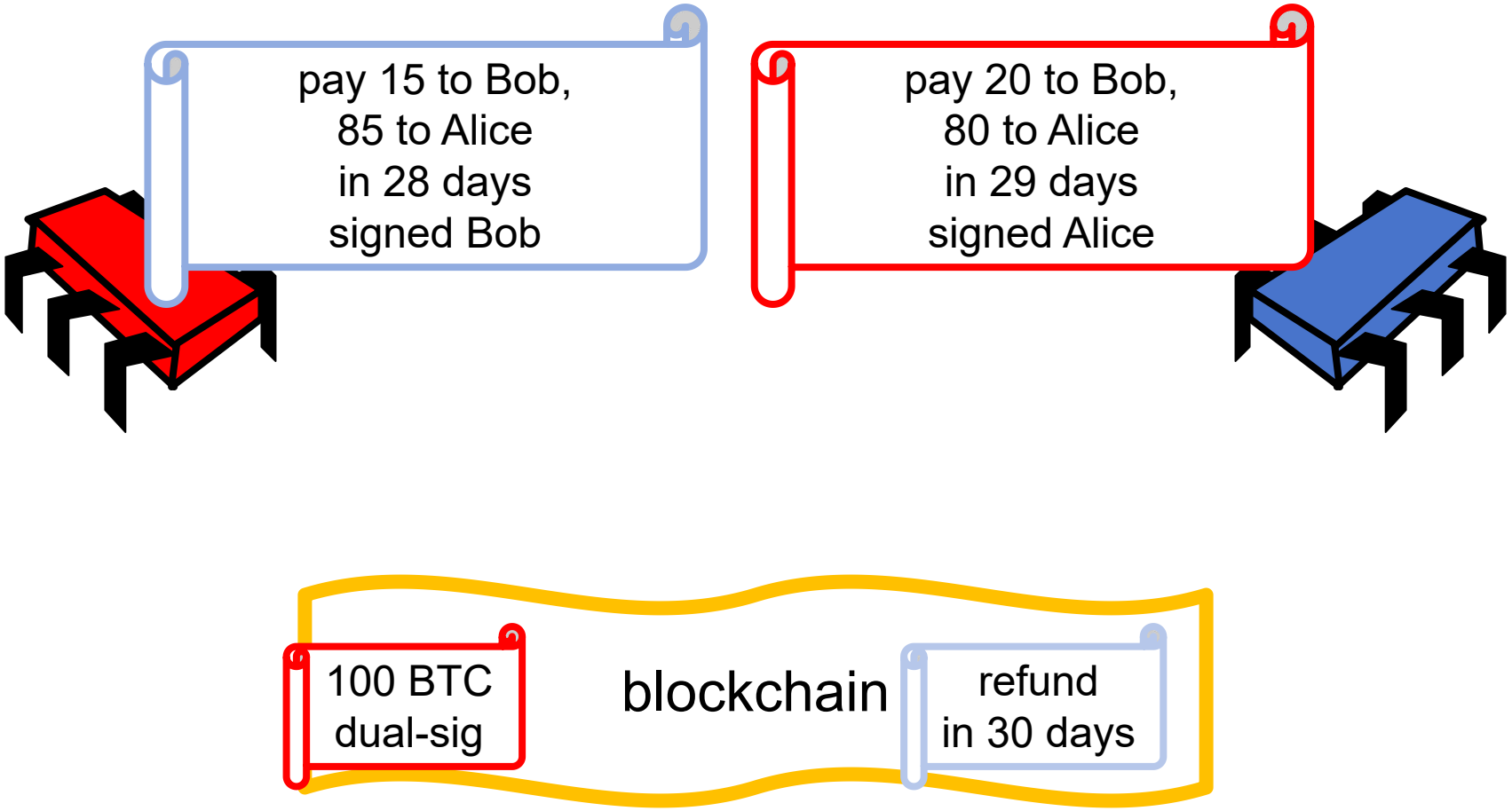
pay 20 to Bob,
80 to Alice
in 29 days
signed Alice

100 BTC
dual-sig

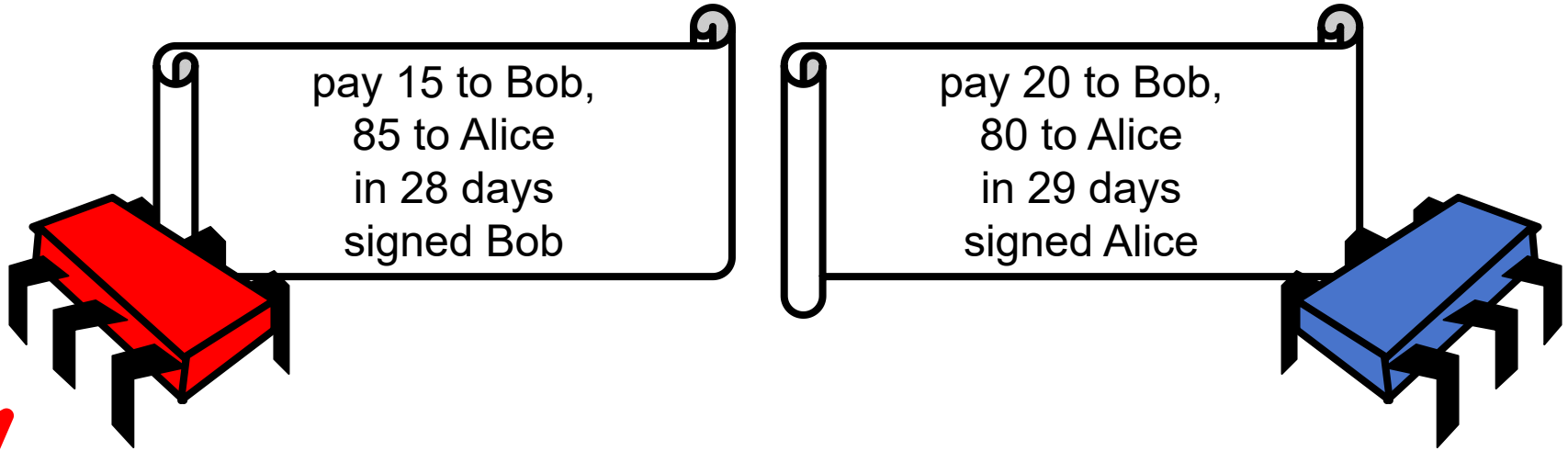
blockchain

refund
in 30 days

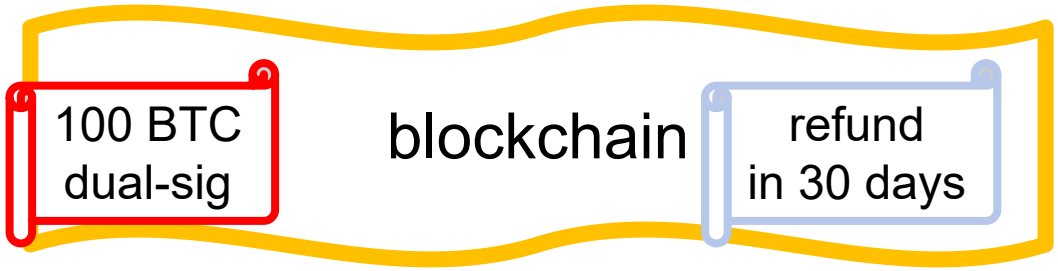
# Exchange


pay 15 to Bob,
85 to Alice
in 28 days
signed Bob
pay 20 to Bob,
80 to Alice
in 29 days
signed Alice
100 BTC
dual-sig
blockchain
refund
in 30 days

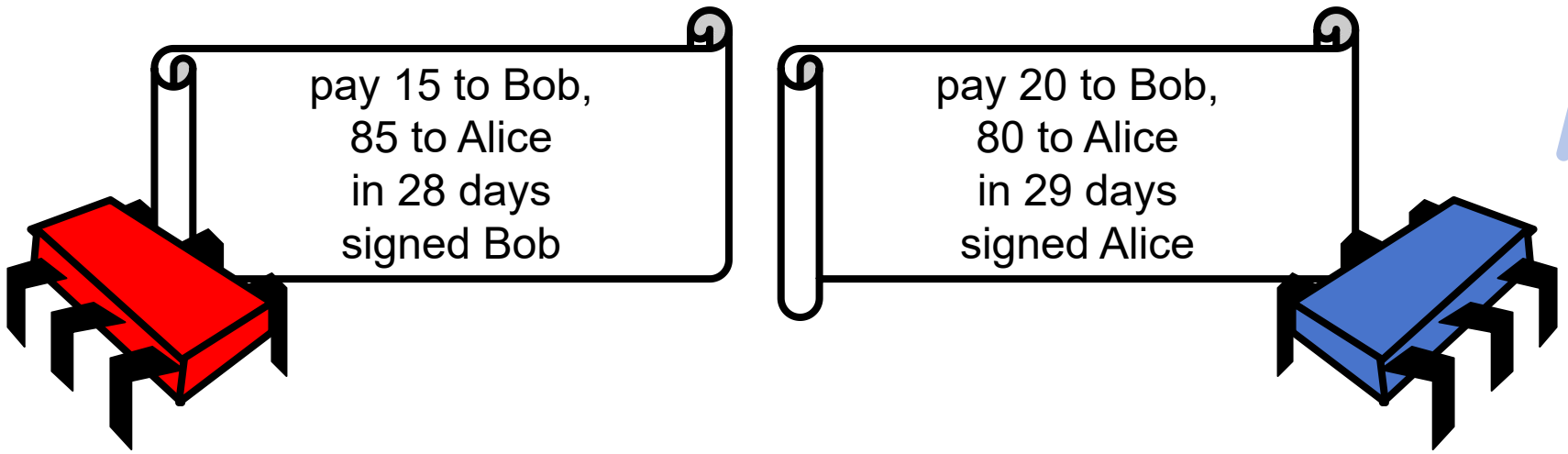
# Exchange

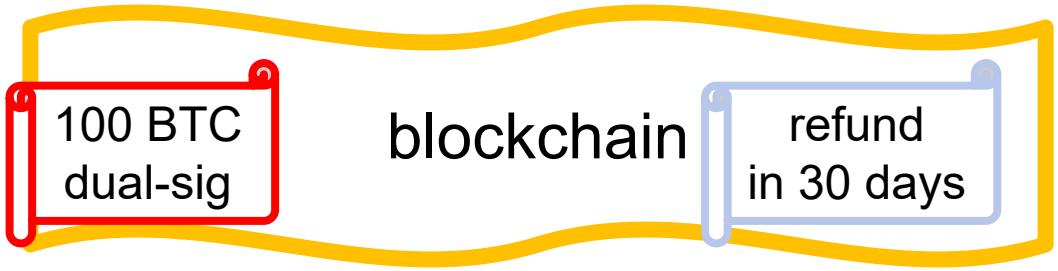



Timelock of 28 days is earlier than my last timelock

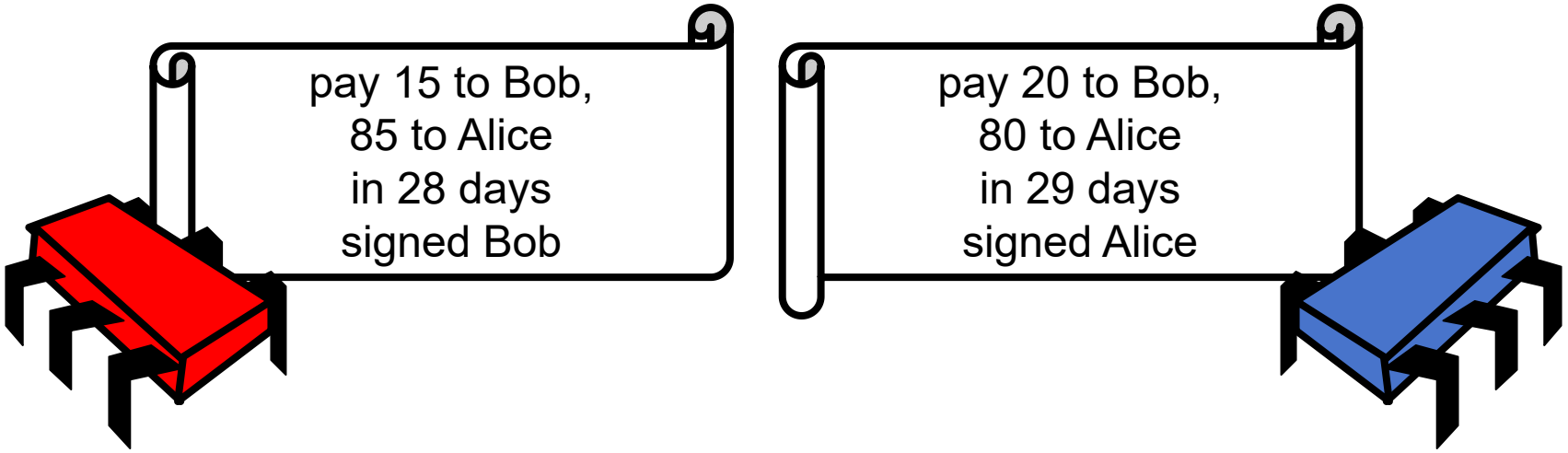


# Exchange

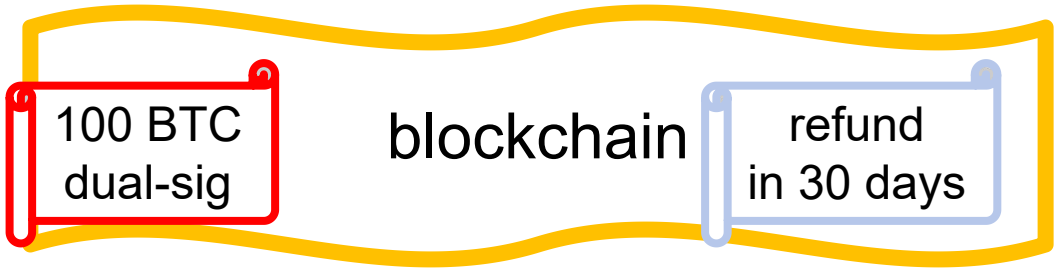

pay 15 to Bob,
85 to Alice
in 28 days
signed Bob
pay 20 to Bob,
80 to Alice
in 29 days
signed Alice
Each time we reverse direction, we shorten the timelocks


100 BTC
dual-sig
blockchain
refund
in 30 days

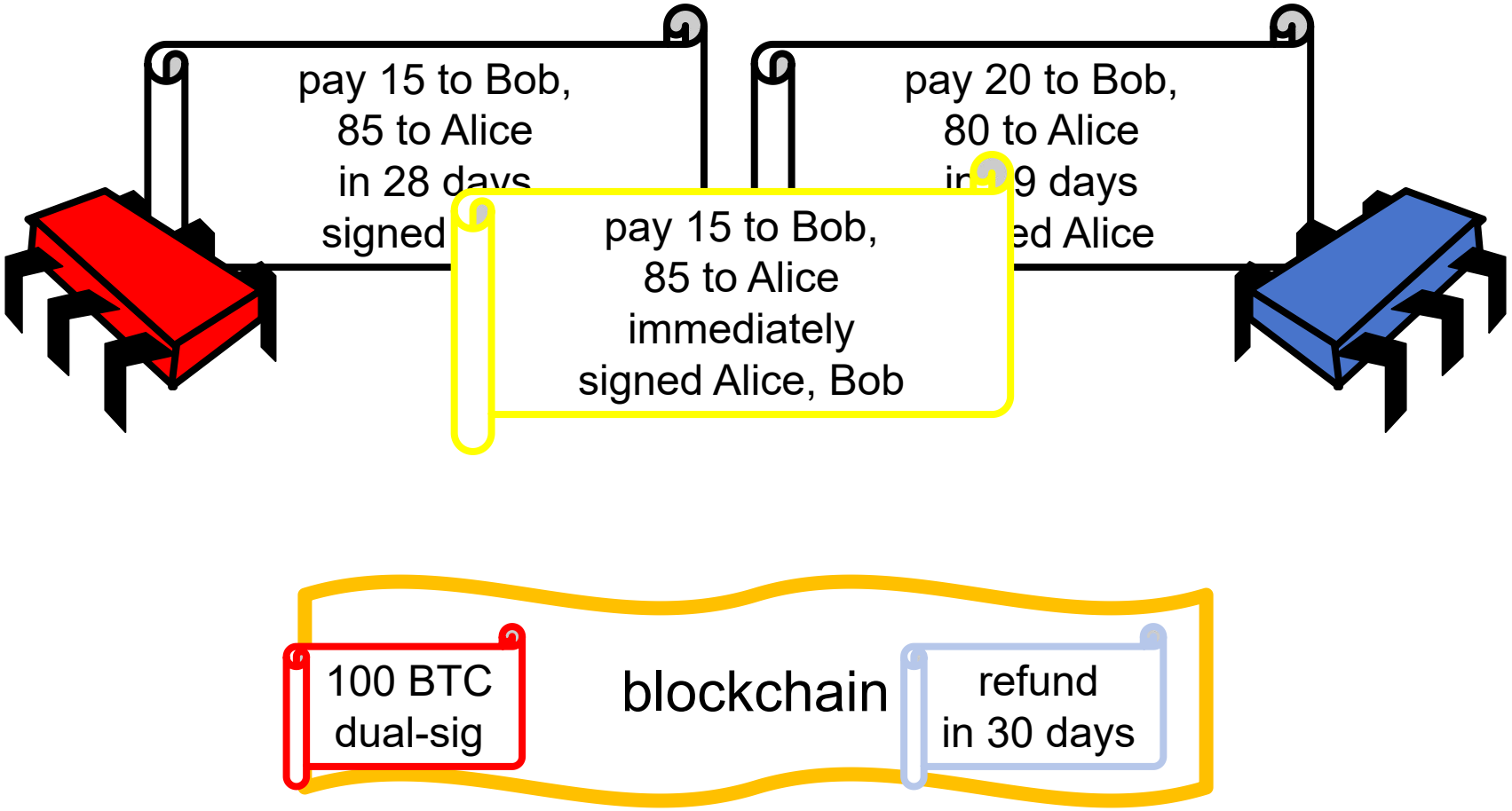
# Settlement



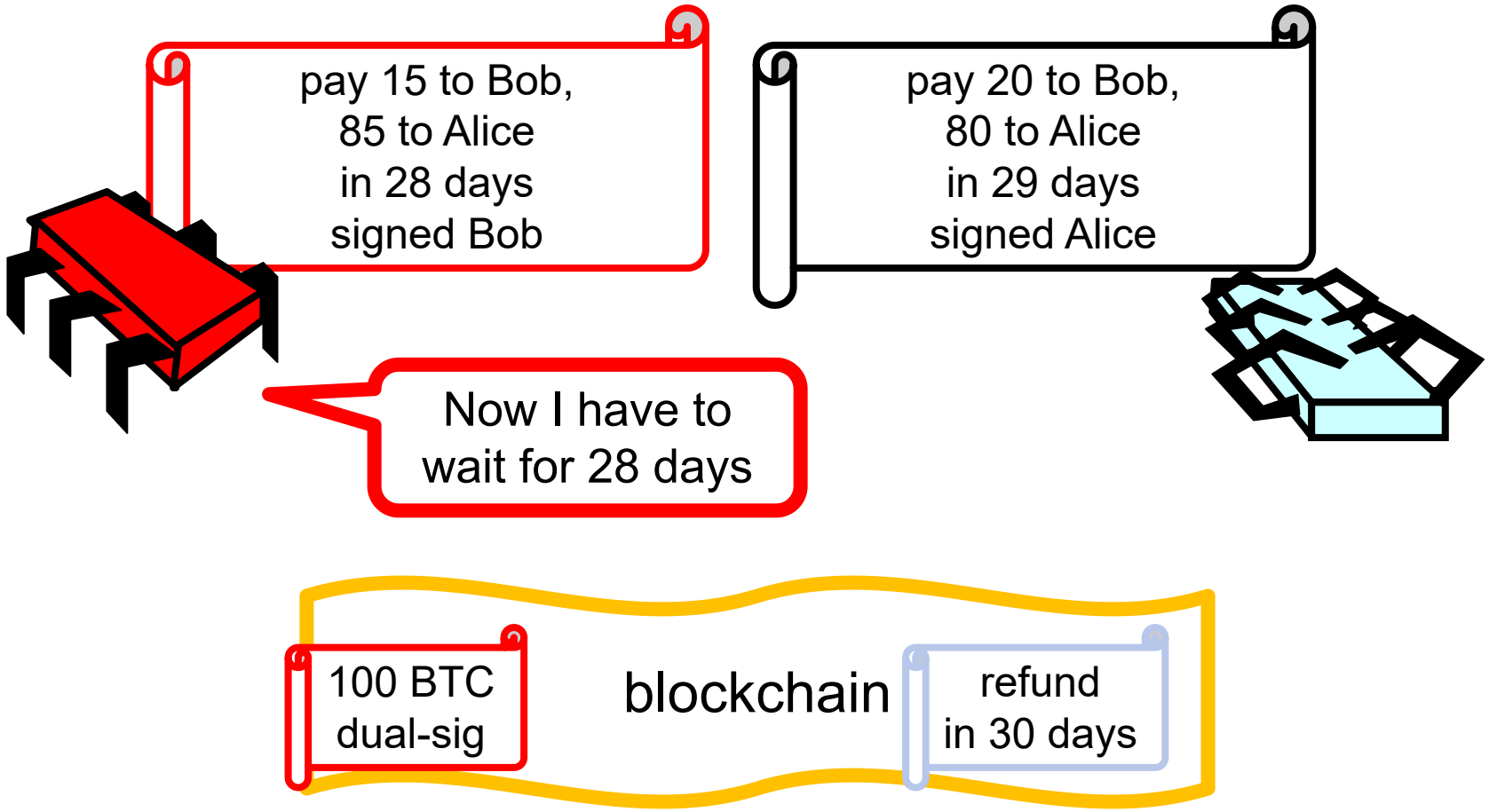

Closing time! Let's draw up a settlement transaction



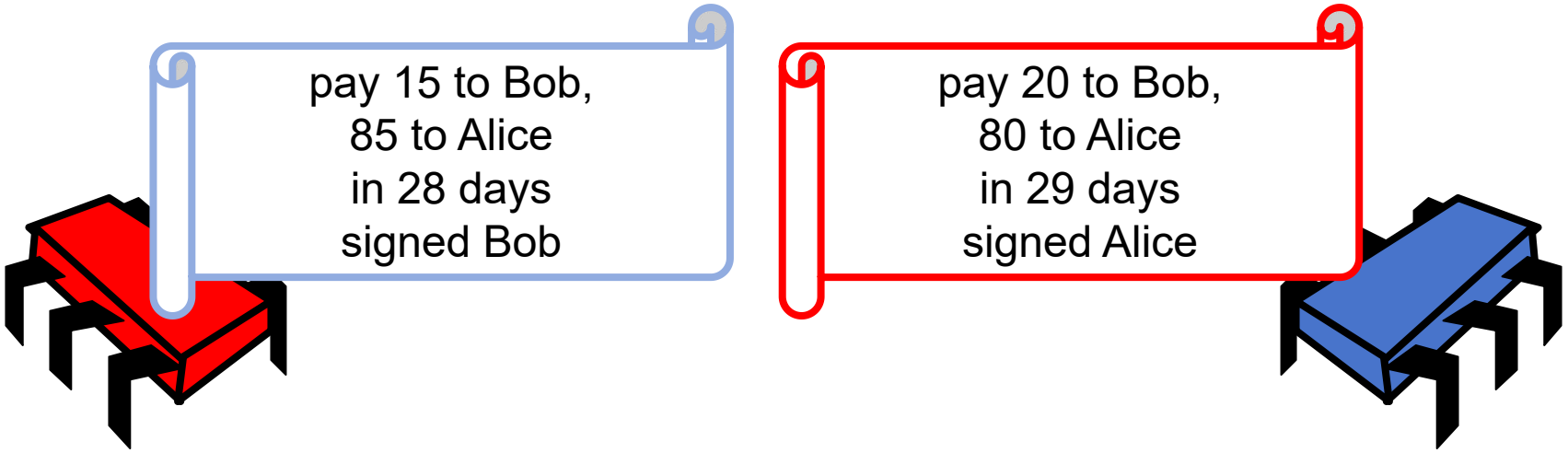
# Settlement

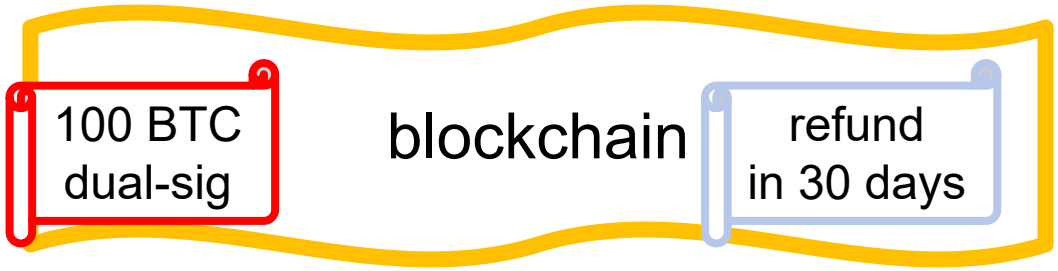

pay 15 to Bob,
85 to Alice
in 28 days
signed
pay 15 to Bob,
85 to Alice
immediately
signed Alice, Bob
pay 20 to Bob,
80 to Alice
9 days
ed Alice
100 BTC
dual-sig
blockchain
refund
in 30 days

# Settlement

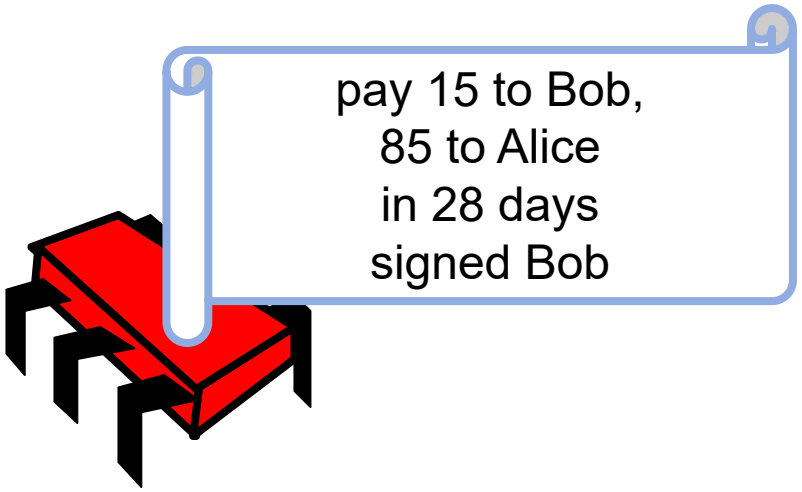

pay 15 to Bob,
85 to Alice
in 28 days
signed Bob
pay 20 to Bob,
80 to Alice
in 29 days
signed Alice
Now I have to
wait for 28 days
100 BTC
dual-sig
blockchain
refund
in 30 days

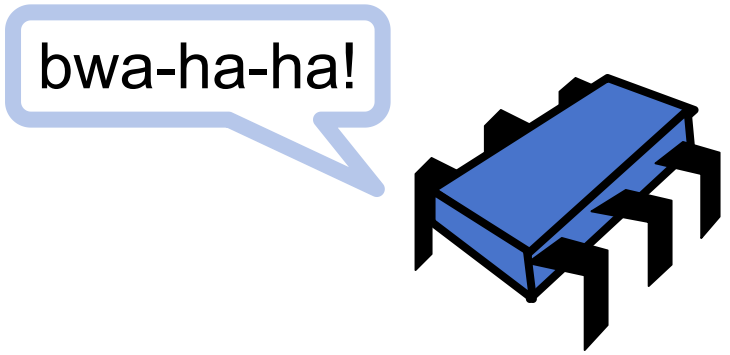
# Bob Tries to Cheat

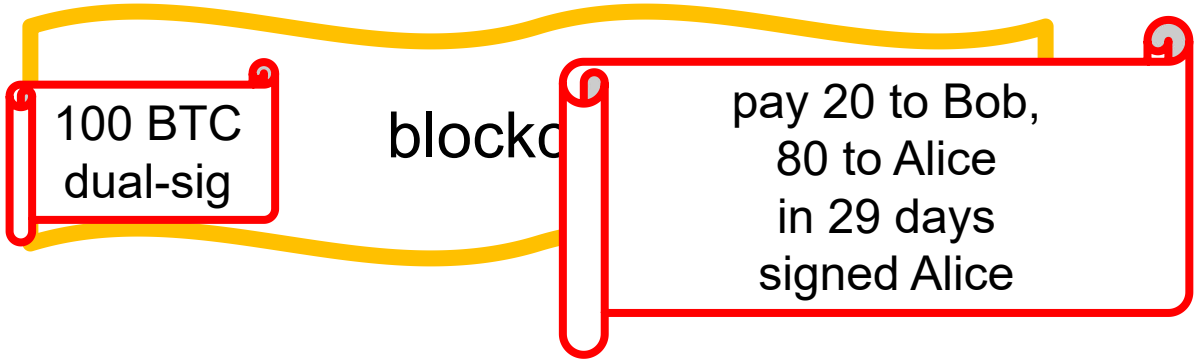


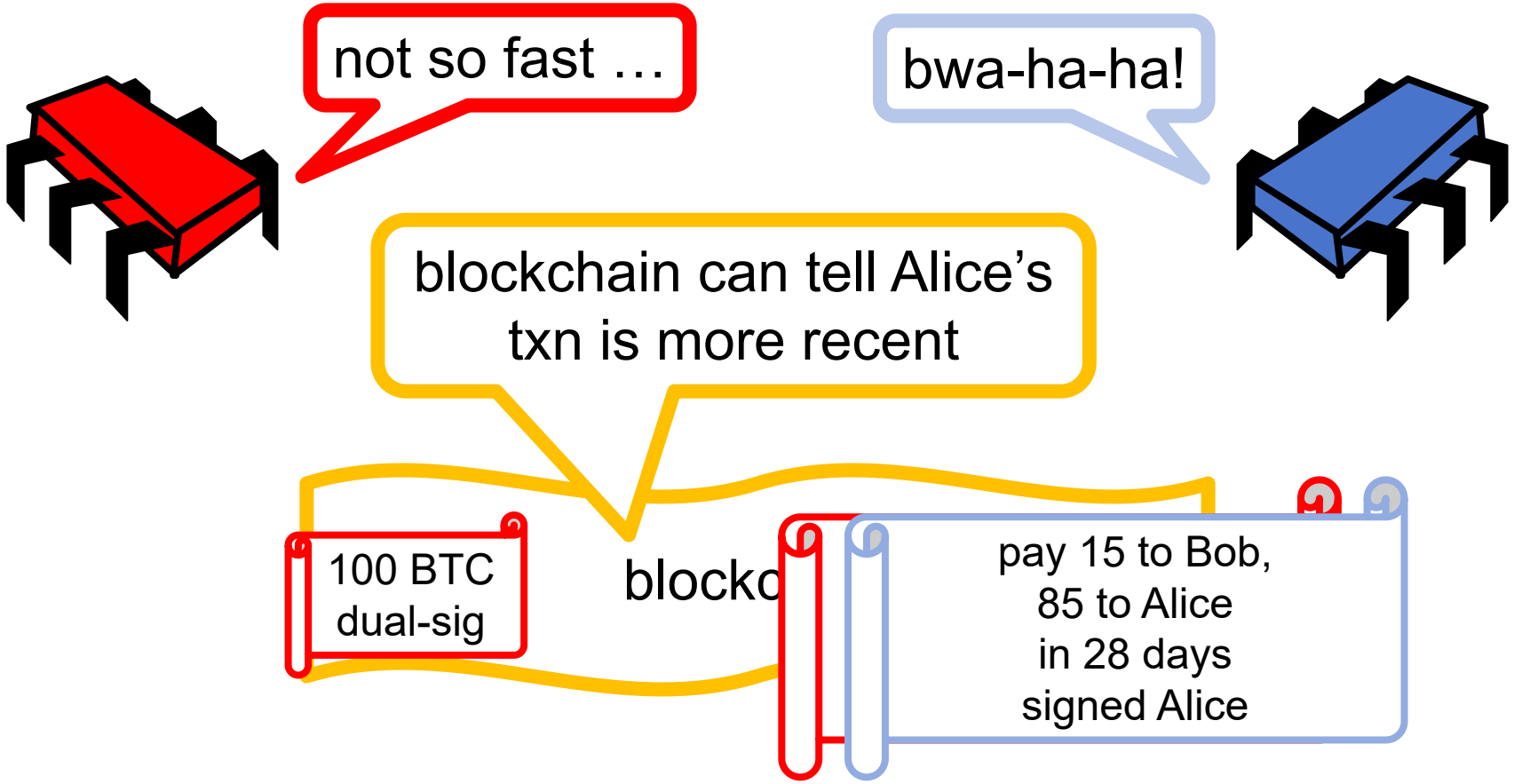
# Bob Tries to Cheat



# Bob Tries to Cheat



# Alice Wants to Pay Carol




Alice can send Carol all the micro-transactions she wants …

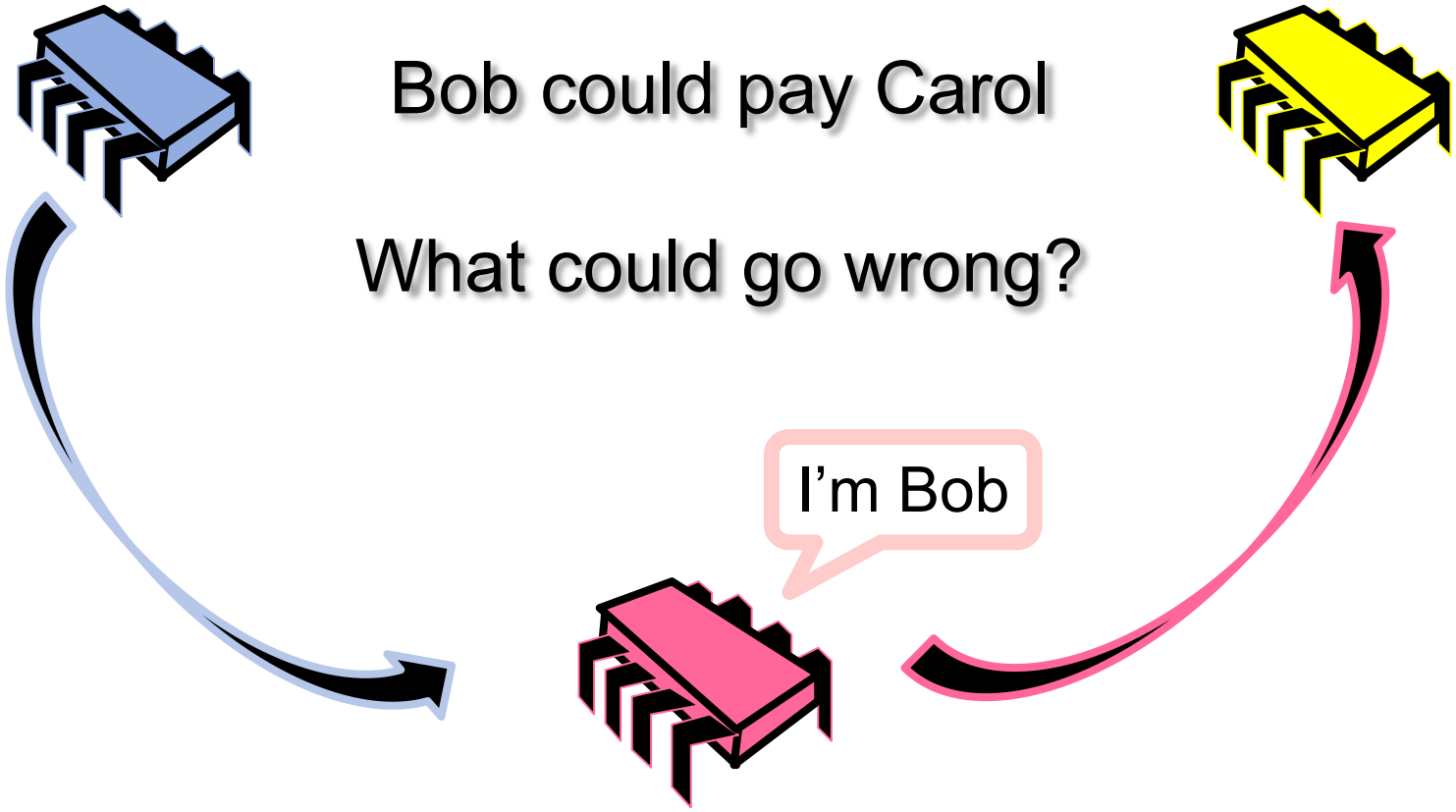
But there is no settlement channel between them

# They Both Have Settlement Channels with Bob

Alice could pay Bob

Bob could pay Carol

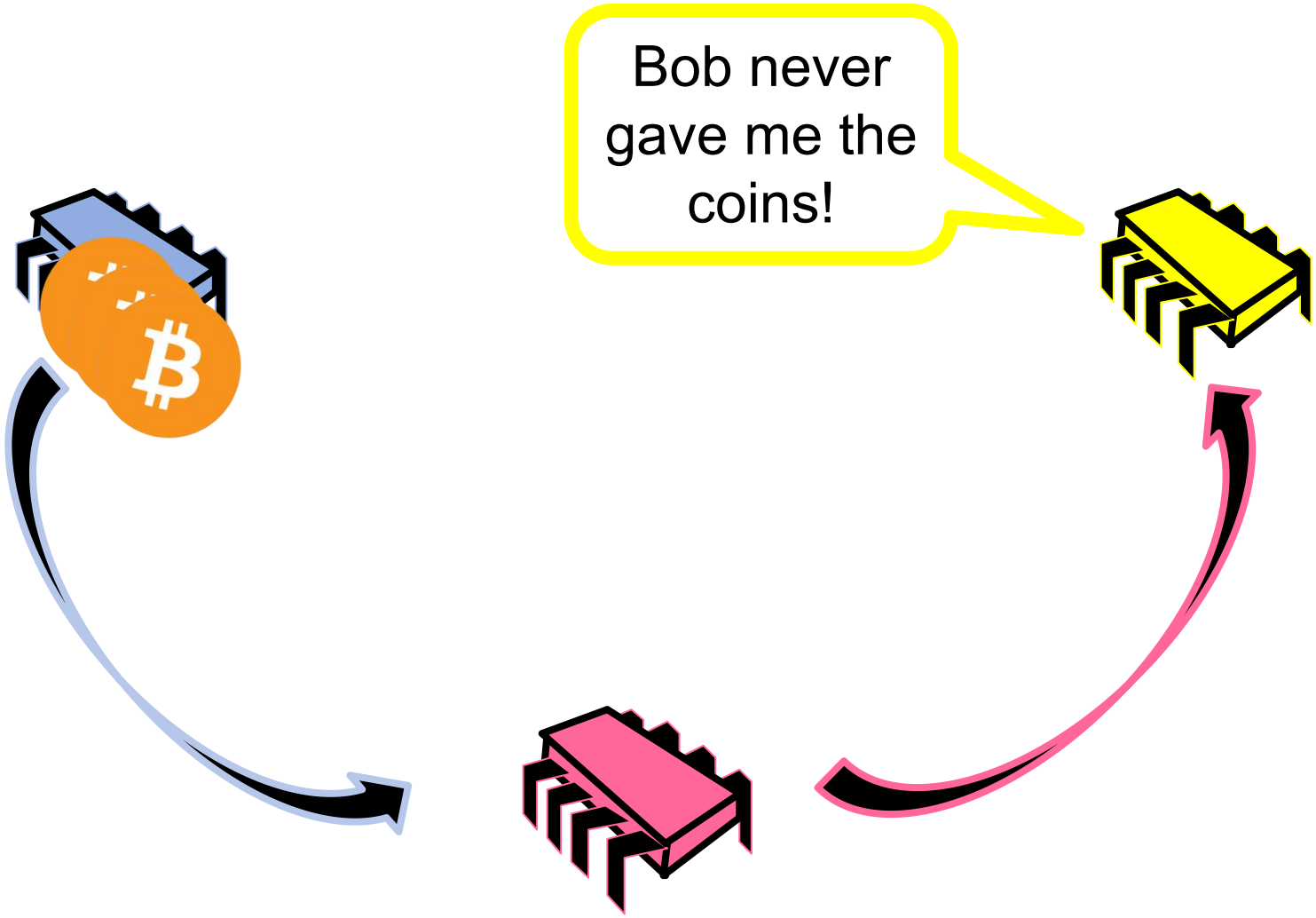
What could go wrong?



# Bob Could Steal the Coins



# Carol Could Deny Everything

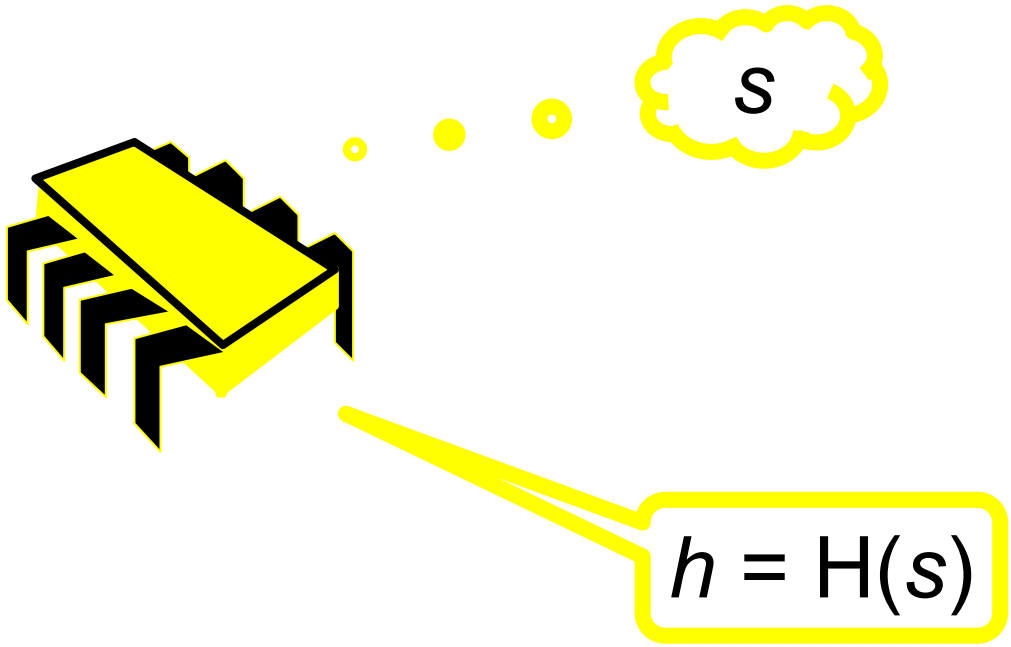


# Hashlock

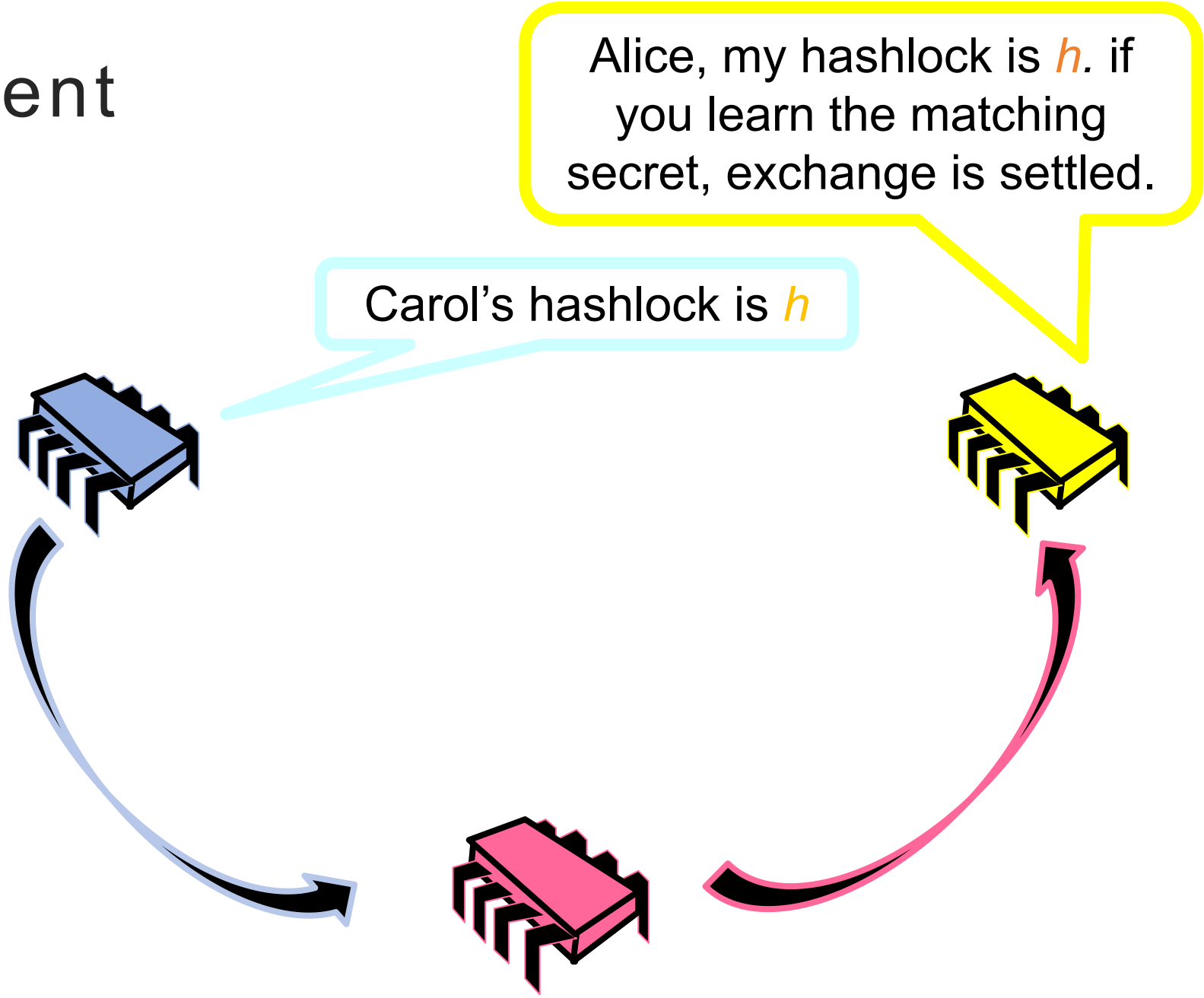



**Contract: I know H(*s*), will pay $$$ if you provide *s***

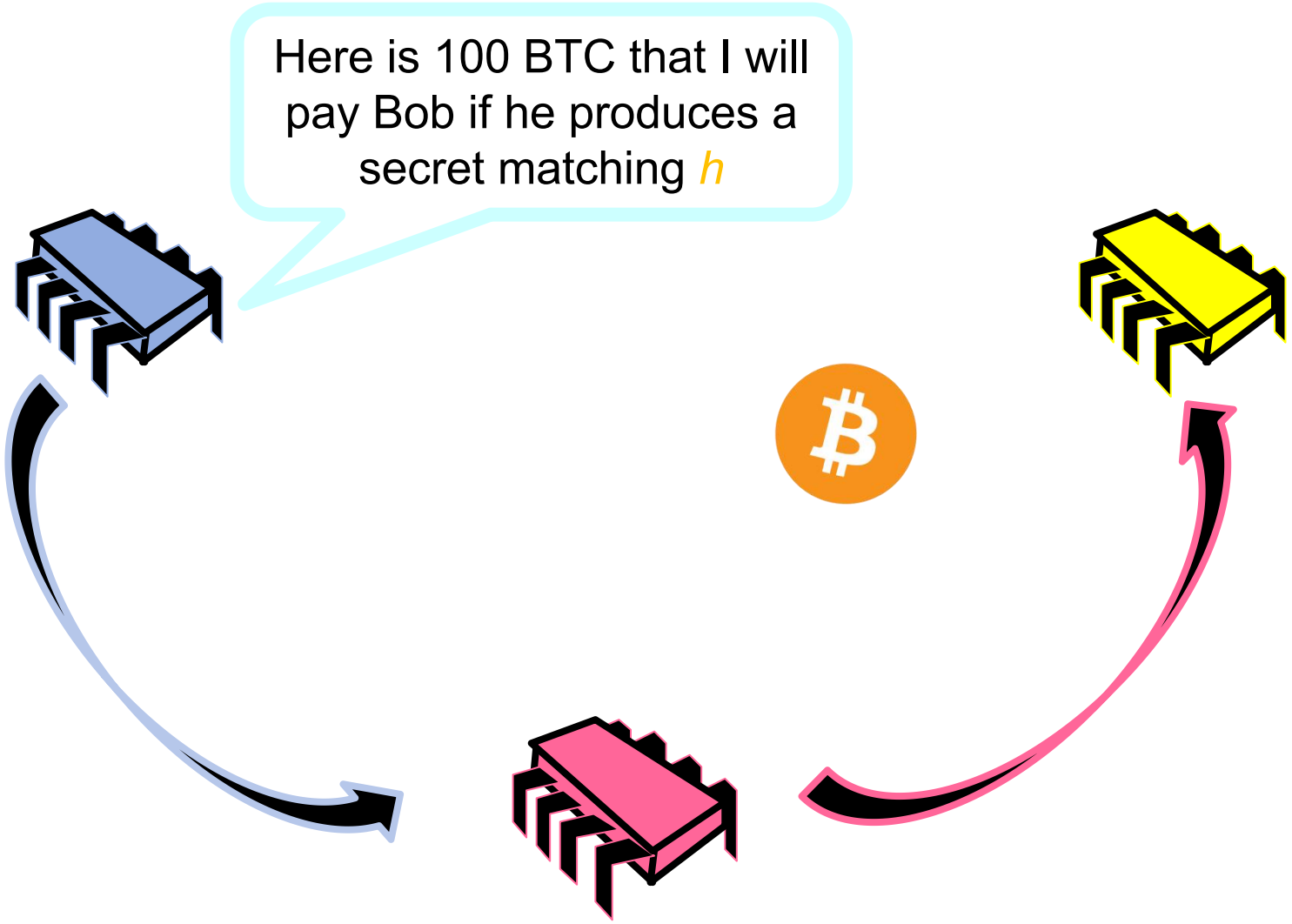
# Carol Has a Secret



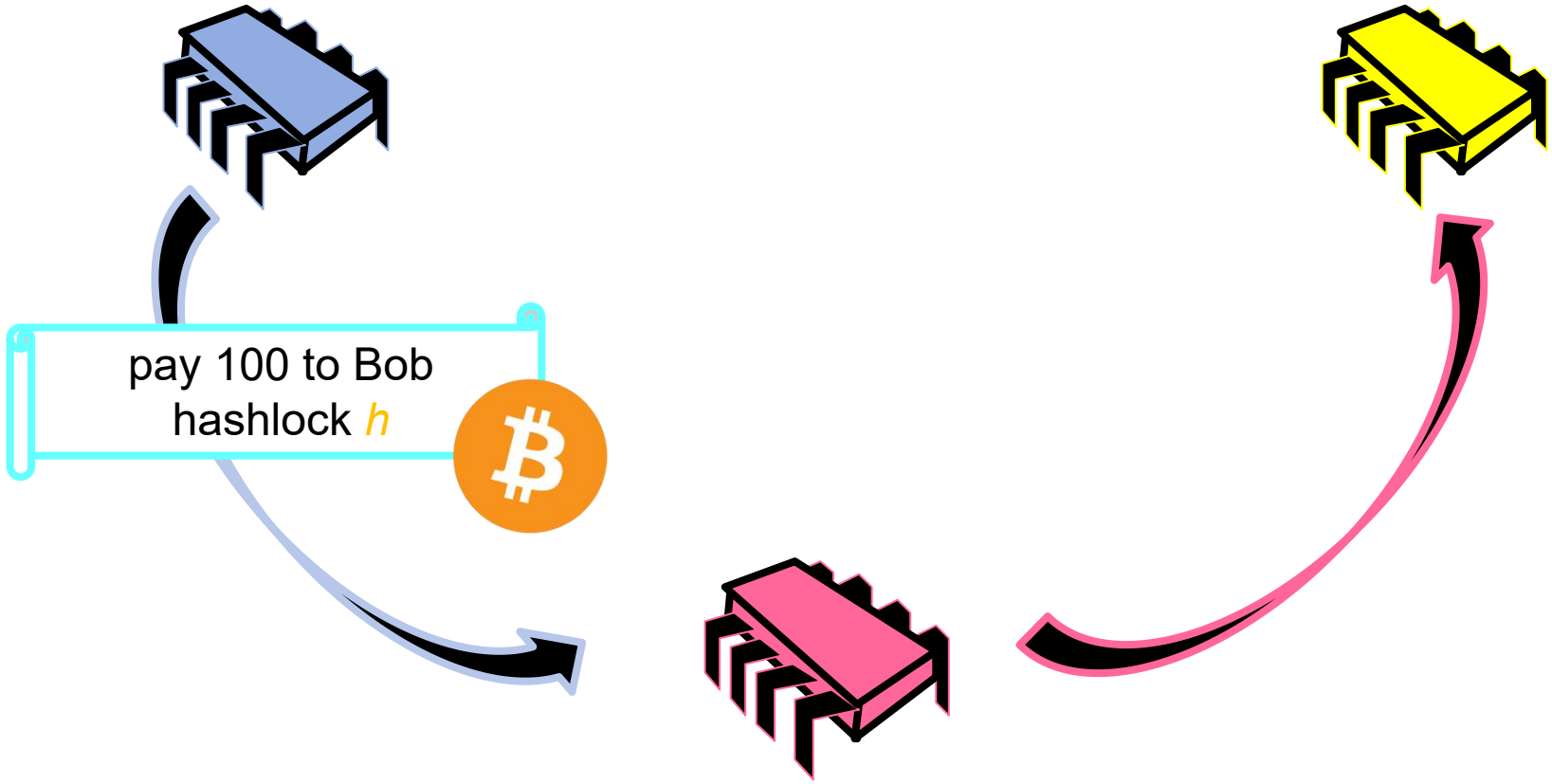
# Settlement


Alice, my hashlock is h. if you learn the matching secret, exchange is settled.
Carol's hashlock is h

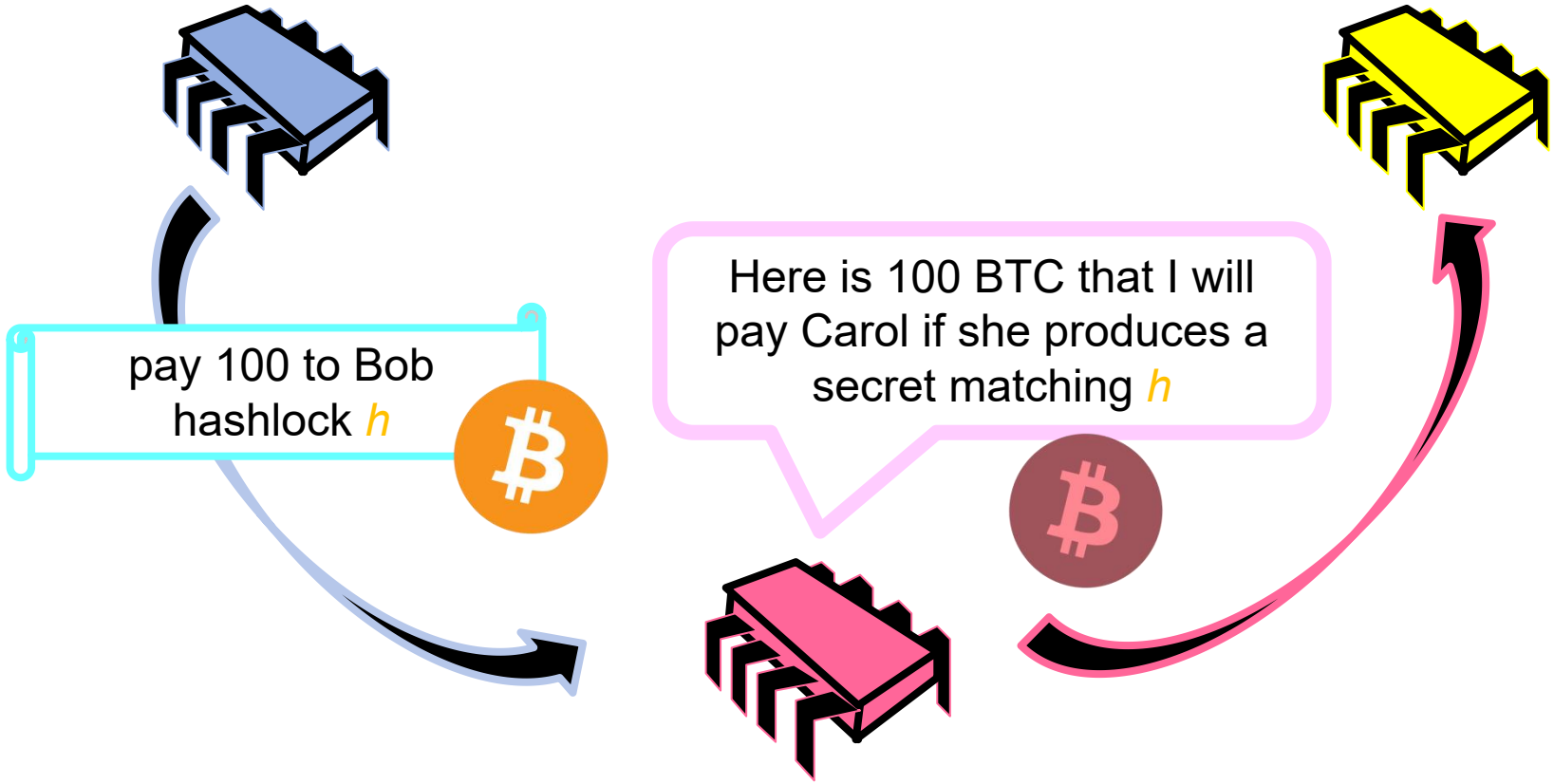
# Settlement


Here is 100 BTC that I will pay Bob if he produces a secret matching *h*

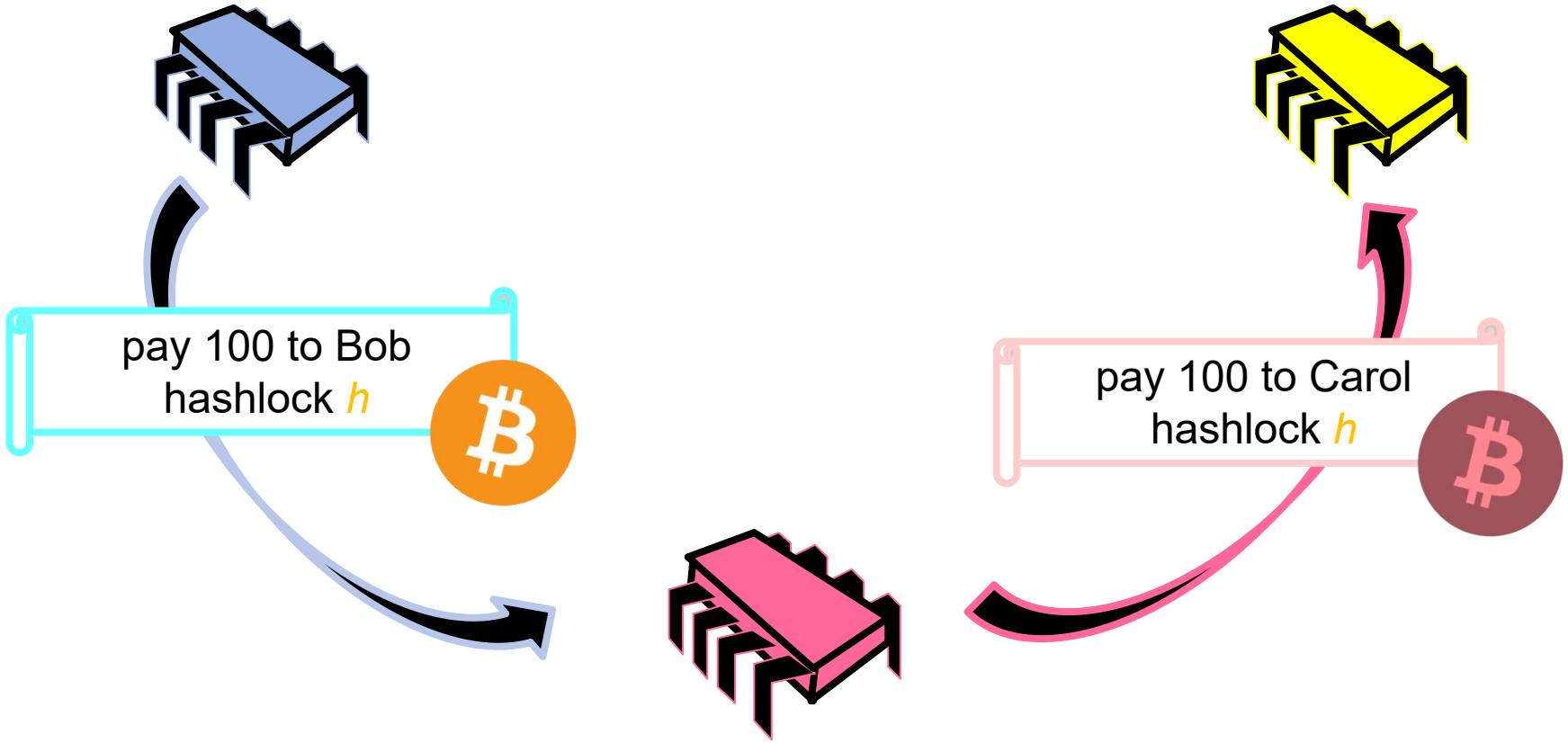
# Settlement



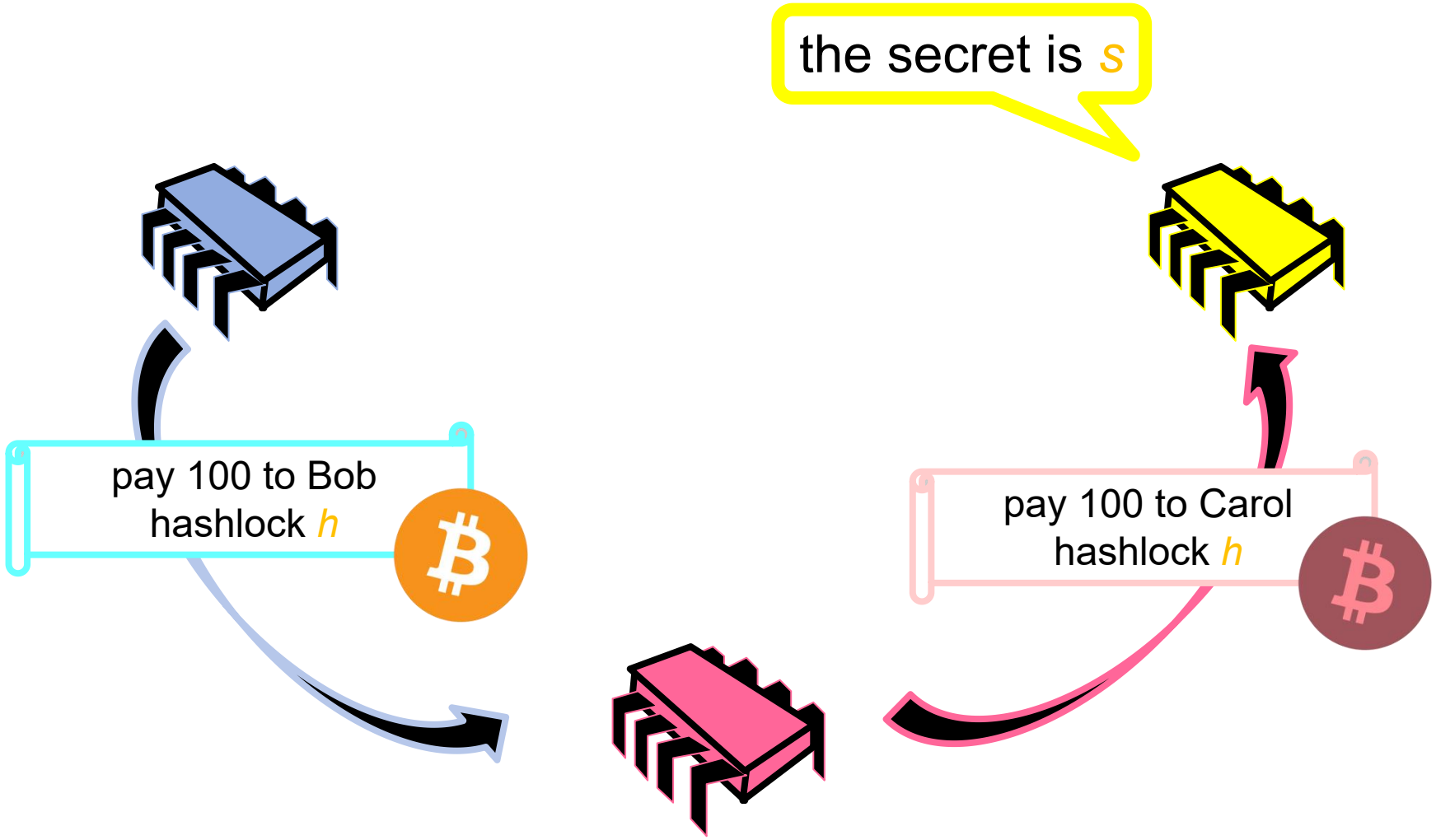
# Settlement


pay 100 to Bob
hashlock h
Here is 100 BTC that I will pay Carol if she produces a secret matching h

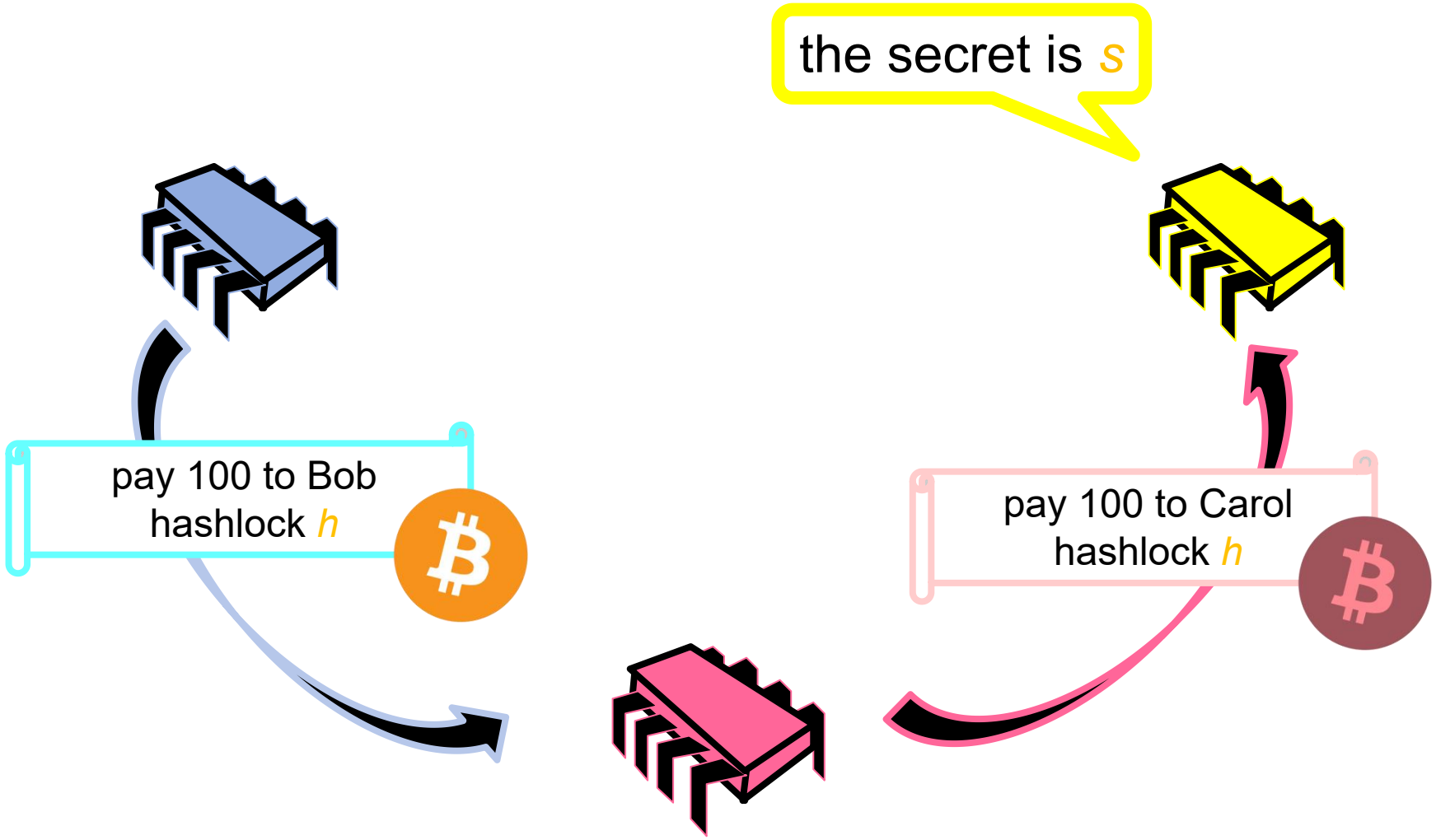
# Settlement



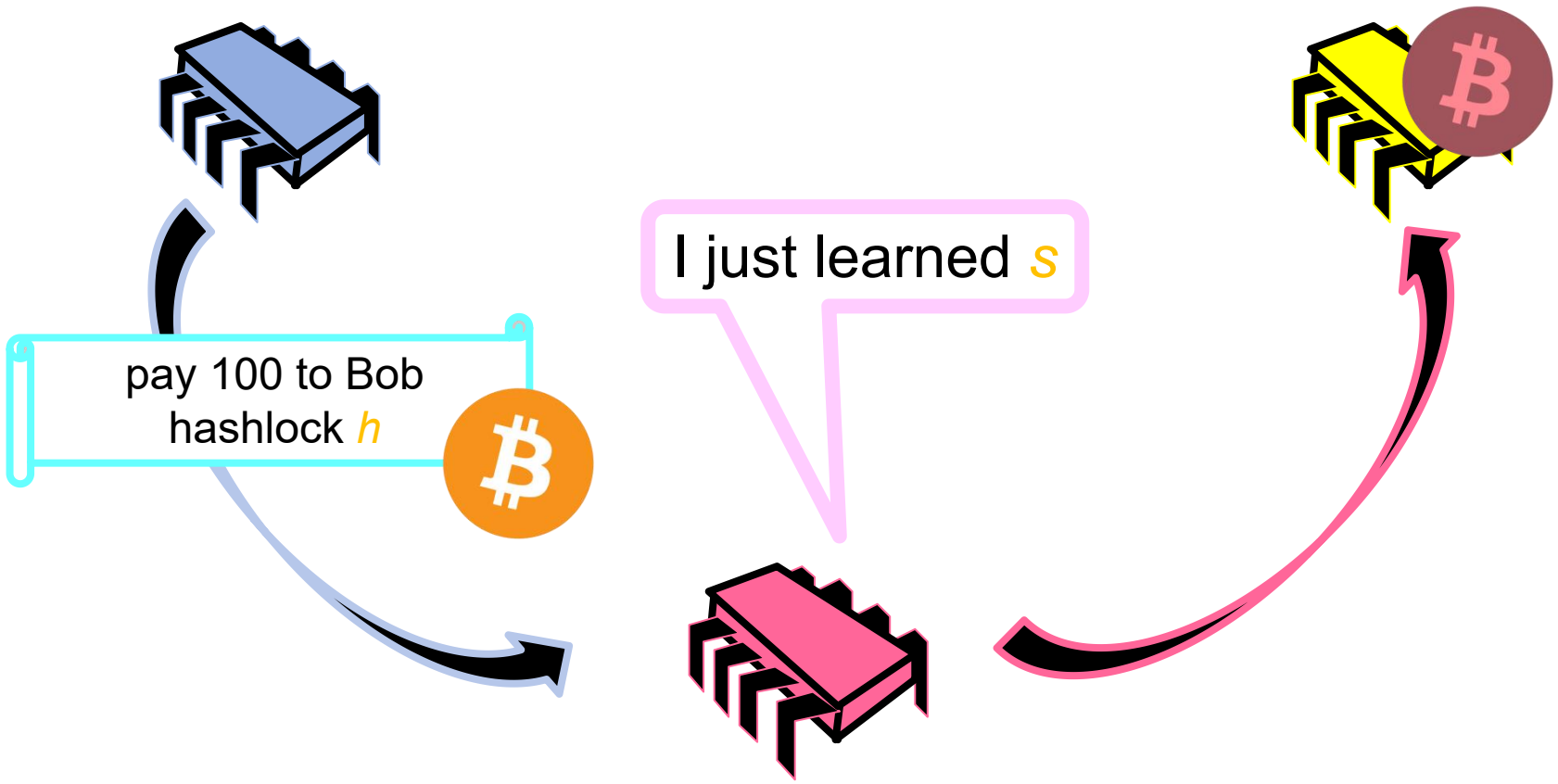
# Settlement


the secret is *s*
pay 100 to Bob
hashlock *h*
pay 100 to Carol
hashlock *h*

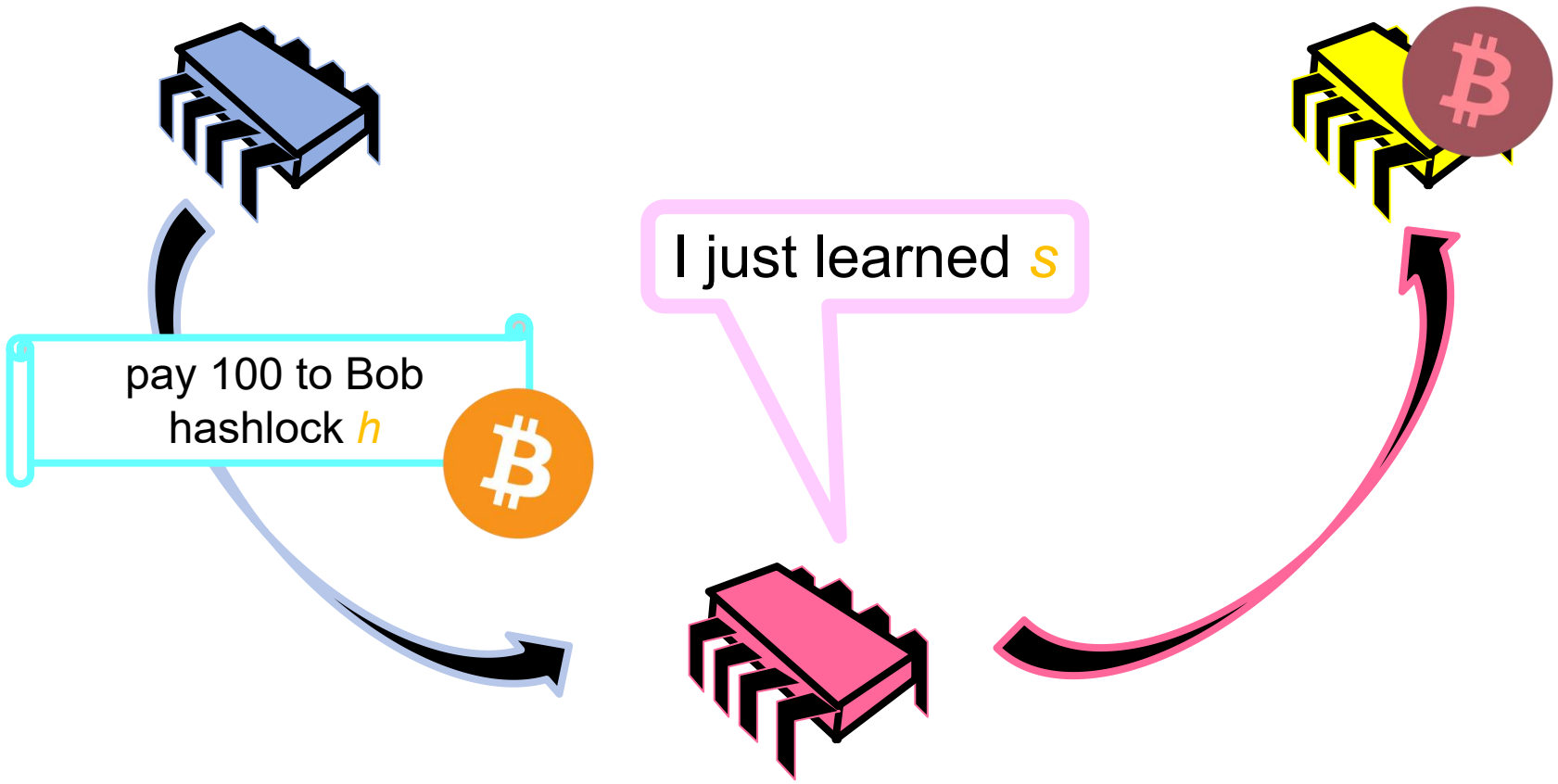
# Settlement


the secret is *s*
pay 100 to Bob
hashlock *h*
pay 100 to Carol
hashlock *h*

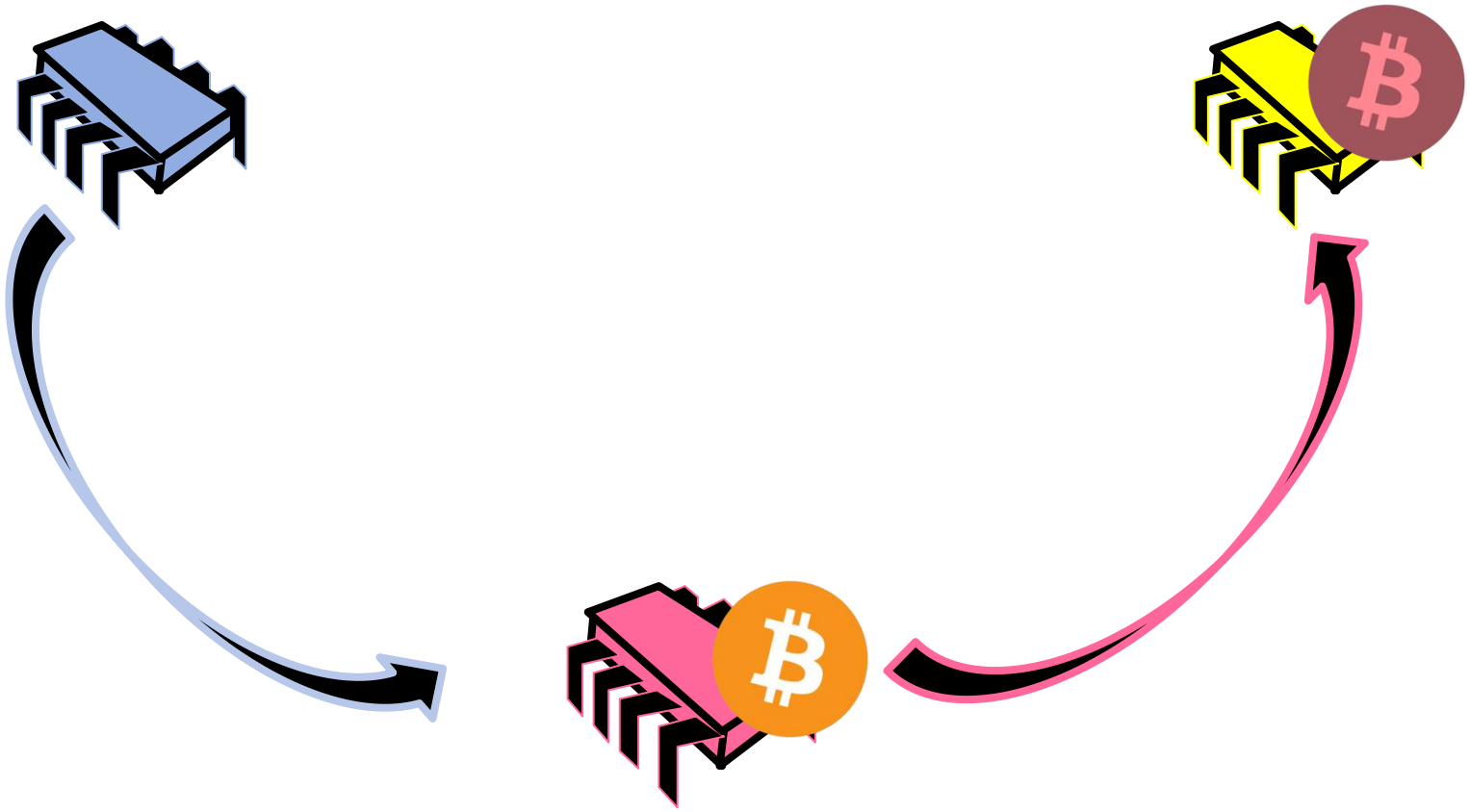
# Settlement



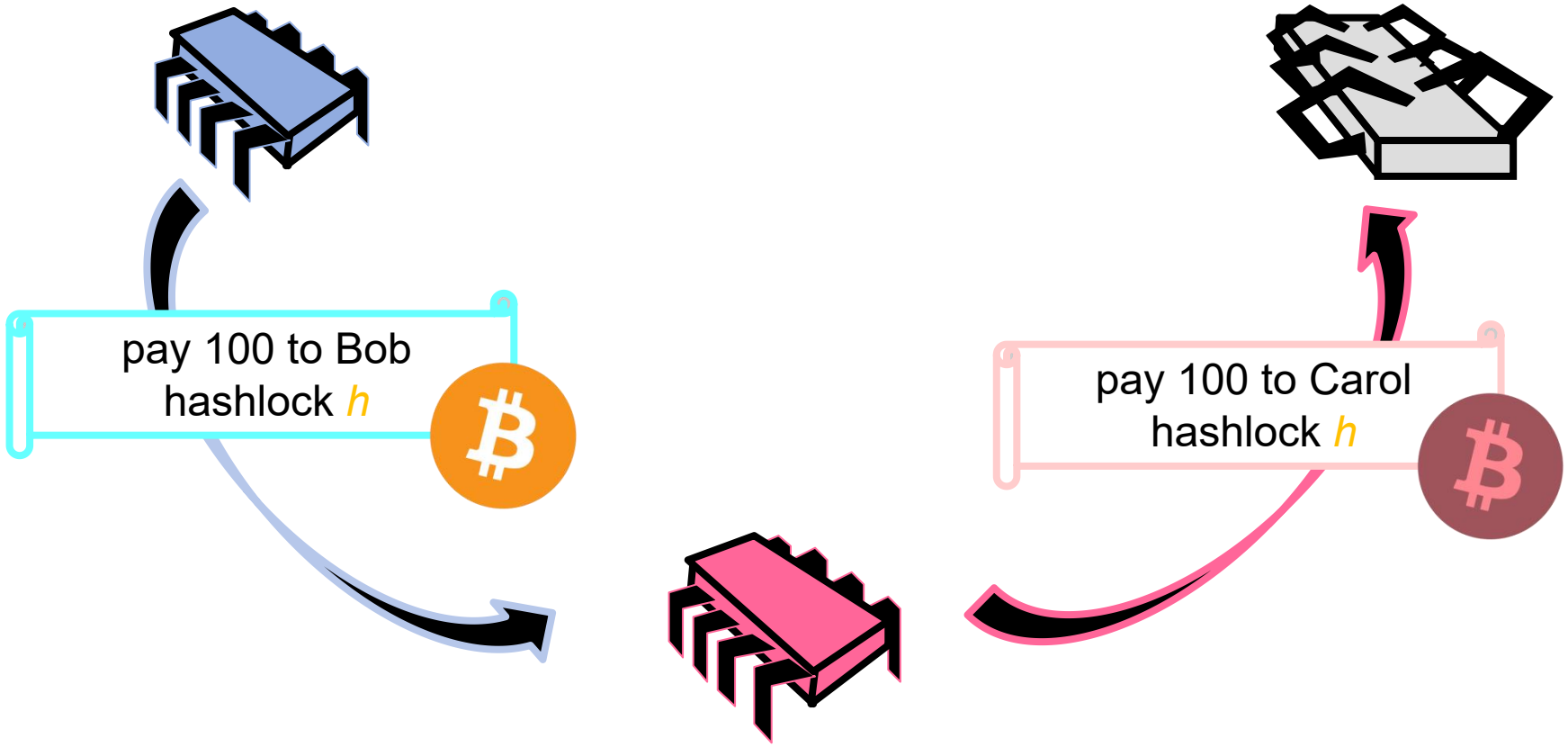
# Settlement



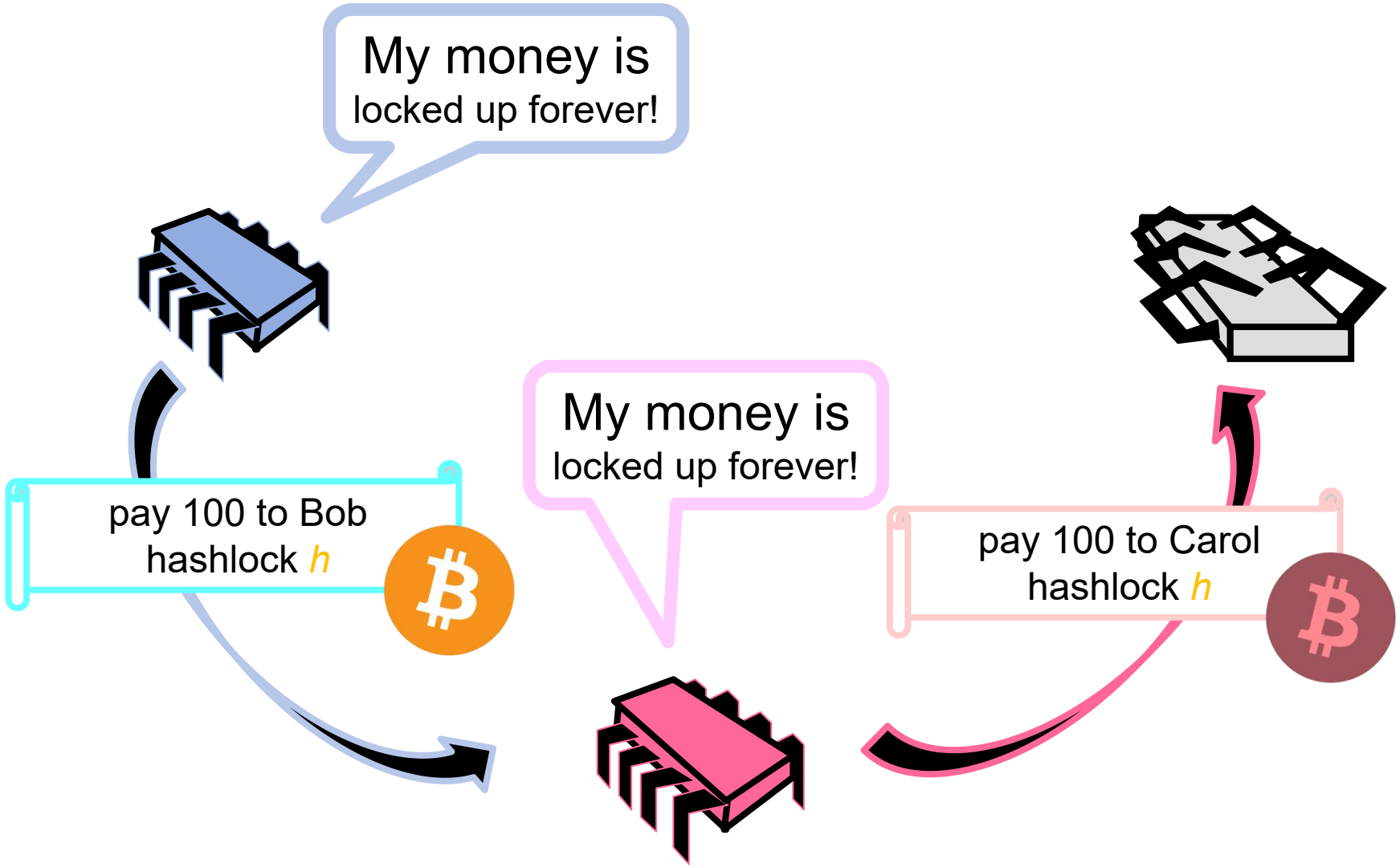
# Settlement Complete



# Oh, Wait ...



# Oh, Wait ...


My money is
locked up forever!
pay 100 to Bob
hashlock h
My money is
locked up forever!
pay 100 to Carol
hashlock h

# Timelock




**Contract:**
**If secret not provided before time *t*, will refund to source!**

# Synchrony

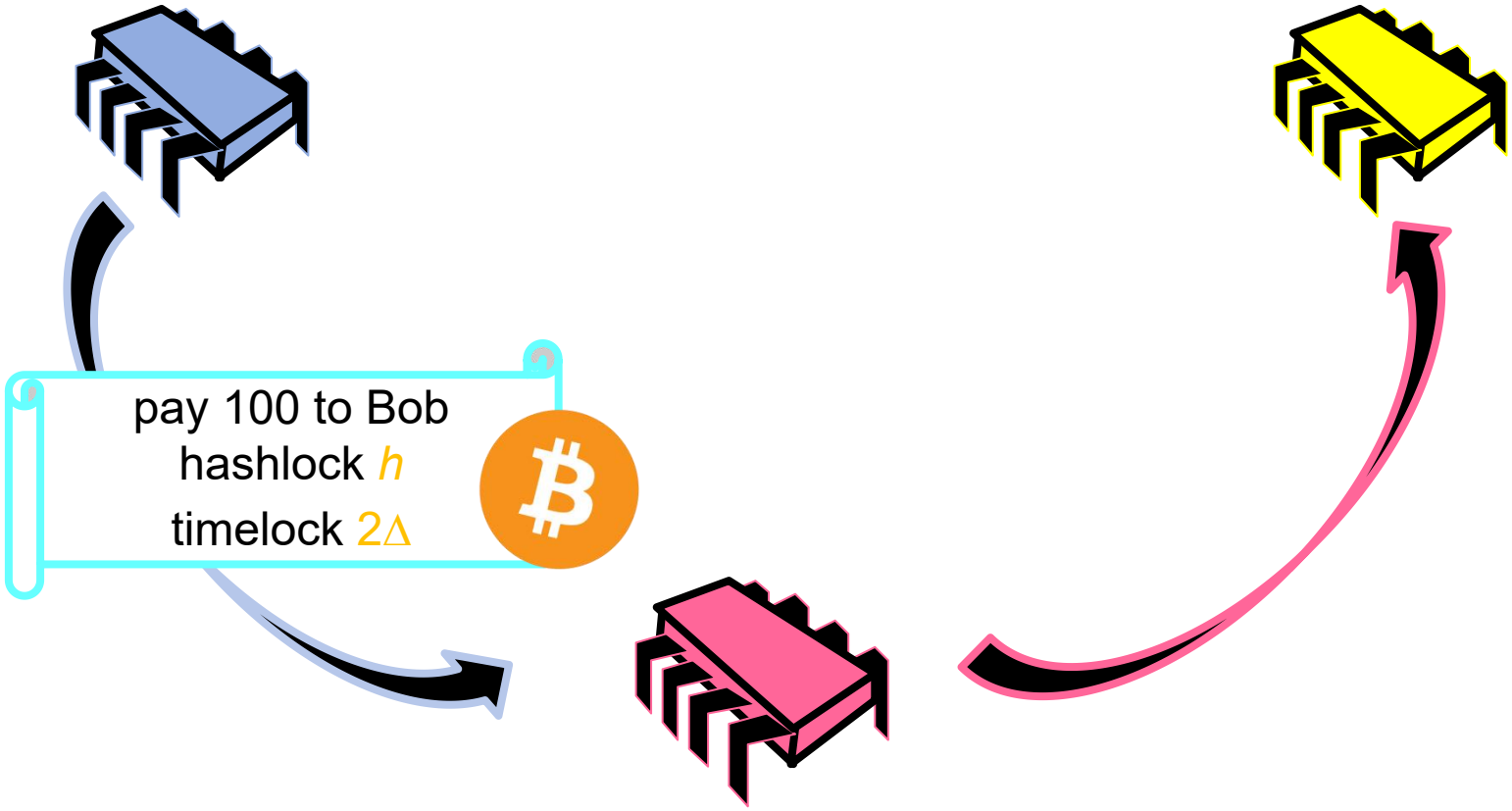
Let $\Delta$ be enough time to finalize a transaction on the blockchain

Can be large cost because only matters when things go wrong

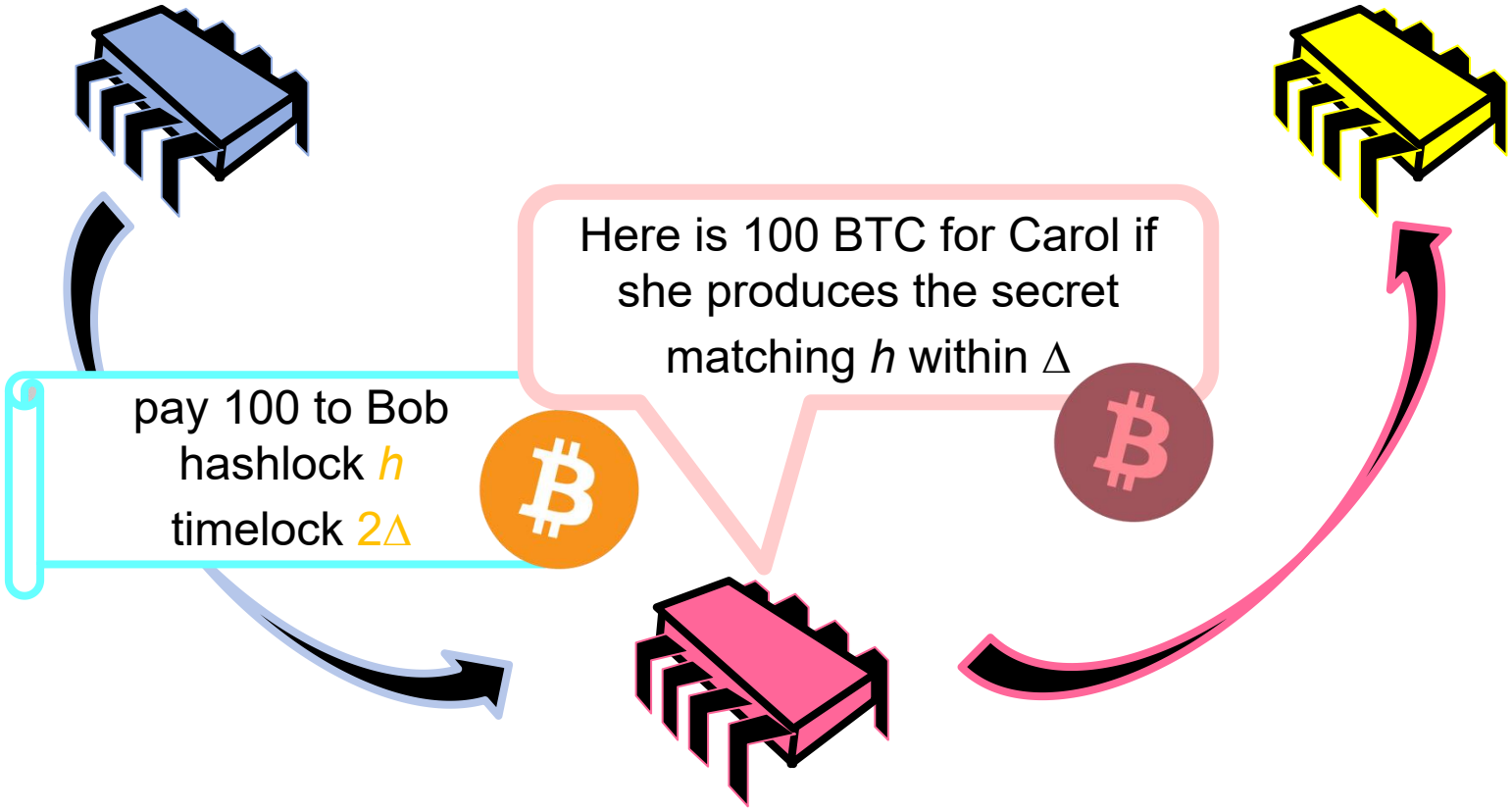
# Settlement


Here is 100 BTC for Bob if he produces the secret matching $h$ within $2\Delta$

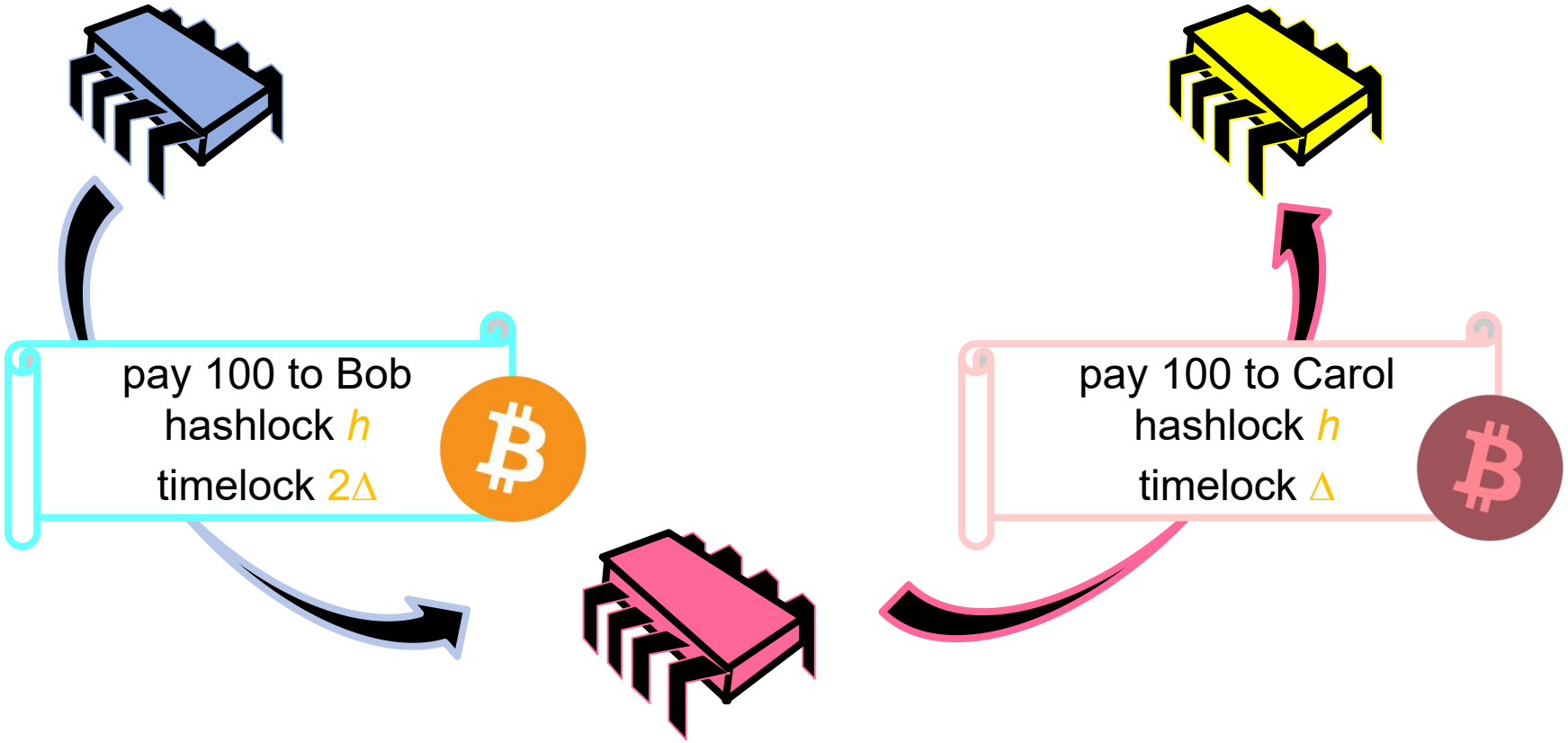
# Settlement



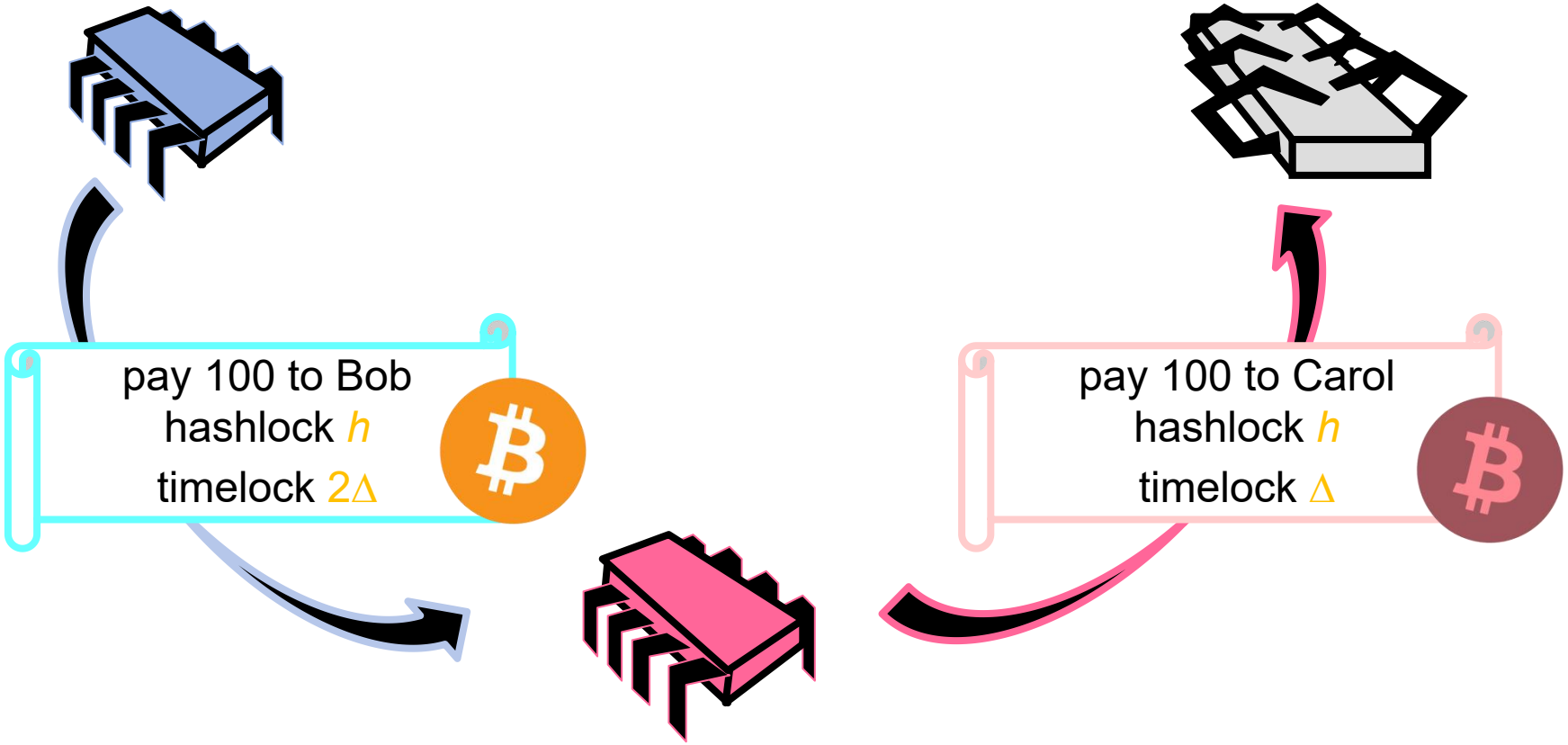
# Settlement


pay 100 to Bob
hashlock h
timelock 2Δ
Here is 100 BTC for Carol if she produces the secret matching h within Δ

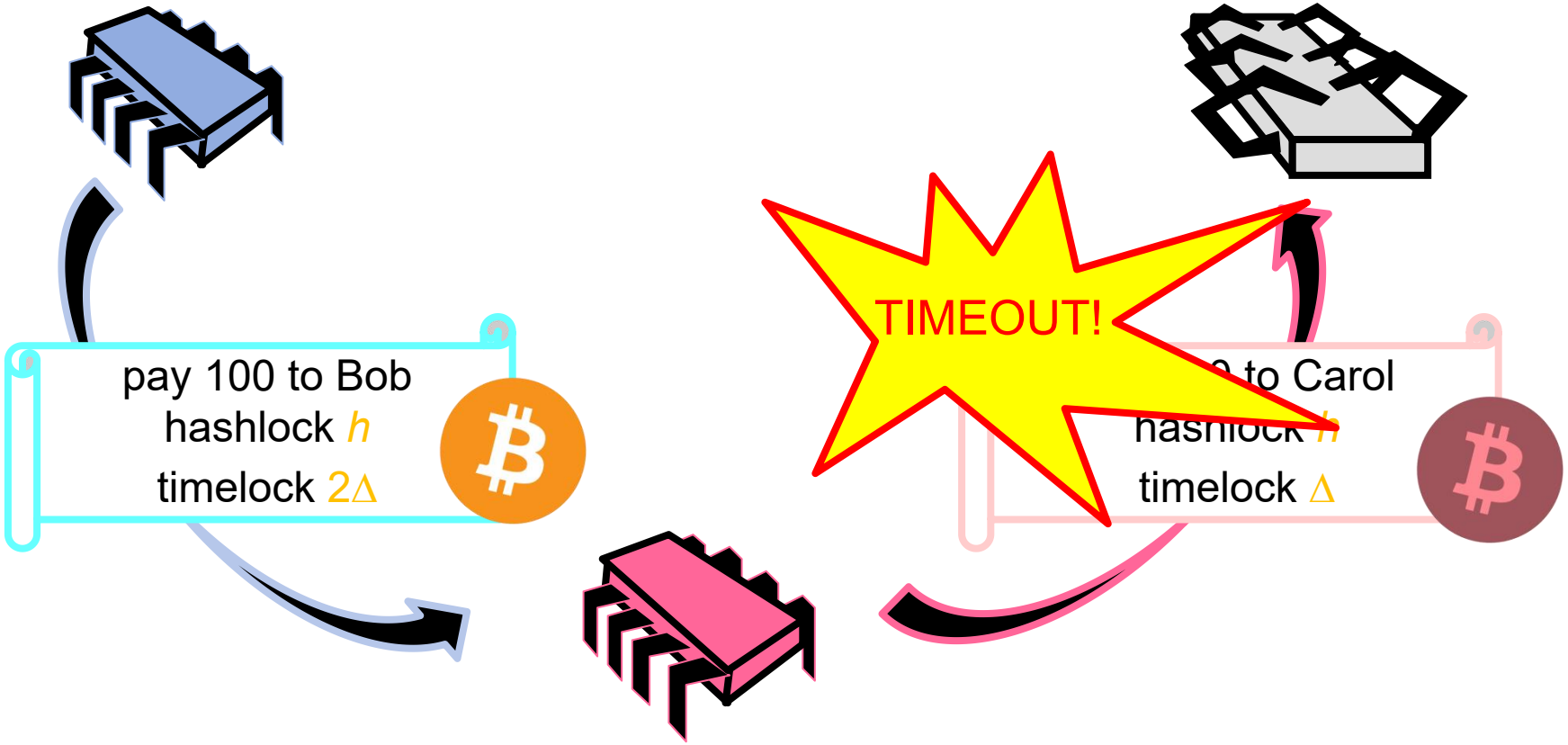
# Settlement


pay 100 to Bob
hashlock h
timelock 2Δ
pay 100 to Carol
hashlock h
timelock Δ

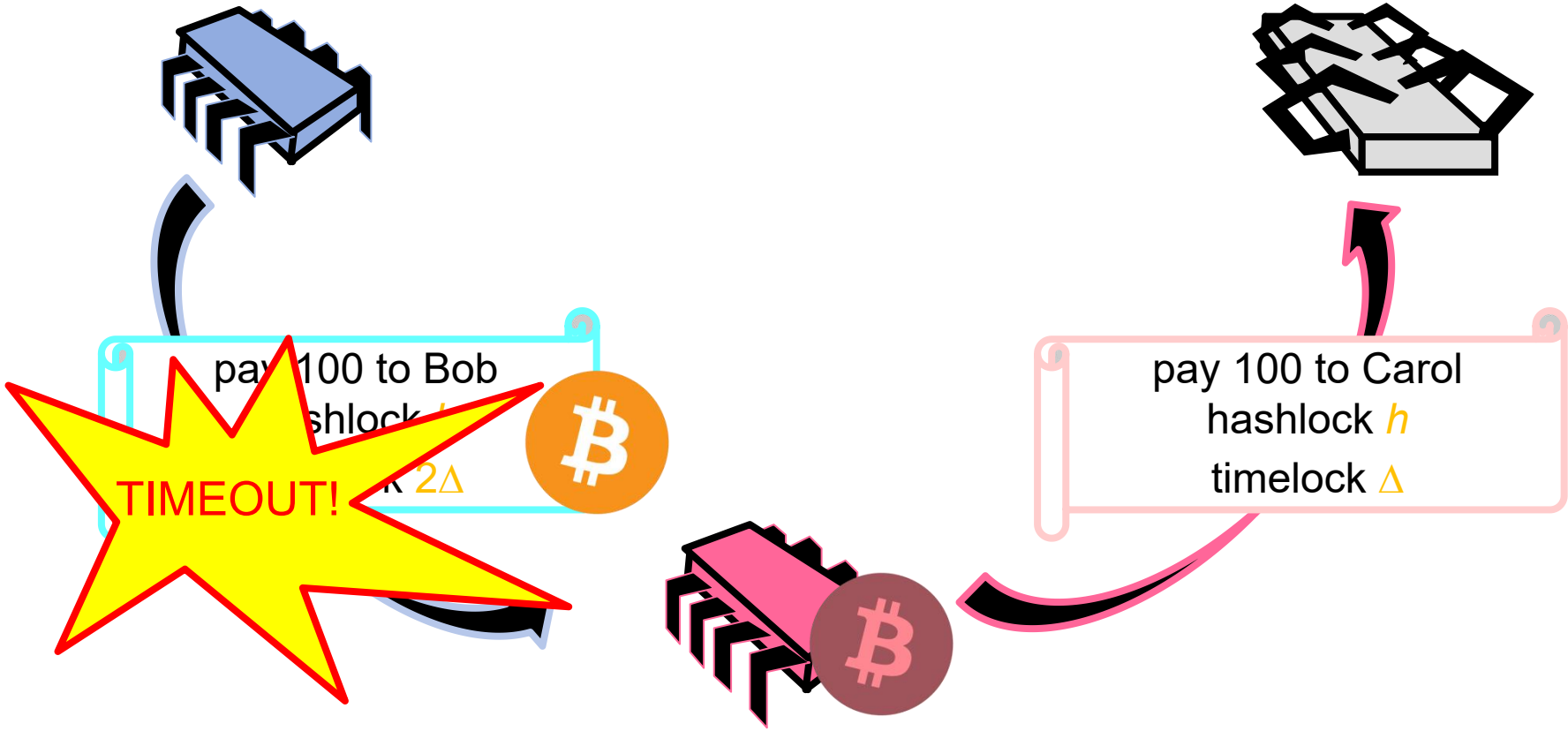
# Settlement


pay 100 to Bob
hashlock h
timelock 2Δ
pay 100 to Carol
hashlock h
timelock Δ

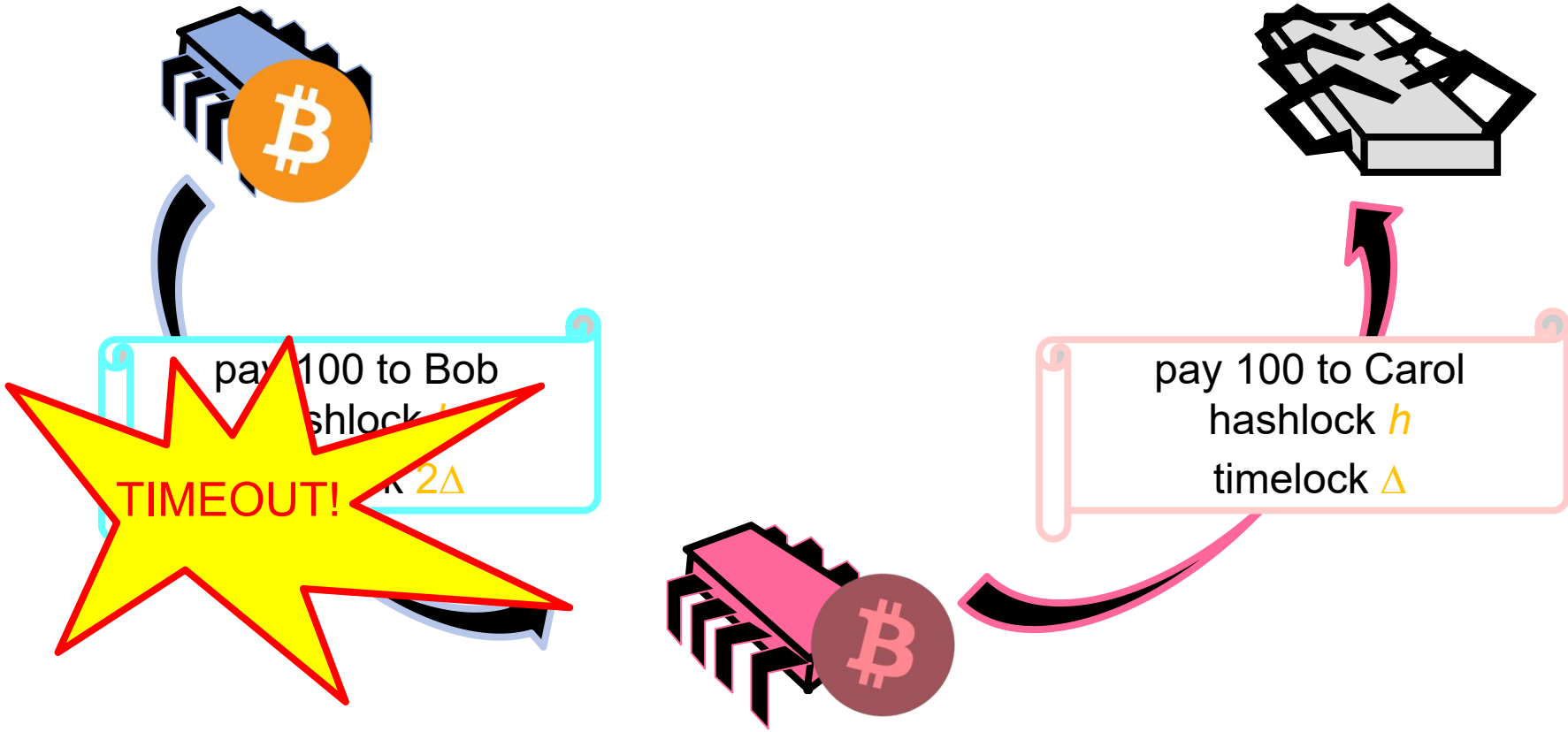
# Settlement


pay 100 to Bob
hashlock h
timelock 2Δ
TIMEOUT!
to Carol
hashlock h
timelock Δ

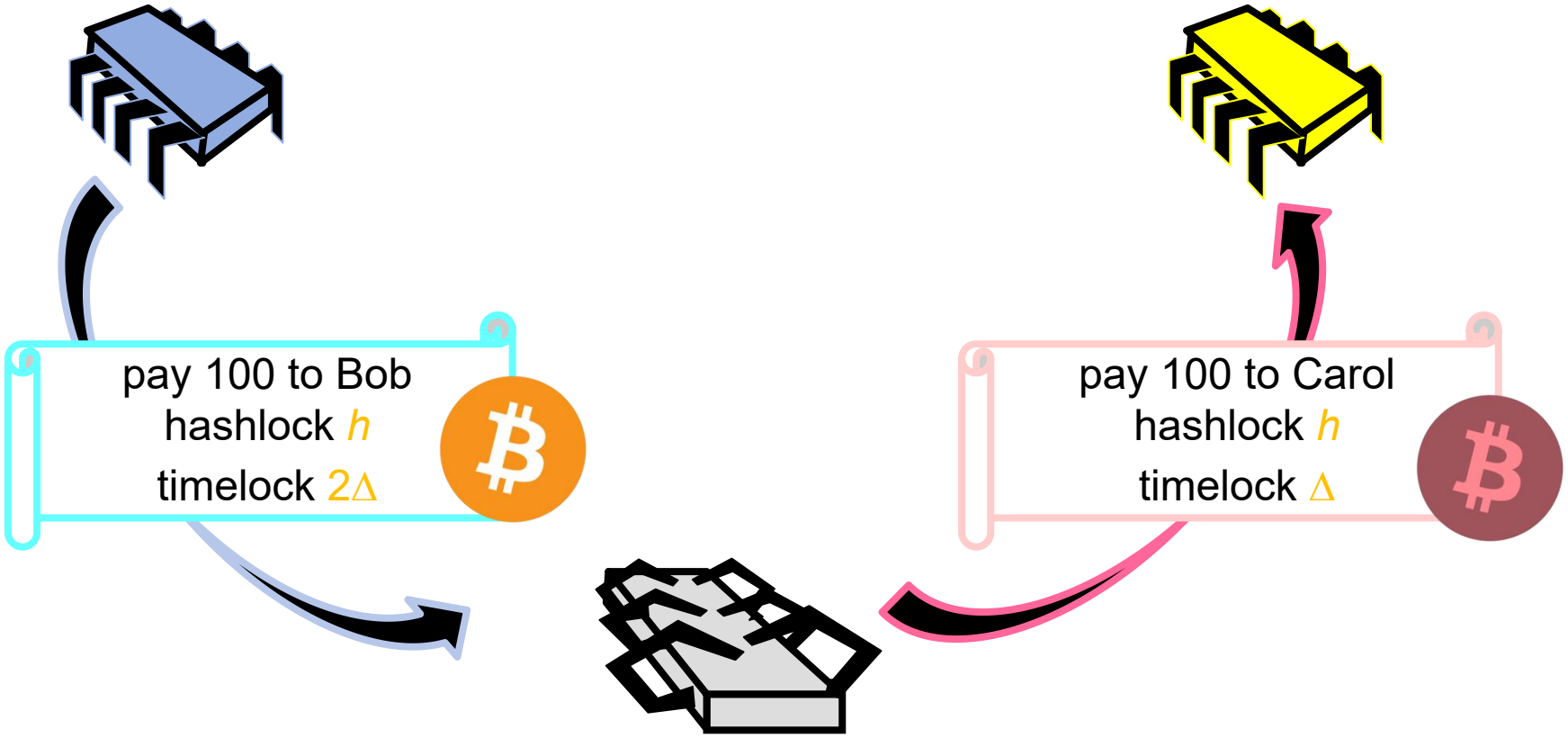
# Settlement



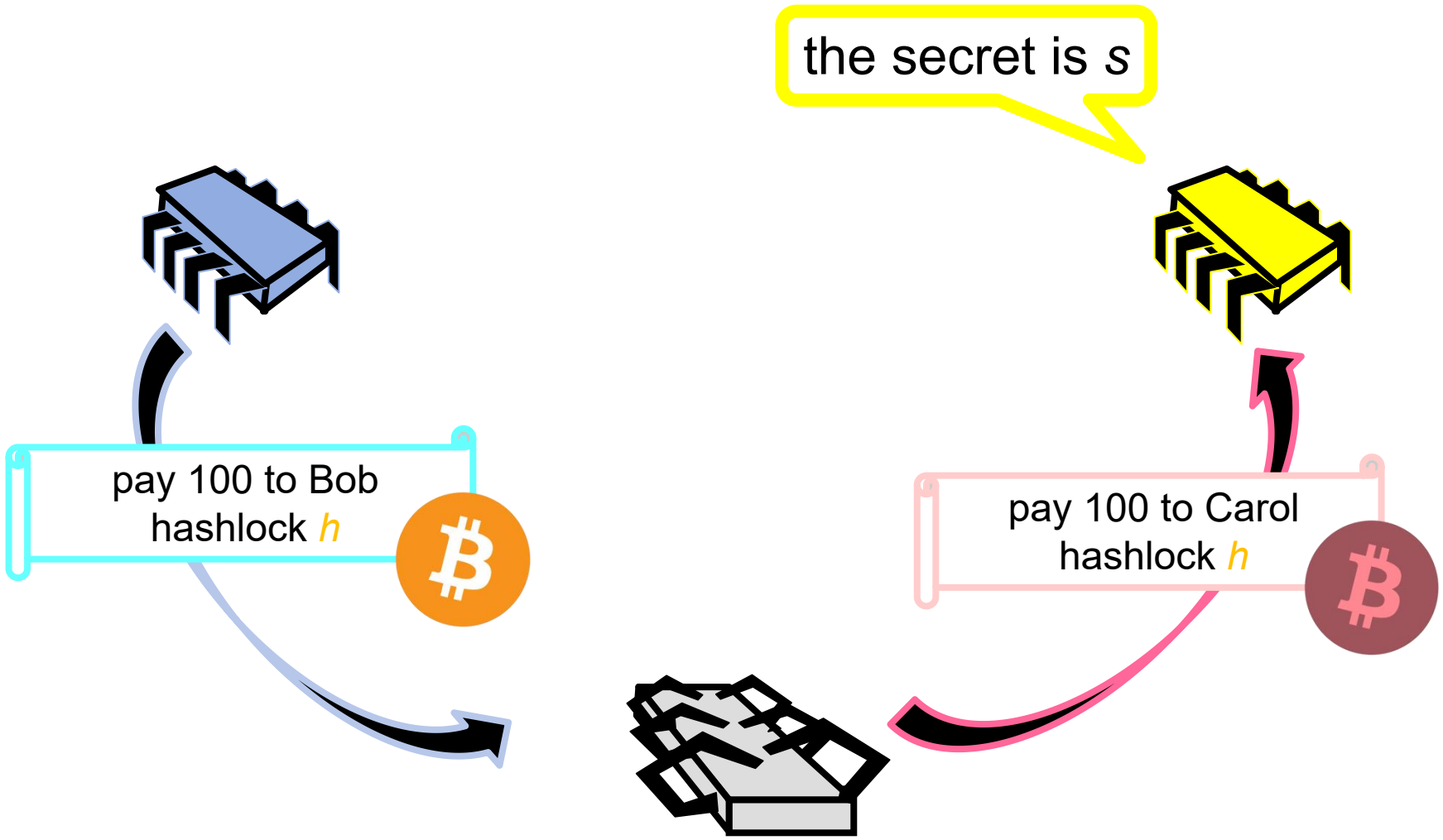
# All Good


pay 100 to Bob
hashlock h
timelock 2Δ
TIMEOUT!
pay 100 to Carol
hashlock h
timelock Δ

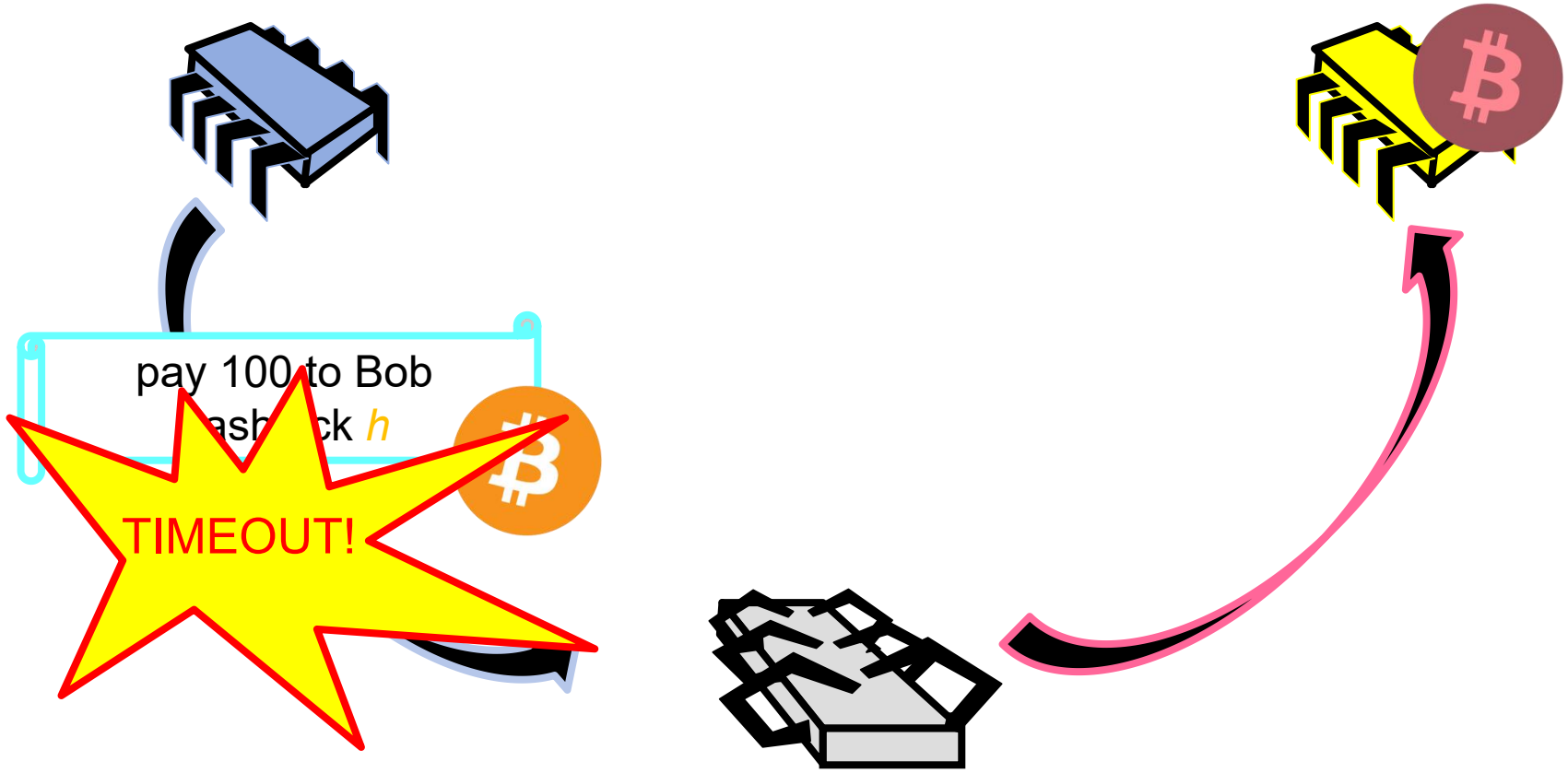
# Settlement


pay 100 to Bob
hashlock h
timelock 2Δ
pay 100 to Carol
hashlock h
timelock Δ

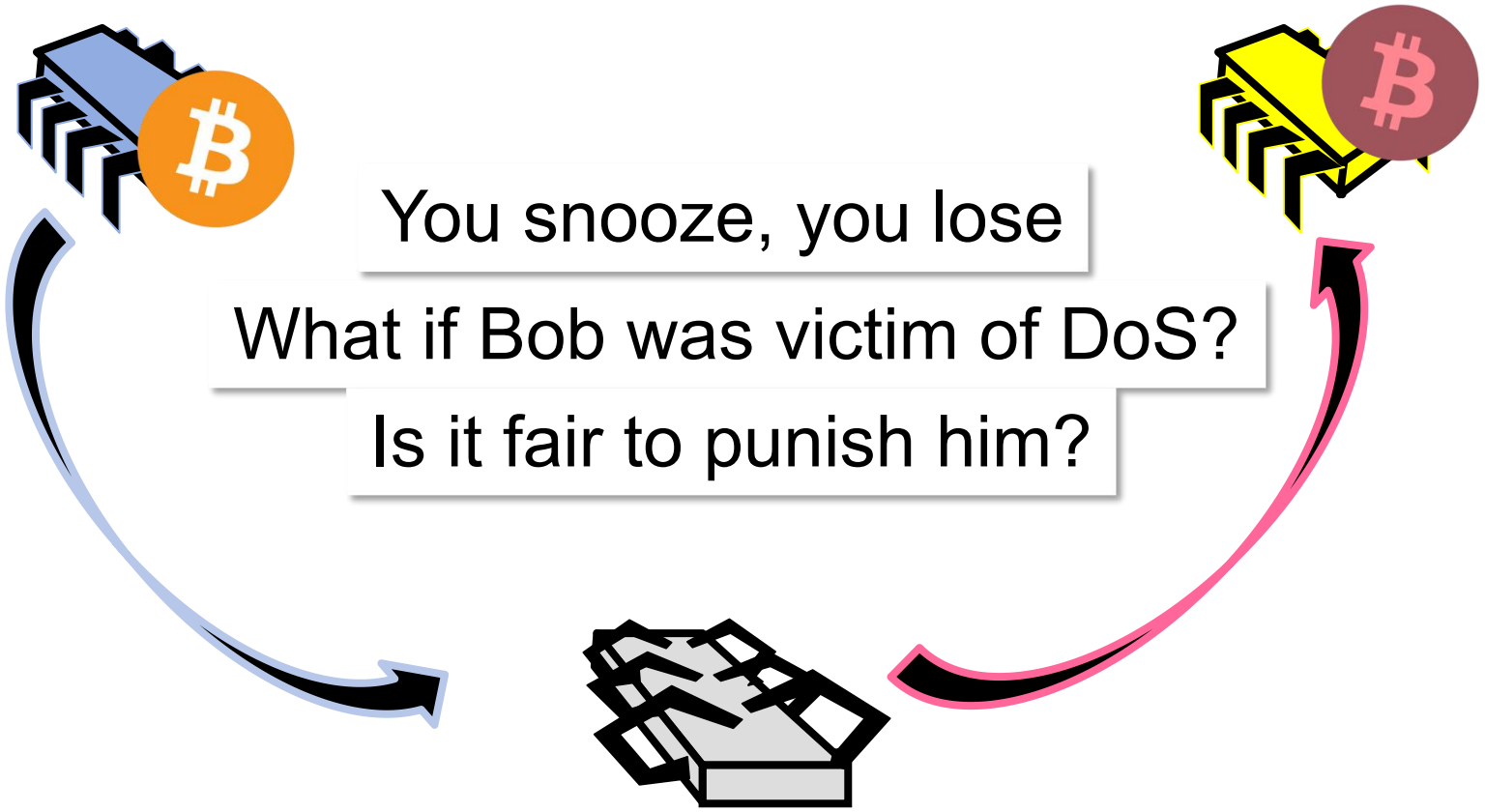
# Settlement


the secret is *s*
pay 100 to Bob
hashlock *h*
pay 100 to Carol
hashlock *h*

# Settlement



# All Good


You snooze, you lose
What if Bob was victim of DoS?
Is it fair to punish him?

# Watchtowers

Participants have to watch the blockchain all the time

When a hashlock is unlocked …

You have to react in time …

Or you lose your stake.

Burdensome



# Watchtowers

A *watchtower* is a service that will watch the blockchain for you …

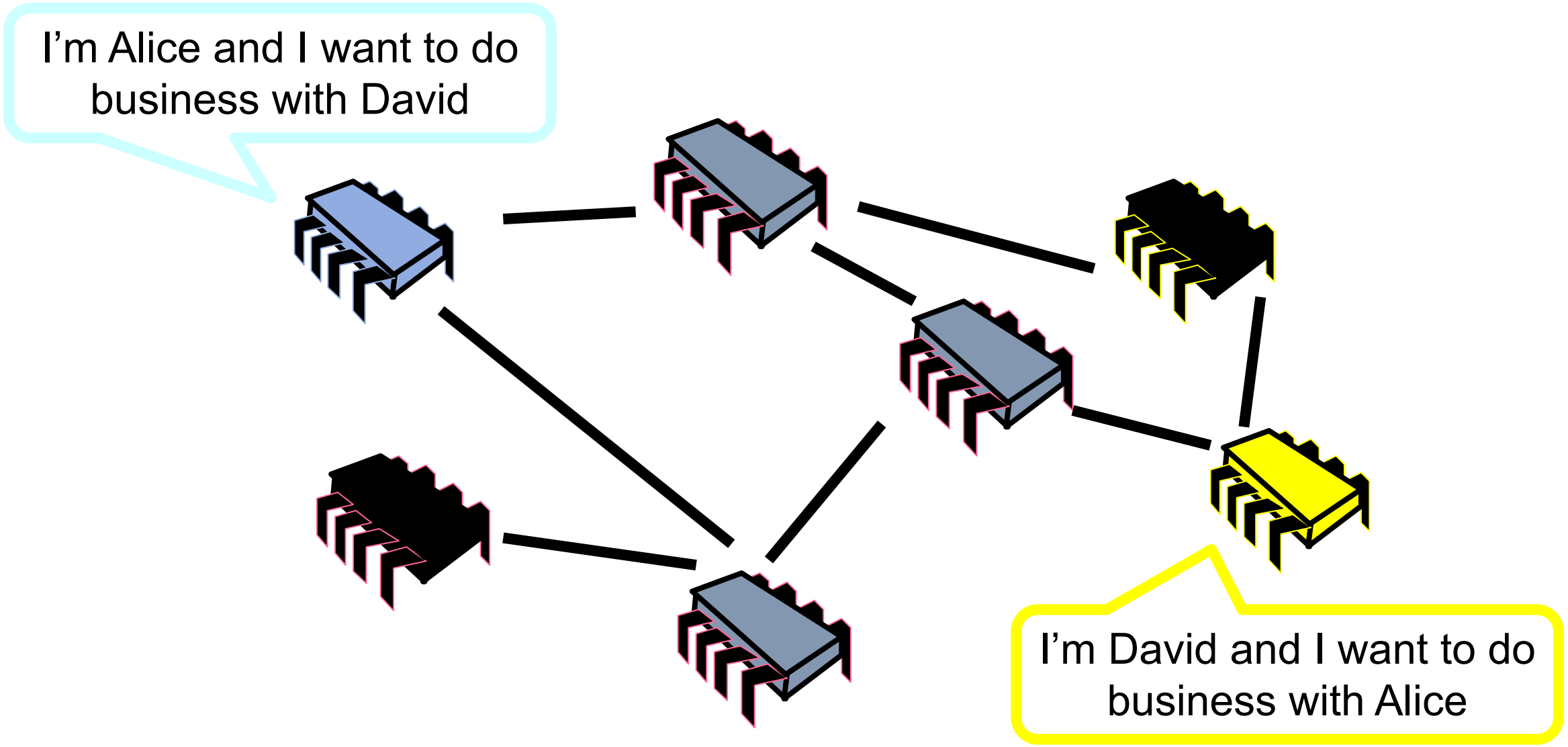
And forward revealed secrets as needed

It does not know your secrets

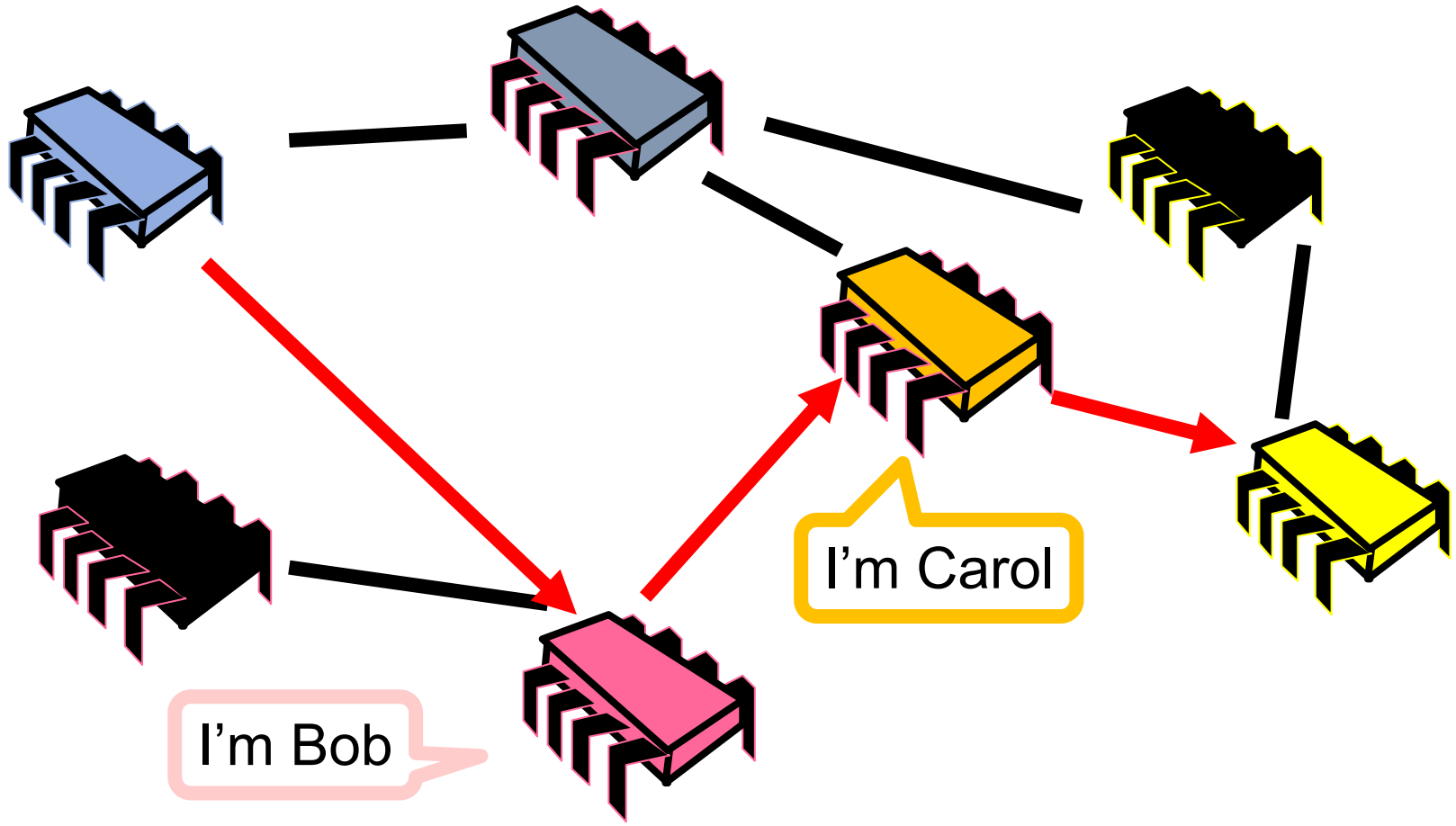
and cannot betray you



# Payment Network



# Payment Network will Find a Way



# BitCoin Lightning Network

Alice sends multiple micro-transactions to David

Settlement:

- Find channel route from Alice to David
- Use generalization of 3-party protocol to settle

Timed Hashlocked contracts

# Lightning Network: The Hope of BitCoin?

- Lightning network is arguably Bitcoin's only hope …
- to become an actual currency
- instead of a speculative instrument
- latency and TPS don't mater if blockchain is settlement-only

# El Salvador make BitCoin Legal Tender

THE DIGEST: NO. 7, JULY 2022 | Subscribe

# El Salvador's Experiment with Bitcoin as Legal Tender

In September 2021, El Salvador became the first country to make bitcoin legal tender, requiring all businesses to accept the cryptocurrency. In an attempt to popularize and regularize its use, the government gave citizens financial incentives to download a special cryptocurrency app.

# Lightning Network in El Salvador



**HOW BITCOIN BROUGHT THE LIGHTNING NETWORK TO EL SALVADOR**

https://cloudtweaks.com/2021/07/how-bitcoin-brought-lightning-network-el-salvador/

# Lightning Network in El Salvador





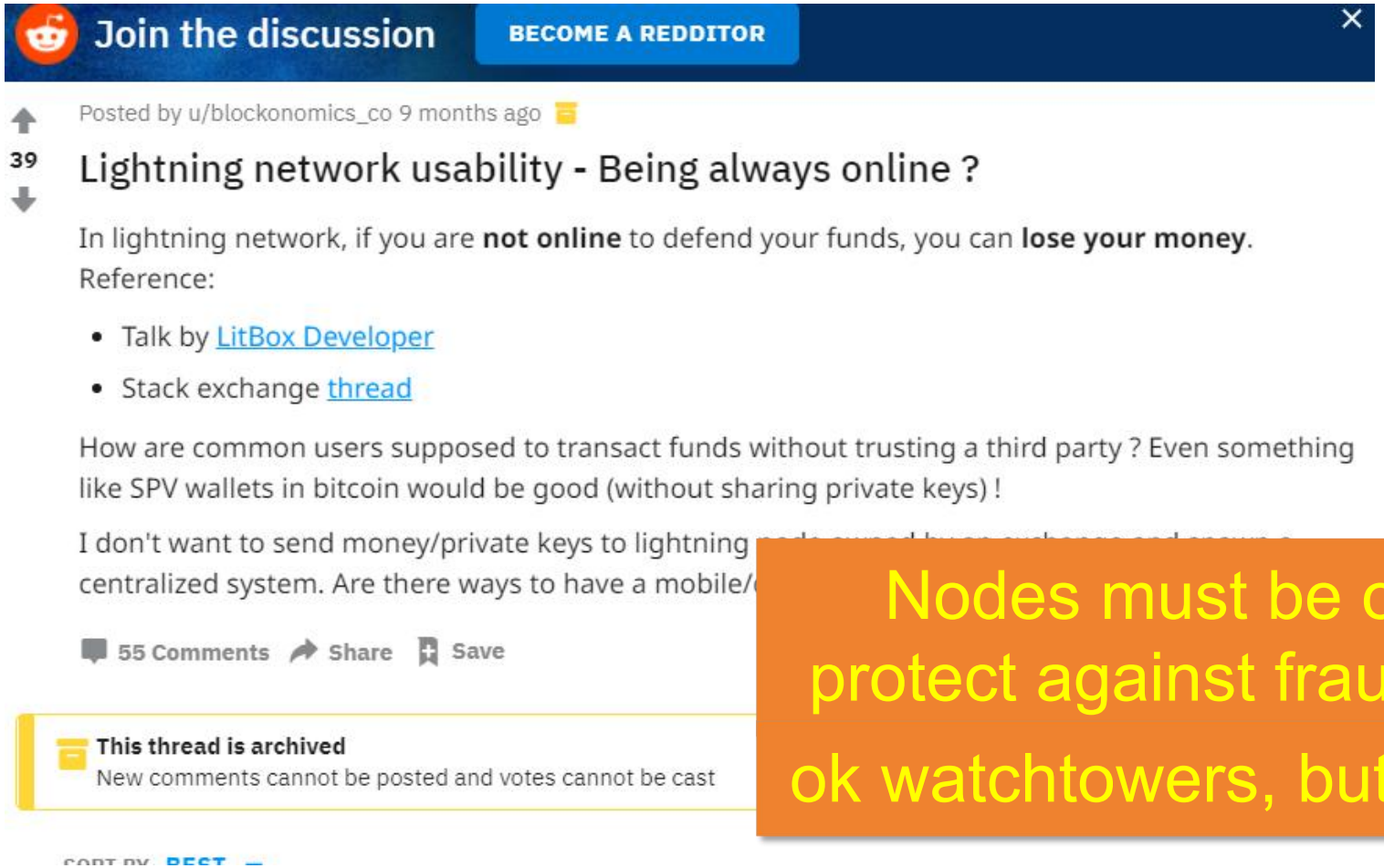

PRASHANT JHA

OCT 12, 2022

## Tap-to-pay Bitcoin Lightning Bolt cards strike El Salvador

CoinCorner, a Bitcoin Lightning-based card payment service provider, is looking to install NFC card readers across El Salvador to offer a seamless retail BTC experience.

# Concerns: Eternal Vigilance

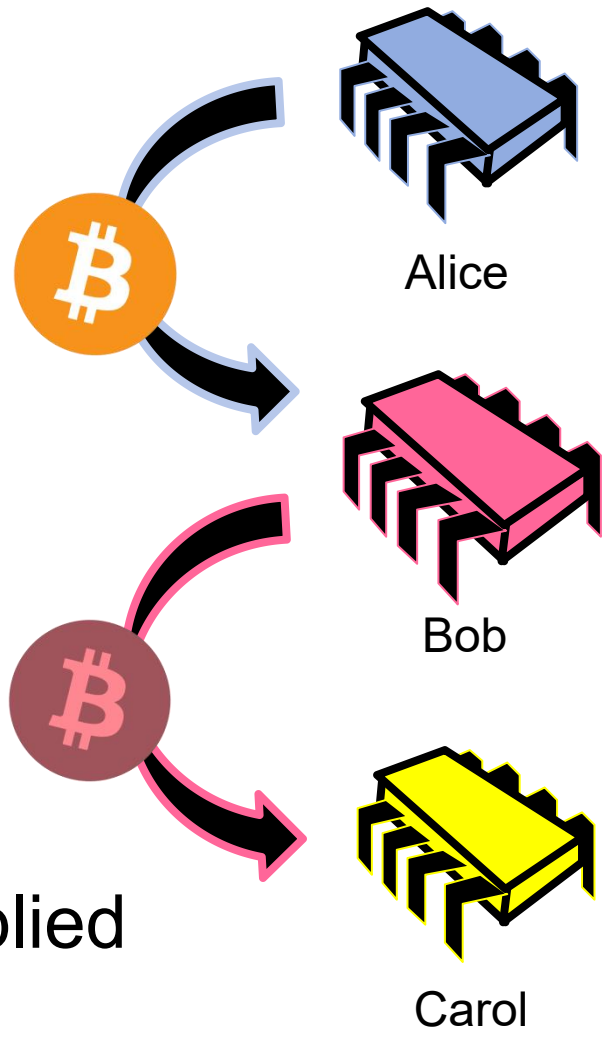



Nodes must be on-line *all the time* to protect against fraudulent channel closing

ok watchtowers, but who really wants that?

# Concerns: Economic Model

- Everyone is lending to everyone else
- Alice cannot give 1BTC to Bob to pass along
- Alice is borrowing from Bob to pay Carol
- Every hop must have the funds available
- More hops used, more lending burden is multiplied



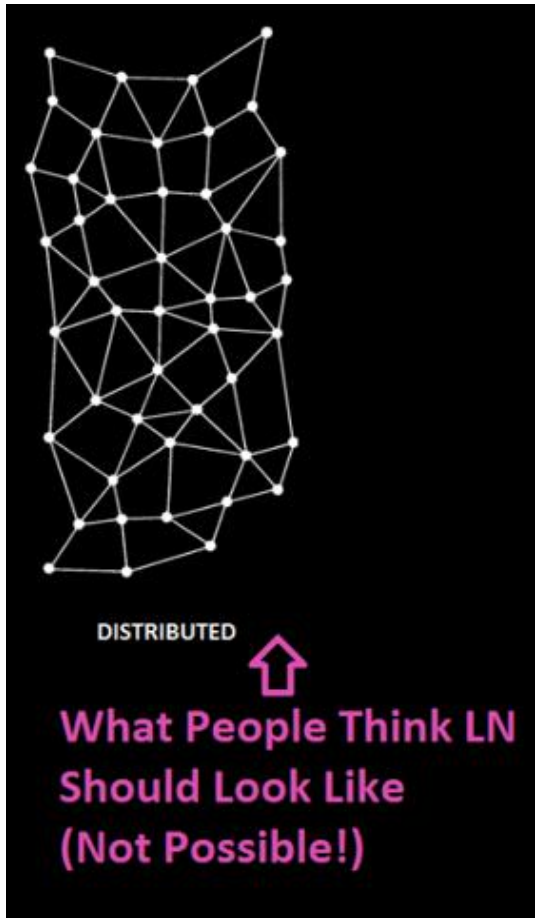
# Concerns: Economic Model

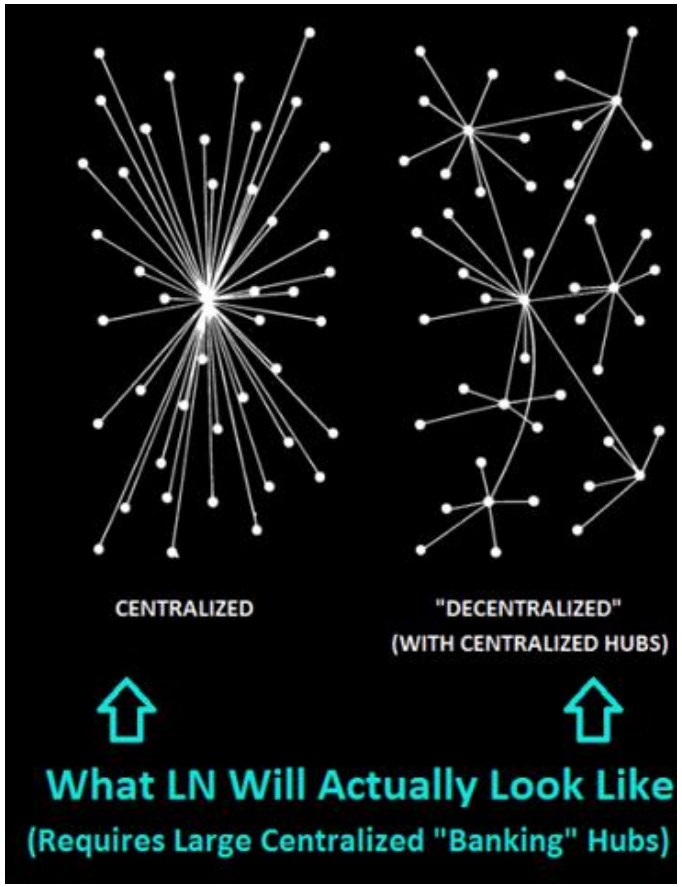
- Makes sense only when endpoints have frequent exchanges
- Must have O($N \times k$) coins tied up in channels
- Counterparty misbehavior can lock coins for a long time
- Could network fees become as bad as blockchain fees?

# Concern: Centralization

- Routing is hard
- You can’t just “link in”, you need a payment channel
- You have to have capital to run a channel
- Channel capacities degrade, must be refreshed
- longer the route, larger chance of delay
- Most natural structure is centralized star

# Concern: Centralization



# Ways To Scale Blockchain

The way is a bit like scaling traditional databases

List of the ways:

1. **Payment Channel**
2. **Layer 2**

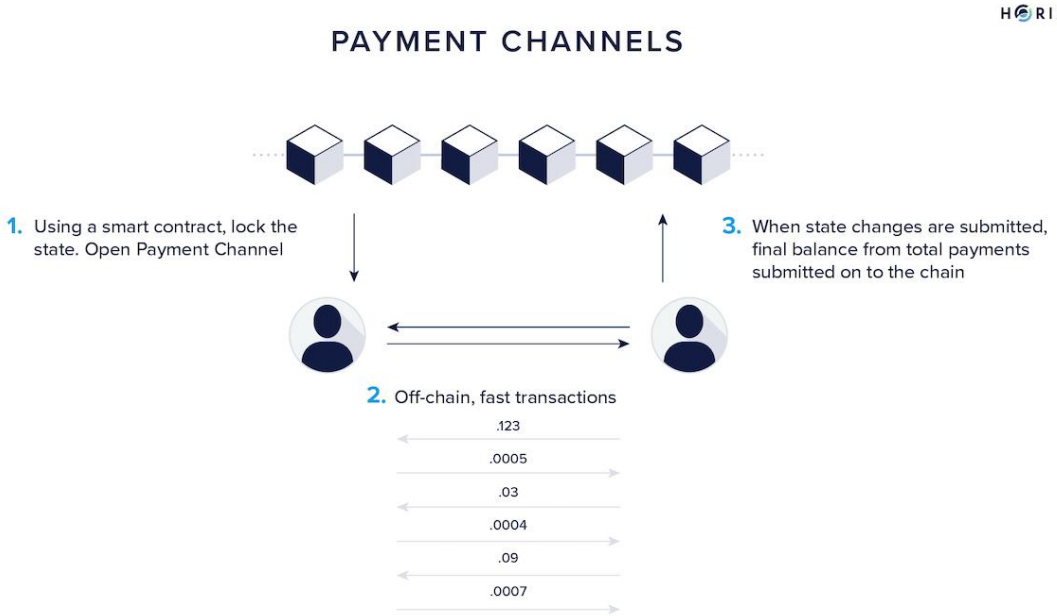
# History of Layer 2

## 1. State channels (2015)

Example: Lightning network on Bitcoin

- Open a 1:1 payment channel between A and B
  - A pre-funds ( “locks up” ) say 1 BTC on L1
  - A signs off-chain "I-promise-to-pay" messages and sends directly to B: "0.1", and then "0.2" etc.
  - To settle the payment, either A or B can close the channel, and L1 ensures they get their dues

- It is a powerful technique, but downsides:
  - Complex to generalize to arbitrary computation
  - Participants (or delegates) have to be online, even if just receiving
  - Unable to run smart contract

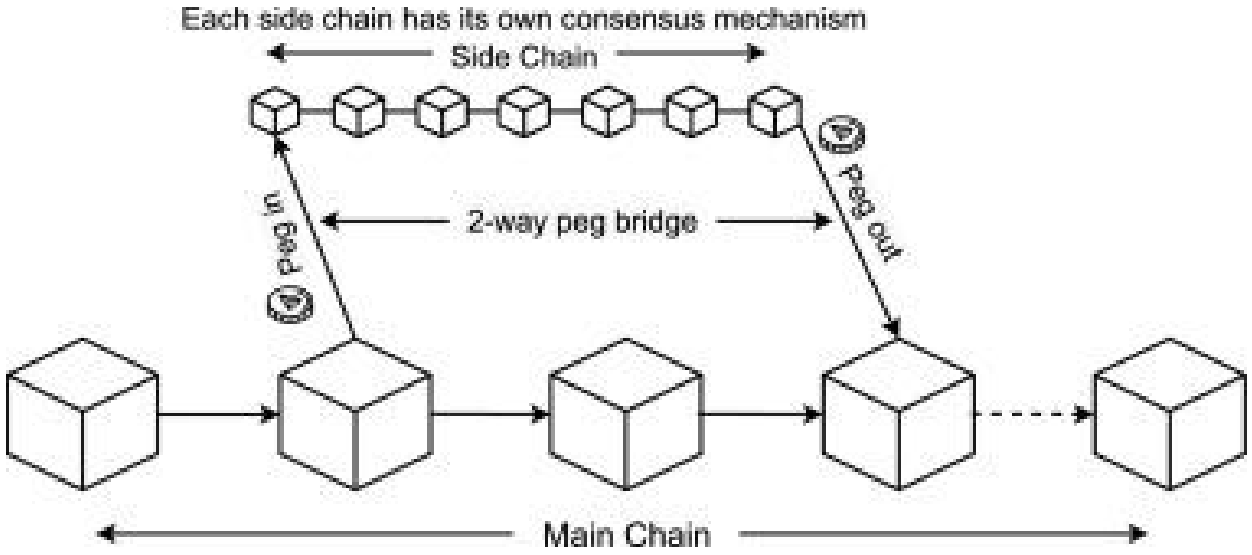


# History of Layer 2

## 2. SideChains

Example: Polygon

- "Pure" off-chain scaling protocols
- Separate blockchains independent of L1
- Connected to L1 by a 2-way bridge
- Easy dApps portability if sidechain uses same VM as L1
- Have own consensus and security model (Do not derive security from L1)



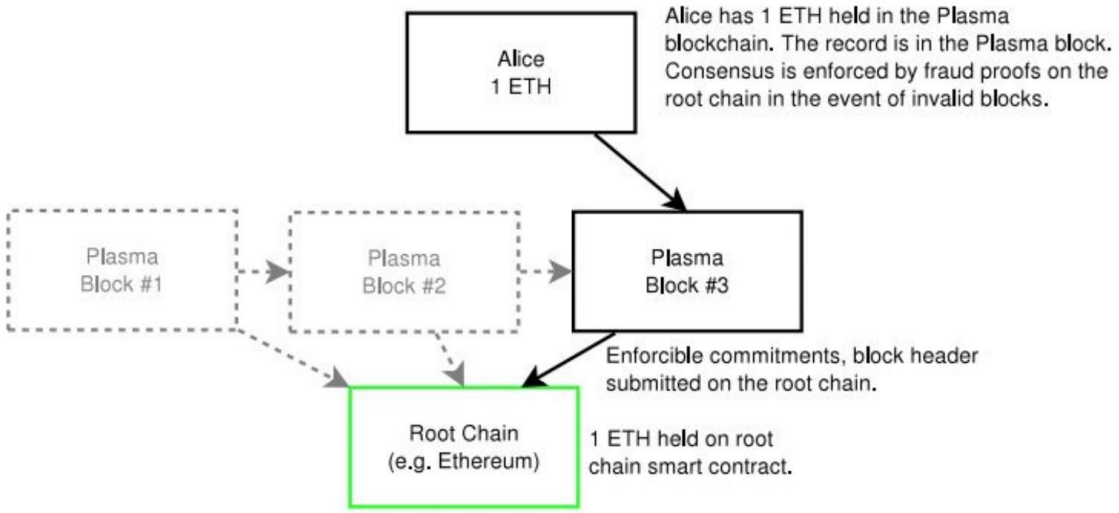
# History of Layer 2

## 3. Plasma, Commit-chains (2017-19)

- Computation + state is in a different chain
  - aka child (L2) chain or plasma chain

- A **smart contract** on root (L1) chain manages the L2 chain

- "Commitments" to L2 chain state (Plasma block header hashes or Merkle roots of balances) are published to L1 periodically




https://eprint.iacr.org/2018/642
https://plasma.io/

# History of Layer 2

## 3. Plasma, Commit-chains (2017-19)

- If there is proof of fraud submitted on L1, then the block is rolled back, and the block creator is penalized
- Any user can deposit into or withdraw from the L2 chain
- On operator misbehavior, a user can do a "forced exit" via the smart contract
- Problem: Child chain' s operator can be untrusted

# History of Layer 2

## 3. Plasma, Commit-chains (2017-19)

**Problem A: Block Withholding Problem**

- What if no L2 block is produced by the operator(s)?

- Forced Exit: A user can completely exit the Plasma chain and withdraw their assets by submitting a Merkle proof of their ownership of assets

# History of Layer 2

## 3. Plasma, Commit-chains (2017-19)

**Problem B: Data Availability (DA) Problem**

- Only commitments are available on L1 **(data off-chain)**

- Is full data to produce those hashes available somewhere for any user to be able to replay all data in order to verify or challenge the commitment?

# History of Layer 2

## 3. Plasma, Commit-chains (2017-19)

**Weakness of Plasma**

- Assets need to have owners

- Complicated to generalize to full EVM

- Fundamental game theory issues around data availability

# History of Layer 2

## 4. Rollups (Current Solutions)

- Hybrid L2 scaling mechanism
- Computation is still off-chain in L2, like Plasma
- But (compressed) per-transaction data is kept on L1
  **(data on-chain, the primary difference from Plasma)**
  - This eliminates the data availability problem
  - Leverage L1' s consensus and security
  - Anyone can locally process all rollup operations and thus withdraw / verify / challenge L2's computation by using L1's data.

| Computation \ Data | On-chain | Off-chain |
|---|---|---|
| **On-chain** | ETH | N/A |
| **Off-chain** | Rollups | Sidechains, State Channels, Plasma |

Block unicorn

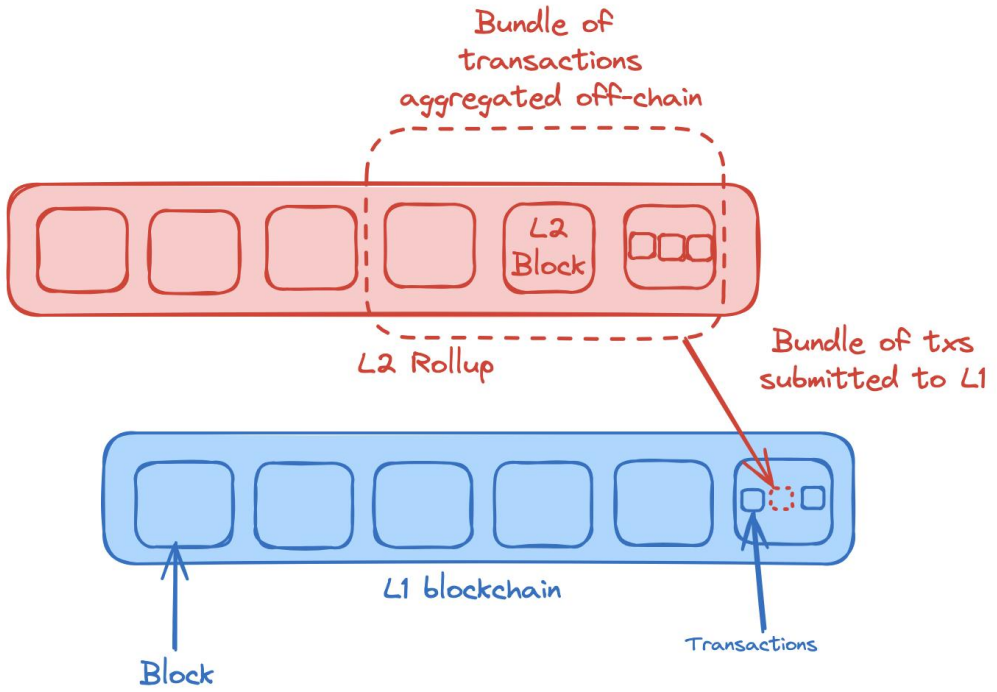
# History of Layer 2

## 4. Rollups (Current Solutions)

- One transaction on L1 can represent numerous transactions within a L2 rollup. However, scaling is limited by L1's data bandwidth

- Fully general EVM computations can be done on L2. In fact, many L2 rollups run EVMs

- Dapps that are running on L1 (Ethereum) can now be easily ported to run on L2s



# Rollups

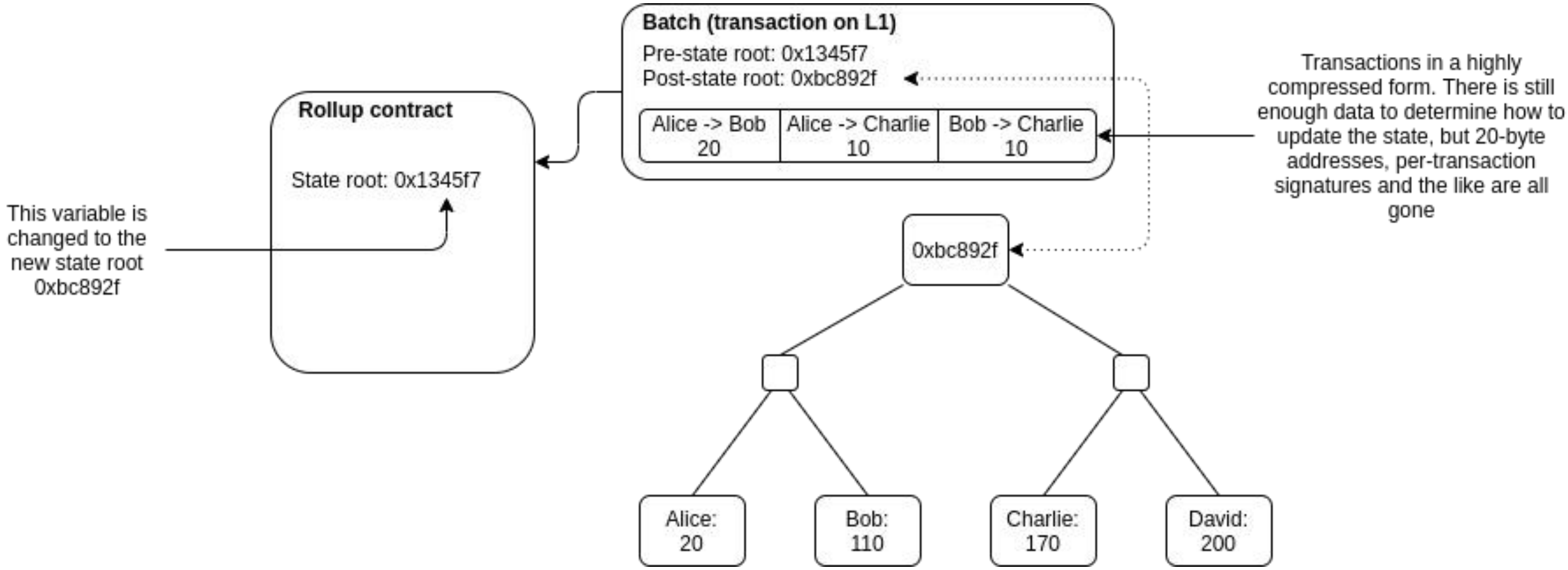
## Architecture of a Rollup

- (Like in Plasma) Smart contracts on L1 that manage the L2, and typically allow:
  - Deposit, Withdrawal
  - Publish "assertions" of new state roots
  - In addition, ability to publish a batch of compressed transaction data, just sufficient to allow anyone to verify correctness

- An operator (aka validator / aggregator / sequencer) on L2 which batches transactions

# Rollups

## Architecture of a Rollup




https://vitalik.eth.limo/general/2021/01/05/rollup.html

# Rollups

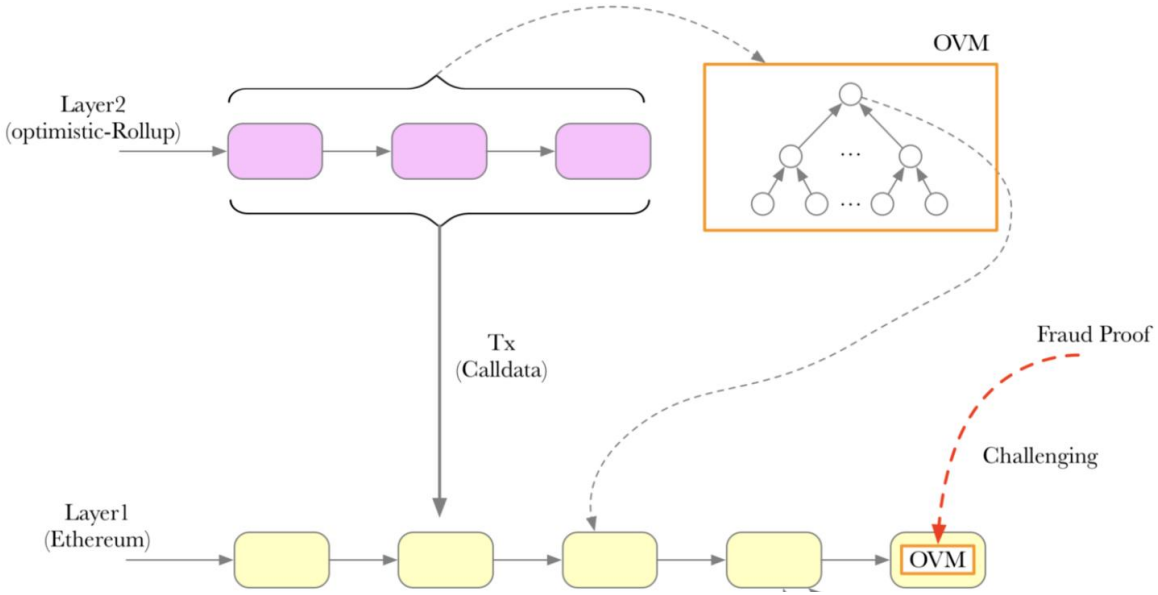
**Problem: How to guarantee batch information is correct?**

2 Solutions to solve this problem:

1. Optimistic Rollups
2. Zero-Knowledge Rollups (ZK Rollups)

# Optimistic Rollup

- Example: Arbitrum, Optimism




https://www.nervos.org/knowledge-base/zk_rollup_vs_optimistic_rollup

- Publish compressed transaction data as "calldata" (before EIP-4844) or "blob" (after EIP-4844)
  - This incurs a "fixed cost" per batch (such as 21,000 gas)
- A challenge window of N (~7-14) days for anyone to provide a "**fraud proof**" to rollup' s smart contract
  - If provided and found correct, state roots from then on are invalidated + Publisher’ s deposit is slashed

# Optimistic Rollup

## Two Different Methods of Fraud Proofs:

### (a) Single-round

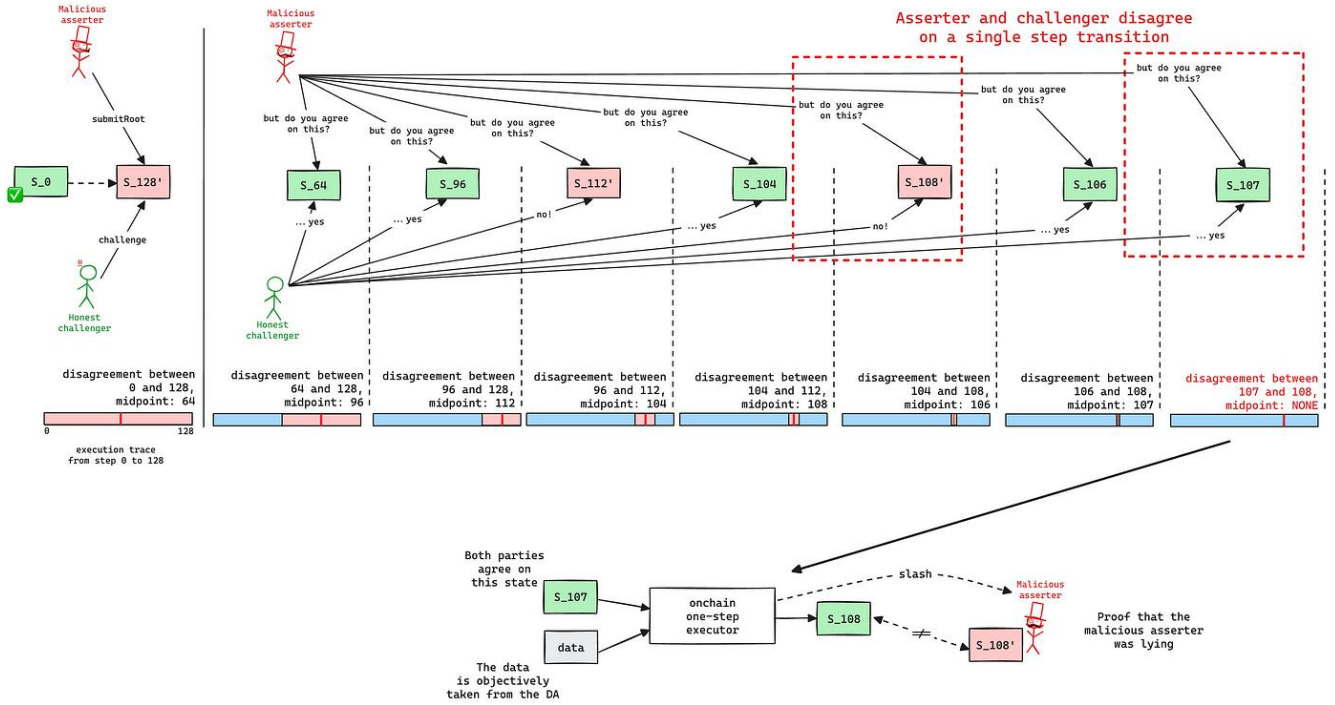
- A verifier L1 contract replays the published transactions on top of latest Merkle root to compute new root
- Expensive

### (b) Multi-round

- To reduce costs
- Interactive protocol between asserter and challenger
- "Bisection protocol" like binary search to zero-in on one transaction of dispute

Single-round

Multi-round




https://medium.com/l2beat/fraud-proof-wars-b0cb4d0f452a

# Optimistic Rollup

## Compression Level in a Batch

| Parameter | Ethereum | Rollup |
|---|---|---|
| Nonce | ~3 | 0 |
| Gasprice | ~8 | 0-0.5 |
| Gas | 3 | 0-0.5 |
| To | 21 | 4 |
| Value | ~9 | ~3 |
| Signature | ~68 (2 + 33 + 33) | ~0.5 |
| From | 0 (recovered from sig) | 4 |
| Total | ~112 Bytes | ~12Bytes |

Compression level: ~10x
Rollup itself scalability improvement: ~10x
=>
Scaling factor of rollup with compression tricks compared to L1: **>100x**

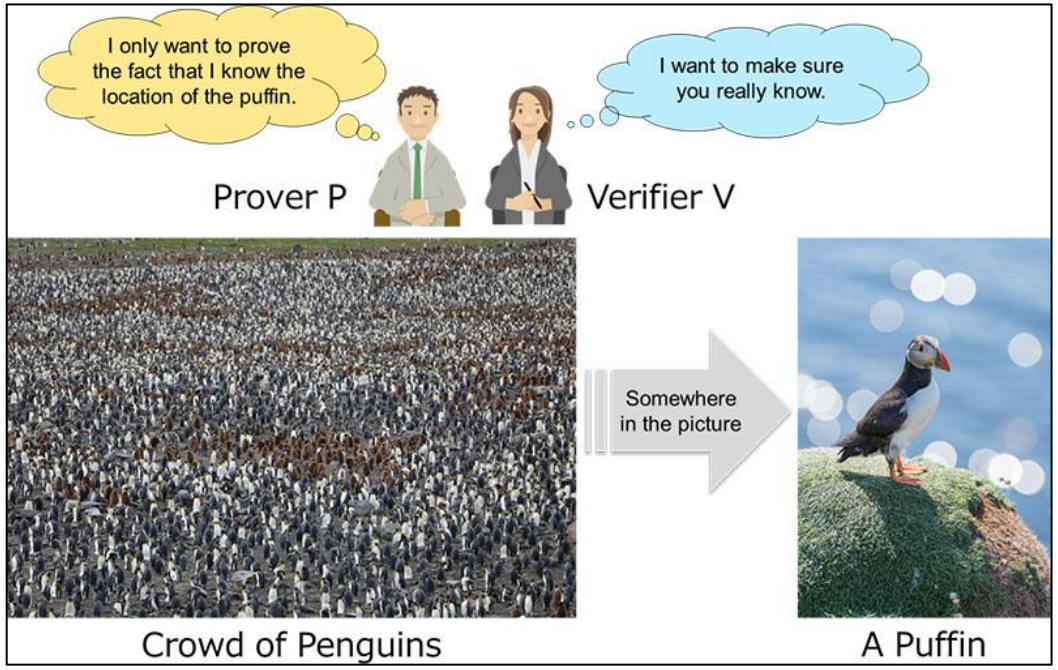
Can further use compression algorithms like Zlib (Optimism) or Brotli (Arbitrum)

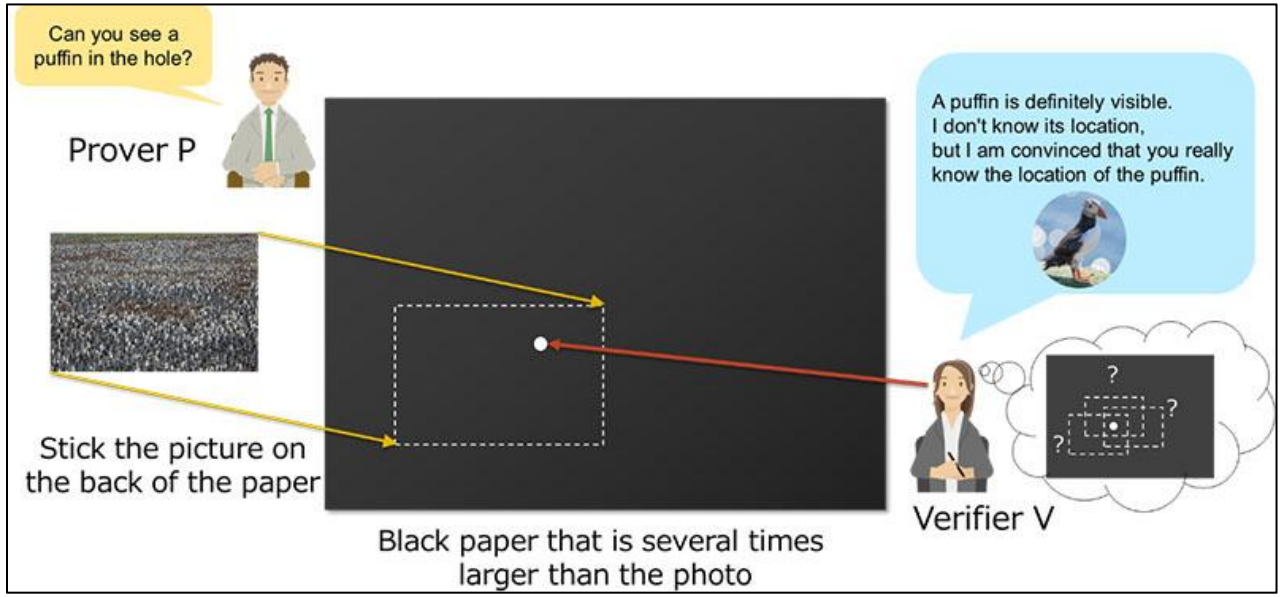
https://vitalik.eth.limo/general/2021/01/05/rollup.html

# Zero-Knowledge Rollup (ZK Rollup)

## Introduction of Zero-Knowledge Proof

Zero-knowledge Proof (ZKP) is a technology in which a person who wants to prove something (Prover) proves the fact that he knows the knowledge **without giving any other knowledge** to the person who wants to verify it (Verifier).

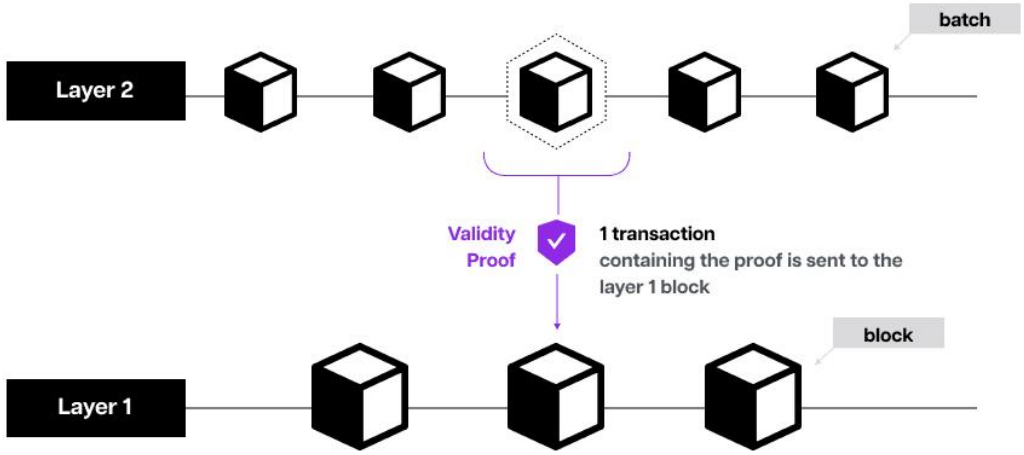



https://www.nttdata.com/global/en/insights/focus/2024/what-is-zero-knowledge-proof

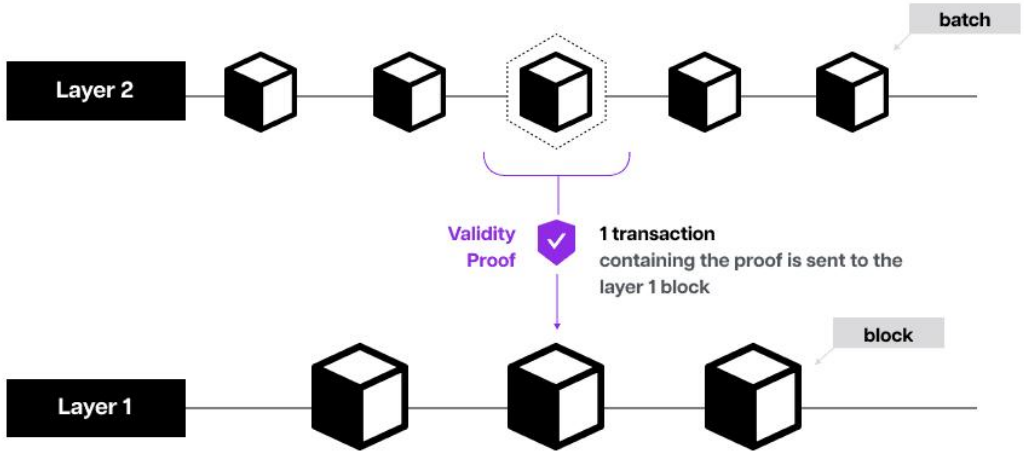
# Zero-Knowledge Rollup (ZK Rollup)

- Examples: Zksync Era, Starknet, Linea
- Prover publishes a "**Validity Proof**" proving that submitted transactions are valid
- Typically via ZK-SNARKS or ZK-STARKS
- Verification of the proof takes much less time than to redo the entire computation in the batch
- Not automatically privacy preserving



# Zero-Knowledge Rollup (ZK Rollup)

- Data availability
  - Changes in balance are often included with proofs
- Instant finality
  - No need to wait for challenge period of 7-14 days
- But proof takes time to construct: O(mins) in every batch
  - Fixed cost is high
- Recursive proofs ( "proofs-of-proofs" ) allow even higher scalability



# Rollup

L2 rollups rely on L1 for:

- Data availability
- Censorship resistance
    - If L2 operator goes offline or misbehaves, user can do a "forced exit" by submitting its own batch on L1 chain
- Dispute resolution (Optimistic) or Validity (ZK)
- Settlement

# Optimistic vs ZK Rollups

| | ZK Rollups | Optimistic Rollups |
|---|---|---|
| Fixed gas cost | ~500K (verification of ZK proof is computationally intensive) | ~40K |
| Finality | Instant (just wait for the next batch) | Challenge period time (~1 week) |
| Technology complexity/immaturity | High (ZK-SNARKs are very new and mathematically complex technology) | Straightforward |
| On-chain per-txn cost | Lower | Higher (extra data need to be published in case it needs to be checked in a fraud proof) |
| Off-chain cost | Higher - creating a ZK proof is expensive. A block of 1K txns could take O(10-20 minutes) | Lower |
| General-purpose? | Harder (ZK-SNARK proving general-purpose EVM execution is much harder than proving simple computations) | Easier |

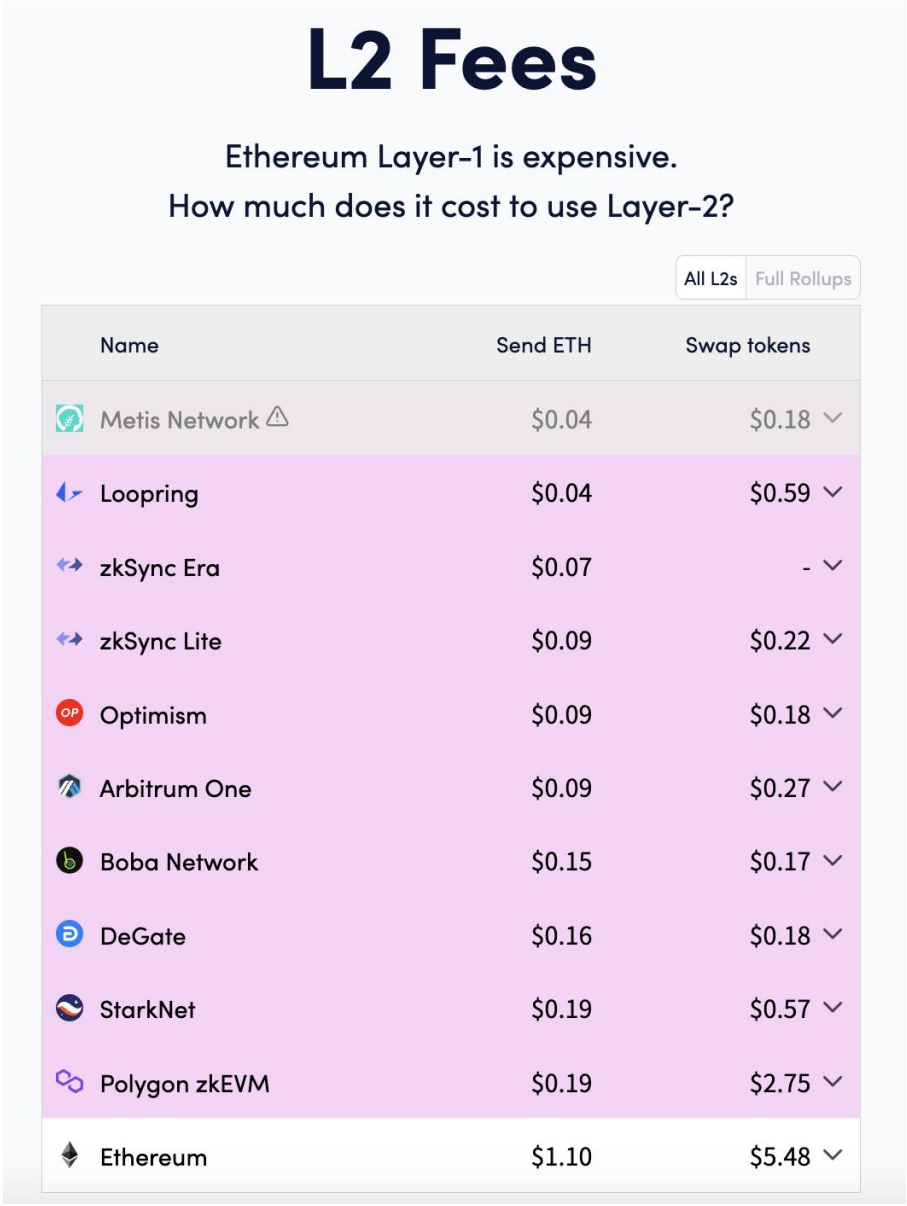
# Scalability with Rollups in Dapps

| Application | Bytes in rollup | Gas cost on layer 1 | Max scalability gain |
|---|---|---|---|
| ETH transfer | **12** | 21,000 | 105x |
| ERC20 transfer | **16** (4 more bytes to specify which token) | ~50,000 | 187x |
| Uniswap trade | **~14** (4 bytes sender + 4 bytes recipient + 3 bytes value + 1 byte max price + 1 byte misc) | ~100,000 | 428x |
| Privacy-preserving withdrawal (Optimistic rollup) | **296** (4 bytes index of root + 32 bytes nullifier + 4 bytes recipient + 256 bytes ZK-SNARK proof) | ~380,000 | 77x |
| Privacy-preserving withdrawal (ZK rollup) | **40** (4 bytes index of root + 32 bytes nullifier + 4 bytes recipient) | ~380,000 | 570x |

*Max scalability gain is calculated as (L1 gas cost) / (bytes in rollup * 16) * 12 million / 12.5 million.*

https://vitalik.eth.limo/general/2021/01/05/rollup.html

# Rollups Transaction Cost




https://l2fees.info/

# Types of Operators / Batch Producers

- Anyone can submit a batch
  - Potentially wasted effort on computing batches in parallel
- Centralized sequencer(s)
  - Allowlist of one or more actors
- Sequencer auction
  - Auction for who is sequencer for next day
- Proof-of-Stake (PoS)
- Delegated PoS (DPoS)

# Conclusions

- L2 Rollups seen as the way forward in the short / medium (may be even long term) for Ethereum scaling because of generality and easy dApps portability

- Current L2 schemes are still immature
    - But lots of activity in the industry / ecosystem

- Sharding is the long term way for scaling blockchain performance

# Workshop

**Part 1. send assets across different blockchains**

**Part 2. Interaction with wrapped ETH contract**

# Send assets across different blockchains

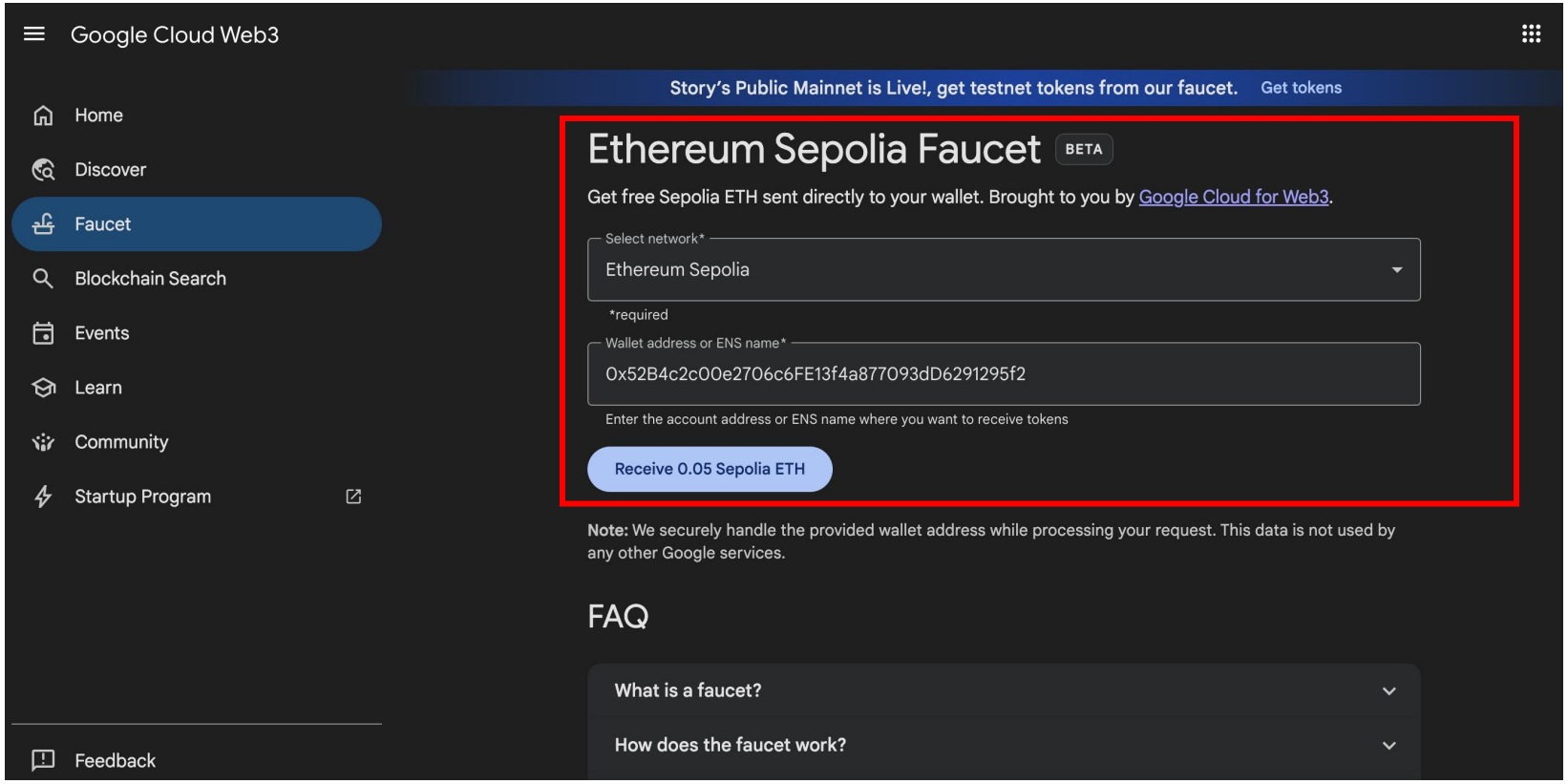
- The objective is to get a wrapped version of ETH tokens on the Amoy chain.
- Two blockchain: from Ethereum to Polygon.
- Use testnet to sent assets across different blockchains.
- Testnet: Ethereum Sepolia and Polygon Amoy.

# Send assets across different blockchains

## ■ Preparation

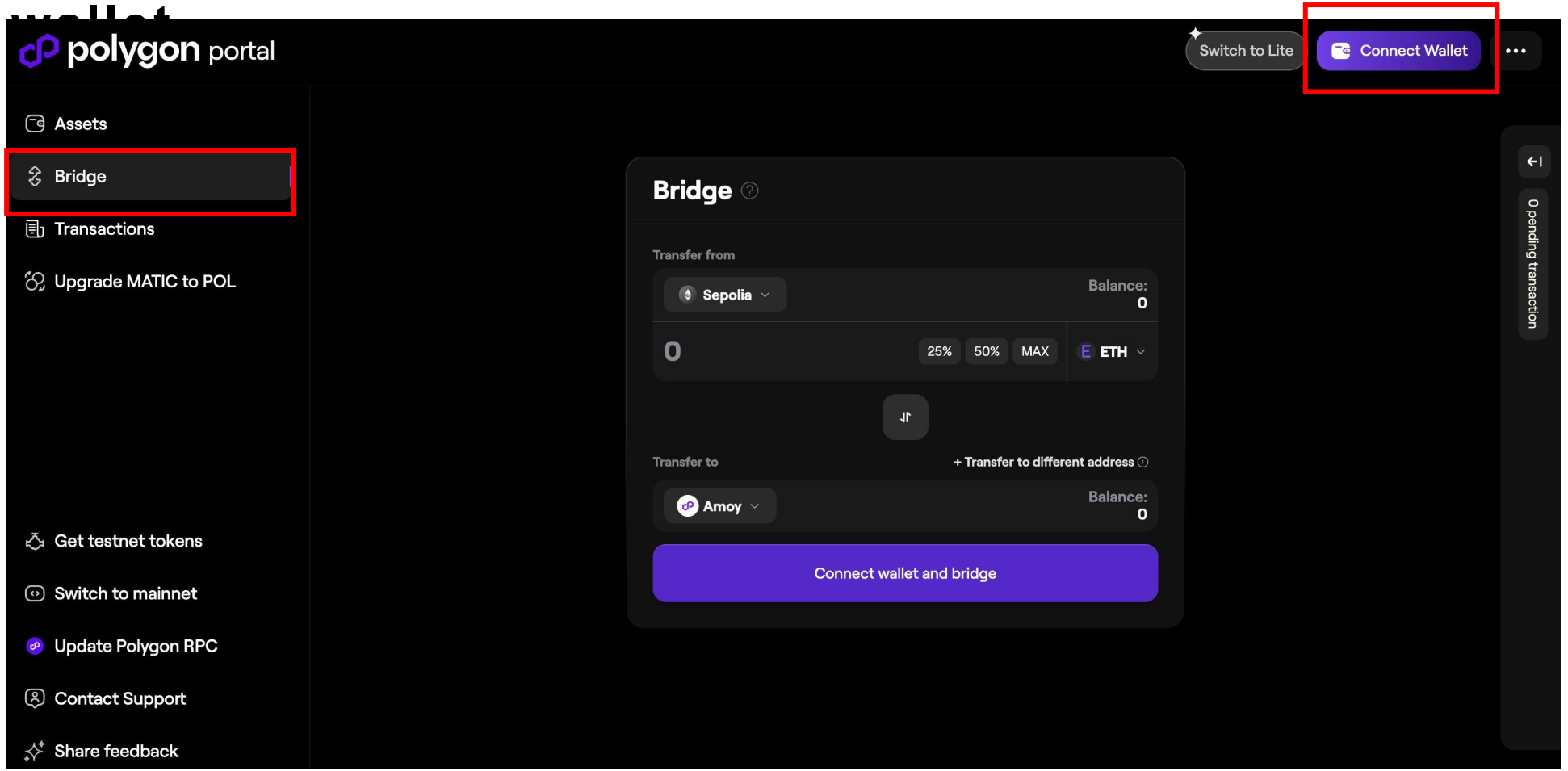
**Obtain test tokens to cover the gas fees required during deployment.**

Example: SepoliaETH for Ethereum Sepolia testnet



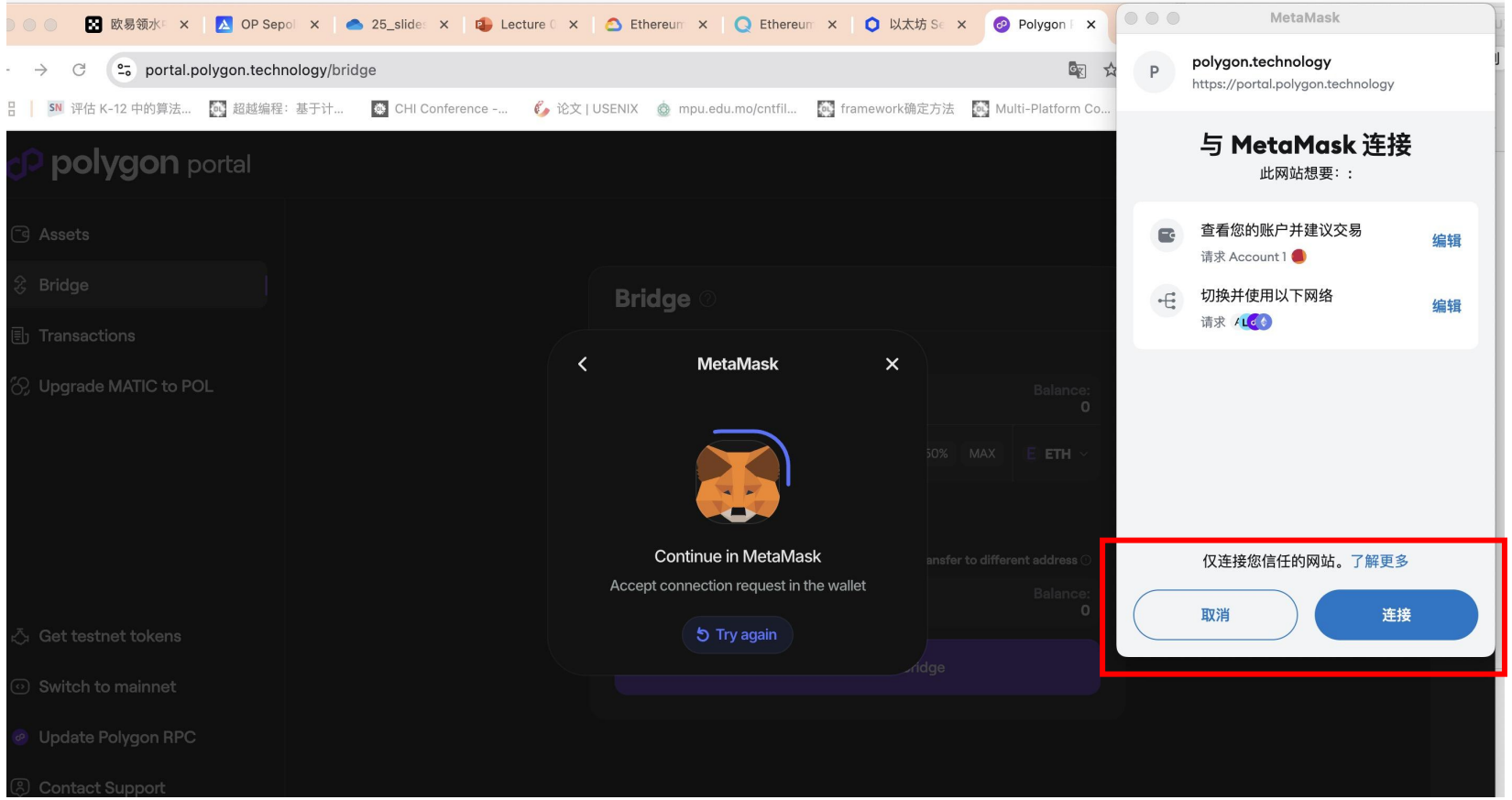
# Send assets across different blockchains

- **Step 1. Go to the GUI for the [bridge between Sepolia and Amoy] (https://portal.polygon.technology/bridge) and connect your wallet**



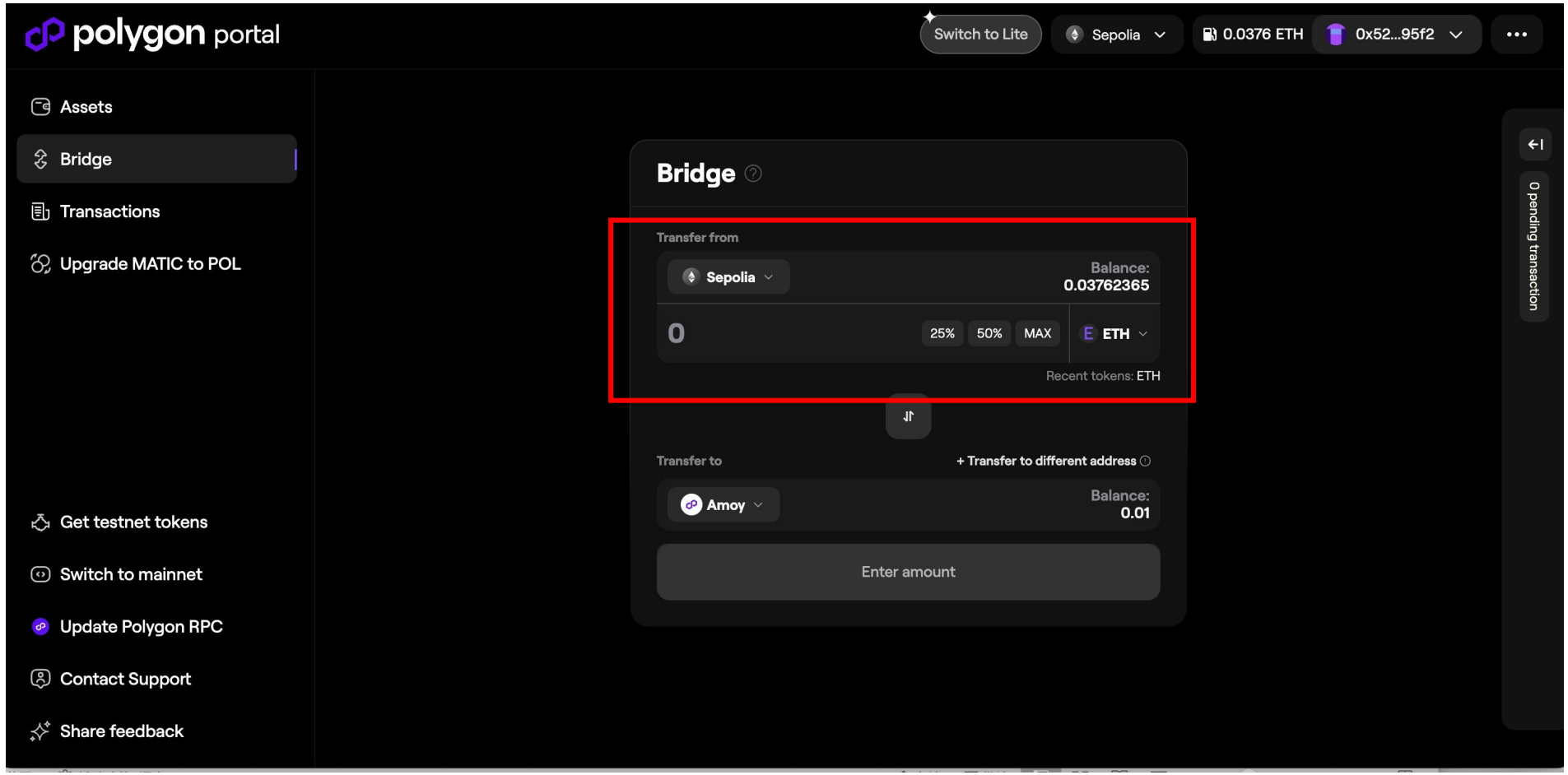
# Send assets across different blockchains

- **Step 1. Go to the GUI for the [bridge between Sepolia and Amoy] (https://portal.polygon.technology/bridge) and connect your wallet.**



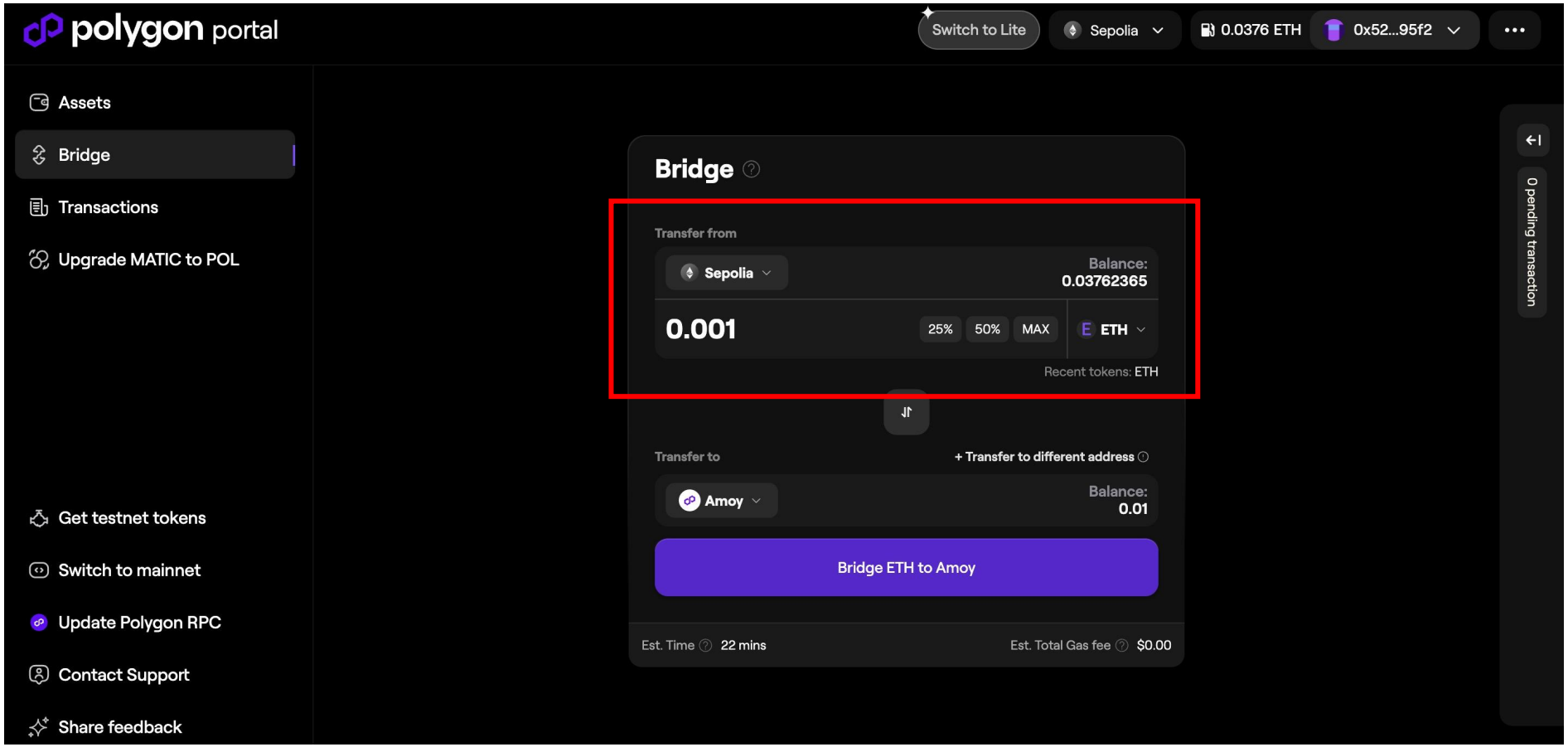
# Send assets across different blockchains

- **Step 2. Select ETH token and the Sepolia testnet in the "From" field.**



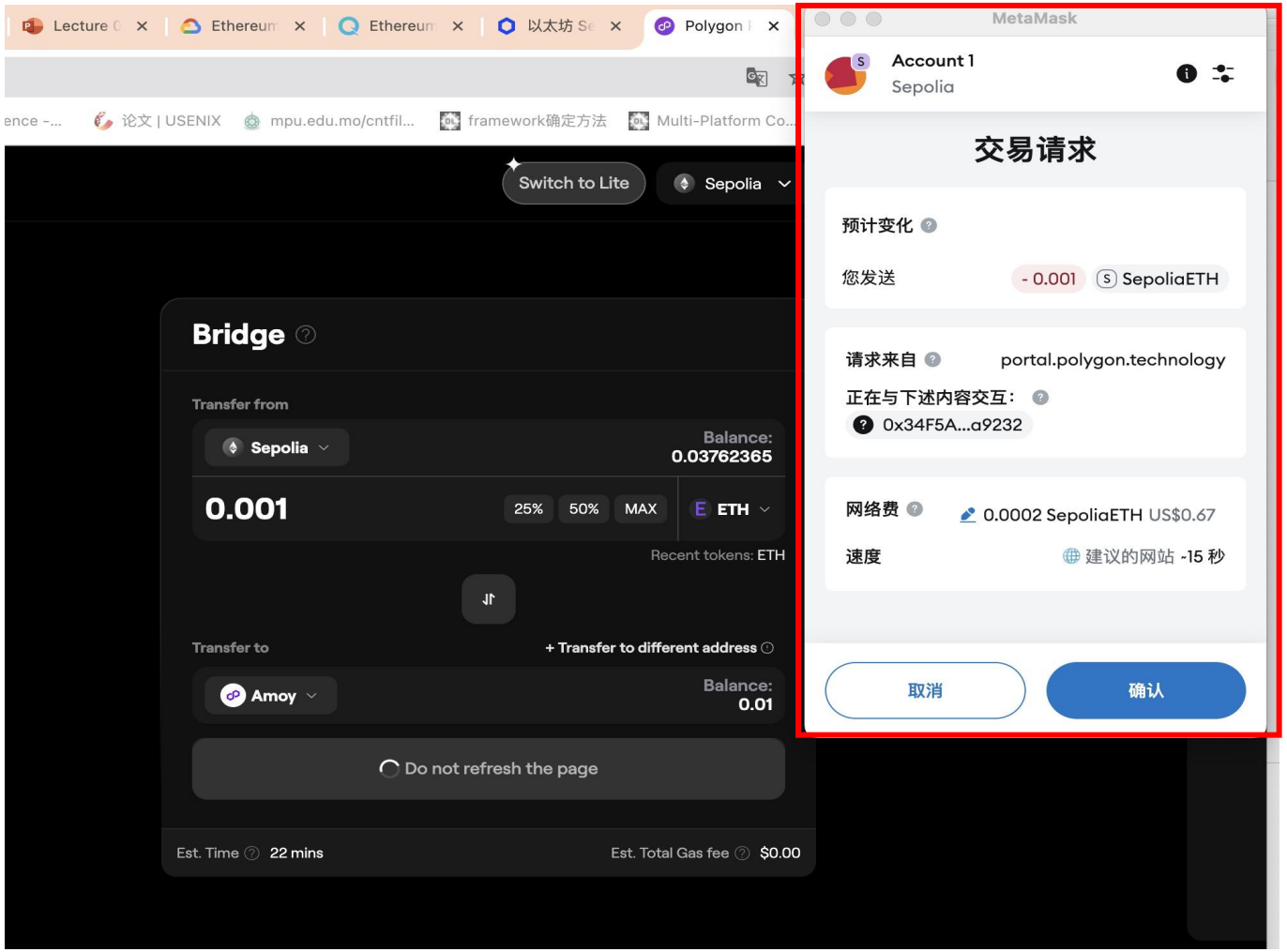
# Send assets across different blockchains

- **Step 3. Enter the amount of ETH you wish to transfer - say 0.001 ETH.**



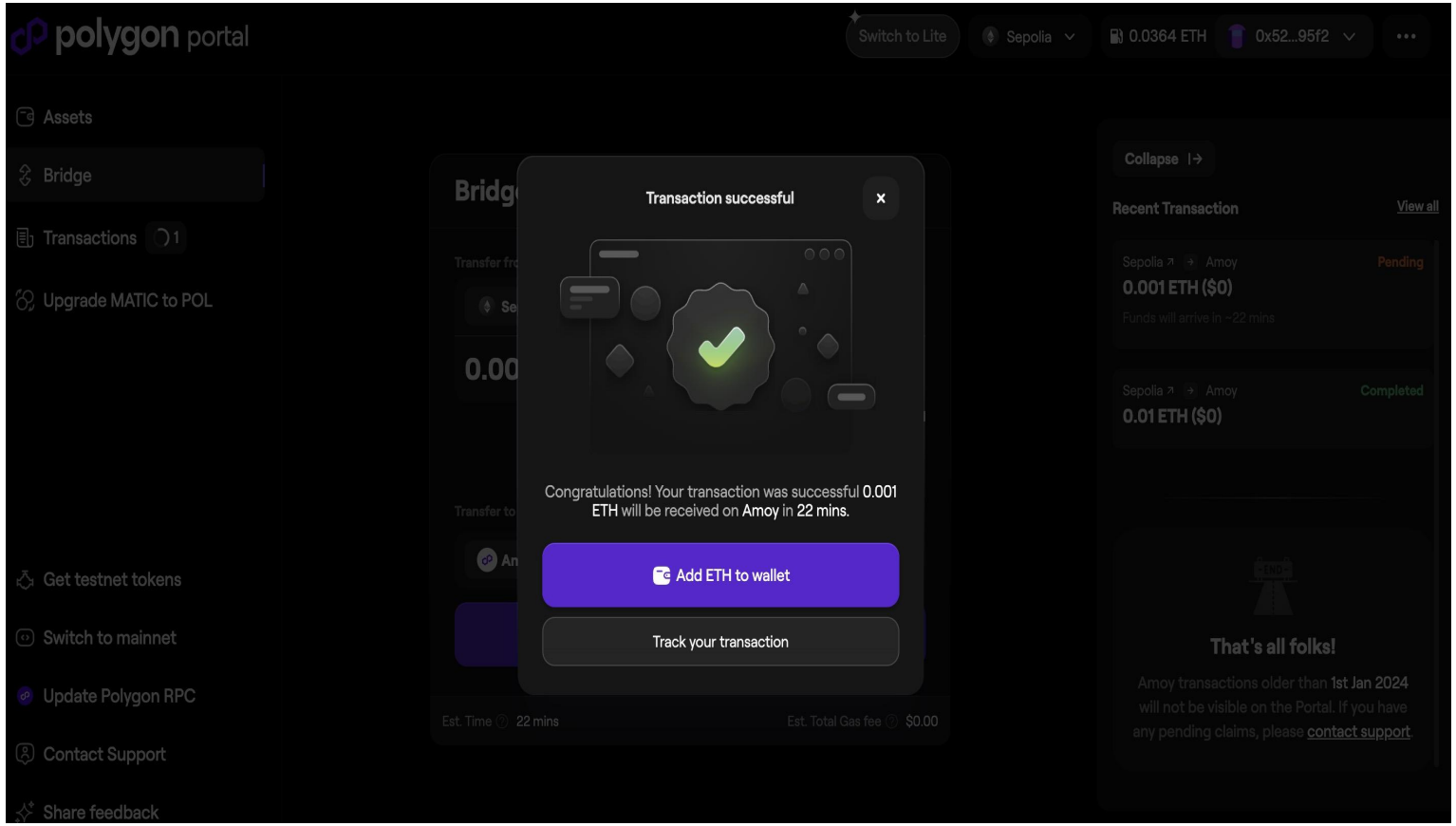
# Send assets across different blockchains

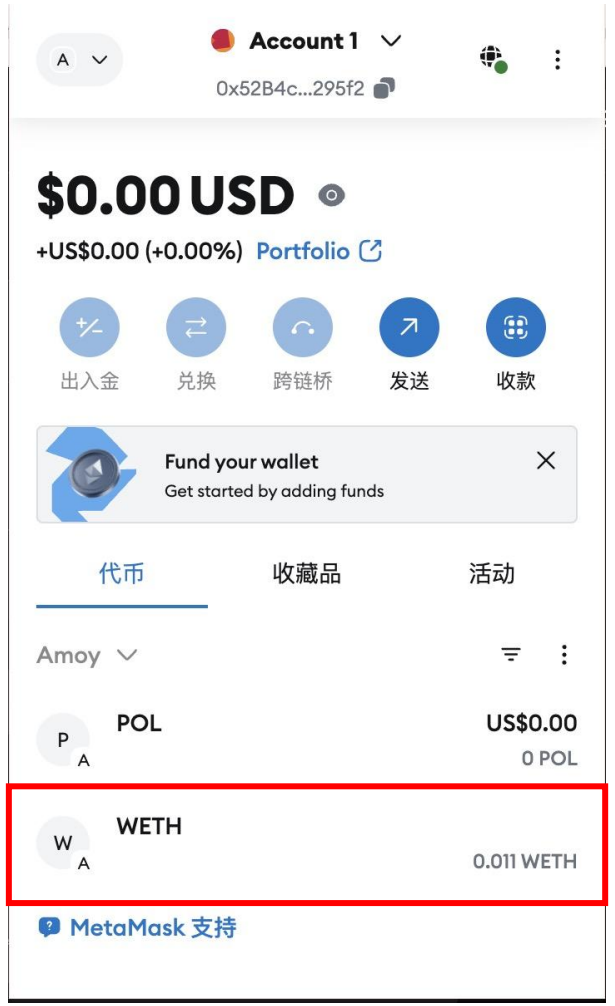
■ **Step 4. Click transfer and confirm the transaction.**



# Send assets across different blockchains

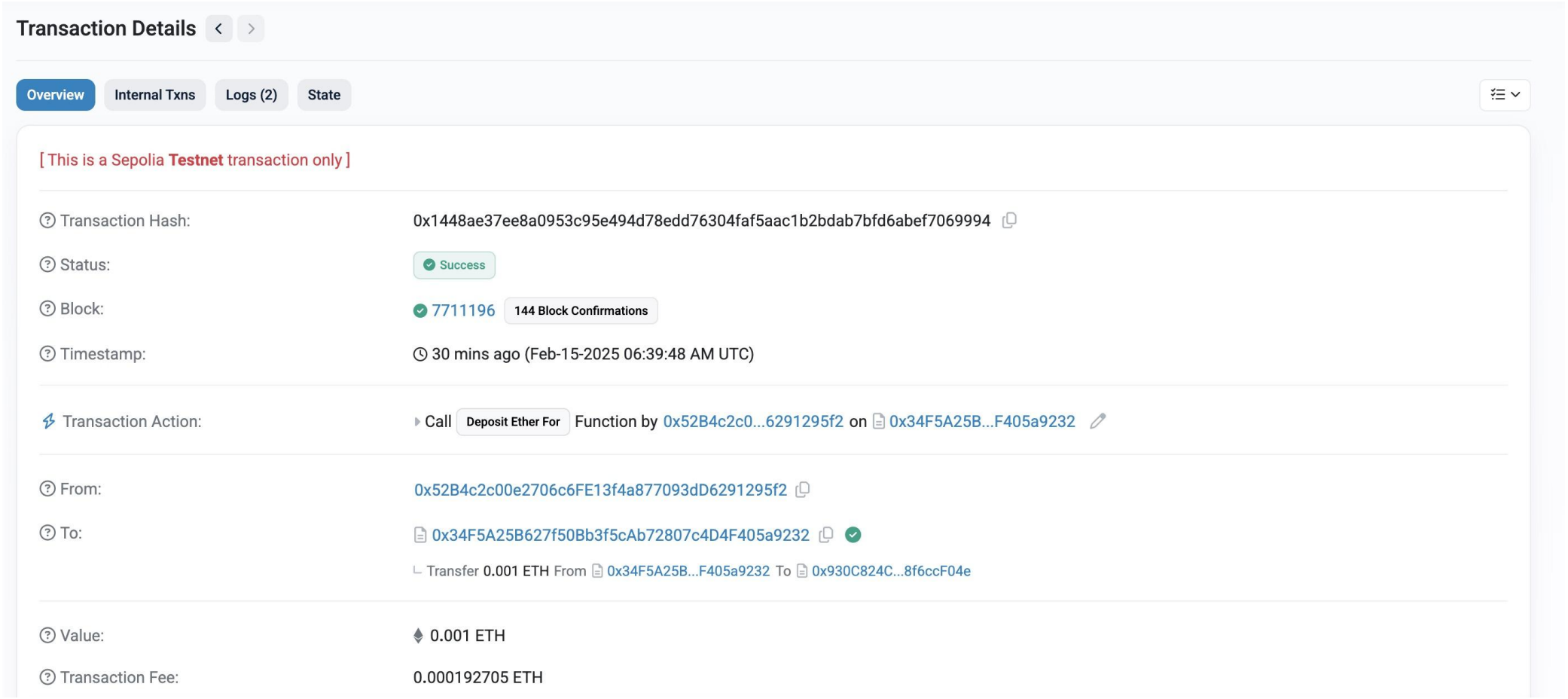
## ■ Step 5. Track the transaction via your wallet.



# Send assets across different blockchains

## ■ View the transaction on Etherscan.



# Interact with the wrapped ETH smart contract

## ■ Step 1. Import WETH to your wallet.

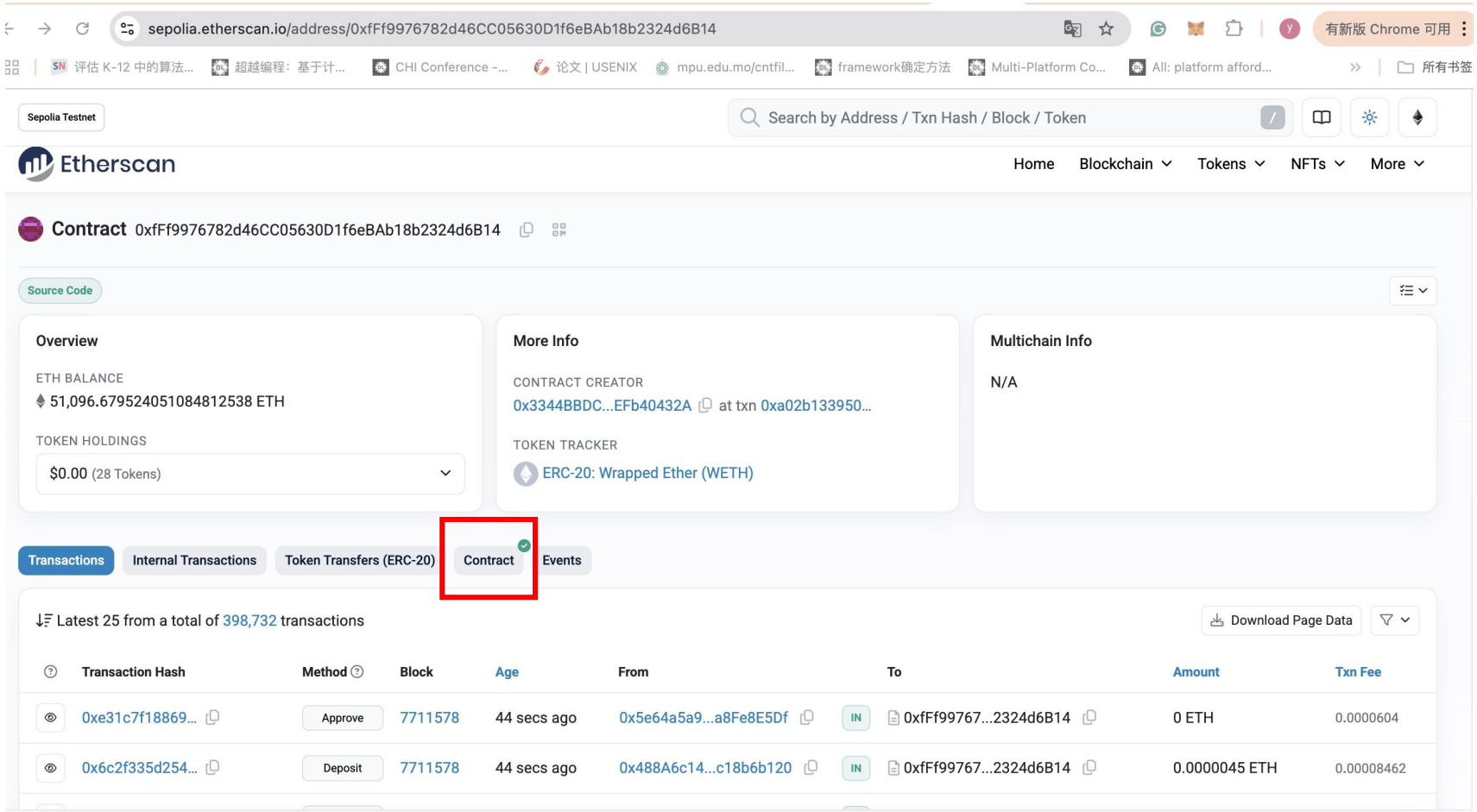
**The contract for converting ETH to WETH is:**
**0xfFf9976782d46CC05630D1f6eBAb18b2324d6B14**





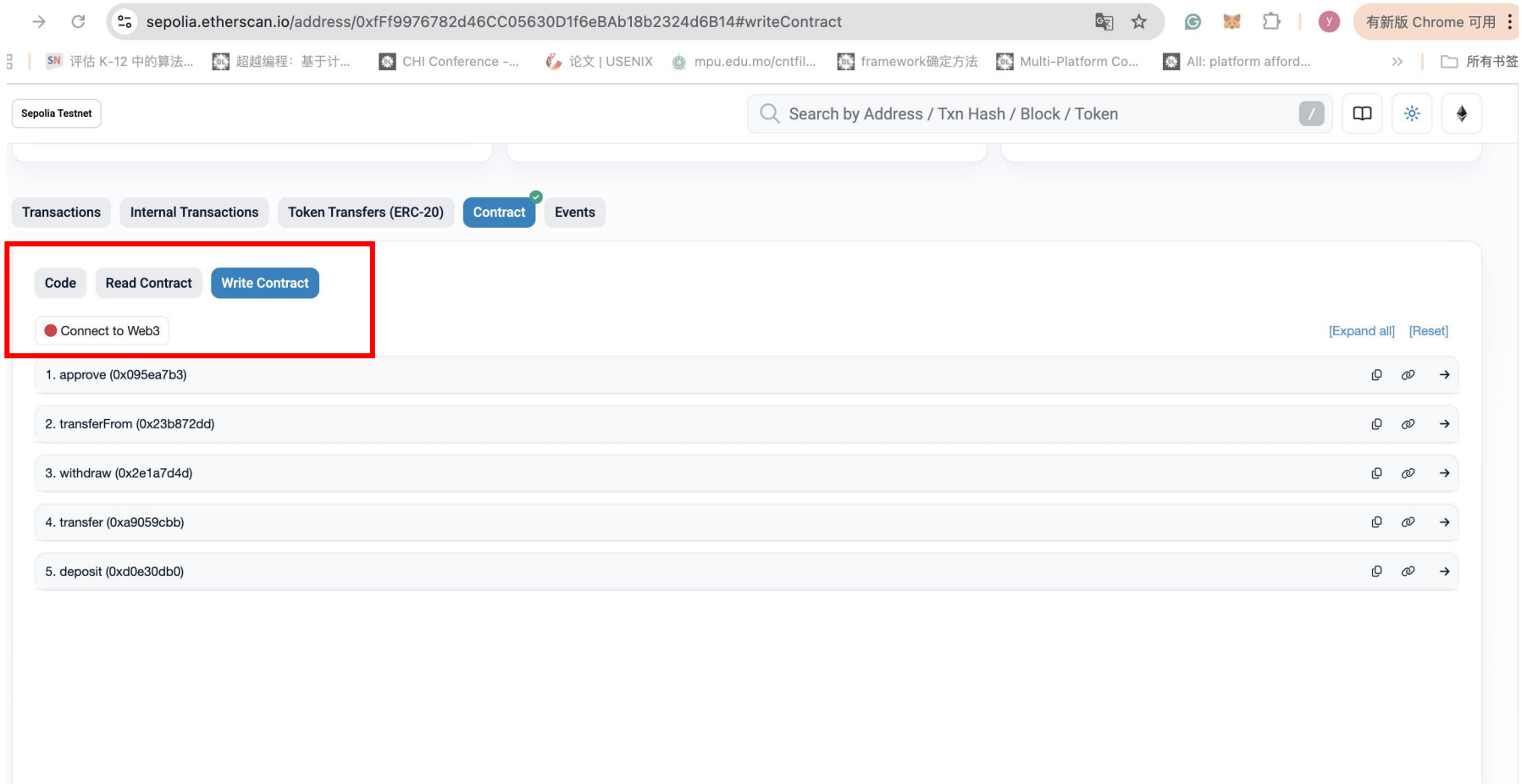

# Interact with the wrapped ETH smart contract

## ■ Step 2. Go to write contract.



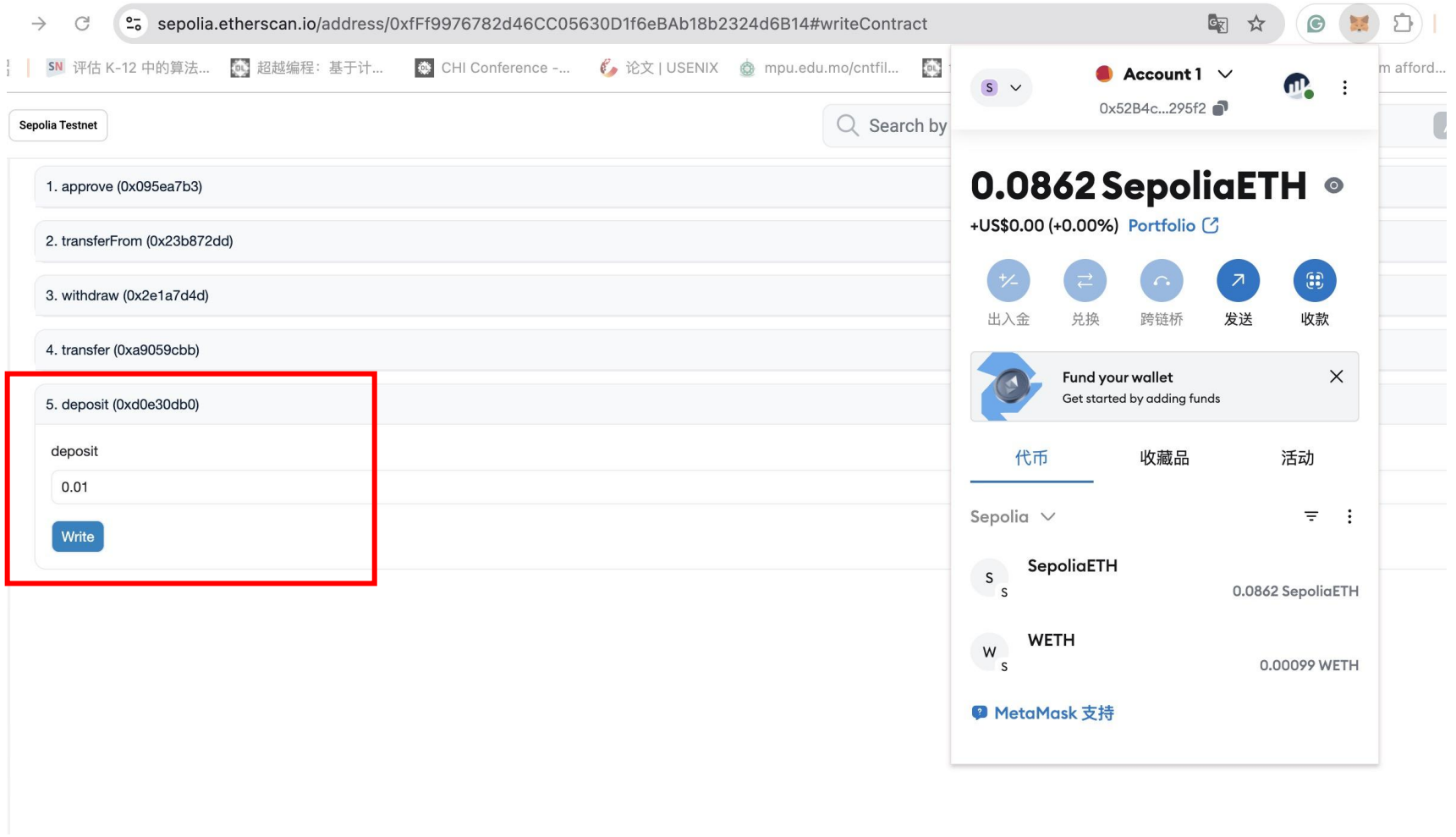
# Interact with the wrapped ETH smart contract

## ■ Step 3. Connect your wallet to Web3.



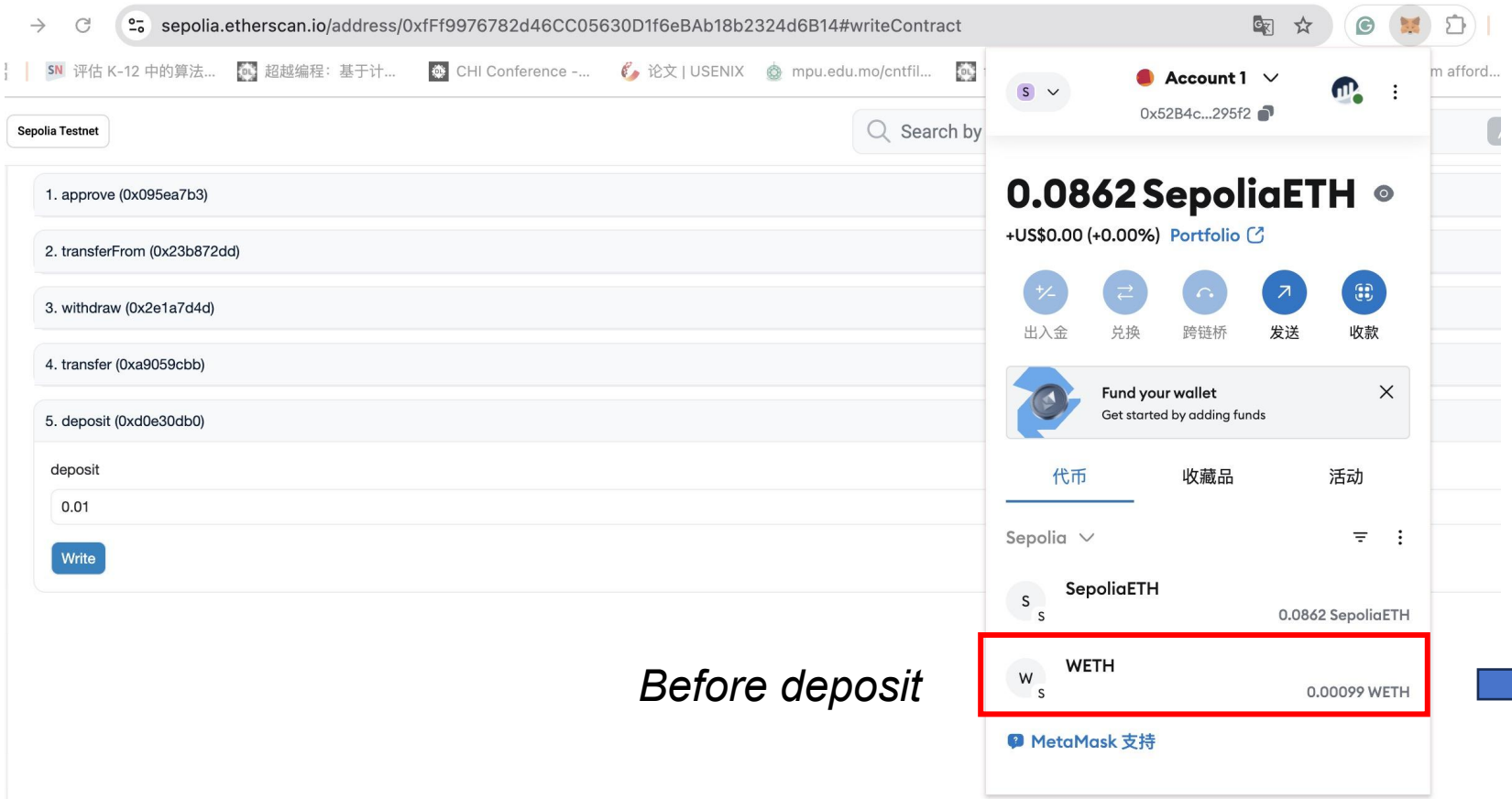
# Interact with the wrapped ETH smart contract

## ■ Step 4. Deposit 0.01 ETH in the contract.



# Interact with the wrapped ETH smart contract

- **Step 5. Check your WETH balance - you should have the same amount of WETH.**




*Before deposit*

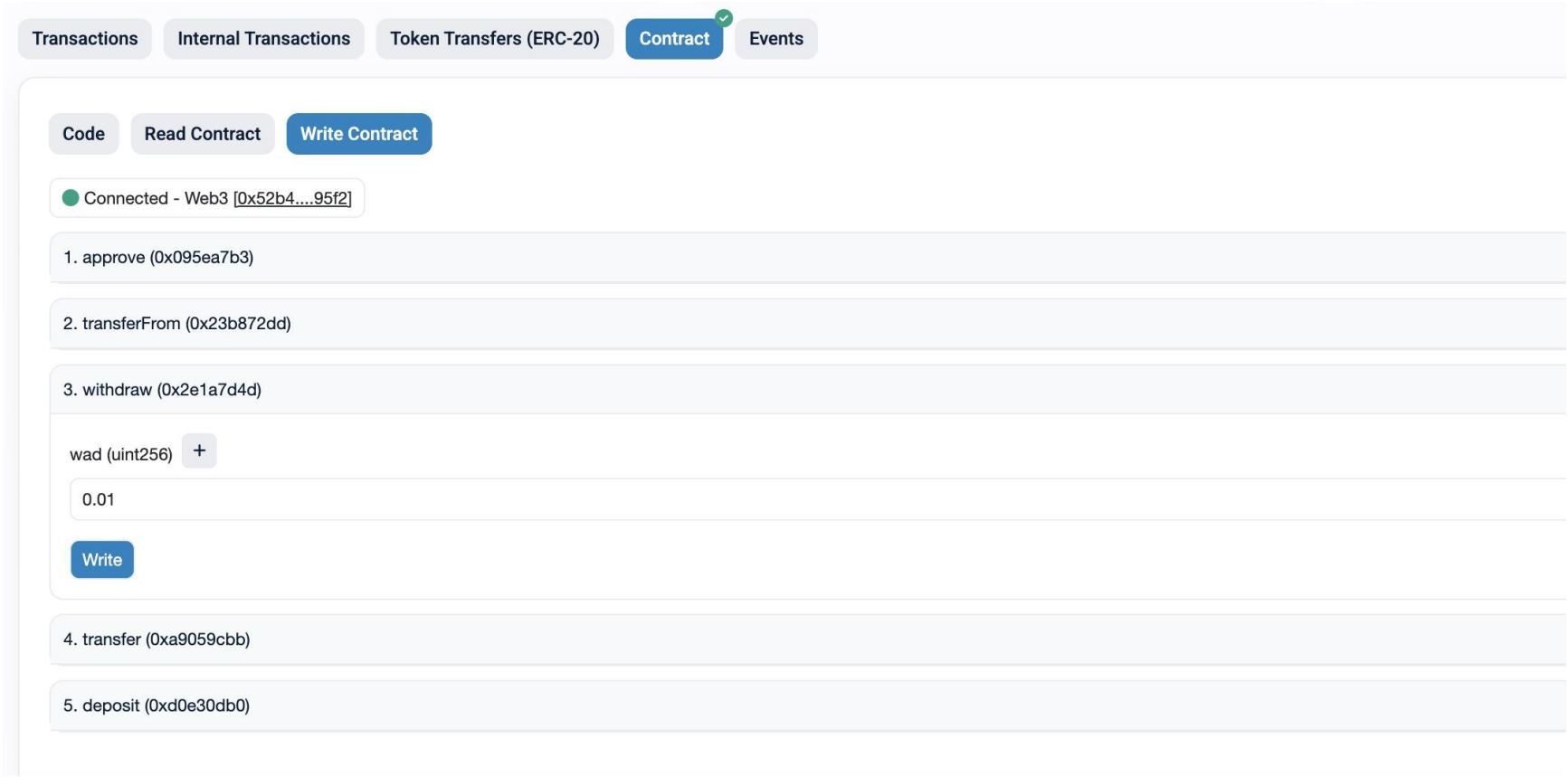



*After deposit*



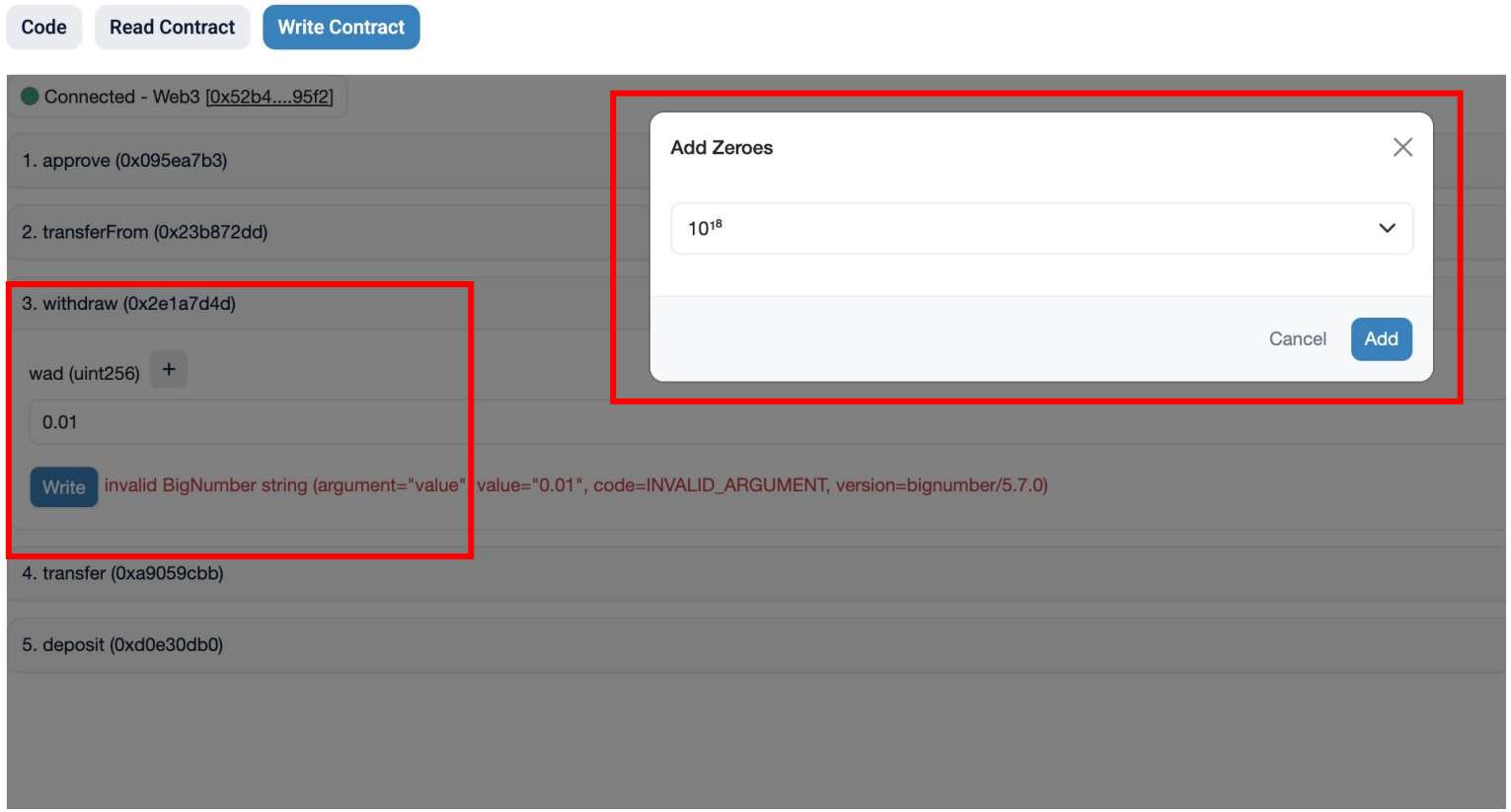
# Interact with the wrapped ETH smart contract

- **Step 6. Use the withdraw function to get your ETH back.**
- **Withdraw 0.01 ETH in the contract.**



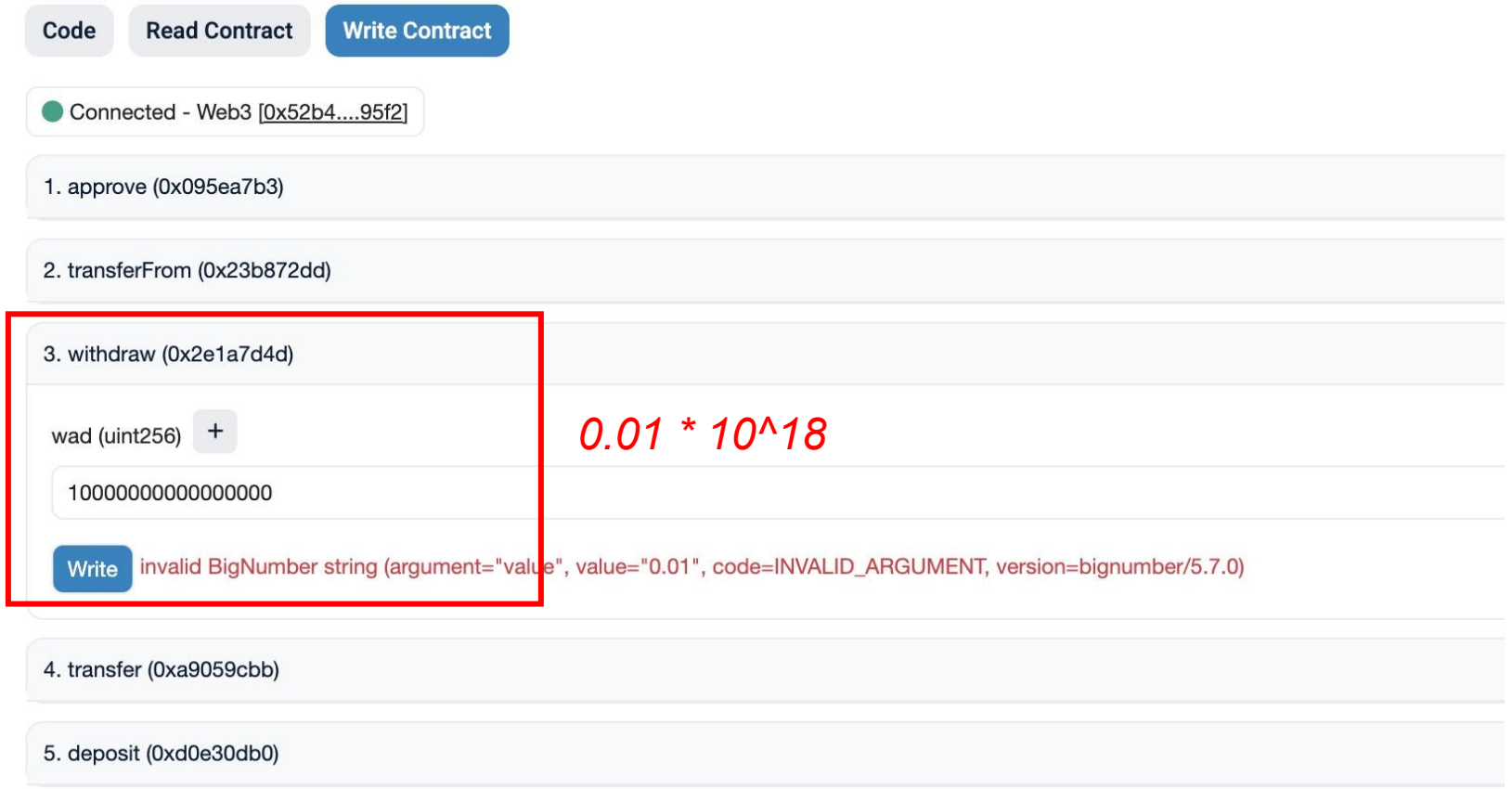
# Interact with the wrapped ETH smart contract

- **Step 6. Use the withdraw function to get your ETH back.**
- A wad is a decimal number with 18 digits of precision that is being represented as an integer.



# Interact with the wrapped ETH smart contract

- **Step 6. Use the withdraw function to get your ETH back.**
- A wad is a decimal number with 18 digits of precision that is being represented as an integer.

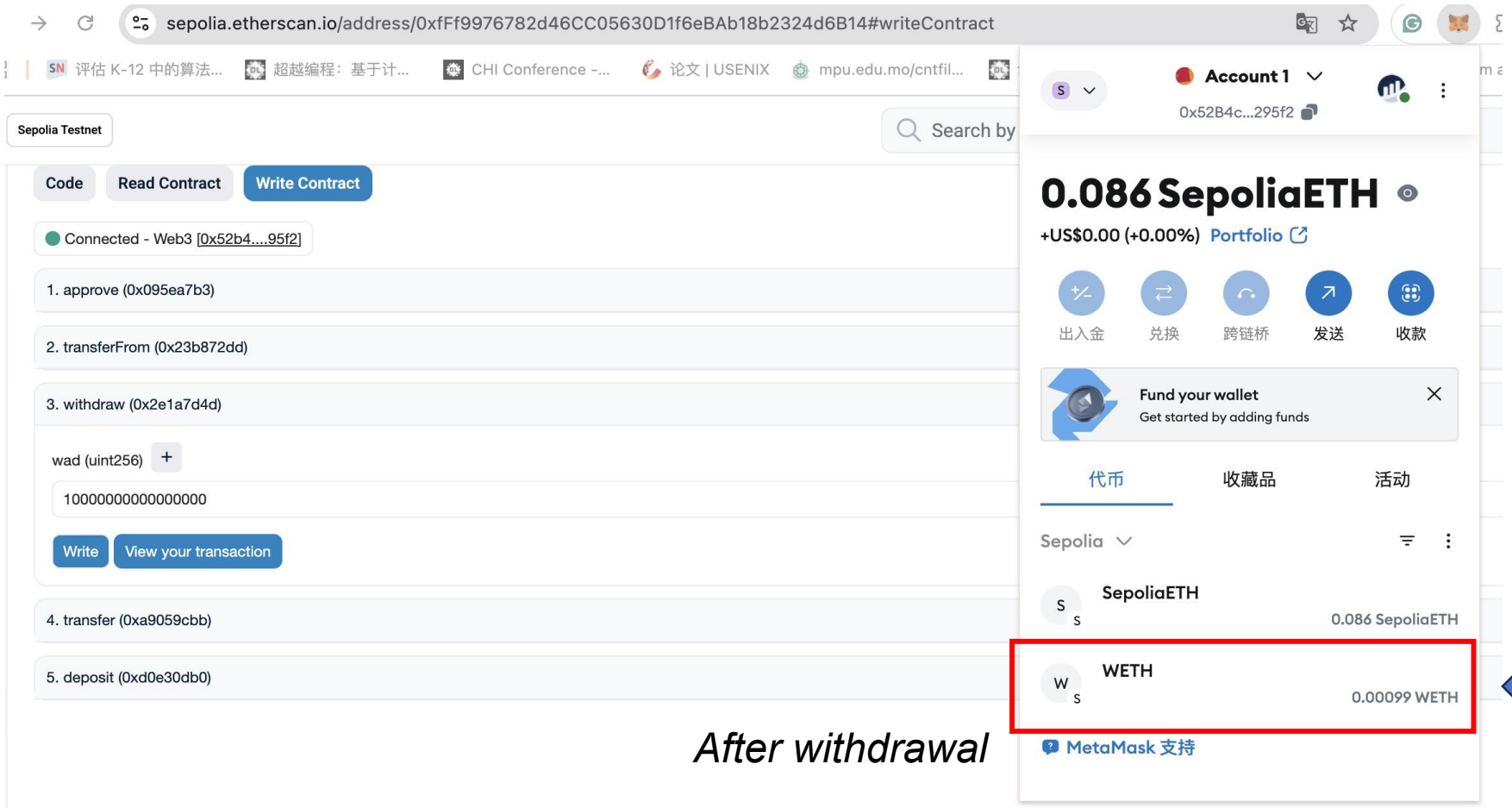



*0.01 * 10^18*



# Interact with the wrapped ETH smart contract

## ■ Step 7. Check your WETH balance.




*After withdrawal*




*Before withdrawal*

